

Proven C Book

proven 라이브러리에 기반한 모던 C 입문

rubidus

rubidus@gmail.com <https://github.com/rubidus-api>

Proven C Book — proven 라이브러리에 기반한 모던 C 입문

지은이 rubidus

rubidus@gmail.com · <https://github.com/rubidus-api>

이 책의 **본문**은 크리에이티브 커먼즈 저작자표시-비영리-동일조건변경허락 4.0 국제 라이선스(CC BY-NC-SA 4.0)에 따라 이용할 수 있다. 출처를 밝히면 자유롭게 공유하고 고칠 수 있으나, 영리 목적 이용은 허용되지 않으며, 고친 결과물에는 같은 라이선스를 적용해야 한다.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

이 책에 수록된 **예제 코드**는 MIT 라이선스로 배포한다 — 배운 것을 자기 프로그램에 제약 없이 가져다 쓸 수 있게 하려는 뜻이다. 예제에서 사용한 proven 라이브러리는 그 자체의 라이선스를 따른다.

이 책의 모든 코드 시연은 실제로 컴파일·실행해 얻은 출력을 그대로 인쇄한 것이다. 조판은 Typst로 했다.

차례

머리말	10
저작권과 연락	10
1 배경설명	12
1.1 프로그래밍이란 무엇인가	12
1.2 C는 어디에 있는가	12
1.3 지금 C의 세계는 요동치고 있다	12
1.4 왜 이 책을 만들었나	13
1.5 이 책을 읽는 법	14
2 단순한 기계 모델 ↔ C의 탄생	16
2.1 컴퓨터를 세 부품으로 그리기	16
2.2 비트 — 정보의 최소 단위	17
2.3 그 단순한 기계의 시절, C가 태어났다	18
3 워드와 주소 ↔ C의 원형	20
3.1 사물함 복도	20
3.2 워드 — 기계의 자연스러운 한 움큼	20
3.3 엔디안 — 여러 칸에 수를 넣는 두 가지 순서	21
3.4 C의 원형이 여기 있다	22
4 주소의 특수 지식 — 0번지, 정렬, 하위 비트	23
4.1 0번 사물함에는 무엇이 있는가	23
4.2 널 삼형제 — 이름만 닮은 세 가지	24
4.3 정렬 — 두 칸짜리 짐은 아무 데나 못 넣는다	24
4.4 하위 비트의 재주 — 태그 포인터	24
5 정수의 표현 — 부호, 오버플로, 시프트	26
5.1 부호 없는 정수 — 시계처럼 도는 수	26
5.2 음수를 담는 세 가지 약속	26
5.3 세 약속의 경쟁, 그리고 C23의 결단	27
5.4 시프트 — 비트를 통째로 밀기	28
5.5 부호 확장 — 좁은 그릇에서 넓은 그릇으로	29
6 수의 표현 ↔ IEEE 754라는 계약	31
6.1 고정소수점 — 소수점을 약속으로 못박다	31
6.2 부동소수점 — 소수점을 데이터에 싣다	31
6.3 근사가 일으키는 세 가지 사건	32
6.4 난립, 그리고 IEEE 754라는 계약	33
7 문자와 텍스트 ↔ 표준 속의 흉터	35
7.1 문자는 번호다	35
7.2 ASCII와 EBCDIC — 두 개의 표	35
7.3 ISO 646 — 국가 변형과 C의 삼중자	35
7.4 8비트의 백가쟁명, 그리고 한글	36
7.5 유니코드와 UTF-8 — 하나의 표, 영리한 담기	36
7.6 같은 글자, 여러 표현 — 텍스트에 잠복한 보안 지형	37
7.7 글자들의 열 — 문자열을 담는 방식들	38
8 스트림의 기원 ↔ 펀치카드, 라인프린터, 프린터 터미널	40
8.1 펀치카드 — 한 장이 한 줄이다	40

8.2	라인프린터 — 줄 단위로 찍는 출력	40
8.3	프린터 터미널 — 종이 위의 대화, 그리고 tty	40
8.4	화면 터미널, 그리고 스트림이라는 유산	41
8.5	띠를 갈아 끼우다 — 리다이렉션과 파이프라인	41
9	기억의 분화 — 레지스터, 캐시, 다층의 사다리	43
9.1	고백 — 그 그림은 과하게 단순했다	43
9.2	레지스터 — 원래부터 있던, CPU의 손안	43
9.3	벌어지는 격차 — 레지스터와 메모리 사이	43
9.4	캐시 — 격차의 틈에 끼어든 중간층	44
9.5	캐시의 분화 — 다층의 사다리	44
10	속도의 기계장치 ↔ 표준의 탄생	46
10.1	파이프라인 — 조립 라인 위의 명령들	46
10.2	분기 예측 — 갈림길을 미리 짐작하다	46
10.3	클록의 한계, 그리고 멀티코어	47
10.4	같은 시대, C는 계약서를 썼다 — 표준 이전의 C와 C89	47
11	컴파일러 최적화 ↔ 추상 기계	49
11.1	편집자의 작업실 — 컴파일러가 하는 일	49
11.2	추상 기계 — 계약서 속의 가상 기계	49
11.3	눈에 보이는 편집 — 값을 언제 읽고 언제 쓰는가	50
11.4	계약의 정밀화 — C99, C11	51
11.5	잘못된 신호 — 엄격한 앨리어싱 위반이 부르는 유령	51
12	그래서 C는 추상적인 언어다	53
12.1	논제의 완성 — 세 조각을 합치다	53
12.2	요즘의 흐름 — 포인터에 붙는 출처	53
12.3	마무리 파노라마 — C는 어느 기계의 언어인가	54
12.4	제2부를 닫으며	54
13	헬로 월드	56
13.1	첫 프로그램	56
13.2	한 줄씩 읽기	56
14	일반적인 컴파일 과정	57
14.1	겉보기 — 명령 한 줄	57
14.2	1주자 — 전처리: 텍스트를 짜깁기한다	57
14.3	2주자 — 컴파일: C를 어셈블리로	57
14.4	3주자 — 어셈블: 기계 명령으로 찍어 내다	58
14.5	4주자 — 링크: 조각을 잇고 구멍을 메운다	58
15	개발환경 구축	59
15.1	필요한 것은 셋뿐이다	59
15.2	경고 — 공짜 검토를 켜 두어라	60
15.3	한글이 깨질 때 — 인코딩의 사슬	60
15.4	디버거 — 멈춰 세워 들여다보기	61
15.5	디버그 빌드의 옵션 — -g와 -O0은 다른 스위치다	62
15.6	디버거의 맹점 — 최적화된 빌드	63
15.7	디버그에선 되는데 릴리스에서만 틀린다	64
15.8	디버거를 믿기 어려울 때의 도구들	64

15.9	새니타이저 — 실행 중에 치는 그물	65
15.10	제3부를 닫으며	66
16	프로그램의 구조	68
16.1	주석 — 사람만 읽는 글	68
16.2	문장 — 한 걸음, 그리고 세미콜론	68
16.3	블록 — 문장들의 묶음	69
16.4	문장이 아닌 줄 — #의 세계	69
16.5	종합 — 두 문장짜리 프로그램	69
17	수식 — 값이 되는 것	70
17.1	리터럴 — 소스에 적는 값	70
17.2	수식 — 계산되어 값이 되는 것	70
17.3	순서 — 우선순위, 그리고 괄호라는 관행	71
17.4	미리 심는 주의 하나 — 계산의 시간 순서는 다를 수 있다	71
18	함수 사용 — 부르는 법	72
18.1	호출 — 이름을 부르면 일꾼이 된다	72
18.2	반환값 — 쓰거나, 버리거나	72
18.3	표준 라이브러리 — 미리 만들어진 일꾼들	73
19	출력	74
19.1	printf — 서식이 있는 출력	74
19.2	짜 맞추기 — 서식과 재료의 계약	75
19.3	제4부를 닫으며 — 헬로 월드 재독	75
20	변수 선언	78
20.1	선언 — 타입, 이름, 그리고 첫 값	78
20.2	대입 — 상태를 바꾸는 부수효과	78
20.3	const — 바꾸지 않겠다는 약속	79
21	함수 선언과 정의	80
21.1	정의 — 일꾼을 만드는 문법	80
21.2	선언 — 계약 서명만 미리	81
21.3	스코프 — 이름이 보이는 범위	81
21.4	외상 청산 — int main(void)	81
22	입력	83
22.1	두 단계 — 읽고, 해석한다	83
22.2	해석은 실패할 수 있다	84
22.3	제5부를 닫으며	84
23	정수 — 유한한 수의 세계	86
23.1	정수 타입 가족	86
23.2	경계를 눈으로 본다	86
24	정수의 연산 — 나눗셈, 비트	88
24.1	나눗셈과 나머지 — 버림의 방향	88
24.2	비트 연산 — 5장의 세계, C의 문법	89
24.3	형변환 — 그릇 사이를 건너다	89
25	암묵적 변환 — 승격과 통상 산술 변환	90
25.1	규칙 1 — 정수 승격	90
25.2	규칙 2 — 통상 산술 변환	91

25.3	규칙 3 — 가변 인자의 기본 진급	91
26	불리언과 비교	93
26.1	bool — 두 값짜리 타입	93
26.2	비교 — bool을 만드는 수식	93
26.3	논리 연산자 — 판단을 엮는다	94
27	결정 — if와 switch	95
27.1	if — 조건이 참이면	95
27.2	switch — 값으로 갈라서는 판	96
28	반복 — 루프와 불변식	98
28.1	루프 3형제	98
28.2	재회 — Duff's device	99
29	함수의 의미 — 값 복사와 부수효과	101
29.1	값 복사 — 원본은 안전하다	101
29.2	부수효과와 평가 순서 — 씨앗의 회수	102
29.3	재귀 — 자신을 부르는 함수	102
29.4	마지막 연산자, 그리고 계약이라는 씨앗	103
30	객체, 주소, 포인터	105
30.1	세 가지 표기 — 선언, &, *	105
30.2	sizeof — 그릇의 크기를 묻다	106
31	널 — 삼형제의 정식 취급	107
31.1	nullptr — 비어 있음의 이름	107
31.2	함정 — 표현을 0으로 가정하는 코드	108
32	포인터의 규칙 — 정렬과 프로버넌스	109
32.1	정렬과 캐스트 — char*의 특권	109
32.2	프로버넌스 — 번호가 같아도 출처가 다르면	109
33	배열	111
33.1	선언, 접근, 순회	111
33.2	배열과 포인터 — 무너짐의 진실	111
33.3	경계 — 생사의 규칙	112
34	문자열	113
34.1	정체 — 배열, 그리고 약속	113
34.2	문자열 리터럴 — 읽기 전용의 땅	114
35	안전한 입력과 proven의 등장	115
35.1	사고의 해부 — 경계 침범이라는 유행병	115
35.2	proven — 검증된 기본기라는 부품	115
36	수명과 저장 기간	117
36.1	두 가지 수명	117
36.2	스택 — 호출의 장부	118
37	동적 메모리	119
37.1	빌리고, 돌려준다	119
37.2	소유 — 누가 돌려줄 책임을 지는가	120
37.3	제7부를 닫으며	120
38	구조체	122
38.1	선언, 초기화, 접근	122

38.2	구조체는 값이다	123
39	공용체와 표현	124
39.1	공용체 — 겹쳐 놓기	124
39.2	표현을 눈으로 — 앤디안과 패딩	124
39.3	제8부를 닫으며	126
40	실수 — 근사의 수학	128
40.1	타입 선택과 비교	128
40.2	특수값 — 무한대와 NaN	129
41	오류와 계약	130
41.1	오류는 값이다 — C의 방식	130
41.2	계약을 코드로 — assert와 방어	131
41.3	const — 가장 값싼 계약	131
42	정의되지 않은 동작	133
42.1	세 가지 회색지대 — UB, 미지정, 구현 정의	133
42.2	왜 존재하는가	133
42.3	“무엇이든 가능함”의 실제 얼굴	133
42.4	피하는 법 — 규율, 도구, 그리고 부품	134
42.5	제9부를 닫으며	135
43	여러 파일 — 분할과 링크	137
43.1	번역 단위와 헤더	137
43.2	연결 — 이름이 파일 경계를 넘는가	137
44	전처리기와 번역 단계	139
44.1	매크로 — 이름을 토큰 열로 바꾸기	139
44.2	# — 토큰을 문자열로 (stringize)	139
44.3	## — 토큰 두 개를 하나로 (token paste)	140
44.4	두 겹 우회 — 왜 한 겹으로는 안 풀리는가	140
44.5	번역 단계 — 표준이 못박은 여덟 걸음	141
45	가변 인자 함수	142
45.1	네 개의 도구	142
45.2	역사 — varargs에서 stdarg로	143
46	표준 라이브러리의 지형	145
46.1	지도 — 도구 상자들	145
46.2	왜 얇은가 — 설계와 역사	145
46.3	출력 서식 문자열의 해부	145
46.4	입력 서식 문자열 — 무엇이 다른가	147
46.5	조심할 자리들	149
47	50년째 출하되는 다섯 가지 버그	152
47.1	하나 — 문자열 함수는 그릇의 크기를 모른다	152
47.2	둘 — 실패를 확인하게 만드는 장치가 없다	153
47.3	셋 — printf는 여러분이 말하는 것을 그대로 믿는다	155
47.4	넷 — 이걸 누가 해제하나	155
47.5	다섯 — 아무도 타입을 검사해 줄 수 없는 콜백	155
47.6	여섯 번째 — 바이트에는 타입이 있다	157
48	시작하기 — 설치할 것이 없다	158

48.1	설치할 것이 없다는 선택	158
48.2	이 책의 예제는 어떻게 빌드되는가	158
48.3	첫 프로그램	159
49	에러는 값이다	161
49.1	두 가지 반환 모양	161
49.2	버리면 컴파일러가 항의한다	163
49.3	실패했을 때 남는 것 — 실패 원자성	163
49.4	돌려줄 상대가 없을 때 — 패닉	164
50	기반 — 바이트, 뷰, 그리고 넘치지 않는 산술	165
50.1	바이트에는 이름이 있다	165
50.2	뷰 — 포인터와 길이를 하나로	165
50.3	넘치지 않는 크기 산술	167
50.4	정렬 — 다음 경계로 밀어 올리기	167
51	할당은 매개변수다	169
51.1	할당자는 값이다	169
51.2	아레나 — 수명이 같은 것들을 한 번에	170
51.3	소유와 빌림, 그리고 자기를 가리키는 상태	172
52	문자열과 텍스트	173
52.1	두 개의 타입, 하나의 규칙	173
52.2	거부하되, 자르지 않는다	174
52.3	찾기와 자르기 — 복사 없는 텍스트 처리	175
52.4	두 세계의 경계 — NUL 종단과 UTF-16	175
53	형식화와 파싱 — 타입을 두 번 적지 않기	177
53.1	{ } — 타입이 없는 자리표시자	177
53.2	반대 방향 — 스캐너	179
54	컨테이너와 알고리즘	181
54.1	자라는 배열	181
54.2	침습적 리스트와 링 버퍼	182
54.3	해시 맵, 그리고 공격받는 자료구조	182
54.4	최악을 보장하는 정렬	184
54.5	바이트를 글자로 — 해시와 인코딩	185
55	바깥 세계 — 파일, 스트림, 시간, 난수	186
55.1	파일 — 열고, 읽고, 닫기	186
55.2	스트림 — 버퍼를 둔 읽고 쓰기	188
55.3	시간 — 두 가지 다른 시계	188
55.4	난수 — 용도가 물건을 정한다	188
55.5	메모리 매핑	190
56	경계 — 겹쳐 돌리기, 그리고 OS가 없을 때	191
56.1	스택리스 코루틴 — 스레드 없이 겹쳐 돌리기	191
56.2	작업 시스템 — 코어를 여럿 쓰기	192
56.3	스레드, 할당자, 그리고 출처	192
56.4	OS가 없을 때 — 프리스탠딩	193
56.5	언제 쓰지 않는 것이 나은가	193
57	실전의 C — 도구, 프로젝트, 그리고 영역	196

57.1	make — 무엇을 다시 만들 것인가	196
57.2	git — 무엇이 언제 바뀌었나	196
57.3	오늘도 C로 도는 것들 — 그리고 왜	196
57.4	임베디드 — C의 본진	197
57.5	한계를 넘는 법 — 실전의 요령	197
58	모던 C 총정리	199
58.1	회고 — 이미 써 온 C23	199
58.2	모던 C 관행 — 한 장의 지침	199
58.3	다음에 읽을 책	200
58.4	마지막으로	201
부록 A	연산자 우선순위와 결합	203
부록 B	printf·scanf 서식 요약	204
	출력 변환	204
	입력 해석	204
부록 C	암묵 변환 요약	205
	정수 승격	205
	통상 산술 변환 (두 피연산자의 공통 타입)	205
	가변 인자의 기본 진급	205
	그 밖의 자주 만나는 변환	205

머리말

이 책은 내가 만든 C 라이브러리 proven의 입문서이자, 그 홍보물이다.

그러나 무엇을 쓰라고 권하려면 먼저 그것이 왜 필요한지를 보여야 한다. proven이 다루는 문제들은 C의 미묘한 자리에 있다 — 표현의 한계, 변환의 규칙, 기억의 수명, 표준이 약속하지 않은 영역. 이 자리들을 모르는 사람에게 proven은 불필요한 물건으로 보인다. 그래서 라이브러리를 소개하기 전에, 그 문제들을 먼저 설명하기로 했다. 이 책의 대부분이 표준 C만으로 쓰인 이유이고, proven이 뒤늦게(제11부) 등장하는 이유다. 앞의 이야기를 읽고 나면 proven이 필요 없다고 판단할 수도 있다. 그것도 좋은 결말이다.

다른 한편으로 이 책에는 C를 좋아하는 사람의 마음도 얼마간 들어가 있다. 쉼 해를 넘긴 언어가 왜 아직 남아 있는지, 그 설계가 무엇을 포기하고 무엇을 얻었는지를 설명하다 보면 자연스럽게 그렇게 된다. 그 부분은 감추지 않았다.

이 책에 실린 모든 코드는 실제로 컴파일되고 실행된 것이다. 인쇄된 실행 결과는 사람이 옮겨 적은 것이 아니라 책을 만들 때마다 기계가 예제를 돌려 얻은 출력이며, 두 컴파일러(gcc와 clang)로 교차 검증했다.

저작권과 연락

본문(이 책의 글과 그림)은 CC BY-NC-SA 4.0을 따른다. 출처를 밝히면 자유롭게 공유하고 고칠 수 있으나, 영리 목적으로는 이용할 수 없고, 고친 결과물에도 같은 라이선스를 적용해야 한다.

예제 코드는 MIT 라이선스를 따른다. 이 책에서 배운 코드는 아무 제약 없이 자기 프로그램에 가져다 쓸 수 있다.

연락은 이메일로 받는다. 오류 신고, 수정 요청, 그 밖의 의사소통은 GitHub 저장소를 통해 하는 편이 낫다 — 기록이 남고, 다른 독자에게도 보인다.

rubidus@gmail.com

<https://github.com/rubidus-api>

제1부 — 바탕

1 배경설명

이 장이 끝나면

프로그래밍이 무엇인지, C가 어떤 언어이고 지금 어떤 처지에 있는지, 그리고 왜 하필 지금 새로운 C 입문서가 필요한지 알게 된다. 이 책이 어떤 방식으로 쓰였고 어떻게 읽으면 되는지도 여기서 정한다.

1.1 프로그래밍이란 무엇인가

컴퓨터는 시키는 일을 하는 기계다. 문제는 시키는 방법이다. 컴퓨터는 사람의 말을 알아듣지 못하고, 지독하게 단순한 지시를 순서대로 받을 때만 움직인다. **프로그래밍**이란 우리가 원하는 일을 그 단순한 지시들의 목록으로 바꾸어 적는 일이고, 그 목록을 적는 약속된 표기법이 **프로그래밍 언어**다.

언어는 수백 가지가 있다. 그중 어떤 언어는 사람 쪽에 바짝 붙어 있어서 쓰기 편하고, 어떤 언어는 기계 쪽에 바짝 붙어 있어서 기계를 속속들이 다룰 수 있다. 이 책이 다루는 C는 후자의 대표다 — 반세기 동안 기계 쪽 자리를 지켜 온, 시스템의 언어다.

1.2 C는 어디에 있는가

C는 눈에 잘 띄지 않는다. 화려한 앱과 웹사이트의 얼굴에는 다른 언어들이 서 있다. 그러나 그 얼굴들 밑을 들추면 거의 어디에나 C가 있다.

● 오늘 하루가 C 위에서 흘러간다

아침에 스마트폰 알람이 울린다 — 그 운영체제의 핵심(커널)은 C로 쓰여 있다. 메시지를 보내면 통신 모듈의 펌웨어가 움직인다 — C다. 웹을 열면 서버와 브라우저가 데이터를 주고받는다 — 그 암호화 라이브러리와 통신 도구(OpenSSL, curl)가 C다. 앱이 무언가를 저장한다 — 세계에서 가장 널리 깔린 데이터베이스 SQLite가 C다. 자동차의 제동 장치, 세탁기의 제어판, 심박 조율기 — 임베디드 기기의 대부분이 C다. 파이썬으로 인공 지능을 돌려도, 그 밑에서 실제 계산을 하는 엔진의 상당 부분이 C(와 그 주변 언어)로 되어 있다. C를 배운다는 것은 이 보이지 않는 층을 읽고 쓸 수 있게 된다는 뜻이다.

1.3 지금 C의 세계는 요동치고 있다

그런데 왜 “지금” 새 입문서인가. 오래된 언어라면 오래된 책으로 배워도 되지 않는가. 그렇지 않다 — 지금 C를 둘러싼 풍경은 조용해 보여도 크게 요동치는 중이고, 이 요동이 이 책의 출발점이다.

첫째, C 안에서 여러 시대가 병립하고 있다. C는 표준이라는 공식 규격으로 관리되는 언어이고, C89(1989), C99, C11, C17을 거쳐 최신판 C23까지 왔다. 문제는 옛 표준이 은퇴하지 않는다는 것이다. 산업 현장에는 C89 시절 관행 그대로의 코드가 여전히 산더미처럼 살아 있고, 많은 교재가 그 시절 어법으로 쓰여 있으며, 그 곁에서 C23은 `bool`과 `nullptr` 같은 새 어휘를 표준에 들였다. 같은 “C”라는 이름 아래 서른 해 넘는 시차의 방언들이 공존한다.

문 옛 표준의 코드가 그렇게 많다면, 옛 어법부터 배우는 것이 실용적이지 않은가?

답 읽는 것과 쓰는 것을 나누면 답이 선다. 옛 코드를 **읽을** 일은 언젠가 생긴다 — 그래서 이 책도 역사와 옛 어법의 사연을 곳곳에서 다룬다. 그러나 새로 **쓰는** 코드가 옛 어법일 이유는 없다. 새 표준은 옛 표준의 함정을 수십 년 치 실전 경험으로 메운 결과물이다. 처음 배우는 사람일수록 가장 안전해진 오늘의 어법으로 시작하는 것이 이득이고, 옛 어법은 “왜 저렇게 썼는지”를 아는 교양으로 충분하다.

둘째, 이웃 언어들이 C의 영역을 잠식하고 있다. 한 지붕에서 출발한 C++은 빠른 주기로 판을 거듭하며 거대한 언어가 됐다 — 표현력은 강력해 졌지만, 언어 전체를 아는 사람이 드물다는 말이 나올 만큼 덩치가 불었다. 다른 쪽에서는 새 세대의 시스템 언어들이 나타났다. Rust는 C가 오래 겪어 온 메모리 사고를 언어 차원에서 봉쇄하겠다는 약속으로, Zig는 “C를 계승하되 더 단순하고 정직하게”라는 구호로, 각각 C가 지키던 자리를 파고 들고 있다. 운영체제 커널에 Rust 코드가 들어가기 시작한 것은 상징적인 사건이다.

셋째, 그 압박 속에서 C 자신도 변화의 조짐을 보이고 있다. C23은 근래 가장 큰 폭의 개정이었고, 표준 위원회는 포인터의 규칙을 더 엄밀하게 다듬는 작업(프로버넌스)과 안전성 논의를 이어 가고 있다. C는 느리게 움직이는 언어지만, 방향은 분명하다 — 더 명확한 계약, 더 안전한 기본값 쪽이다.

△ “C는 낡은 언어라서, 배워도 곧 쓸모없어진다”

그렇듯한 걱정이다 — 새 언어들이 저렇게 몰려오고 있으니까. 그러나 현실의 수치는 반대를 가리킨다. 위에서 본 것처럼 세상의 기반 시설이 C 위에서 있고, 이 층은 교체 비용이 너무 커서 수십 년 단위로 움직인다. 새 언어들조차 C와 대화하는 능력(C 인터페이스)을 생존 조건으로 갖추고 태어난다 — C는 시스템 세계의 공용어라서, Rust를 쓰는 Zig를 쓰는 결국 C를 읽고 이해해야 한다. 낡기는커녕, C는 이웃들이 늘어날수록 더 중요해지는 종류의 언어다.

문 그러면 차라리 Rust나 Zig부터 배우는 것이 낫지 않은가?

답 정직하게 답하면, 목적에 따라 다르다. 당장 안전한 응용 프로그램을 하나 만드는 것이 목표라면 다른 선택지도 좋다. 그러나 컴퓨터가 실제로 어떻게 움직이는지를 바닥부터 이해하는 것이 목표라면 C만 한 교재가 없다. C는 기계를 가장 얇게 감싼 언어라서, C를 배우는 과정이 곧 기계와 메모리와 컴파일러를 배우는 과정이 된다. 그리고 그 이해는 Rust로 가든 Zig로 가든 그대로 이고 갈 수 있는 밑천이다. 이 책은 그 밑천을 오늘의 어법으로 마련해 주는 책이다.

1.4 왜 이 책을 만들었나

이 요동치는 풍경이 이 책의 첫 번째 이유다. 여러 표준이 병립하고, 이웃 언어들이 경쟁을 걸어오고, C 자신이 변하고 있는 지금 — 옛 어법의 관성으로 쓰인 입문서가 아니라, **오늘의 C(C23)와 오늘의 관행에서 출발하는** 입문서가 필요하다고 판단했다. 그래서 이 책은 목차부터 다시 짰다. 문법 항목을 순서대로 나열하는 대신, 컴퓨터라는 기계의 기본부터 시작해 개념이 필요해지는 순서대로 C를 만난다.

두 번째 이유는 밝혀 두는 것이 정직하겠다. 저자는 **proven**이라는 C 라이브러리를 만들어 공개하고 있다. proven은 C 프로그래밍에서 가장 사고가 잦은 기본기 — 문자열 다루기, 입력 처리, 수의 변환 같은 일 — 를 검증된 형태로 제공하는 것을 목표로 하는 라이브러리다. C의 유명한 함정들은 대부분 이 기본기 층에서 터지는데, 표준 라이브러리는 역사적 이유로 그 함정을 그대로 안고 있다. 이 책의 뒷부분(제7부부터)에서는 그 함정이 실제로 왜 위험한지 보인 다음, proven이 그것을 어떻게 막는지 예제의 기반으로 사용한다. 이 책은 그 라이브러리를 알리려는 목적도 겸해서 만들어졌다 — 다만 순서는 지킨다. 먼저 표준 C만으로 개념을 온전히 배우고, proven은 그 필요가 스스로 증명된 시점에 등장한다.

문 라이브러리를 홍보하는 책이라면, 내용이 그쪽으로 치우치지 않는가?

답 치우치지 않도록 구조로 막아 두었다. 이 책의 앞 30여 개 장은 proven을 한 줄도 쓰지 않는다 — 순수하게 컴퓨터의 기본과 표준 C만으로 진행된다. proven이 등장하는 것은 “왜 이런 도구가 필요

한가”가 본문 속에서 스스로 드러난 뒤이고, 그때도 표준적인 방법을 먼저 보인 다음 비교로 제시한다. proven 없이도 이 책의 C 지식은 온전히 성립한다.

1.5 이 책을 읽는 법

이 책은 읽는 책이다. 당연한 말 같지만, 프로그래밍 책으로서는 드문 약속이다.

술술 읽히는 것을 목표로 삼았다. 그래서 연습문제도, “직접 해 보라”는 지시도 넣지 않았다. 컴퓨터 앞에 앉아 있지 않아도 된다 — 소파에서, 지하철에서, 그냥 소설처럼 앞에서 뒤로 읽으면 된다. 모든 코드는 지면 위에서 처음부터 끝까지 시연하고, 실행 결과까지 지면에 인쇄되어 있다. 그 실행 결과는 장식이 아니라 실제 결과다 — 이 책에 실린 모든 예제는 기계가 실제로 컴파일하고 실행해서 얻은 출력을 그대로 실는다.

연습을 원한다면 다른 책이 낫다. 과제를 잘 설계하고 단계별로 풀려 주는 일은 기존의 좋은 C 입문서들이 훨씬 잘한다. 이 책이 맡은 자리는 그 옆이다 — 왜 그렇게 생겼는지를 읽어 나가는 쪽.

문답으로 진행한다. 본문 곳곳에서 방금 읽은 내용에 대해 독자가 품었을 법한 질문을 그 자리에서 묻고, 바로 답한다. 질문을 만난 순간 잠깐 스스로 답을 떠올려 보고 다음 줄을 읽으면 가장 좋지만, 그냥 내리읽어도 된다 — 문답 자체가 설명의 일부다.

외우게 하지 않는다. 기억할 가치가 있는 것은 뒤 장의 서두에서 “돌아 보기”라는 이름으로 다시 찾아온다. 이전 장의 내용을 조금 더 깊은 질문으로 다시 만나게 되는데, 이것이 이 책의 복습 장치다. 잊었어도 괜찮다 — 답이 바로 옆에 있다.

작은 장으로 빠르게 간다. 장 하나는 짧다. 한 번에 하나의 착상만 다루고 다음으로 넘어간다. 큰 주제는 여러 장에 걸쳐 나선처럼 여러 번, 매번 조금 더 깊게 지나간다. 한 장에서 모든 것을 말하지 않으니, 어딘가 궁금증이 남았다면 그것은 대개 의도된 것이다 — 몇 장 뒤에서 그 궁금증을 다시 만나게 된다.

이제 시작한다. 첫걸음은 C가 아니라 컴퓨터 그 자체다 — C가 왜 이렇게 생겼는지는, C가 태어난 기계를 보아야 이해되기 때문이다. 다음 부에서 컴퓨터를 아주 단순한 그림으로 그리는 것부터 시작한다.

제2부 — 전산의 기본과 배경지식

이 부는 C 문법을 하나도 가르치지 않는다. 대신 이후의 모든 장이 딛고 설 바닥을 깎는다. 세 걸음으로 오른다.

기계와 기억 (2-4장) — 컴퓨터를 세 부품의 단순한 그림으로 그리고, 기억이라는 사물함 복도를 걷는다. 손에 잡히는 이야기다.

표현의 사다리 (5-8장) — 그 사물함에 정수를, 소수점 있는 수를, 글자를, 그리고 글자의 흐름을 담는 법. 한 단씩 오른다.

속도와 추상 (9-12장) — 기계가 빨라지며 기억이 어떻게 갈라졌는지 (레지스터·캐시), 속도의 기계장치들, 컴파일러라는 편집자, 그래서 C가 왜 “추상적인 언어”인지. 이 부의 결론이 여기서 완성된다.

각 걸음마다 C의 역사가 나란히 걷는다. 연대와 인명을 외우게 하려는 것이 아니다 — 오늘의 C가 왜 이런 모양인지, 역사가 가장 짧은 설명이기 때문이다.

2 단순한 기계 모델 ↔ C의 탄생

이 장이 끝나면

컴퓨터를 세 가지 부품 — 계산하는 곳, 기억하는 곳, 박자를 맞추는 것 — 의 단순한 그림으로 그릴 수 있게 된다. 정보의 최소 단위인 비트가 무엇인지, 왜 하필 2진법인지 알게 된다. 그리고 그 단순한 기계의 시절에 C라는 언어가 어떻게 태어났는지 보게 된다.

돌아보기 1장에서 C가 운영체제와 임베디드 기기, 인터넷 기반시설 속에 지금도 살아 있다고 했다. 반세기가 넘은 언어가 어떻게 아직 그 자리에 있는가?

답 C가 서 있는 자리가 특별하기 때문이다. 그 자리는 기계와 사람이 만나는 가장 낮은 층이고, 기계는 반세기 동안 빨라졌을 뿐 일하는 방식의 뼈대는 놀랄 만큼 그대로다. 그 뼈대를 이 장에서 직접 들여다본다. 뼈대가 그대로인 한, 뼈대의 언어도 그대로 남는다.

2.1 컴퓨터를 세 부품으로 그리기

컴퓨터는 복잡한 물건이다. 그러나 복잡한 것을 이해하는 첫걸음은 언제나 과감하게 단순한 그림을 그리는 것이다. 지금부터 컴퓨터를 딱 세 부품으로 그린다.

첫째, **CPU**. 계산하는 곳이다. 더하고, 비교하고, 다음에 무엇을 할지 정한다.

둘째, **메모리**. 기억하는 곳이다. 계산에 쓸 재료와 계산의 결과가 여기에 놓인다.

셋째, **클럭**. 박자를 맞추는 것이다. 메트로놈처럼 일정한 간격으로 똑딱이고, CPU는 그 박자에 맞춰 한 걸음씩 일한다.

이 그림에서 컴퓨터가 하는 일은 이것이 전부다: **클럭이 똑딱일 때마다, CPU가 메모리에서 다음 할 일을 하나 가져와서, 그대로 한다.** 이것을 끝없이 반복한다.

문 “1초에 40억 번” 같은 광고 문구의 기가헤르츠(GHz)가 이 클럭인가?

답 그렇다. 4GHz는 메트로놈이 1초에 40억 번 똑딱인다는 뜻이다. 박자마다 CPU가 일을 한 걸음씩 진행하니, 박자가 빠를수록 같은 시간에 더 많은 걸음을 걷는다. 다만 걸음 수가 전부는 아니라는 것을 10장에서 보게 된다 — 한 걸음에 여러 일을 하는 요령이 현대 CPU의 진짜 무기다.

문 겨우 “가져와서 그대로 한다”의 반복으로 게임과 동영상과 인공지능이 어떻게 가능한가?

답 벽돌 한 장은 단순하지만 벽돌로 성을 쌓을 수 있는 것과 같다. 단순한 걸음이라도 1초에 수십억 번이면, 그리고 그 걸음들을 겹겹이 조직하면 — 작은 일들이 모여 함수가 되고, 함수가 모여 프로그램이 되고, 프로그램이 모여 운영체제가 되면 — 어떤 복잡한 일이든 지을 수 있다. 이 “겹겹이 쌓기”가 전산학의 심장인 **추상화**이고, 이 책 전체가 그 층계를 오르는 이야기다.

CPU가 “다음 할 일”로 가져오는 것을 **명령**이라 부른다. 명령은 대단한 것이 아니다. “이 수와 저 수를 더해라”, “이 값을 저 칸에 넣어라”, “방금 결과가 0이면 다른 곳으로 건너뛰어라” — 이 정도 수준의, 지독하게 단순한 지시다. 프로그램이란 이런 명령을 순서대로 늘어놓은 목록이고, 프로그래밍이란 그 목록을 사람이 감당할 수 있는 방식으로 적는 일이다.

2.2 비트 — 정보의 최소 단위

메모리가 “기억하는 곳”이라 했다. 그러면 무엇을, 어떤 모양으로 기억하는가.

전자 회로가 가장 잘하는 일은 두 가지 상태를 구별하는 것이다. 전압이 높거나, 낮거나. 스위치가 켜져 있거나, 꺼져 있거나. 이 둘 중 하나를 담는 가장 작은 기억 단위를 **비트(bit)**라 부른다. 우리는 두 상태에 0과 1이라는 이름을 붙인다.

△ “컴퓨터 안 어딘가에 0과 1이 글자처럼 적혀 있다”

그럴듯한 생각이다 — 책과 화면마다 컴퓨터는 0과 1로 가득하다고 그려지니까. 그러나 기계 안에 숫자 글자가 있는 것이 아니다. 실제로 있는 것은 높거나 낮은 전압, 차 있거나 빈 전하 같은 물리 상태 뿐이다. 0과 1은 그 물리 상태를 사람이 읽기 위해 붙인 **이름**, 즉 해석이다. 이 구별은 한가한 트집이 아니다 — “기억된 것 자체”와 “그것을 어떻게 읽을 것인가”의 분리는 이 책에서 계속 되돌아올 주제이고, 나중에 같은 비트 열을 수로도 글자로도 읽게 되는 이유다(6장, 7장).

문 왜 하필 두 상태인가? 스위치 하나에 0부터 9까지 열 가지 상태를 담으면 훨씬 효율적이지 않은가?

답 실제로 초기에는 10진 컴퓨터가 만들어지기도 했다. 그러나 열 단계의 전압을 구별하려면 단계 사이 간격이 좁아지고, 간격이 좁으면 잡음이나 온도 변화에 이웃 단계로 넘어가 버린다 — 즉 틀리기 쉽다. 두 상태는 간격이 가장 넓어서, 회로가 싸고 빠르고 잡음에 강하다. 2진법은 수학적 우아함 때문이 아니라 **공학적 정직함** 때문에 이겼다.

비트 하나는 두 가지를 구별한다. 비트를 여러 개 묶으면 구별할 수 있는 가짓수가 곱으로 불어난다. 두 개면 네 가지(00, 01, 10, 11), 세 개면 여덟 가지다. 여덟 개를 묶은 것에는 **바이트(byte)**라는 이름이 붙어 있고, 한 바이트는 256가지를 구별한다.

문 바이트는 왜 하필 여덟 개 묶음인가? 처음부터, 그리고 어디서나 여덟 개였는가?

답 아니다 — 바이트의 크기는 8비트로 고정된 적이 없다. 원래 “바이트”는 **문자 하나를 담는 묶음**이라는 뜻으로 쓰인 말이고, 그 크기는 기계마다 달랐다. 초기 컴퓨터들에는 6비트, 7비트, 9비트 바이트가 실제로 있었고, 워드가 36비트라서 바이트를 6비트씩 여섯 개로 자르던 기계도 있었다. 8비트가 대세가 된 것은 1960년대 IBM의 대형 컴퓨터(System/360)가 8비트를 채택하고 산업이 그 뒤를 따르면서다. 그래서 정확함이 생명인 통신 규격들은 지금도 “바이트” 대신 **옥텟(octet, 정확히 8비트)**이라는 말을 쓴다 — 바이트라는 말만으로는 크기가 보장되지 않았던 역사의 흔적이다.

그리고 이것은 옛날이야기만이 아니다. 지금도 일부 신호 처리용 프로세서(DSP)는 가장 작은 기억 단위가 16비트나 32비트라서, 그 위의 C에서는 바이트가 16비트·32비트다. 흔히 만나는 기계에서는 사실상 전부 8비트지만, “바이트 = 무조건 8비트”는 약속이 아니라 관행이다.

C 표준(C23)의 정의도 그래서 신중하다. C23에서 **바이트**란 “기본 문자 하나를 담을 수 있는, 주소를 붙일 수 있는 가장 작은 기억 단위”이고, 그 비트 수는 `CHAR_BIT`라는 이름으로 공개하되 **8 이상**이라고만 못박는다 — 정확히 8이 아니라 “최소 8”이다. 이 책은 이후 흔한 기계들을 따라 바이트 = 8비트로 놓고 진행하되, 그것이 표준의 보장이 아니라 사실상의 관행임을 여기 적어 둔다. (표준 위원회는 차기 표준에서 8비트로 아예 못박는 방향을 논의하고 있다 — 관행이 반세기를 버티면 약속으로 승격되는 셈이다.)

Σ 묶음의 크기와 가짓수

비트 n 개의 묶음이 구별할 수 있는 상태의 가짓수는, 각 자리가 독립적으로 두 값을 가지므로

$$2 \times 2 \times \cdots \times 2 = 2^n$$

이다. $2^8 = 256$, $2^{16} = 65536$, $2^{32} \approx 4.3 \times 10^9$. 이 책에서 2^n 은 계속 나타난다 — 기억의 크기, 표현 가능한 수의 범위, 주소의 개수가 전부 이 꼴이다.

묶음으로 수를 나타내는 방법은 우리가 쓰는 십진법과 같은 원리인 위치 기수법이다. 십진수 $304 = 3 \cdot 10^2 + 0 \cdot 10^1 + 4 \cdot 10^0$ 이듯, 2진수 $101_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 5$ 다. 자리 하나가 10 대신 2의 거듭제곱을 짊어질 뿐이다.

문 256가지밖에 안 되는 바이트로 어떻게 큰 수를 다루는가?

답 십진법에서 한 자리로 부족하면 자리를 늘리듯, 바이트도 여러 개를 이어 붙여 큰 수를 담는다. 몇 개를 이어 붙이는 것이 자연스러운가는 기계마다 정해져 있다 — 그 “자연스러운 묶음”이 다음 장의 주인공인 워드다.

2.3 그 단순한 기계의 시절, C가 태어났다

지금까지 그린 세 부품의 그림은 오늘의 컴퓨터에게는 지나치게 단순한 그림이다(그 이야기가 9·10장이다). 그런데 1970년대 초의 컴퓨터에게는 이 그림이 거의 사실 그대로였다. 클럭은 초당 수십만 번 수준 이었고, CPU는 정말로 한 박자에 한 걸음씩 걸었고, 메모리는 정말로 한 줄로 늘어난 사물함이었다.

C는 바로 그 시절, 그 단순한 기계 위에서 태어났다 — 그런데 무(無)에서 솟은 것이 아니라, 짧고 뚜렷한 족보의 끝에서 태어났다. 그 족보를 알면 C의 생김새 몇 가지가 단번에 설명되므로, 여기 간추려 둔다.

출발점은 1960년대 영국의 CPL이라는 야심찬 언어였다 — 너무 야심차서 당대 기계로는 완성이 어려웠고, 케임브리지의 마틴 리처즈가 그 정수만 추려 BCPL(1967)이라는 작고 실용적인 언어를 만들었다. BCPL의 급진적 단순함은 이것이다: **타입이 하나뿐이다**. 모든 값은 그냥 워드 한 칸이고, 그것을 수로 쓸지 주소로 쓸지는 연산이 정한다. 3장에서 배운 “칸에 든 것은 그저 비트이고 해석이 정한다”를 언어 전체로 밀어붙인 셈이다.

벨 연구소의 켄 톰프슨은 낡은 PDP-7 위에서 초기 Unix를 만들며, BCPL을 자기 취향으로 더 줄인 B(1969년경)를 만들었다 — 여전히 타입은 하나였고, 오늘날까지 살아남은 간결한 어법들(+= 같은)이 이 손질에서 왔다. 그런데 연구소가 새 기계 PDP-11을 들이자 B의 전제가 깨졌다. PDP-11의 메모리는 워드가 아니라 **바이트** 단위로 주소가 붙었고(3장의 복도가 바로 이 모습이다), 크기가 다른 데이터 — 1바이트짜리 문자, 2바이트짜리 정수, 그리고 부동소수점 — 를 구별해 다뤄야 했다. “모든 것이 워드 한 칸”인 B로는 이 기계를 제대로 부릴 수 없었던 것이다.

그래서 데니스 리치가 B에 **타입**을 들였다 — 문자와 정수를 구별하고, 무엇의 주소인지 아는 포인터를 만들고, 구조체를 보냈다. 이 개조가 1971-73년 사이에 쌓여 새 이름을 얻은 것이 C다. 즉 C의 타입 시스템은 철학적 설계가 아니라 **바이트 주소 기계라는 현실**에 대한 응답이었다. C가 뻗속까지 바이트 지향인 것(2장의 바이트, 3장의 복도), 포인터가 언어의 한복판에 있는 것, 그러면서도 타입이 험거운 편인 것(타입 하나 짜리 조상의 유산) — 전부 이 족보의 흔적이다.

덧붙여 두 가지. 오늘의 눈에 익숙한 어법이 처음부터 있었던 것은 아니다 — 이를테면 지금의 +=는 초기 C에서 +=라고 적었고, 몇 해 뒤에야 지금 꼴이 됐다(헛갈림 때문이었다: $x=-1$ 이 “빠기”인지 “대입”인지). 언어는 이렇게 실사용의 상처를 먹으며 다듬어진다. 그리고 이 시절의 C에는 아직 표준도, 규격서도 없었

다 — 언어의 정의는 사실상 “리치의 컴파일러가 받아 주는 것”이었다. 이 험거움이 어떤 소동으로 자라 표준(C89)을 부르게 되는지는 10장에서 있다.

그래서 초기의 C는 과장을 조금 보태면 **그 시절 기계의 정직한 별명**이었다. C의 연산 하나가 기계 명령 한두 개에 그대로 대응했고, 프로그래머는 C를 쓰면서도 기계가 무엇을 할지 훤히 내다볼 수 있었다. C가 “기계에 가까운 언어”라는 평판은 여기서 왔다 — 그리고 이 평판이 오늘날 절반만 맞는 말이 된 사연이 이 부의 뒷장들(9-12장)이다.

● 이식성 혁명 — Unix를 C로 다시 쓰다

1973년경 Unix의 심장부가 어셈블리어에서 C로 다시 쓰였다. 당시로서는 파격이었다 — 운영체제는 기계어 수준에서 짜야 한다는 것이 상식이었기 때문이다. 효과는 압도적이었다. 어셈블리 Unix는 PDP-11에 묶여 있었지만, C로 쓰인 Unix는 “C 컴파일러만 있으면” 다른 기계로 옮길 수 있었다. 운영체제가 특정 기계의 소유물이 아니게 된 것이다. 오늘날 Linux, Windows, macOS의 커널이 모두 C(또는 그 후예)로 쓰여 있고, 새 프로세서가 나오면 가장 먼저 C 컴파일러부터 이식되는 전통이 여기서 시작됐다.

문 50년 전 기계에 맞춘 언어를 지금 배우는 것이 손해는 아닌가?

답 거꾸로다. 기계의 뼈대 — 메모리, 주소, 바이트, 명령의 순차 실행 — 는 50년 동안 바뀌지 않았고, C는 그 뼈대를 가장 얇게 감싼 언어로 남아 있다. 그래서 C를 배우는 일은 특정 언어의 문법 암기가 아니라 컴퓨터 그 자체를 배우는 일에 가깝다. 이후에 어떤 언어를 쓰게 되든, 그 언어의 밑바닥에는 거의 언제나 C가 깔려 있다.

이 장의 그림을 정리하면 이렇다. 기계는 클럭의 박자에 맞춰 메모리에서 명령을 가져와 그대로 실행하는 일을 반복하고, 기억은 두 상태의 최소 단위인 비트로 이루어진다. C는 이 그림이 거의 사실이던 시절에, 이 그림을 언어로 옮겨 태어났다.

다음 장은 세 부품 중 메모리를 자세히 들여다본다. 사물함 복도를 걸으며 칸마다 붙은 번호 — **주소** — 를 읽고, 기계가 한 번에 집는 자연스러운 묶음 — **워드** — 을 재고, 여러 바이트짜리 수를 사물함에 넣는 두 가지 순서 — **엔디안** — 를 구경한다. C의 포인터라는 유명한 개념이 바로 이 복도에서 태어났다.

3 워드와 주소 ↔ C의 원형

이 장이 끝나면

메모리를 번호 붙은 사물함들의 긴 복도로 그릴 수 있게 된다. 그 번호가 주소이고, 기계가 사물함을 한 번에 몇 칸씩 다루는지가 워드다. 여러 칸에 걸치는 수를 넣는 두 가지 순서 — 리틀 엔디안과 빅 엔디안 — 를 그림으로 구별하게 된다.

돌아보기 2장에서 바이트 하나는 256가지밖에 구별하지 못하며, 큰 수는 바이트를 여러 개 이어 붙여 담는다고 했다. 그러면 기계는 이어 붙인 바이트들이 “한 덩어리”라는 것을 어떻게 아는가?

답 모른다 — 이것이 이 장의 출발점이다. 메모리 자체는 어느 칸이 어느 칸과 한 덩어리인지 전혀 기록하지 않는다. 덩어리를 아는 것은 메모리가 아니라 **읽는 쪽**이다. 프로그램이 “여기서부터 네 칸을 하나의 수로 읽겠다”고 정할 뿐이다. 2장의 오개념 — 기억된 것 자체와 읽는 방법의 분리 — 이 여기서 첫 실전을 치른다.

3.1 사물함 복도

메모리를 그림으로 그리면 이렇다. 긴 복도에 같은 크기의 사물함이 한 줄로 끝없이 이어져 있고, 사물함마다 번호가 붙어 있다. 사물함 한 칸의 크기는 1바이트다. 번호는 0부터 시작해 1씩 늘어난다. 이 번호가 **주소(address)**다.

01001000	01101001	00100001	00000000	11111111
100	101	102	103	104

칸 안에 든 것은 언제나 그저 비트 여덟 개다. 그 여덟 비트가 수인지, 글자인지, 더 큰 무엇의 조각인지는 칸 자신도 복도도 모른다 — 읽는 쪽의 해석이 정한다.

주소에 관해 첫눈에 안 보이는 중요한 사실이 하나 있다. **주소 자체도 수다**. 사물함 번호는 그냥 정수이고, 정수이므로 그것을 다른 사물함에 넣어 둘 수도 있다. “347번 칸에, 512라는 번호를 적어 둔다” — 이상할 것이 하나도 없다. 이 평범한 착상이 나중에 C에서 가장 유명한 개념인 **포인터**가 된다(30장). 지금은 “주소도 수라서 어디에든 적어 둘 수 있다”까지만 챙기면 된다.

문 복도의 길이, 그러니까 사물함의 개수는 얼마나 되는가?

답 주소를 몇 비트로 적느냐가 정한다. 주소가 n 비트 수라면 사물함은 최대 2^n 개까지 구별할 수 있다(2장의 2^n 이 바로 다시 나왔다). 주소를 32비트로 적던 시절의 복도는 최대 약 43억 칸 — 4GiB — 이었고, 오늘의 64비트 주소는 사실상 다 쓸 수 없을 만큼 긴 복도를 만든다. “32비트 시스템에서 메모리 4GB 한계” 같은 말의 정체가 이것이다.

3.2 워드 — 기계의 자연스러운 한 움큼

사물함은 한 칸씩 있지만, 기계가 일할 때 한 칸씩만 집으면 너무 느리다. 그래서 CPU는 여러 칸을 한 **움큼**에 집도록 만들어져 있다. 그 움큼의 크기 — 기계가 가장 자연스럽게, 한 번에 다루는 비트 수 — 를 **워드(word)**라 부른다.

“64비트 컴퓨터”라는 말이 바로 이것이다. 그 기계의 워드가 64비트, 즉 8바이트라는 뜻이다. CPU 안의 임시 보관함(레지스터)이 그 크기이고, 메모리를 오가는 통로(버스)가 그 폭에 맞춰져 있고, 주소도 대

개 그 크기의 수로 다룬다. 워드는 기계의 “손 크기”라 해도 좋다 — 손이 큰 기계는 한 번에 더 큰 수를 집는다.

문 그러면 64비트 기계에서는 모든 것이 8바이트 단위로 저장되는가?

답 아니다. 저장은 여전히 바이트 단위로 자유롭다 — 1바이트짜리 글자도, 4바이트짜리 수도 같은 복도에 산다. 워드는 “기계가 가장 편하게 집는 크기”이지 “모든 것의 크기”가 아니다. 실제로 C에는 크기가 다른 여러 수 타입이 있고(22장), 어떤 크기의 데이터를 어디에 두는 것이 편한가라는 문제는 4장(정렬)에서 다시 만난다.

3.3 엔디안 — 여러 칸에 수를 넣는 두 가지 순서

이제 이 장의 하이라이트다. 4바이트짜리 수 하나를 사물함 네 칸에 넣어야 한다. 수를 16진법으로 12 34 56 78이라는 네 바이트 조각으로 나눴다고 하자(16진법 두 자리가 1바이트다). 100번 칸부터 넣는다면 — 어느 조각을 100번 칸에 넣어야 하는가?

두 학파가 있다. **빅 엔디안(big-endian)**은 큰 자리부터 넣는다. 사람이 종이에 수를 적는 순서 그대로다.

12	34	56	78
100	101	102	103

리틀 엔디안(little-endian)은 작은 자리부터 넣는다. 뒤집힌 것처럼 보이지만, “*i*번째 칸에 *i*번째로 작은 자리가 있다”는 나름의 정연한 규칙이다.

78	56	34	12
100	101	102	103

지금 이 글을 읽는 컴퓨터와 스마트폰은 거의 확실히 리틀 엔디안이다(인텔·AMD·대부분의 ARM 운영). 날짜 표기와 똑같은 상황이다 — 2026-08-04라고 적는 나라와 04-08-2026이라고 적는 나라가 있고, 어느 쪽도 틀리지 않았지만, **서로의 편지를 그대로 읽으면 사고가 난다.**

문 사람 눈에는 빅 엔디안이 자연스러운데, 왜 대부분의 기계는 “뒤집힌” 리틀 엔디안을 택했는가? 무슨 장점이 있는가?

답 두 가지 실속 때문이다.

첫째, **시작 칸이 흔들리지 않는다.** 리틀 엔디안에서는 수의 가장 작은 자리가 언제나 시작 주소 그 칸에 있다. 위 그림의 수를 “4바이트 전부”로 읽든 “아래 2바이트만”·“아래 1바이트만”으로 줄여 읽든, 읽기 시작하는 칸은 똑같이 100번이다. 빅 엔디안에서는 읽는 폭을 바꿀 때마다 시작 칸을 옮겨 다시 계산해야 한다. 같은 값을 크기가 다른 눈으로 읽는 일이 잦은 기계에게 “시작 주소 불변”은 꽤 큰 단 순함이다.

둘째, **계산의 진행 방향과 맞는다.** 손으로 덧셈을 할 때를 떠올리면 된다 — 일의 자리부터 더하고 반 아올림을 위로 넘긴다. 여러 바이트짜리 수를 더하는 기계도 마찬가지로 작은 자리부터 처리하는데, 리틀 엔디안에서는 그것이 “낮은 주소부터 순서대로”와 정확히 일치한다. 초기의 작은 CPU들은 첫 바이트를 가져오자마자 덧셈을 시작할 수 있었고, 이 이점이 초창기 설계(인텔 계열의 조상들)에 스며서 오늘날까지 이어졌다.

빅 엔디안의 장점도 공정하게 적으면 — 덤프가 사람 눈에 그대로 읽히고, 바이트를 앞에서부터 통째로 비교하면 수의 대소 비교와 일치한다. 그래서 통신 규약처럼 “사람과 기계가 함께 들여다보는 자리”에는 빅 엔디안이 남았다. 어느 쪽도 우월하지 않다 — 자주 하는 일이 다를 뿐이다.

△ “메모리를 앞에서부터 읽으면 적힌 순서대로 수가 보인다”

그렇듯하다 — 종이에 적힌 수는 그렇게 읽으니까. 그러나 리틀 엔디안 기계에서 12 34 56 78이라는 수는 메모리에 78 56 34 12 순서로 놓인다. 메모리를 칸 순서대로 들여다보면(디버거나 파일 덤프에서 실제로 이렇게 본다) 수가 “뒤집혀” 보인다. 뒤집힌 것이 아니라, 넣는 순서의 약속이 다를 뿐이다. 기억할 것: 메모리 덤프는 종이에 적은 수가 아니다.

● NUXI — 순서가 낳은 유령

초기 Unix를 다른 회사의 컴퓨터로 이식하던 엔지니어들이 화면에서 기묘한 것을 보았다. UNIX라고 적혀야 할 곳에 NUXI가 찍힌 것이다. 두 기계의 엔디안이 달라서, 2바이트 묶음 안의 글자 순서가 통째로 뒤집힌 결과였다. 이 사건은 “NUXI 문제”라는 이름으로 엔디안 사고의 대명사가 됐다. 같은 종류의 사고는 지금도 난다 — 파일을 한 기계에서 저장해 다른 기계에서 읽을 때, 네트워크로 수를 주고 받을 때, 엔디안을 정하지 않으면 수가 깨진다. 그래서 인터넷 규약은 “통신할 때는 빅 엔디안으로 보낸다”는 **네트워크 바이트 순서**를 합의해 두었다.

문 내 컴퓨터가 리틀 엔디안이라는 것을 프로그램으로 확인할 수 있는가?

답 있다 — 여러 바이트짜리 수를 메모리에 넣고, 그 첫 칸을 1바이트만 따로 읽어 보면 된다. 첫 칸에 작은 자리 조각이 있으면 리틀 엔디안이다. 이 확인을 C로 실제로 해 보이는 것이 39장(공용체)의 시연이다 — 같은 기억을 다른 눈으로 읽는 도구가 그때 생긴다.

3.4 C의 원형이 여기 있다

이 장의 복도 그림은 C라는 언어의 설계도이기도 하다. C가 태어난 시절 (2장), 언어를 설계한 사람들은 기계의 이 모습을 감추는 대신 언어의 개념으로 그대로 들어 올렸다. 메모리는 바이트의 열이라는 것 — C의 모든 데이터는 바이트 단위 크기를 갖는다. 칸마다 주소가 있다는 것 — C에서는 거의 모든 것의 주소를 물을 수 있다. 주소도 수라는 것 — 그 수를 담는 타입(포인터)이 언어의 한복판에 있다. 다른 많은 언어가 “사물함 복도”를 프로그래머에게 보이지 않게 봉인해 둔 것과 대조적이다.

이 정직함은 양날이다. 기계를 훤히 내다보며 일할 수 있는 힘이자, 복도에서 길을 잃으면 그대로 사고가 되는 위험이다. 이 책의 제7부가 그 힘과 위험을 정면으로 다룬다.

다음 장은 이 복도의 구석에 있는 흥미로운 특례들을 구경한다. 0번 사물함에는 무엇이 있는가, 왜 두 칸짜리 짐은 아무 번호에나 못 넣는가(정렬), 그리고 번호의 끝자리가 항상 0이라는 사실을 이용해 몰래 정보를 숨기는 재주(태그 포인터)까지 — 주소의 세계가 생각보다 훨씬 사람 사는 동네 같다는 것을 보게 된다.

4 주소의 특수 지식 — 0번지, 정렬, 하위 비트

이 장이 끝나면

사물함 복도의 구석 지식 세 가지를 챙긴다. 0번 사물함은 왜 특별한지, 왜 큰 짐은 아무 번호에나 못 넣는지(정렬), 그리고 번호의 끝자리에 정보를 숨기는 재주(태그 포인터)까지. “널”이라는 이름의 세 가지 다른 물건도 여기서 처음 구별해 둔다.

돌아보기 3장에서 “주소도 수라서 다른 사물함에 적어 둘 수 있다”고 했다. 그런데 적어 둔 주소가 **아직 없음**을 나타내고 싶으면 — “아무 데도 가리키지 않는다”는 표시는 어떻게 하는가?

답 특별한 값 하나를 “없음”의 표시로 약속하면 된다. 그리고 거의 모든 시스템이 그 표시로 0을 골랐다 — 0번 사물함을 아무도 안 쓰는 칸으로 비워 두고, “0이 적혀 있으면 아무 데도 안 가리키는 것”으로 읽는 약속이다. 이 약속된 값이 **널 포인터(null pointer)**다. 그런데 왜 하필 0번 칸을 비워 둘 수 있었는가 — 그 이야기가 이 장의 첫 절이다.

4.1 0번 사물함에는 무엇이 있는가

답부터 말하면, **기계마다 다르다**. 그리고 그 차이가 재미있다.

데스크톱과 서버, 스마트폰처럼 운영체제가 보호 모드로 도는 환경에서는, 운영체제가 0번지 근처를 일부러 **접근 금지 구역**으로 만들어 둔다. 어떤 프로그램이 0번지를 읽거나 쓰려 하면 그 즉시 붙잡혀 종료된다. 이것은 친절이다 — “없음” 표시(널)를 실수로 진짜 주소처럼 쓰는 사고는 아주 흔한데, 0번지가 금지 구역이라서 그런 실수가 조용히 지나가지 않고 그 자리에서 요란하게 잡힌다.

임베디드의 세계는 다르다. 운영체제 없이 도는 작은 칩에서는 0번지가 멀쩡한, 심지어 중요한 메모리인 경우가 많다 — 여러 마이크로컨트롤러가 0번지 부근에 인터럽트 벡터 테이블(비상 연락망 같은 표)을 둔다. 그런 기계에서 0번지 읽기는 합법이고 의미가 있다.

문 0번지가 멀쩡한 메모리인 기계에서는, “없음” 표시와 “진짜 0번지”를 어떻게 구별하는가?

답 구별이 흐려지는 것이 사실이고, 그래서 임베디드 프로그래머는 이 경계를 의식하며 일한다. 더 흥미로운 것은 역사의 답이다 — 아예 0이 아닌 값을 “없음” 표시로 쓴 기계들이 실제로 있었다. 다음 사례가 그 이야기다.

● 널 포인터가 0이 아니었던 기계들

C 표준은 처음부터 신중했다. 소스 코드에 적은 상수 0이 널 포인터를 뜻한다는 것은 표준의 약속이지만, 그 널 포인터가 기계 안에서 **실제로 어떤 비트 패턴**으로 표현되는지는 기계의 자유로 남겨 두었다. 그리고 그 자유를 실제로 쓴 기계들이 있었다. Prime 50 시리즈는 세그먼트 07777, 오프셋 0이라는 값을 널로 썼고, CDC Cyber 180은 0xB00000000000이라는 특별한 패턴을, Lisp에 최적화된 Symbolics 머신은 “NIL 객체의 주소”라는 0 아닌 값을 널로 썼다. 이런 기계에서 널 포인터의 비트를 들여다보면 0이 하나도 아니다 — 그래도 소스 코드에서 포인터를 0과 비교하면 표준의 약속대로 참이 나온다. 소스의 0은 **기호**이고, 기계 속 표현은 **구현**이다 — 2장부터 이어 온 “기호와 표현의 분리”가 여기서도 반복된다.

오늘날의 주류 기계(인텔·AMD·ARM 등)는 전부 “모든 비트 0”을 널로 쓰므로 이 구별을 체감할 일은 드물다. 그러나 뒤에 볼 태그 포인터의 세계에서는 “없음을 나타내는 값이 전부 0은 아닌” 상황이 지금도 멀쩡히 살아 있다.

4.2 널 삼형제 — 이름만 닮은 세 가지

“널”이라 불리는 것이 C 주변에 셋 있다. 지금 한 번 구별해 두면 나중에 큰 혼란을 던다. 정식 취급은 제7부(31장)에서 하고, 여기서는 얼굴만 익힌다.

- **널 포인터** — 방금 본 것. “아무 데도 가리키지 않는다”는 약속된 포인터 값이다. 포인터의 세계에 산다.
- **NULL** — C 소스 코드에서 널 포인터를 적을 때 쓰는 이름(매크로)이다. 최신 C(C23)에는 더 명확한 `nullptr` 라는 표기도 들어왔다. 역시 포인터의 세계에 산다.
- **NUL 문자** — 전혀 다른 물건이다. 문자표(7장)의 0번 자리에 있는, 값이 0인 문자 하나로, C에서는 `'\0'`이라 적는다. 크기 1바이트짜리 데이터이고, 문자열의 끝 표시로 쓰인다(34장). 문자의 세계에 산다.

셋 다 이름에 “널”이 들어가고 셋 다 어딘가에 0이 얹혀 있어서 뒤섞기 쉽지만, 포인터의 널과 문자의 NUL은 사는 세계가 다르다. “값이 우연히 다 0이라서 이름이 닮았을 뿐인 남남”이라 기억해 두면 된다.

4.3 정렬 — 두 칸짜리 짐은 아무 데나 못 넣는다

사물함 복도의 두 번째 구석 지식이다. 3장에서 기계는 여러 칸을 한 움큼에 집는다고 했다(워드). 그런데 기계의 손은 아무 위치에서나 움큼을 집도록 만들어져 있지 않다 — 대개 **자기 크기의 배수 번호에서만** 편하게 집는다. 4바이트짜리 수는 4의 배수 주소(100, 104, 108, ...)에, 8바이트짜리는 8의 배수 주소에 놓여 있을 때 한 번에 집힌다. 이 규칙이 **정렬(alignment)**이다.

78	56	34	12				
100	101	102	103	104	105	106	107

위처럼 100번(4의 배수)에서 시작하는 4바이트는 정렬이 맞다. 만약 같은 수를 102번에서 시작해 넣으면 — 정렬이 어긋난다. 그때 무슨 일이 일어나는지도 기계마다 다르다. 관대한 기계(인텔 계열)는 두 움큼으로 나눠 집느라 **느려질 뿐** 일은 해 준다. 엄격한 기계(옛 SPARC, 일부 ARM 시절)는 그 자리에서 오류(버스 폴트)를 내며 프로그램을 세웠다. “되는 기계에서만 되는 코드”라는 이식성 문제의 고전적 원천 중 하나다.

문 왜 기계는 아무 데서나 못 집는가? 사물함 비유로 무엇이 걸리는가?

답 기계의 손(버스)이 복도를 4칸짜리 격자로 보고 움직이기 때문이다. 100-103이 한 격자, 104-107이 다음 격자다. 102에서 시작하는 4바이트는 두 격자에 걸쳐 있어서, 한 번에 집으려면 격자 두 개를 모두 열고 필요한 조각을 이어 붙여야 한다 — 두 번 일하거나, 아예 거부하거나. 두 칸짜리 짐은 짝수 번호부터 넣으라는 사물함 관리 규칙과 같다.

4.4 하위 비트의 재주 — 태그 포인터

정렬 규칙에는 뜻밖의 부산물이 있다. 4의 배수인 수를 2진법으로 적으면 끝의 두 비트가 **항상 0**이고, 8의 배수라면 끝의 세 비트가 항상 0이다. 즉, 정렬된 주소의 하위 비트들은 언제나 0이라서 — **정보를 담는 공짜 공간**으로 쓸 수 있다.

이 재주를 **태그 포인터(tagged pointer)**라 부른다. 주소를 적어 두는 김에, 어차피 0인 끝자리에 작은 딱지(태그)를 끼워 넣는 것이다. 실제로 널리 쓰인다 — Lisp 계열 언어와 OCaml의 런타임은 “이 값은 정수인가, 객체 주소인가”를 하위 비트 태그로 구별하고, 자바스크립트 엔진과 가비지 컬렉터들도 같은

재주로 상태 표시를 포인터에 얹는다. 물론 공짜는 아니다 — 그 주소를 진짜로 쓰기 전에는 딱지를 떼야 (하위 비트를 도로 0으로) 한다.

여기서 앞의 널 이야기와 다시 만난다. 태그가 얹힌 세계에서는 “없음”을 나타내는 특별한 값조차 태그를 품고 있어서 **모든 비트가 0이 아닐 수 있다**. 예컨대 Lisp의 “없음”(NIL)은 실재하는 특별 객체의 주소다. C의 널 포인터가 주류 기계에서 전부 0으로 통일된 오늘날에도, 한 층 위의 런타임 세계에서는 “0 아닌 없음”이 여전히 현역이라는 이야기다.

문 프로그래머가 직접 태그 포인터 재주를 부려도 되는가?

답 C에서 기술적으로 가능하고 실제로 쓰이는 기법이지만, 함정이 많은 고급 기술이다 — 정렬 보장이 있어야 하고, 쓰기 전에 반드시 태그를 떼야 하고, 뒤에 배울 포인터 규칙(12장 프로버넌스, 32장)과도 얽힌다. 지금 단계의 결론은 하나면 된다: 주소의 하위 비트는 “그냥 수”가 아니라 정렬이 만들어 준 특별한 자리라는 것.

이 장의 세 가지 구석 지식 — 0번지의 특별함, 정렬, 하위 비트 — 은 모두 하나의 사실에서 나왔다. 주소는 수이지만, **아무 수나 똑같이 대접받는 것은 아니라는 것**. 복도에도 지리(地理)가 있다.

다음 장부터는 사물함에 담기는 내용물 쪽으로 눈을 돌린다. 표현의 사다리 첫 단은 정수다 — 음수를 비트로 담는 세 가지 방법이 경쟁했던 사연과 C23의 결단, 넘치는 수(오버플로), 그리고 비트를 통째로 밀고 당기는 시프트까지.

5 정수의 표현 — 부호, 오버플로, 시프트

이 장이 끝나면

표현의 사다리 첫 단이다. 부호 없는 정수가 어떻게 순환하는 수(모듈러) 인지, 음수를 비트로 담는 세 가지 방법이 어떻게 겨루다 2의 보수로 정리됐는지 — 그리고 C23이 마침내 무엇을 못박았는지 — 를 본다. 넘치는 수(오버플로)와, 비트를 통째로 밀고 당기는 시프트까지 챙기면, 정수에 관한 배경지식이 완성된다.

돌아보기 3장에서 여러 바이트를 이어 붙여 큰 수를 담는다고 했고, 엔디안(넣는 순서)까지 보았다. 그런데 지금까지의 수는 전부 0 이상이였다. 음수는 — 비트 어디에 마이너스 기호를 담는가?

답: 담을 곳이 없다 — 비트에는 0과 1뿐, 마이너스 기호가 없다(2장). 그래서 음수 역시 **약속**으로 만든다. 어떤 비트 패턴을 음수로 “읽기로” 정하는 것이다. 그 약속을 정하는 방법이 하나가 아니었다는 것, 그리고 그 경쟁의 결말이 이 장의 한복판이다.

5.1 부호 없는 정수 — 시계처럼 도는 수

먼저 마이너스 걱정이 없는 세계부터 정리한다. n 비트를 그냥 2진수로 읽으면 0부터 $2^n - 1$ 까지, 2^n 개의 수가 담긴다(2장). 이것이 **부호 없는(unsigned)** 정수다. 8비트면 0~255다.

이 수의 세계에서 꼭 챙길 성질이 하나 있다. **끝이 처음과 이어져 있다**는 것이다. 255에서 1을 더하면 256이 아니라 — 256을 담을 아홉 번째 비트가 없으므로 — 0이 된다. 12시에서 한 시간이 지나면 1시가 되는 시계와 같은 구조다. 수학에서는 이런 셈을 **모듈러 산술**이라 부른다.

Σ 모듈러 산술 — 유한한 수의 정확한 수학

n 비트 부호 없는 정수의 덧셈·뺄셈·곱셈은 결과를 2^n 으로 나눈 나머지를 취하는 연산과 정확히 같다:

$$\text{결과} = (a + b) \bmod 2^n$$

8비트에서 $250 + 10 = 260 \bmod 256 = 4$ 다. 중요한 것은 이것이 “틀린 덧셈”이 아니라 **다른 덧셈** — 완전히 정의된, 예측 가능한 수학 — 이라는 점이다. C 표준도 부호 없는 정수의 넘침을 오류가 아니라 이 수학으로 정의한다.

이 순환에는 이름이 있다 — **오버플로(overflow)**, 정확히는 “감아 돌기”(wrap-around)다. 정의된 동작이라 해도, 예상하지 못한 자리에서 일어나면 사고가 된다.

● 256판의 벽 — 팩맨 킬 스크린

오락실 게임 팩맨에는 유명한 벽이 있다. 255판까지는 멀쩡히 진행되다가 256판째에 화면 오른쪽 절반이 뜻 모를 기호로 뒤덮이며 게임이 불가능해 진다. 판 번호를 **8비트**로 세던 코드가 255를 넘는 순간 0으로 감아 돌았고, “지금 몇 판인지”를 전제로 과일을 그리던 루틴이 엉뚱한 개수를 그리며 화면을 부순 것이다. 설계자가 “256판까지 갈 사람은 없다”고 여겼던 그릇의 크기가, 최고수들의 도전 앞에서 벽이 된 사례다 — 그릇의 크기는 언제나 누군가에게는 닿는 벽이 된다.

5.2 음수를 담는 세 가지 약속

이제 음수다. 관건은 n 비트 패턴의 절반쯤을 음수로 “읽기로” 약속하는 방식인데, 역사상 세 가지 약속이 실제로 쓰였다. 8비트로 -5를 담는 경우로 비교한다.

첫째, 부호-크기(sign-magnitude). 사람의 표기를 그대로 흉내 낸다 — 맨 앞 비트를 마이너스 기호로 쓰고(1이면 음수), 나머지에 크기를 담는다. -5 는 $1_0000101$. 직관적이지만 두 가지 대가가 있다. $00000000(+0)$ 과 $10000000(-0)$, **0이 두 개 생긴다**. 그리고 덧셈 회로가 골치다 — 부호가 다른 두 수를 더하려면 “크기를 비교해서 큰 쪽에서 작은 쪽을 빼고 부호를 정하는” 별도 절차가 필요하다.

둘째, 1의 보수(ones' complement). 음수를 만들 때 모든 비트를 뒤집는다. 5 가 0000101 이니 -5 는 11111010 . 덧셈 회로가 한결 단순해지지만, 여전히 0이 두 개다(00000000 과 11111111) — 비교할 때마다 “두 0은 같은 것으로 친다”는 예외를 끌고 다녀야 한다.

셋째, 2의 보수(two's complement). 비트를 뒤집고 **1을 더한다**. -5 는 $11111010 + 1 = 11111011$. 언뜻 임의로 보이는 이 규칙이 사실은 가장 우아하다 — 0이 하나뿐이고, 무엇보다 **부호 없는 덧셈 회로를 그대로 쓰면 부호 있는 덧셈이 저절로 맞는다**. 별도 절차도 예외도 없다.

문 그런데 왜 이름이 “보수”인가? 그리고 1의 보수, 2의 보수라는 이름에서 1과 2는 무엇을 가리키는가?

답 **보수(補數, complement)**는 “채워서 어떤 기준을 완성해 주는 수”다. 십진법으로 먼저 감을 잡으면 쉽다 — 세 자리 수 304에 대해, 각 자리를 9까지 채워 주는 695를 **9의 보수**라 하고(모든 자리에서 $9 - \text{그 자리}$), 1000까지 채워 주는 696을 **10의 보수**라 한다($10^3 - 304$). 둘의 관계는 “9의 보수 + 1 = 10의 보수”다.

2진법에서 똑같은 일을 하면 이름의 뜻이 풀린다. 각 자리를 1까지 채워 주는 수 — 즉 11111111 에서 빼는 것 — 가 **1의 보수**인데, 2진법에서 $1 - \text{비트}$ 는 곧 비트 뒤집기라서 “뒤집는다”는 규칙이 나온 것이다. 그리고 2^n — 이진법의 “1000...0”, 즉 **2의 거듭제곱** — 에서 빼는 것이 **2의 보수**다. “뒤집고 1을 더한다”는 요령은 십진법의 “9의 보수 + 1 = 10의 보수”와 똑같은 관계다.

영어 표기도 여기서 정리해 둔다. 통용 표기는 **one's complement**와 **two's complement**다 (“1s”나 “2s” 같은 표기는 없다). 다만 전산학자 커누스(Knuth)는 재치 있는 구별을 주장했다 — 1의 보수는 “모든 자리의 1들에 대한 보수”이므로 복수 소유격 **ones' complement**가 옳고, 2의 보수는 “ 2^n 이라는 하나의 수에 대한 보수”이므로 단수 소유격 **two's complement**가 옳다는 것이다. 문법 트집 같지만 두 보수의 수학적 정의 차이(자리별 기준 vs 전체 기준)를 정확히 담은 구별이고 — 실제로 **C 표준 문서 자신이 이 구별을 채택했다**. 표준의 표현 조항은 세 방식을 “sign and magnitude”, “two's complement”, “**ones' complement**”라고 적는다. 커누스의 트집이 법전에 이긴 셈이다. 교과서 용어로는 2의 보수를 **기수 보수(radix complement)**, 1의 보수를 **감소 기수 보수(diminished radix complement)**라고도 부른다.

Σ 2의 보수가 우아한 이유 — 모듈러 산술의 재활용

2의 보수의 정체는 위의 모듈러 산술이다. $-x$ 를 $2^n - x$ 라는 패턴으로 담는 약속이므로, 8비트에서 -5 는 $256 - 5 = 251$, 즉 11111011 이다. 그러면 $7 + (-5)$ 는 회로 입장에서 $7 + 251 = 258 \bmod 256 = 2$ — 답이 저절로 맞는다. 음수란 “모듈러 시계에서 반대 방향으로 세는 것”일 뿐이라서, 회로는 부호를 아예 몰라도 된다. 유일한 비대칭은 범위다 — 8비트에서 -128 부터 $+127$ 까지로, 음수가 하나 더 많다(-128 의 짝 $+128$ 이 없다).

5.3 세 약속의 경쟁, 그리고 C23의 결단

세 방식 모두 실제 기계에 쓰였다 — 부호-크기와 1의 보수는 초기 대형기들(UNIVAC, CDC 계열 등)에 실존했다. C가 표준화된 1989년에도 그런 기계들이 현역이었기 때문에, C 표준은 어느 편도 들지 않았다. **세 가지 표현을 모두 허용한 것이다**. 이 중립은 공짜가 아니었다 — 표현이 다르면 같은 연산의 결

과 비트가 달라지므로, 표준은 부호 있는 정수의 많은 동작을 “기계마다 다르다”로 남겨 둘 수밖에 없었다. 부호 있는 오버플로가 **정의되지 않은 동작**(42장)이 된 사연의 절반이 여기에 있다.

그사이 현실은 한쪽으로 수렴했다. 2의 보수의 회로 단순성이 압도적이라 수십 년간 새로 만들어진 CPU는 사실상 전부 2의 보수였고, 나머지 두 방식은 박물관으로 갔다. 그리고 **C23이 마침내 결단했다 — 부호 있는 정수의 표현은 2의 보수다**. 반세기의 관행이 표준의 약속으로 승격된 것이다(2장의 “바이트=8비트” 논의와 똑같은 무늬다).

문 표현이 2의 보수로 못박혔으니, 이제 부호 있는 오버플로도 “감아 돌기”로 정의된 것인가?

답 아니다 — 여기가 미묘하고 중요한 지점이다. C23이 못박은 것은 **표현** (음수가 어떤 비트 패턴인가)이지, **오버플로의 의미**가 아니다. 부호 있는 정수의 오버플로는 C23에서도 여전히 정의되지 않은 동작으로 남았다. 이유는 표현이 아니라 최적화다 — “부호 있는 수는 넘치지 않는다”는 전제가 컴파일러(11장)의 루프 분석과 재배열에 요긴해서, 표준은 그 전제를 유지하는 쪽을 택했다. 요약하면: 부호 없는 넘침 = 정의된 감아 돌기, 부호 있는 넘침 = 여전히 계약 밖. 실무 규칙은 23장에서 다룬다.

△ “오버플로가 나면 컴퓨터가 오류를 알려 준다”

그렇듯한 기대다 — 잘못된 일이 생기면 알려 주는 것이 도리이니까. 그러나 CPU의 덧셈 회로는 넘침의 순간 내부 신호(플래그)를 올릴 뿐, **프로그램을 세우거나 알리지 않는 것이 기본**이다. C도 마찬가지다 — 부호 없는 수는 조용히 감아 돌고, 부호 있는 수는 계약 밖(무슨 일이든 일어날 수 있음)이다. 팩맨의 화면이 요란하게 부서진 것은 오버플로 “경보”가 아니라 감아 둔 값이 낳은 **후속 사고**였다. 넘침의 감시는 기계가 아니라 프로그래머의 일이다 — 이것이 뒤에서 proven 같은 검증 도구가 산술을 검사하는 이유이기도 하다(35장, 50장).

5.4 시프트 — 비트를 통째로 밀기

정수의 비트를 다루는 기본 연산이 하나 더 있다 — **시프트(shift)**, 비트 열을 통째로 왼쪽이나 오른쪽으로 미는 것이다. 밀고 나면 두 가지 질문이 남는다. **밀려난 비트는 어디로 가고, 빈자리는 무엇으로 채우는가**.

왼쪽 시프트는 답이 하나다. 위로 밀려난 비트는 버려지고, 아래 빈자리는 0으로 채운다. 00010110을 왼쪽으로 한 칸 밀면 00101100 — 십진법에서 끝에 0을 붙이면 10배가 되듯, 2진법의 왼쪽 한 칸은 **2배**다.

오른쪽 시프트는 답이 둘이다. 빈자리(맨 위)를 무엇으로 채우는가에 따라 갈린다.

- **논리 시프트(logical)**: 0으로 채운다. 부호 없는 수에 맞는 방식이고, 오른쪽 한 칸은 2로 나눈 몫이 된다.
- **산술 시프트(arithmetic)**: **부호 비트를 복사해** 채운다. 2의 보수 음수는 맨 위 비트들이 1로 차 있으므로(위의 $-5 = 1111011$), 1을 채워야 “2로 나누기”라는 뜻이 유지된다. 0을 채워 버리면 음수가 갑자기 거대한 양수처럼 읽힌다.

그래서 CPU들은 오른쪽 시프트 명령을 두 벌(논리/산술) 갖고 있고, C에서는 부호 없는 수의 오른쪽 시프트는 논리로, 부호 있는 음수의 오른쪽 시프트는 — 오랫동안 “기계마다 다르다”였다가 사실상 모든 구현이 산술을 쓰는 관행으로 수렴했다. 2의 보수 확정과 나란히, 관행을 약속으로 승격하는 방향의 정리다.

문 비트를 밀고 채우는 규칙까지 알아야 할 만큼, 시프트가 중요한 연산인가?

답 중요하다 — 두 가지 이유에서다.

첫째, **가장 싼 연산이기 때문이다**. 시프트는 회로 입장에서 배선을 옆으로 옮기는 수준의 일이라, 거의 모든 CPU에서 한 박자짜리 최속 연산에 속한다. 방금 본 대로 왼쪽 k 칸은 2^k 배, 오른쪽 k 칸은 2^k 으로 나눈 몫이므로 — 2의 거듭제곱 곱셈·나눗셈을 시프트로 바꾸는 것은 고전적인 속도 요령이었다. 다만 오늘의 C에서는 그 요령을 **사람이 부릴 필요가 없다**. 소스에는 뜻 그대로 $x * 2$, $x / 8$ 이라 쓰면 되고, 컴파일러(11장의 편집자)가 알아서 시프트로 바꿔 준다. 가독성을 내주고 얻을 것이 없는 자리다.

둘째, **비트의 세계를 다루는 기본 동작이기 때문이다**. 여러 값을 한 정수의 비트 자리들에 나눠 담고 꺼내는 일 — 7장에서 본 UTF-8 바이트 조립, 4장의 태그 포인터, 색상값(RGB)의 분해, 하드웨어 레지스터의 플래그 읽기 — 이 전부가 “원하는 자리로 밀고, 필요한 비트만 남기는” 시프트+마스크의 조합이다. 수의 곱셈으로서의 시프트는 컴파일러에게 넘어갔지만, **자리 배치 도구**로서의 시프트는 시스템 프로그래머의 일상 언어로 남아 있다. C 문법에서의 실제 사용은 24장에서 다룬다.

문 8비트짜리 수를 여덟 칸, 혹은 그 이상 밀면 어떻게 되는가? 상식적으로는 전부 밀려나 0이 될 것 같은데.

답 바로 그 “상식”이 기계마다 달랐다는 것이 함정이다. 시프트 칸수는 CPU 안에서 몇 비트짜리 회로가 처리하는데, 꼭 이상의 칸수가 들어왔을 때의 대응이 갈렸다 — 어떤 CPU 계열(인텔 x86)은 칸수의 아래 몇 비트만 보고 나머지를 무시해서 32비트 수를 32칸 밀면 **그대로**이고, 다른 계열(옛 ARM 등)은 정말로 다 밀어서 **0**이 된다. 같은 코드가 기계마다 다른 답을 내는 것이다. C 표준의 대응은 이제 익숙한 패턴이다 — 어느 편도 들 수 없으니, **꼭 이상의 시프트는 정의되지 않은 동작**으로 계약 밖에 두었다. “기계들이 서로 다르게 대응하는 곳은 표준이 약속을 포기한다” — 이 무늬는 42장에서 정식으로 다시 만난다.

5.5 부호 확장 — 좁은 그릇에서 넓은 그릇으로

시프트의 “무엇으로 채우는가”라는 질문은 한 군데서 더 나타난다. 8비트 그릇에 든 수를 16비트 그릇으로 옮겨 담을 때다. 늘어난 위쪽 여덟 칸을 무엇으로 채울 것인가.

부호 없는 수는 답이 자명하다 — **0으로 채운다**(zero extension). 8비트의 11111011(=251)은 16비트의 00000000 11111011(=251)이 된다. 값이 그대로다.

부호 있는 수에서는 같은 방법이 사고를 낸다. 8비트의 11111011은 2의 보수로 -5인데, 위를 0으로 채우면 16비트의 00000000 11111011 — 맨 앞 비트가 0이니 **양수 251**로 읽힌다. -5가 그릇을 옮기다가 251이 된 것이다. 올바른 답은 산술 시프트와 같은 요령이다 — **부호 비트를 복사해 채운다**. 11111111 11111011, 여전히 -5다. 이것이 **부호 확장**(sign extension)이다.

문 1을 잔뜩 채웠는데 왜 값이 그대로인가? 우연인가?

답 우연이 아니라 모듈러 수학의 필연이다. 2의 보수에서 8비트의 -5는 $2^8 - 5 = 251$ 이라는 패턴이었고, 16비트의 -5는 $2^{16} - 5 = 65531$ 이라는 패턴이다. 그런데 $65531 = 251 + 255 \cdot 256$ — 이진법으로 적으면 정확히 “원래 패턴 위에 1을 여덟 개 얹은 것”이다. 부호 비트 복사를 “ 2^8 에 대한 보수를 2^{16} 에 대한 보수로 갈아 끼우는” 산술을 비트 복사 한 번으로 해치우는 요령이다. 2의 보수의 우아함이 여기서도 일한다 — 부호-크기나 1의 보수였다면 이런 공짜 확장은 없다.

거꾸로 넓은 그릇에서 좁은 그릇으로 **줄여** 담으면 위쪽 비트들이 그냥 잘려 나간다 — 값이 그릇에 안 들어가면 소리 없이 망가진다는 점에서 오버플로의 사촌이다. C는 계산 전에 작은 정수를 자동으로 넓히

는 규칙 (정수 승격)을 갖고 있고, 넓히기·줄이기가 언제 일어나며 무엇이 위험한지 는 C의 정수 타입과 함께 23·24장에서 정식으로 다룬다 — 이 장의 그림 (0 채움 / 부호 복사 / 잘림)이 그때의 밑천이다.

정수의 배경지식이 완성됐다. 부호 없는 수는 시계처럼 도는 모듈러의 세계이고, 음수는 세 가지 약속의 경쟁 끝에 2의 보수로 정리되어 C23이 못박았으며, 넘침은 조용하고, 시프트와 그릇 옮기기(확장)는 채움의 방식 까지 알아야 뜻이 선다. C의 정수 **타입들**과 실무 규칙은 23·24장에서 이 배경 위에 세운다.

다음 장은 사다리의 다음 단이다 — 정수 너머, 소수점 있는 수를 담는 두 가지 방법과 IEEE 754라는 계약을 만난다.

6 수의 표현 ↔ IEEE 754라는 계약

이 장이 끝나면

표현의 사다리 첫 단이다. 소수점 있는 수를 사물함에 담는 두 가지 방법 — 고정소수점과 부동소수점 — 을 구별하게 되고, 부동소수점의 난립을 통일한 IEEE 754라는 계약을 만난다. 컴퓨터의 수가 유한하고 근사적이라는, 이 책에서 가장 중요한 사실 하나가 여기서 자리 잡는다.

돌아보기 5장에서 부호 있는 정수까지 비트에 담았다 — 음수를 담는 세 가지 방법이 겨루다 2의 보수로 정리되는 것까지 보았다. 그런데 여전히 전부 정수다. 1.5 같은 수는 — 비트로 어떻게 담는가?

답 비트 자체에는 소수점이 없다. 그래서 소수점은 **약속**으로 만든다 — 음수를 “약속”(2의 보수)으로 만들었던 것과 같은 이치다. “소수점이 어디에 있다고 치고 읽을 것인가”를 정하는 방식이 두 갈래로 갈리는데, 소수점을 못박아 두는 쪽(고정)과 소수점을 데이터에 실어 움직이는 쪽(부동)이다. 이 장이 그 두 갈래의 이야기다.

6.1 고정소수점 — 소수점을 약속으로 못박다

첫 번째 방법은 놀랄 만큼 단순하다. **그냥 정수를 쓰되, 단위를 바꾼다.**

돈 계산이 좋은 예다. 1,500.25원을 저장하고 싶으면, “우리 장부는 전부 전(錢, 0.01원) 단위로 적는다”고 약속하고 정수 150025를 저장하면 된다. 소수점은 어디에도 저장되지 않는다 — 읽고 쓰는 모든 사람이 “끝에서 두 자리 앞에 소수점이 있다고 친다”는 약속을 공유할 뿐이다. 이것이 **고정소수점(fixed point)**이다. 소수점의 위치가 약속으로 고정되어 있다.

고정소수점의 미덕은 정수 그 자체라는 것이다. 더하기와 빼기가 정수 연산만큼 빠르고 정확하며, 어렵지 않다. 그래서 금액 계산과, 소수점 하드웨어가 없는 작은 임베디드 칩에서 지금도 현역이다. 약점은 뺏함이다 — 아주 큰 수와 아주 작은 수를 한 약속으로 감당할 수 없다. 전 단위 장부에 은하의 질량과 원자의 질량을 같이 적을 수는 없는 노릇이다.

6.2 부동소수점 — 소수점을 데이터에 싣다

두 번째 방법은 과학 시간에 배운 **과학적 표기법**을 기계에 옮긴 것이다. 6.02×10^{23} 처럼 수를 “알맹이 숫자”와 “10을 몇 번 곱했는가”로 나눠 적는 방식 말이다. 알맹이(가수, significand)와 곱한 횟수(지수, exponent)를 둘 다 데이터로 저장하면, 소수점이 지수를 따라 **떠다닌다** — 그래서 **부동소수점(floating point)**이다. 기계는 10 대신 2를 쓴다: 수를 가수 $\times 2^{\text{지수}}$ 꼴로 담는다.

이 방식의 힘은 **같은 비트 수로 어마어마한 범위를 감당한다**는 것이다. 은하도 원자도 한 형식에 담진다. 대가는 정밀도다 — 알맹이 숫자의 자리 수가 유한하므로, 그 자리 수를 넘는 부분은 **반올림으로 버려진다**.

△ “컴퓨터는 수학을 잘하니까, 소수 계산은 당연히 정확하다”

그렇듯하다 — 계산기가 틀리는 것을 본 적이 없으니까. 그러나 부동소수점이 담을 수 있는 수는 **유한 개**뿐이고, 담지 못하는 수는 가장 가까운 이웃으로 반올림된다. 놀랍게도 0.1조차 그렇다 — 0.1은 2진법으로 무한소수라서 (아래 수학 박스), 기계 속의 “0.1”은 사실 0.1에 아주 가까운 다른 수다. 작은 어긋남은 계산을 거치며 쌓일 수 있다. 이것은 컴퓨터의 결함이 아니라 유한한 표현의 필연이고, 잘 다루는 법이 따로 있다(40장). 지금 기억할 것은 하나다: **부동소수점은 정확한 수가 아니라 성실한 근사다.**

Σ 왜 0.1은 2진법으로 못 떨어지는가

유한 소수로 떨어지는 분수는 분모가 기수의 거듭제곱 인수로만 이루어진 경우뿐이다. 십진법(기수 $10 = 2 \times 5$)에서 $\frac{1}{10}$ 은 한 자리로 끝난다. 그러나 2진법의 기수는 2뿐이라서, 분모에 5가 들어 있는 $\frac{1}{10}$ 은 절대 유한하게 끝나지 않는다:

$$0.1_{10} = 0.0001100110011..._2$$

$\frac{1}{3}$ 이 십진법에서 0.333...으로 무한한 것과 같은 이치다. 유한한 가수는 이 무한을 어딘가에서 잘라야 하고, 그 잘림이 근사의 근원이다.

6.3 근사가 일으키는 세 가지 사건

“성실한 근사”가 실전에서 어떤 얼굴로 나타나는지, 수치로 미리 구경해 둔다. (여기서는 지면 계산으로 보이고, C 코드로 직접 실행해 보이는 것은 40장이다.)

사건 1 — $0.1 + 0.2 \neq 0.3$. 프로그래밍 세계에서 가장 유명한 등식 아닌 등식이다. 방금 본 대로 0.1도 0.2도 0.3도 2진법으로는 무한소수라서, 기계에는 각각 “가장 가까운 이웃”이 대신 담긴다. 그런데 0.1의 이웃과 0.2의 이웃을 더한 결과가, 하필 0.3의 이웃과 **다른 쪽**으로 반올림된다. 64비트 형식으로 실제 값을 적으면 —

- 기계 속 $0.1 + 0.2 = 0.30000000000000004440...$
- 기계 속 $0.3 = 0.29999999999999998889...$

둘 다 0.3에 성실하게 가깝지만, **서로 같지는 않다**. 그래서 “같은가?”라고 물으면(==) 기계는 정직하게 “아니오”라고 답한다.

내부 표현을 직접 들여다보면 사건의 전모가 한눈에 보인다. 64비트 형식은 [부호 1비트 | 지수 11비트 | 가수 52비트]로 나뉘는데, 이 네 수의 64비트를 16진법으로 적으면 —

수	64비트 내부 표현 (16진)
0.1	3FB9 9999 9999 999A
0.2	3FC9 9999 9999 999A
0.3	3FD3 3333 3333 3333
$0.1 + 0.2$	3FD3 3333 3333 3334

읽을거리가 세 겹이다. 첫째, 0.1과 0.2의 가수에 늘어선 9999...는 2진 무한소수 0011 의 순환이 16진법으로 보인 모습이고, **끝자리가 9가 아니라 A인 것은** 무한을 52비트에서 자르며 반올림해 올린 흔적이다 — 수학 박스에서 본 “잘림”이 비트에 그대로 찍혀 있다. 둘째, 0.3의 가수 3333...도 같은 순환의 다른 위상이며, 이쪽은 마지막에서 내림이 됐다. 셋째 — 핵심이다 — 0.3과 $0.1+0.2$ 는 **마지막 비트 하나만** 다르다. 눈금 하나(전문 용어로 1 ulp) 차이의 바로 옆 이웃인 것이다. ==는 이 한 비트의 차이도 다름으로 답하므로, 부동소수점의 “같음”은 눈금 하나의 어긋남에도 깨진다.

사건 2 — 그러면 비교는 어떻게 하는가. 부동소수점의 세계에서 “같다”는 질문 자체를 바꿔야 한다 — “정확히 같은가”가 아니라 “충분히 가까운가”로. 허용 오차(전통적으로 **엡실론**, ϵ 이라 부른다)를 하나 정하고, 두 수의 차이가 그 안에 들면 같다고 치는 것이다:

$$|a - b| < \epsilon \quad (\text{예: } \epsilon = 10^{-9})$$

이것이 기본형이고, 실전에는 한 겹이 더 있다 — 수가 커지면 이웃 사이 간격도 커지므로(아래 사건 3), 고정된 ϵ 대신 **수의 크기에 비례하는** 허용치(상대 오차)를 쓰는 것이 안전하다. 두 방식의 구체적인 사용

법과 함정은 40장에서 C 코드로 다룬다. 지금 기억할 것은 한 문장이다: **부동소수점을 ==로 비교하는 코드는 거의 언제나 의심 대상이다.**

사건 3 — 큰 수 곁에서 작은 수는 사라진다. 유효숫자가 유한하다는 것은, 크기가 너무 다른 두 수를 한 그릇에서 만나게 하면 작은 쪽이 **통째로 사라진다**는 뜻이기도 하다. 32비트 float(유효 십진 약 7자리) 로 시연하면 —

- 8.000000001을 담는 순간: 유효 7자리를 넘는 꼬리가 잘려 그냥 8.0이 된다. **더하기도 전에, 저장에서 이미 사라졌다.** 내부 표현으로 보면 더 선명하다 — float의 8.0은 4100 0000인데, 8.000000001을 담아도 가장 가까운 눈금이 8.0이라서 **똑같은 4100 0000**이 된다. 십진 표기로는 다른 두 수가, 비트로는 완전히 같은 하나의 수다.
- $8.0 + 0.000000008$: 결과는 그대로 8.0, 비트도 그대로 4100 0000 이다. 왜인가 — 8.0 크기에서 float의 눈금 간격(이웃 사이 거리)을 계산하면 $2^3 \times 2^{-23} = 2^{-20} \approx 0.000000095$ 다. 더한 0.000000008은 눈금 반 칸에도 못 미쳐서, 반올림하면 제자리다. 이것을 **흡수**라 부른다.
- 반대로 $8.0000001 - 8.0$ 처럼 **거의 같은 두 수의 뺄셈**은 앞자리들이 전부 지워지고 끝자리의 어림만 남는다 — 유효숫자 7자리로 시작한 계산이 유효숫자 한두 자리짜리 답을 내는 것이다. 이쪽은 **상쇄(cancellation)**라 부르며, 수치 계산에서 정밀도를 갉아먹는 주범이다.

세 사건의 뿌리는 하나다 — 표현 가능한 수들이 **띄엄띄엄한 눈금**이고, 눈금 간격은 수가 클수록 넓어진다는 것. 0 근처에서는 눈금이 촘촘해 10^{-300} 같은 수도 구별하지만, 10^{16} 쯤 가면 눈금 간격이 1을 넘어 **정수조차 건너뛰다**. “몇 자리까지 믿을 수 있는가”(유효숫자)라는 감각이 부동소수점 사용의 전부라 해도 과언이 아니다.

6.4 난립, 그리고 IEEE 754라는 계약

부동소수점이라는 착상은 일찍부터 있었지만, **어떻게** 담을 것인가 — 가수와 지수에 몇 비트씩 줄 것인가, 반올림은 어느 쪽으로 할 것인가 — 는 오랫동안 회사마다 달랐다. IBM 대형기, DEC의 VAX, Cray 슈퍼컴퓨터가 저마다 다른 형식을 썼다. 같은 프로그램이 기계를 옮기면 **다른 답**을 내놓았고, 수치 계산 프로그래머들은 기계마다 계산의 버릇을 새로 익혀야 했다.

1985년, 이 난립을 IEEE 754라는 표준이 통일했다. 형식(부호 1비트 + 지수 + 가수), 반올림 규칙, 무한대와 “수 아님”(NaN) 같은 특수값의 처리까지 못박은 정밀한 계약이다. 오늘날 사실상 모든 CPU와 GPU가 이 계약을 따른다 — 32비트 형식(C의 float)과 64비트 형식(C의 double)이 그 대표다.

시대를 눈여겨보면 재미있다. 부동소수점의 계약(IEEE 754, 1985)과 C 언어의 계약(ANSI C, 1989)은 같은 몇 해 사이에 태어났다. 벤더마다 제멋대로이던 것을 건디다 못해 산업 전체가 “계약서를 쓰자”로 돌아선 시대였다 — 이 “계약의 시대” 이야기는 10장에서 다시 만난다.

문 32비트 float와 64비트 double은 실무에서 어떻게 갈라 쓰는가?

답 기본값은 double이다. 64비트 형식은 십진 유효숫자로 약 15-16자리를 감당해서 대부분의 계산에 여유가 있다. float(약 7자리)는 메모리와 대역폭이 아쉬운 곳 — 대량의 좌표, 그래픽, 기계학습 — 에서 크기의 이득으로 선택한다. “정밀도가 필요해서 double, 물량이 필요해서 float” 라고 기억하면 대강 맞다. C에서의 실제 사용은 40장에서 다룬다.

문 5장의 정수 오버플로와 이 장의 반올림은 둘 다 “유한한 그릇” 문제인데, 성격이 같은 문제인가?

답 뿌리는 같고 증상이 다르다. 정수의 그릇은 **범위**가 유한해서 끝에서 넘치고(오버플로 — 넘치기 전까지는 완전히 정확하다), 부동소수점의 그릇은 **정밀도**가 유한해서 어디서나 조금씩 어림된다(대

신 범위는 어마어마하다). 정확하지만 좁은 그릇과, 넓지만 어림하는 그릇 — 어느 그릇을 쓸지 고르는 감각이 수를 다루는 기본기이고, C의 타입 선택 (23장, 40장)이 바로 그 고르기다.

표현의 사다리 첫 단을 정리하면 이렇다. 소수점은 비트에 없고 약속에 있다. 약속을 못박으면 고정소수점, 데이터에 실으면 부동소수점이고, 부동소수점의 약속은 IEEE 754라는 계약으로 통일되어 있다. 그리고 그 계약 아래의 수는 정확한 수가 아니라 성실한 근사다.

다음 단은 글자다. 사물함에 “안녕”을 담으려면 무엇을 약속해야 하는가 — 문자와 텍스트의 세계, 그리고 그 약속들이 어긋나며 남긴 흉터들의 이야기다.

7 문자와 텍스트 ↔ 표준 속의 흉터

이 장이 끝나면

사물함에 글자를 담는 법을 배운다 — 문자는 결국 번호라는 것, 그 번호표 (문자 코드)가 어떤 역사를 거쳐 유니코드와 UTF-8에 이르렀는지, 그리고 그 역사가 C 문법에 남긴 흉터(삼중자)까지. 한국어를 쓰는 독자에게는 유난히 실감 나는 장이다.

돌아보기 6장에서 소수점은 비트에 없고 약속에 있다고 했다. 그러면 글자는 — 비트에 “가”라는 모양이 들어 있는 것인가?

답 아니다, 같은 이치다. 비트에는 모양도 소리도 없다. 있는 것은 수뿐이고, “어떤 수가 어떤 글자인가”라는 **번호표의 약속**이 따로 있다. 글자를 담는다는 것은 번호를 담는 것이고, 텍스트란 번호의 열이다. 이 장은 그 번호표들의 이야기다.

7.1 문자는 번호다

기계에 “A”를 저장하는 방법은 하나뿐이다 — “A는 65번으로 한다”는 약속을 만들고, 65라는 수를 저장하는 것이다. 이런 약속의 표, 즉 글자와 번호의 대응표를 **문자 집합**(character set) 또는 문자 코드라 부른다.

문제는 이 표가 **여러 개**였다는 것이다. 회사마다, 나라마다 자기 표를 만들었다. 같은 번호가 표마다 다른 글자를 가리켰고, 표가 다른 두 기계가 텍스트를 주고받으면 글자가 엉켰다. 이 장의 나머지는 그 엉킴의 역사이고, 역사의 각 굽이가 오늘의 C와 컴퓨팅에 흔적을 남겼다.

7.2 ASCII와 EBCDIC — 두 개의 표

1960년대에 두 갈래의 표가 자리 잡았다. IBM의 대형 컴퓨터 세계는 **EBCDIC**이라는 표를 썼다 — 천공카드 시절의 코드에서 자라난 표라서 알파벳이 연속 번호가 아닌 등 지금 눈에는 기묘한 배치였지만, IBM의 세계에서는 그것이 법이었다.

다른 한쪽에서 통신 업계를 중심으로 만들어진 표가 **ASCII**다. 7비트, 즉 128칸짜리 작은 표에 영어 대소문자·숫자·문장부호, 그리고 화면에 안 보이는 **제어 문자**들을 담았다. 제어 문자는 “종이를 한 줄 올려라”, “경고음을 울려라”처럼 글자가 아니라 장치에 내리는 지시다 — 이들의 사연은 다음 장(스트림)에서 만난다. 그리고 이 표의 **0번 자리**에 앉은 것이 4장에서 얼굴을 익힌 **NUL 문자**다. “아무 글자도 아님”을 뜻하는 문자로, 나중에 C가 문자열의 끝 표시로 채용한다(34장).

ASCII가 이긴 표가 되었다 — 오늘날 거의 모든 문자 코드가 ASCII를 안쪽에 품고 있다. 그러나 128칸은 영어만으로도 빠듯했고, 세계의 글자를 담기에는 어림도 없었다. 여기서 흉터의 역사가 시작된다.

7.3 ISO 646 — 국가 변형과 C의 삼중자

첫 번째 미봉책은 ASCII의 국제판인 **ISO 646**이었다. 아이디어는 이랬다: “표는 하나로 쓰되, 나라마다 덜 중요한 몇 칸을 자기 글자로 바꿔 쓰게 허용한다.” 그렇게 바꿔 쓰도록 지정된 칸에 하필 `[ ] { } \ | #` 같은 기호들이 있었다.

결과는 프로그래머에게 재앙이었다. 덴마크의 단말기에서 C 코드를 열면 중괄호 자리에 덴마크 글자가 보였다. 같은 번호인데 표가 달라 다른 글자가 찍힌 것이다.

● **₩ 표시** — 한국에 남은 ISO 646의 흉터

한국도 이 역사의 당사자다. 한국판 표(KS X 1003)는 백슬래시 \ 자리를 원화 기호 ₩로 바꿨다. 그 시절 한국 컴퓨터에서 파일 경로는 C:\Program Files\W...로 보였고 — 놀랍게도 이 흉터는 지금도 만질 수 있다. 오늘날의 한국어 윈도우에서도 일부 글꼴은 백슬래시 번호를 ₩로 그린다. 같은 번호, 같은 비트인데 표(글꼴)가 ₩를 그리는 것이다. “기억된 것과 그것을 읽는 방법은 별개”라는 이 책의 후렴구를, 한국 독자는 화면에서 매일 목격해 온 셈이다.

C는 이 혼란의 한복판에서 표준화됐다(10장). 중괄호가 안 보이는 나라에서도 C를 쓸 수 있어야 했으므로, C89는 기괴한 우회로를 마련했다 — **삼중자(trigraph)**다. ??<라고 적으면 {로, ??/라고 적으면 \로 읽어 주는 문자 세 개짜리 암호였다. 언어 문법에 새겨진, 문자 코드 전쟁의 흉터다.

문 삼중자는 지금도 쓰이는가?

답 아니다 — 그리고 그 최후가 근래 C 역사의 상징적 장면이다. 유니코드 시대가 오며 삼중자는 존재 이유를 잃었고, 오히려 ??? 같은 문자열이 뜻밖에 변환되는 함정으로만 남았다. C23이 삼중자를 표준에서 **제거**했다. 1989년에 새겨진 흉터가 2023년에야 지워진 것이다 — 표준이 무언가를 들이는 일보다 **빼는** 일이 얼마나 느린지 보여 주는 사례이기도 하다.

7.4 8비트의 백가쟁명, 그리고 한글

바이트가 8비트로 자리 잡자(2장) 표를 256칸으로 늘린 **ISO 8859** 계열이 나왔다 — 앞 128칸은 ASCII 그대로, 뒤 128칸에 각 언어권 글자를 담는 방식이다(서유럽용 Latin-1 등). 그러나 뒤 128칸을 언어권마다 다르게 쓰는 구조라, 표를 잘못 짚으면 글자가 엉키는 문제는 그대로였다.

그리고 256칸으로는 어림도 없는 글자들이 있었다. 한글, 한자, 가나 — 동아시아의 글자는 수천, 수만이다. 해법은 **한 글자에 바이트 여러 개**를 쓰는 멀티바이트 인코딩이었다(한국의 EUC-KR 등). 글자마다 길이가 다른 데다 나라마다 방식이 달라, 표를 잘못 짚은 한글 문서가 뜻 모를 기호의 행렬로 깨지는 일 — 이른바 “글자 깨짐” — 은 이 시대 한국 컴퓨터 사용자의 일상이었다.

7.5 유니코드와 UTF-8 — 하나의 표, 영리한 답기

근본 해법은 결국 하나뿐이었다. **세상 모든 글자를 하나의 표에 넣자**. 그것이 1990년대의 **유니코드**다. 글자마다 고유 번호(코드 포인트)를 주고 U+AC00(가)처럼 적는다. 표의 크기는 백만 칸이 넘는다.

남은 문제는 그 큰 번호를 바이트에 **어떻게 담느냐**였다. 모든 글자를 4바이트씩 쓰면 낭비가 크고, 무엇보다 산더미 같은 기존 ASCII 텍스트와 호환되지 않는다. 답이 된 것이 **UTF-8**이라는 가변 길이 인코딩이다 — ASCII 범위의 글자는 1바이트 그대로, 그 밖의 글자는 2-4바이트로 담는다. 기존 ASCII 파일은 그 자체로 올바른 UTF-8이라는 절묘한 설계 덕에, UTF-8은 오늘날 웹과 소스 코드의 사실상 유일한 표준이 됐다. 이 책의 예제와 C23의 `u8` 문자열이 쓰는 것도 UTF-8이다(34장).

Σ UTF-8의 바이트 구조

코드 포인트의 크기에 따라 길이가 달라진다. 첫 바이트의 앞 비트들이 “이 글자가 몇 바이트짜리인지”를 알리고, 이어지는 바이트는 전부 10으로 시작한다:

범위	바이트 수	비트 배치
U+0000 U+007F	1	0xxxxxxx (= ASCII)
U+0080 U+07FF	2	110xxxxx 10xxxxxx
U+0800 U+FFFF	3	1110xxxx 10xxxxxx 10xxxxxx

U+10000	U+10FFFF	4	11110xxx 10xxxxxx x3
---------	----------	---	----------------------

이어지는 바이트가 항상 10으로 시작하므로, 텍스트 중간 아무 데서 깨어나도 글자의 경계를 되찾을 수 있다(자기 동기화). 한글 음절(U+AC00 부근)은 3바이트 구간에 산다.

㉠ “한 글자는 1바이트다”

ASCII 시대의 직관이 남긴 오개념이고, C를 배울 때 특히 위험하다 — C의 char 타입은 이름과 달리 “글자”가 아니라 바이트이기 때문이다(34장). UTF-8에서 영문 a는 1바이트지만 한글 가는 3바이트다. “안녕”이라는 두 글자는 여섯 바이트다. 글자 수와 바이트 수는 다른 물건이고, 이 구별이 무너진 코드는 한글 처리에서 반드시 사고를 낸다.

7.6 같은 글자, 여러 표현 — 텍스트에 잠복한 보안 지형

유니코드가 표를 통일했다고 문제가 끝난 것은 아니다. 오히려 표가 커지고 표현이 여러 겹이 되면서 새로운 함정이 생겼다. 텍스트를 다루는 프로그램 이라면 언젠가 만나게 되는 지형이라, 지도 삼아 훑어 둔다. 뿌리는 하나다 — 같은 것을 두 가지 이상으로 적을 수 있으면, 검사하는 쪽과 해석하는 쪽이 어긋날 수 있다.

① 정규화(normalization) — 한글의 NFC와 NFD. “각”이라는 글자는 유니코드에서 두 가지로 적을 수 있다. 완성된 음절 하나로 적는 방식(NFC, U+AC01 한 코드 포인트)과, 자모를 이어 적는 방식(NFD, ㄱ + ㅏ + ㄱ 세 코드 포인트)이다. 눈에는 똑같이 보이지만 바이트 열은 완전히 다르다 — 그래서 단순 바이트 비교는 “각 ≠ 각”이라 답한다. 한국 사용자에게 이것은 교과서 이야기가 아니다: macOS가 파일을 NFD 계열로 저장해 온 탓에, 한글 파일 이름이 다른 OS로 건너가며 자모가 풀려 보이거나 압축 파일에서 이름이 어긋나는 일이 흔했다. 해법은 **경계에서 한 형태로 정규화**하는 것 — 그리고 보안 문맥에서는 이 정규화의 순서가 결정적이다(아래 ④).

② 오버롱 인코딩 — UTF-8이 한때 열어 둔 뒷문. 7장 수학 박스의 UTF-8 규칙대로면 /(U+002F)는 1바이트 2f다. 그런데 초기 구현들은 같은 문자를 2바이트(c0 af)나 3바이트로 “길게” 적은 것도 관대하게 받아 주었다 — 오버롱(overlong) 표현이다. 여기서 고전적 공격이 나왔다: 보안 검사는 2f만 찾아 경로 조작(..)을 걸러 내는데, 파일 시스템은 오버롱 c0 af도 /로 해석한 것이다. 검사와 해석의 어긋남 — 2001년 IIS 서버를 휩쓴 디렉터리 탈출 취약점의 정체가 이것이었다. 그래서 오늘의 표준은 **오버롱을 불법으로 못박고 디코더가 거부하도록** 정했다.

③ 사라진 인코딩의 유령 — UTF-7. 7비트 통로(초기 전자우편)로 유니코드를 보내려고 만든 UTF-7은, 평범해 보이는 ASCII 글자들로 다른 문자를 표현 한다(+ADw-가 <가 되는 식이다). 브라우저들이 인코딩을 자동 추측하던 시절, 공격자는 이 성질로 필터를 통과시켰다 — 검사할 때는 무해한 글자 열이었다가, 브라우저가 UTF-7로 해석하는 순간 태그로 부활하는 것이다. UTF-7은 이 사고들 때문에 사실상 퇴출됐고, 오늘의 규범은 **인코딩을 추측하지 말고 명시하라**로 굳었다.

④ 유니코드 자체의 함정 — 동형 문자와 방향 제어. 표가 크다는 것은 똑같이 생긴 다른 글자가 있다는 뜻이기도 하다. 라틴 a와 키릴 a는 화면에서 구별되지 않는다 — 이를 이용해 진짜와 똑같이 보이는 가짜 도메인을 만드는 것이 **호모그래프 공격**이다(그래서 브라우저는 의심스러운 혼용 도메인을 원래 표기로 되돌려 보여 준다). 더 교묘한 것은 **방향 제어 문자**다: 유니코드에는 글자의 표시 순서를 뒤집는 보이지 않는 문자들이 있어서, **소스 코드에서 사람이 읽는 순서와 컴파일러가 읽는 순서를 다르게** 만들 수 있다 — 2021년 공개된 “Trojan Source”가 이 기법으로, 리뷰어에게는 멀쩡해 보이는 코드가 실제로는 다른 논리로 컴파일되게 만들 수 있음을 보였다. 이후 컴파일러들이 이런 문자에 경고를 내기 시작했다.

⑤ **층이 겹칠 때 — 이스케이프와 다중 해석.** 웹에서 <를 글자로 보이게 하려면 <로 적는다(HTML 이스케이프). 이 층이 여럿 겹치면 사고가 난다 — 한 번 이스케이프한 것을 다른 층이 다시 해독하면 원래 기호가 부활하고, 반대로 검사만 하고 이스케이프를 잊으면 입력이 코드가 된다. 19장에서 볼 서식 문자열 취약점, 데이터베이스의 SQL 주입, 웹의 XSS가 전부 같은 무늬다 — **데이터를 틀(코드)로 오해하게 만드는 것.** C의 삼중자도 이 무늬의 조상 격 사례였다: 문자열 안에 우연히 ??/가 들어 있으면 전처리기가 그것을 백슬래시로 바꿔 버려, 프로그래머가 적은 적 없는 이스케이프가 생겨났다(C23의 삼중자 제거가 반가운 이유의 하나다).

문 이 함정들에서 프로그래머가 챙길 원칙을 하나로 줄이면 무엇인가?

답 **검사와 해석 사이에 표현이 바뀌지 않게 하라**는 것이다. 실무 수칙으로 펼치면 셋이다 — 첫째, **인코딩을 추측하지 말고 명시한다**(입출력 경계 에서 UTF-8로 못박기). 둘째, **정규화·디코딩을 먼저 하고 검사는 그 뒤에 한다** — 순서가 뒤집히면 오버롱·UTF-7 사고가 재현된다. 셋째, **데이터를 틀에 섞지 않는다** — 값은 언제나 값의 자리에 둔다(19장의 `printf("%s", 입력)` 관용구가 이 원칙의 가장 작은 실천이다). 세 수칙 모두 “표현과 해석은 별개”라는 이 책의 후렴구에서 나온다.

7.7 글자들의 열 — 문자열을 담는 방식들

글자 하나를 담는 법을 알았으니, 마지막 질문이 남는다. 글자들의 **열** — **문자열(string)** — 을 사물함에 담을 때, “어디부터 어디까지가 이 문자열인가”를 어떻게 아는가. 3장의 후렴구가 다시 나온다: 메모리는 덩어리의 경계를 모르고, 아는 것은 읽는 쪽의 약속이다. 그 약속의 방식이 여럿이라, 주요 갈래를 구경해 둔다.

방식 1 — 길이를 앞에 적는다(길이-버퍼). 문자열 앞에 “이 뒤로 몇 글자”라는 수를 붙여 두는 방식이다. 파스칼 계열 언어가 써서 **파스칼 문자열**이라고도 부른다. 길이를 묻는 데 셈이 필요 없고(적힌 수를 읽으면 끝), 어떤 바이트든 — 0번 문자조차 — 내용물로 담을 수 있다. 대가는 길이 칸의 크기를 미리 정해야 한다는 것 — 옛 파스칼은 길이를 1바이트에 담아서 문자열이 255자를 넘을 수 없었다(2장의 그릇 이야기가 여기서도).

방식 2 — 끝에 표지를 세운다(NUL 중단). 길이를 적지 않는 대신, 열의 끝에 특별한 문자 — 4장에서 얼굴을 익힌 값 0의 NUL 문자 — 를 세워 두는 방식이다. **C가 택한 방식이 이것이다.** 미덕은 극단적 단순함이다 — 시작 지점만 알면 되고, 열의 중간 어디를 가리켜도 “거기부터 NUL까지”가 곧 문자열이다. 대가는 셋: 길이를 알려면 NUL까지 **일일이 세어야** 하고, 내용물에 NUL을 담을 수 없으며, 무엇보다 — 표지 세우기를 잊으면 읽는 쪽이 남의 땅까지 폭주한다. 이 마지막 대가가 C 역사상 가장 많은 사고를 낸 지형이고, 34장이 그 현장이다.

방식 3 — 길이와 용량을 함께 관리한다. 현대 언어들의 동적 문자열은 대개 [내용물의 위치, 지금 길이, 그릇의 용량] 세 값을 한 묶음으로 관리한다 — 길이 조회는 즉시, 덧붙이기는 용량 안에서 자유, 모자라면 더 큰 그릇으로 이사한다. C++의 `std::string`, Rust의 `String`이 이 구조이고, 뒤에서 만날 `proven`의 문자열 처리도 길이를 명시적으로 다루는 이 계열의 사고방식이다(35장).

● 15글자의 마법 — MS STL의 작은 문자열 최적화

방식 3의 실제 구현에는 영리한 겹장치가 있다. 이웃 언어 C++의 마이크로소프트 표준 라이브러리 구현(MS STL)의 `std::string`은, 짧은 문자열 — 구체적으로 **15글자 이하**(15바이트 + 끝 표지) — 이면 별도의 큰 창고(동적 메모리)를 빌리지 않고 **문자열 객체 자신의 몸 안에 그냥 담아 버린다**. “작은 문자열 최적화”(SSO, small string optimization)라 불리는 기법이다. 왜 15인가 — 객체 안에 어차피 있어야 할 위치·길이·용량 칸들의 자리를 짧은 문자열일 때 내용물 버퍼로 재활용하면 딱 그 크

기가 나오기 때문이다. 효과는 9장의 지식으로 설명된다: 실무 문자열의 대다수는 짧은데, 그것들이 창고 왕복(할당) 없이 객체와 한 몸으로 붙어 다니니 캐시 라인 위에서 함께 움직인다. 다른 구현들(GCC·Clang의 표준 라이브러리)도 수치만 다를 뿐 같은 기법을 쓴다 — “표현 방식의 선택이 곧 속도”라는 것을 보여 주는 현대의 교과서적 사례다.

이 밖에도 지형은 넓다 — 큰 문서를 조각의 나무로 관리하는 편집기용 표현(로프), 원본을 복사하지 않고 [시작, 길이]만 가리키는 뷰(view) 방식 등, 쓰임마다 알맞은 표현이 따로 있다. 기억할 것은 하나다: **문자열이 메모리에 놓이는 모양은 하나가 아니고, 언어와 라이브러리의 선택이며, 그 선택이 성능과 안전의 성격을 정한다.** C의 선택(NUL 종단)이 어떤 성격이고 어떻게 다뤄야 하는지는 34장에서 정면으로 만난다.

문 이 어지러운 역사를 입문자가 왜 지금 알아야 하는가? UTF-8만 쓰면 되는 것 아닌가?

답 새로 만드는 것은 UTF-8만 쓰면 된다 — 그것이 오늘의 관행이고 이 책의 전제다. 그러나 두 가지 이유로 역사가 필요하다. 첫째, 세상의 파일과 시스템에는 옛 인코딩이 여전히 흐르고 있어서, 글자 깨짐을 만났을 때 “표를 잘못 짚었구나”를 진단할 수 있어야 한다. 둘째, C 자신이 이 역사의 산물이라 — `char`라는 이름, NUL 종단, 삼중자의 잔해 — 역사를 알면 C의 기묘한 구석들이 전부 사연 있는 상처로 읽힌다. 외울 것은 없다. “문자는 번호이고, 표가 여러 개였다가 유니코드로 통일됐고, 담은 방식은 UTF-8이 이겼다” — 이 세 문장이면 충분하다.

다음 장은 표현의 사다리 마지막 단이다. 글자를 담을 줄 알게 됐으니, 이제 글자가 흐르는 이야기다 — 컴퓨터의 입출력이 왜 “문자가 한 줄씩 흘러가는 것”으로 생겼는지, 그 답이 종이 카드와 타자기의 시대에 있다.

8 스트림의 기원 ↔ 펀치카드, 라인프린터, 프린터 터미널

이 장이 끝나면

컴퓨터의 입출력이 왜 “문자가 한 줄씩 흐르는 것”의 모습인지 알게 된다. 답은 종이의 시대에 있다 — 펀치카드, 라인프린터, 그리고 타자기를 닮은 프린터 터미널. `\n`이라는 두 글자와 `tty`라는 이상한 이름의 유래도 여기서 풀린다. 이 장이 끝나면 곧 만날 첫 프로그램의 `printf`가 어디에 대고 말하는 것인지 미리 이해하게 된다.

돌아보기 7장에서 제어 문자 — “종이를 한 줄 올려라” 같은, 장치에 내리는 지시 — 가 문자표의 한 구석에 산다고 했다. 글자들의 표에 왜 장치 조작 지시가 끼어 있는가?

답 그 표가 만들어진 시대에는 텍스트가 곧 **장치의 움직임**이었기 때문이다. 글자는 화면에 그려지는 것이 아니라 종이에 물리적으로 찍히는 것이었고, “다음 줄로”는 실제로 기계 부품이 움직이는 동작이었다. 그 시대의 장치들을 구경하면, 오늘의 입출력이 왜 이런 모양인지 전부 풀린다.

8.1 펀치카드 — 한 장이 한 줄이다

초기 컴퓨팅의 입력은 **펀치카드**였다. 뿔뿔한 종이 카드에 구멍을 뚫어 글자를 기록한다 — 구멍의 조합이 글자 하나이고, 카드 한 장에 글자가 **80칸** 들어간다. 프로그램 한 줄을 카드 한 장에 뚫고, 프로그램 전체는 카드 뭉치가 된다. 뭉치를 기계에 먹이면 카드가 한 장씩 빨려 들어가며 읽힌다.

여기서 두 가지 유산이 태어났다. 첫째, “**한 장 = 한 줄**”이라는 감각. 텍스트는 줄들의 열이라는, 지금은 공기처럼 당연한 관념이 물리적 실물에서 왔다. 둘째, **80이라는 수**. 카드의 80칸은 이후 터미널 화면의 80컬럼으로, “코드 한 줄은 80자 안쪽으로”라는 오늘날의 코딩 관행으로 반세기를 건너 살아남았다.

문 카드 뭉치를 떨어뜨리면 어떻게 되는가?

답 대참사다 — 순서가 섞인 카드 수백 장이 곧 프로그램이 뒤섞인 것이다. 그래서 카드 구석에 일련 번호를 뚫어 두고 기계로 다시 정렬하는 요령이 있었다. 카드 한 귀퉁이를 사선으로 잘라 뒤집힌 카드를 눈으로 찾게 한 것도 같은 지혜다. 우스개 같지만, “데이터에 순서 번호를 붙여 두면 섞여도 복구할 수 있다”는 착상은 지금도 통신과 데이터베이스 설계의 기본기다.

8.2 라인프린터 — 줄 단위로 찍는 출력

출력 쪽의 실물은 **라인프린터**였다. 이름 그대로 **한 번에 한 줄**을 통째로 종이에 찍는 인쇄기다. 계산 결과는 화면이 아니라 연속 용지에 줄줄이 찍혀 나왔다. 입력은 카드 한 장씩(= 한 줄씩), 출력은 프린터 한 줄씩 — 컴퓨터의 입출력은 태생부터 **줄 단위의 흐름**이었다.

8.3 프린터 터미널 — 종이 위의 대화, 그리고 tty

시분할의 시대가 오며(여러 사람이 한 컴퓨터를 동시에 쓰는 방식) 사람과 컴퓨터가 **대화**할 장치가 필요해졌다. 그 자리에 들어온 것이 전신 타자기 — **텔레타이프**(Teletype)다. 타자기처럼 생긴 이 기계로 글쇠를 치면 그 글자가 컴퓨터로 가고, 컴퓨터의 응답이 같은 기계의 종이에 찍힌다. 화면은 없다. 대화 전체가 종이 두루마리에 인쇄된다.

Unix가 바로 이 장치들 곁에서 태어났고(2장), 단말 장치를 가리키는 이름을 텔레타이프의 약자에서 따 왔다 — **tty**. 오늘날의 터미널 창 밑바닥에 여전히 살아 있는 이름이다.

타자기의 몸에서 온 유산이 또 있다. 타자기에서 줄을 바꾸는 동작은 사실 두 개다 — 활자 뭉치(캐리지)를 왼쪽 끝으로 **되돌리는** 동작과, 종이를 한 줄 **올리는** 동작. 문자표는 이 두 동작에 각각 제어 문자를

배정했다. 캐리지 리턴(CR, C 표기 \r)과 라인 피드(LF, \n)다. 줄 하나를 바꾸는 데 지시 두 개가 필요했던 것은 기계 부품이 실제로 둘 움직였기 때문이다.

● CRLF — 타자기가 남긴 백 년짜리 숙제

물리적 이유가 사라진 뒤에도 두 문자는 남았고, 세계는 줄바꿈 표기를 통일하지 못했다. Windows 계열은 두 동작을 다 적는 \r\n(CRLF)을, Unix 계열은 \n 하나를 줄의 끝으로 삼았다. 그 결과가 오늘날의 일상적 사고들이다 — 한쪽에서 만든 텍스트 파일을 다른 쪽에서 열면 줄 끝에 유령 문자가 붙거나 온 파일이 한 줄로 보이고, 버전 관리 도구(git)는 “CRLF를 LF로 바꿀 것”이라는 경고를 낸다. 타자기 부품의 움직임이, 부품이 사라진 지 반세기 뒤의 협업 현장에서 아직 경고문을 띄우고 있는 셈이다.

그리고 이 두 문자는 보안에도 얼굴을 내민다. 인터넷 규약들(HTTP, 전자우편)은 **줄바꿈으로 항목을 나누는** 텍스트 규약이라서 — 사용자가 준 값에 CR·LF가 섞여 들어가면 공격자가 **새 항목을 만들어 끼워 넣을 수 있다**. 응답 헤더에 남의 값을 그대로 넣었다가 헤더가 통째로 조작되거나(HTTP 응답 분할), 로그 한 줄에 남의 값을 넣었다가 가짜 로그 줄이 위조되는 식이다(로그 위조). 7장의 후렴구가 여기서도 반복된다 — **데이터를 틀에 섞지 말 것**, 그리고 경계에서 제어 문자를 걸러 낼 것.

8.4 화면 터미널, 그리고 스트림이라는 유산

이윽고 종이가 화면으로 바뀌었다 — 그러나 새 화면 장치들은 프린터 터미널을 **홍내 내는** 방식으로만 들어섰다. 글자가 아래에서 위로 밀려 올라가는 오늘의 터미널 창은, 종이가 위로 밀려 올라가던 텔레타이프의 유리판 버전이다. 겉모습만 바뀌고 대화의 문법은 그대로 남은 것이다.

이 물리적 계보 전체가 하나의 관념으로 응축됐다. 입출력이란 **문자가 한 줄씩, 순서대로 흘러가는** 띠라는 관념 — 이것이 **스트림(stream)**이다. 카드가 한 장씩 빨려 들어가듯 입력이 흘러 들어오고, 프린터가 한 줄씩 찍듯 출력이 흘러 나간다. C는 이 관념을 언어의 입출력 모델로 그대로 받아들였다. 프로그램마다 기본으로 주어지는 **표준 입력**과 **표준 출력**이라는 두 흐름이 그것이고, 이 책의 다음 부에서 만날 printf는 바로 표준 출력이라는 띠에 글자를 흘려보내는 함수다.

8.5 띠를 갈아 끼우다 — 리다이렉션과 파이프라인

스트림이라는 관념에는 뜻밖의 힘이 하나 숨어 있었다. 프로그램이 자기 출력의 **끝이 어디인지 모른다**면 — 그 끝을 바깥에서 갈아 끼울 수 있다는 뜻이다.

Unix가 이 착상을 제도로 만들었다. 프로그램은 표준 입력·표준 출력이라는 띠에 대고 읽고 쓸 뿐이고, 그 띠의 반대편을 무엇에 연결할지는 **프로그램을 실행하는 쪽**이 정한다. 터미널에서 명령 뒤에 > 파일.txt라고 적으면 출력의 띠가 화면 대신 파일로 이어지고(**리다이렉션**), < 입력.txt라고 적으면 입력의 띠가 키보드 대신 파일에서 흘러온다. 프로그램의 코드는 한 글자도 바뀌지 않는다.

여기서 한 걸음 더 나간 것이 **파이프라인**이다. 어떤 프로그램의 출력 띠를 다른 프로그램의 입력 띠에 곧장 잇는 것 — 터미널의 | 기호가 그 연결이다.

\$ 목록내기 | 걸러내기 | 세기

세 프로그램이 각각 자기 띠만 보고 일하는데, 셋을 이어 붙이면 하나의 새 도구가 된다. 이 연결을 발명한 더글러스 매클로이(Doug McIlroy)와 Unix 사람들이 남긴 원칙이 이후 소프트웨어 설계의 격언이 됐다 — **“한 가지 일을 잘하는 작은 프로그램을 만들고, 텍스트 스트림으로 협력하게 하라.”** 프로그램들을 부품으로 쓰는 조립의 문법이, 겨우 이 “띠를 갈아 끼우는” 단순함에서 나온 것이다.

이 자유의 대가로 프로그래머가 지켜야 할 것도 생긴다. 내 프로그램의 출력이 언제나 사람의 화면으로 간다고 가정하면 안 된다는 것 — 장식용 글자나 진행 표시가 뒤 프로그램의 입력을 오염시킬 수 있다. 그

래서 Unix 전통은 통로를 하나 더 둔다: 진짜 결과는 표준 출력으로, 사람에게 하는 말(경고·오류)은 **표준 오류**라는 별도의 띠로 보내는 것이다. 세 번째 띠의 존재 이유가 이것이다.

● 이 책의 실행 결과도 리다이렉션의 산물이다

추상적인 이야기가 아니다 — 지금 읽고 있는 이 책이 그 증거다. 이 책의 모든 코드 시연은 기계가 예제를 실행해 얻은 진짜 출력인데, 그 출력을 사람이 받아 적은 것이 아니라 **출력의 띠를 파일로 돌려** (리다이렉션) 조판 도구가 그 파일을 읽어 인쇄한 것이다. 입력이 필요한 예제도 마찬가지로 파일을 표준 입력의 띠에 이어 준다 — 그래서 키보드를 두드리는 사람이 없어도 예제가 재현된다. 22장에서 “입력은 키보드에서 온다”는 오개념을 깰 때, 그 산 증거가 바로 이 책의 제작 과정이다.

문 요즘 프로그램은 창과 버튼으로 소통하는데, 스트림은 옛날이야기 아닌가?

답 화면의 앞쪽만 보면 그렇게 보이지만, 뒤쪽에서는 스트림이 오히려 전성기다. 서버 프로그램들의 기록(로그)은 스트림이고, 프로그램과 프로그램을 이어 붙이는 파이프라인도, 인공지능 챗봇이 답을 한 글자씩 “흘려보내는” 것도 스트림이다. 무엇보다 프로그래밍을 배우는 자리에서는 스트림이 가장 단순하고 정직한 소통로다 — 이 책의 예제가 전부 스트림 위에서 도는 이유다.

표현의 사다리가 완성됐다. 사물함에 수를 담는 법(6장), 글자를 담는 법(7장), 그리고 글자를 흘려보내는 법(8장)까지 — 이제 기계에 무엇이든 담고 내보낼 수 있다.

다음 장부터는 이 부의 마지막 등반이다. 지금까지의 단순한 기계 그림이 사실 얼마나 과감한 거짓말이었는지 — 기억이 갈라지고(9장), 실행이 겹쳐지고(10장), 컴파일러가 끼어들면서(11장), C가 왜 “추상적인 언어”일 수밖에 없는지(12장)의 이야기가 남았다.

9 기억의 분화 — 레지스터, 캐시, 다층의 사다리

이 장이 끝나면

고백으로 시작하는 장이다 — 2장의 세 부품 그림이 사실 얼마나 과감한 단순화였는지 밝히고, 그림을 수리한다. CPU 안에 원래부터 있던 레지스터, 바깥의 큰 메모리, 그리고 둘 사이의 속도 격차가 벌어지며 그 틈에 끼어든 다층의 캐시까지 — 기억이 하나가 아니라 사다리라는 것을 알게 된다.

돌아보기 2장에서 “CPU가 메모리에서 다음 할 일을 가져와 그대로 한다”고 했다. 그러면 계산 자체도 메모리 위에서 하는가 — 100번 칸과 104번 칸을 더한 결과가 바로 108번 칸으로 가는가?

답 아니다 — 그리고 이 질문이 이 장의 문을 연다. CPU는 메모리 위에서 직접 계산하지 않는다. 계산은 CPU 안의 별도 공간에서 일어난다. 2장의 그림에는 그 공간이 아예 그려져 있지 않았다. 이제 정직해질 시간이다.

9.1 고백 — 그 그림은 과하게 단순했다

2장의 CPU·메모리·클록 모델은 배움의 첫 계단으로는 훌륭하지만, 오늘의 컴퓨터에 대해서는 **과하게 간소화된** 그림이다. 어느 정도인가 하면 — 그 그림대로라면 오늘의 CPU는 시간 대부분을 **멍하니 기다리며** 보내야 한다. 클록은 반세기 동안 수십만 배 빨라졌는데 메모리는 그만큼 빨라지지 못했기 때문이다. 그런데 실제 컴퓨터는 멍하니 있지 않다. 그 사이에 무슨 장치가 들어섰는지가 이 장과 다음 장의 이야기다. 먼저, 처음부터 그림에서 빠져 있던 것부터 채워 넣는다.

9.2 레지스터 — 원래부터 있던, CPU의 손안

CPU 안에는 **레지스터(register)**라는 작은 기억 장소들이 있다. 처음부터 있었다 — 2장의 그림이 생략했을 뿐이다. 개수는 겨우 수십 개, 크기는 워드 하나씩(3장). 대신 속도는 **즉시**다. 클록 박자 안에서 바로 읽고 쓴다. 손안에 쥐 동전이라 해도 좋다 — 주머니(메모리)보다 가깝고, 꺼내는 데 시간이 걸리지 않는다.

중요한 것은 이것이다. **모든 계산은 레지스터 위에서 일어난다**. 두 수를 더하려면 먼저 메모리에서 레지스터로 가져오고(load), 레지스터끼리 더하고, 결과를 메모리로 되돌린다(store). “100번 칸 + 104번 칸”처럼 보이는 계산의 실제 동선은 언제나 [메모리 → 레지스터 → 계산 → 메모리]다.

문 그렇게 중요한 것이 왜 겨우 수십 개뿐인가? 잔뜩 만들면 안 되나?

답 빠름과 많음은 맞바꿈 관계이기 때문이다. 레지스터가 즉시인 이유는 CPU의 계산 회로 바로 곁에, 가장 비싼 방식으로 만들어져 있어서다. 수를 늘리면 거리가 멀어지고 고르는 시간이 붙어서 “즉시”가 무너진다. 그래서 설계는 “아주 빠른 것을 조금”으로 귀결됐고 — 이 맞바꿈이 이 장 전체를 지배하는 원리다: **빠른 기억일수록 작고, 큰 기억일수록 느리다**.

C의 표면에도 이 지층의 흔적이 있다. C에는 register라는 키워드가 있는데 — “이 변수를 되도록 레지스터에 두라”는 옛 시절의 부탁이다. 오늘날에는 컴파일러가 사람보다 배치를 훨씬 잘해서 이 부탁은 장식이 됐지만, 변수를 누가 레지스터에 올리고 언제 메모리로 내리는가라는 문제 자체는 여전히 살아 있고, 11장(컴파일러 최적화)의 핵심 소재가 된다.

9.3 벌어지는 격차 — 레지스터와 메모리 사이

이제 두 기억을 나란히 세워 보면 그림이 선명해진다.

- 레지스터: 수십 개 × 워드 크기. 즉시.

- **메모리(DRAM):** 수백억 칸. 오늘의 CPU 기준으로 수백 클럭 박자를 기다려야 답이 온다.

처음부터 이랬던 것은 아니다. C가 태어난 시절(2장)에는 CPU와 메모리의 걸음이 얼추 맞았다. 그런데 이후 수십 년, CPU의 클럭은 폭주하듯 빨라졌고 메모리는 커지는 쪽으로 진화하느라 속도가 그만큼 따라오지 못했다. 격차는 해마다 벌어져 수백 배가 됐다 — 손안의 동전과, 왕복 한 시간짜리 창고의 관계가 된 것이다. 계산은 즉시인데 재료 조달이 수백 박자라면, 기계는 일하는 시간보다 기다리는 시간이 길어진다.

△ “메모리 접근은 어디를 읽든 같은 비용이다”

단순한 그림이 심어 주는 자연스러운 오개념이고, 프로그램의 속도를 이해하려 할 때 가장 먼저 부수어야 할 착각이다. 실제로는 방금 만졌던 근처를 다시 만지는 것과 처음 가 보는 먼 곳을 만지는 것의 비용이 수십-수백 배 다르다 — 왜 그런지가 바로 다음 절의 캐시다. 같은 일을 하는 두 프로그램이 메모리를 어떤 순서로 만지느냐만으로 몇 배씩 차이 나는 이유가 여기에 있다.

9.4 캐시 — 격차의 틈에 끼어든 중간층

수백 배 격차의 해법으로 공학이 내놓은 것이 캐시(cache)다. 착상은 도서관에서 일하는 사람의 요령 그대로다. 서고(메모리)까지 왕복이 느리다면 — **방금 가져온 책을 책상 위에 얼마간 두라.** 다시 찾을 때 책상에 있으면(적중) 즉시고, 없으면(미스) 그때만 서고에 다녀온다.

이것이 통하는 이유는 프로그램의 버릇 때문이다. 프로그램은 메모리를 아무 데나 무작위로 만지지 않는다 — **방금 만진 곳을 곧 다시 만지고** (시간 지역성, temporal locality), **만진 곳의 이웃을 이어서 만진다**(공간 지역성, spatial locality). 루프가 같은 변수를 반복해 만지고, 배열을 앞에서부터 차례로 훑는 것을 떠올리면 된다. 이 버릇 덕에, 작은 책상 하나로도 서고 왕복의 대부분을 없앨 수 있다.

공간 지역성을 활용하는 요령이 하나 더 있다. 캐시는 메모리에서 물건을 **한 칸씩 가져오지 않는다**. 요청한 칸의 이웃까지 묶어 — 오늘날 대개 64바이트, 사물함 예순네 칸을 상자째 — 한 번에 나른다. 이 운반 단위가 **캐시 라인(cache line)**이다. 배열을 차례로 훑을 때 첫 칸에서 한 번만 서고에 다녀오면 다음 예순네 칸이 공짜인 이유이고, 거꾸로 멀리멀리 건너뛰며 만지는 프로그램은 상자를 매번 새로 나르느라 느려지는 이유다. 이 “상자” 개념은 다음 장에서 뜻밖의 사고(false sharing)의 주인공으로 다시 나온다.

9.5 캐시의 분화 — 다층의 사다리

캐시에도 레지스터와 같은 맛바꿈이 적용된다 — 빠르려면 작아야 하고, 크려면 느려진다. 그래서 캐시는 하나가 아니라 **층**으로 분화했다. CPU 바로 곁의 작고 가장 빠른 L1, 그 뒤의 중간 L2, 여러 코어가 함께 쓰는 크고 상대적으로 느린 L3 — 그리고 그 너머의 메모리. 대략의 감각만 숫자로 적으면 이렇다(기계마다 다르다).

층	크기의 규모	대략의 대기	비유
레지스터	수십 개	0 (즉시)	손안의 동전
L1 캐시	수십 KiB	박자 몇 개	책상 위
L2 캐시	수백 KiB-MiB	십수 박자	책장
L3 캐시	수-수십 MiB	수십 박자	같은 층 서가
메모리(DRAM)	수십 GiB	수백 박자	창고

읽는 법은 하나다 — **아래로 한 층 내려갈 때마다 크게 느려진다.** 기억은 “한 덩어리”가 아니라 이 사다리 전체이고, 프로그램의 속도는 상당 부분 “필요한 것이 사다리의 몇 층에서 발견되는가”로 정해진다.

● 같은 일, 다섯 배의 시간 — 2차원 표 훑기

큰 2차원 표(예: 수천×수천의 수)를 전부 더하는 단순한 일을 생각하면 된다. 행을 따라 — 메모리에 놓인 순서 그대로 — 훑으면 캐시 라인 상자가 매번 알뜰하게 쓰인다. 열을 따라 — 매번 멀리 건너뛰며 — 훑으면 상자를 나르고 버리기를 반복한다. 하는 일(덧셈의 횟수)은 완전히 같은데, 실측하면 몇 배에서 수십 배까지 차이가 난다. 코드 두 줄의 순서 차이가 낳는 이 격차는, 기억이 사다리라는 사실을 모르면 설명할 길이 없다 — 그리고 이런 실측을 이 책의 예제로 직접 보게 된다(28장).

문 프로그래머가 캐시를 직접 조종할 수는 없는가 — “이건 L1에 넣어 뒀” 같은 지시는?

답 일반적으로는 없다. 캐시는 하드웨어가 자동으로 운용하고, C 언어에도 캐시를 직접 만지는 표준 문법은 없다. 프로그래머가 하는 일은 조종이 아니라 **협조**다 — 데이터를 이웃끼리 모아 두고, 만질 때 순서대로 만지는 것. 기억이 사다리임을 아는 사람의 코드는 자연스럽게 그렇게 생긴다. 그것으로 충분히, 방금 본 몇 배의 차이가 난다.

그림 수리의 전반부가 끝났다 — 기억은 하나가 아니라 레지스터에서 창고까지의 사다리다. 그러나 아직 절반이다. CPU는 기다림을 줄이는 것으로 만족하지 않고, **실행 자체를 겹치는** 쪽으로도 진화했다 — 조립 라인처럼 명령들을 포개고, 갈림길을 미리 짐작하고, 끝내 일꾼(코어)의 수를 늘렸다. 그 이야기, 그리고 그 요동의 한복판에서 C가 처음으로 계약서(C89)를 쓰게 된 사연이 다음 장이다.

10 속도의 기계장치 ↔ 표준의 탄생

이 장이 끝나면

기다림을 줄인 것이 캐시였다면, 이번에는 실행 자체를 겹치는 장치들이다 — 파이프라인, 분기 예측, 그리고 클럭의 한계가 부른 멀티코어까지. 코어가 여럿이 되며 생기는 기묘한 사고(false sharing)도 구경한다. 나란히, 같은 요동의 시대에 C가 처음으로 계약서(C89)를 쓰게 된 사연을 본다.

돌아보기 2장의 문답에서 “걸음 수(클럭)가 전부는 아니다 — 한 걸음에 여러 일을 하는 요령이 현대 CPU의 진짜 무기”라고 예고했다. 한 걸음에 여러 일이란 어떻게 가능한가? 한 명령이 끝나야 다음 명령을 하는 것 아닌가?

답 “끝나야 시작한다”를 버리면 된다. 명령 하나의 처리를 여러 단계로 쪼개고, 단계가 다른 여러 명령을 **동시에** 진행하는 것이다 — 한 명령이 계산되는 동안 다음 명령을 가져오고, 그다음 명령을 해독한다. 이 장의 첫 장치, 파이프라인이다.

10.1 파이프라인 — 조립 라인 위의 명령들

세탁소를 떠올리면 된다. 세탁 30분, 건조 30분, 다림질 30분이라 할 때, 한 벌을 끝내고 다음 벌을 시작하면 세 벌에 270분이 걸린다. 그러나 첫 벌이 건조기로 넘어가는 순간 둘째 벌을 세탁기에 넣으면 — 장비 세 대가 **서로 다른 벌을 동시에** 처리하며, 흐름이 차오른 뒤에는 30분마다 한 벌씩 완성되어 나온다.

CPU가 명령을 처리하는 것도 똑같이 단계로 나뉜다 — 가져오고(fetch), 해독하고(decode), 계산하고(execute), 결과를 쓴다(write). 이 단계들을 조립 라인으로 만든 것이 **파이프라인(pipeline)**이다. 명령 하나하나가 빨라지는 것이 아니다 — **단위 시간에 완성되는 명령의 수가** 늘다. 현대 CPU의 파이프라인은 열 단계가 넘고, 여기에 라인을 여러 개 두어 박자당 여러 명령을 완성하는 데까지 나아갔다. 2장의 “한 박자에 한 걸음”은 이렇게 무너진다 — 걸보기 순서는 지키되, 속은 겹쳐 돌아간다.

같은 시대의 요령 하나를 여기 적어 둔다. 루프 한 바퀴마다 드는 관리 비용(횃수 세기, 갈림길 판단)을 아끼려고 **한 바퀴에 여러 개씩** 일을 처리하는 기법이 있다 — 루프 언롤링(loop unrolling)이라 한다. 1983년, Tom Duff라는 프로그래머는 이 기법을 C의 두 문법을 기괴하게 겹쳐 쓰는 방법으로 극한까지 밀어붙였고, 그 코드는 **Duff's device**라는 이름의 전설이 됐다. 코드 자체는 아직 보여 줄 수 없다 — 이 책이 그 문법들을 아직 소개하지 않았기 때문이다. 재회를 예약해 둔다(28장). 지금은 한 문장이면 된다: 그 시절에는 사람이 이런 곡예로 기계의 박자를 짜냈다는 것, 그리고 오늘날 그 곡예는 컴파일러의 일이 됐다는 것(11장).

10.2 분기 예측 — 갈림길을 미리 짐작하다

파이프라인에는 아픈 곳이 있다. **갈림길**이다. “결과가 0이면 저쪽으로 건너뛰어라” 같은 명령(분기)을 만나면, 어느 쪽 길의 명령을 라인에 실어야 할지 결과가 나오기 전에는 모른다. 라인을 세우고 기다리면 겹침의 이득이 사라진다.

그래서 CPU는 **짐작**한다. 단골 손님's 주문을 미리 만들어 두는 카페처럼, 지금까지의 이력으로 “이 갈림길은 대개 이쪽”을 학습해 그쪽 명령을 미리 라인에 싣는다 — **분기 예측(branch prediction)**이다. 맞으면 공짜고, 틀리면 미리 만든 주문을 전부 버리고 라인을 비운 뒤 다시 시작한다. 이 벌금이 파이프라인 깊이만큼 크기 때문에, 현대 CPU는 예측기에 진지한 회로를 투자하고 적중률은 일상 코드에서 90%를 훌쩍 넘는다.

● 정렬된 배열이 더 빨랐던 사건

프로그래머들 사이에 널리 회자된 실측이 있다 — 큰 배열에서 “값이 일정 기준보다 큰 것만 더하라”는 같은 코드가, 배열을 **미리 정렬해 두면** 몇 배 빨라진다는 관찰이다. 덧셈의 횟수는 같은데 왜인가. 답은 분기 예측이다. 뒤섞인 배열에서는 “크다/작다” 갈림길이 매번 무작위라 예측이 반타작으로 틀리고, 그때마다 라인을 비우는 벌금을 문다. 정렬된 배열에서는 갈림길의 답이 한동안 같은 쪽으로 이어져 예측이 거의 다 맞는다. 눈에 보이지 않는 짐작 회로 하나가, 같은 코드의 속도를 몇 배 가른 것이다 (27장에서 이 감각을 다시 쓴다 — “if 는 공짜가 아니다”).

10.3 클록의 한계, 그리고 멀티코어

이 모든 겹침에도 불구하고, 2000년대 중반에 벽이 왔다. 클록을 더 올리면 전력과 발열이 감당할 수 없이 치솟는 물리적 한계였다. 수십 년 이어지던 “내년에는 더 빠른 클록”이 멈춘 것이다.

산업의 응답은 방향 전환이었다 — 한 일꾼을 더 빠르게 만드는 대신, **일꾼의 수를 늘린다**. CPU 하나에 독립적으로 명령을 실행하는 엔진 — **코어(core)** — 를 두 개, 네 개, 수십 개씩 넣기 시작했다. 오늘날의 스마트폰조차 코어 여덟 개쯤은 예사다. 공짜 점심은 끝났다 — 프로그램이 저절로 빨라지던 시대가 끝나고, 여러 일꾼에게 일을 **나누어 주는** 문제가 프로그래머의 몫이 된 것이다(이 책의 범위 밖이지만, 지형은 알아 둔다).

그리고 일꾼이 여럿이 되자, 9장의 캐시가 뜻밖의 사고 현장이 된다.

문 코어마다 자기 캐시(책상)가 있다면, 두 코어가 같은 데이터를 만질 때는 무슨 일이 일어나는가?

답 하드웨어가 뒤에서 책상들 사이의 일관성을 맞춰 준다 — 한 코어가 값을 고치면 다른 코어의 책상에 놓인 사본을 무효로 만드는 식이다. 정확성은 이렇게 지켜지는데, 문제는 이 정리 정돈이 **공짜가 아니라는 것**, 그리고 그 단위가 변수 하나가 아니라 9장의 **캐시 라인 상자**라는 것이다. 여기서 기묘한 사고가 태어난다.

False sharing(거짓 공유)이라 불리는 사고다. 두 코어가 **서로 다른** 변수를 만지는데 — 공유한 것이 하나도 없는데 — 하필 두 변수가 **같은 캐시 라인 상자**에 나란히 놓여 있으면, 하드웨어의 눈에는 같은 상자를 두 코어가 다투는 것으로 보인다. 한쪽이 쓸 때마다 다른 쪽 책상의 상자가 무효가 되고, 상자가 두 책상 사이를 핑퐁처럼 오가며, 두 코어 모두 수십 배 느려진다. 코드만 보면 아무것도 공유하지 않으므로 원인이 보이지 않는다 — 기억이 상자 단위로 움직인다는 9장의 사실을 알아야만 진단이 되는, 현대적인 사고의 대표 사례다.

10.4 같은 시대, C는 계약서를 썼다 — 표준 이전의 C와 C89

기계가 이렇게 요동치던 시기, C의 세계에도 다른 종류의 요동이 있었다.

C에는 오랫동안 **공식 표준이 없었다**. 1978년에 나온 K&R의 책 “The C Programming Language”가 **사실상의 표준** 노릇을 했다 — 언어의 규격서가 아니라, 잘 쓰인 한 권의 책이 법전을 대신한 것이다. Unix와 함께 C가 대학과 업계로 번지자 회사마다 자기 컴파일러를 만들었고, 책이 못박지 않은 구석마다 방언이 자랐다. 그 시절의 C는 지금 눈에는 험잡다 — 함수를 선언할 때 인자의 타입을 적지 않았고(원형이 없었다), 타입을 안 적은 이름은 슬그머니 int로 통했다. 어느 컴파일러에서 되던 코드가 다른 컴파일러에서 안 되는 일이 예사였고, 이식은 번번이 모험이었다.

건디다 못한 산업이 1983년 표준화 위원회(ANSI X3J11)를 꾸렸고, 6년의 격론 끝에 1989년 **C89** — 최초의 공식 C 표준 — 가 나왔다. 함수 **원형**(인자 타입까지 적는 선언, 21장)을 들여 컴파일러가 호출의 실수를 잡을 수 있게 했고, 험거운 구석들을 조였으며, 무엇보다 “프로그래머는 무엇을 약속받고, 구현은 어

디까지 자유로운가”를 처음으로 **문서**로 못박았다. 6장의 IEEE 754(1985)와 같은 몇 해의 일이다 — 하드웨어의 수도, 언어의 문법도, 벤더마다 제멋대로이던 것을 계약서로 다스리기 시작한 “계약의 시대”였다.

이 계약이라는 관념이 왜 갈수록 중요해지는가 — 기계는 이 장에서 본 것처럼 갈수록 겹치고 짐작하고 나뉘는데, 프로그래머가 그 소용돌이를 일일이 알 수는 없다. 소용돌이와 프로그래머 사이에 **또 하나의 층**이 서 있기 때문이다. 소스 코드를 받아 기계의 말로 옮기면서, 뜻만 지킨다면 무엇이든 뜯어고칠 권리를 가진 층 — 컴파일러다. 다음 장의 주인공이다.

11 컴파일러 최적화 ↔ 추상 기계

이 장이 끝나면

컴파일러가 소스 코드를 “그대로” 번역하지 않는다는 것 — 뜻만 지키면 마음껏 재배열하고 지우고 바꿔 쓴다는 것 — 을 알게 된다. 그 자유의 근거인 “관찰 가능한 결과”라는 개념과, 표준 문서 속의 가상 기계인 추상 기계를 만난다. C99·C11이 이 계약을 어떻게 정밀하게 다듬었는지도 본다.

돌아보기 10장 끝에서 컴파일러를 “뜻만 지킨다면 무엇이든 뜯어고칠 권리를 가진 층”이라 불렀다. 그런데 3장에서 C는 “기계의 정직한 별명”이라 하지 않았나 — 내가 쓴 C 코드는 기계 명령에 그대로 대응하는 것 아니었나?

답 그 시절에는 대체로 그랬고, 지금은 아니다. 오늘의 컴파일러는 직역가가 아니라 **편집자**다 — 뜻을 지키는 한에서 문장을 통째로 다시 쓴다. “그대로 대응”이라는 초기 C의 감각이 어떻게, 왜 무너졌는가가 이 장이고, 그 무너짐이야말로 C를 추상적인 언어로 만든 마지막 조각이다.

11.1 편집자의 작업실 — 컴파일러가 하는 일

몇 가지 실제 편집 사례를 보면 감각이 잡힌다. 컴파일러는 일상적으로 이런 일을 한다.

미리 계산한다. 소스에 $\text{초} = 24 * 60 * 60$ 이라 적혀 있으면, 곱셈 명령을 만들지 않고 그냥 86400을 적어 넣는다. 실행 시점에 곱할 이유가 없기 때문이다.

죽은 일을 지운다. 계산해 놓고 아무도 안 쓰는 값, 도달할 수 없는 갈림길 — 통째로 사라진다. 소스에 있는 코드가 실행 파일에는 존재하지 않을 수 있다.

순서를 바꾼다. 서로 상관없는 계산이라면, 파이프라인(10장)이 잘 차도록 앞뒤를 재배열한다. 소스의 줄 순서는 실행 순서의 약속이 아니게 된다.

메모리 왕복을 없앤다. 루프가 도는 내내 변수 하나를 매번 메모리에서 읽고 쓰는 대신, 레지스터(9장)에 올려 두고 끝날 때 한 번만 내려놓는다. 소스에는 천 번의 메모리 접근이 적혀 있어도 실제로는 두 번 일 수 있다.

이 편집들이 겹치고 쌓이면, 실행 파일 속 기계 명령과 소스 코드는 문장 대 문장으로 대응하지 않는 별개의 문서가 된다. 디버거로 최적화된 프로그램을 들여다보면 “이 줄은 최적화로 사라졌음”이라는 안내를 만나는 것이 그래서다.

문 몇대로 고쳐도 되는 근거가 무엇인가? 어디까지 고쳐도 “뜻을 지켰다”고 치는가?

답 바로 그 경계선을 긋는 것이 표준의 일이고, 경계선의 이름이 **관찰 가능한 결과**다. 대략 이렇다 — 프로그램이 바깥세상과 주고받는 것(출력한 내용, 읽은 입력, 특별히 보호된 접근)만 약속대로면, 그 결과를 만들어 내는 속사정은 전부 컴파일러의 재량이다. 중간 계산을 몇 번 하는지, 변수가 메모리에 실제로 있는지, 어떤 순서로 일하는지 — 바깥에서 관찰되지 않는 한 아무래도 좋다. 겉보기만 같으면 속은 자유 — 이것이 계약의 문장이다.

11.2 추상 기계 — 계약서 속의 가상 기계

그러면 “겉보기”의 기준은 누가 정하는가. 실제 기계일 수는 없다 — 실제 기계는 수천 종이고 저마다 다르다. 그래서 표준은 문서 안에 **가상의 기계**를 하나 정의해 두었다. C 프로그램의 의미란 “이 가상 기계 위에서 이렇게 실행되는 것”이라고 못박는, 종이 위에만 존재하는 기계 — **추상 기계**(abstract machine)다.

이제 세 층의 관계가 선명해진다.

- **프로그래머**는 추상 기계를 상대로 코드를 쓴다. 코드의 의미는 추상 기계 위에서 실행이 정한다.
- **컴파일러**는 그 의미의 관찰 가능한 결과만 지키면서, 실제 기계에 맞는 가장 빠른 코드를 자유롭게 지어낸다.
- **실제 기계**는 캐시가 몇 층이든 파이프라인이 몇 단이든(8·10장), 그 소용돌이를 겉으로 드러내지 않는다.

3장에서 “C는 기계의 정직한 별명”이라는 평판이 오늘날 절반만 맞다고 했던 것의 정답이 이것이다. C가 별명 노릇을 해 주는 대상은 이제 실제 기계가 아니라 **추상 기계**다. 실제 기계와의 대응은 컴파일러가 그때그때 지어내는, 겉보기만 보장된 번역이다.

11.3 눈에 보이는 편집 — 값을 언제 읽고 언제 쓰는가

편집자가 가장 즐겨 손대는 자리가 **메모리 왕복**이다. 9장에서 배운 대로 메모리는 느리고 레지스터는 즉시라서, 컴파일러는 값을 되도록 레지스터에 붙잡아 둔다. 두 장면으로 감을 잡는다.

장면 1 — 반복되는 읽기를 한 번으로. 이런 루프를 생각하면 된다:

```
for (int i = 0; i < n; i += 1) {  
    total += table[i] * scale; /* scale은 루프 내내 안 바뀐다 */  
}
```

소스대로라면 `scale`을 매 바퀴 메모리에서 읽어야 하지만, 컴파일러는 “이 루프 안에서 `scale`을 바꾸는 코드가 없다”고 판단해 **루프 시작 전에 한 번 읽어 레지스터에 얹어** 둔다(루프 불변 코드 이동). 천 번의 읽기가 한 번이 된다. `total`도 마찬가지로 레지스터에 두었다가 루프가 끝난 뒤 한 번만 메모리에 내려놓는다.

장면 2 — 쓰기를 미루고 순서를 바꾸기. 서로 무관한 두 변수에 값을 넣는 코드라면, 컴파일러는 파이프라인(10장)이 잘 차도록 순서를 바꾸거나 쓰기를 뒤로 미룰 수 있다. 소스의 줄 번호는 실행 시각의 약속이 아니다.

두 편집 모두 **관찰 가능한 결과**가 같으므로 계약 안이다. 문제는 — “관찰 가능”의 기준이 표준의 것이지 프로그래머의 기대가 아니라는 데서 생긴다.

● 안 도는 루프 — “메모리를 곧이곧대로” 믿었을 때

임베디드 프로그래밍의 고전적 사고를 보자. 하드웨어의 상태 레지스터가 메모리의 어떤 주소에 나타난다고 하고(4장의 그 세계다), “준비될 때까지 기다리는” 코드를 이렇게 적었다고 하자:

```
int flag = *status; /* 장치가 바꿔 주는 값 */  
while (flag == 0) { flag = *status; } /* 준비될 때까지 대기 */
```

프로그래머의 그림은 “매 바퀴 메모리를 다시 읽는다”이다. 그러나 컴파일러의 눈에는 이 루프 안에서 `*status`를 바꾸는 코드가 하나도 없다 — 그러면 장면 1의 편집이 적용된다: 값을 **한 번만** 읽어 레지스터에 얹고, 이후로는 그 레지스터만 본다. 장치가 아무리 메모리를 바꿔도 프로그램은 영원히 옛 값을 보며 돈다. 무한 루프다.

이유는 계약에 있다. 표준의 추상 기계는 “이 프로그램 밖의 무언가가 메모리를 바꿀 수 있다”는 것을 알지 못한다 — 그런 사정이 있다면 **프로그래머가 알려 주어야** 하고, 그 표기가 `volatile`이다(“이 접근은 매번 실제로 하라”는 지시). 같은 무늬가 멀티코어 세계에서도 재현된다: 다른 코어가 바꾸는 값을 평범한 변수로 폴링하면 같은 이유로 굳는다(그쪽의 정답은 `volatile`이 아니라 C11의 원자적 타입이다 — 아래 참조). 교훈은 하나다: **컴파일러는 소스에 적힌 것을 실행하지 않는다. 계약이 보장한 결과만 만든다.**

11.4 계약의 정밀화 — C99, C11

C89가 계약서의 초판이었다면(10장), 이후의 표준들은 조항을 정밀하게 다듬어 온 개정판들이다. 기계와 컴파일러가 공격적으로 진화할수록 “어디 까지 고쳐도 되는가”를 더 날카롭게 그어야 했기 때문이다. 두 번의 큰 개정만 짚는다.

C99(1999)는 편집자에게 더 많은 재량 근거를 주는 쪽으로 언어를 다듬었다 — 대표적으로, 타입이 다른 이름끼리는 같은 기억을 가리키지 않는다고 **전제해도 좋다**는 규칙(엄격한 앨리어싱, strict aliasing)이 명문화됐다. 이 전제가 있어야 편집자가 “이 둘은 서로 무관하다”고 보고 재배열·캐싱을 할 수 있다. 대가는 프로그래머 쪽 의무다 — 그 전제를 깨는 코드(같은 기억을 다른 타입의 눈으로 읽는 일)는 계약 위반이 된다(39장에서 안전한 방법과 함께 재론한다).

C11(2011)은 10장의 멀티코어 시대에 대한 응답이다 — 일꾼이 여럿일 때 “한 코어의 쓰기가 다른 코어에 언제 보이는가”를 계약 조항으로 못박은 **메모리 모델**과, 그 통신에 쓰는 원자적 타입(_Atomic)이 표준에 들어 왔다. 그 전까지 다중 코어 위의 C는 표준 바깥의 관행에 기대던, 말 그대로 계약서 없는 동업이었다.

11.5 잘못된 신호 — 엄격한 앨리어싱 위반이 부르는 유령

엄격한 앨리어싱은 이 장에서 가장 조심스러운 조항이라, 무늬를 하나 보아 둔다. 규칙을 다시 적으면 — **타입이 다른 두 포인터는 같은 기억을 가리키지 않는다고 컴파일러가 전제해도 좋다**(char 계열은 예외, 32장).

문제는 이 전제가 프로그래머의 코드에서 **신호**로 읽힌다는 데 있다. 부동소수점의 비트를 들여다보려고 이런 옛 기법을 썼다고 하자:

```
float f = 1.0f;
uint32_t bits = *(uint32_t *)&f; /* 계약 위반: float를 uint32의 눈으로 */
```

프로그래머의 의도는 “같은 4바이트를 다른 눈으로 읽자”이지만, 컴파일러가 받은 신호는 정반대다 — “**이 float*와 저 uint32_t*는 서로 다른 기억을 가리킨다(그렇게 전제해도 좋다)**”. 그러면 편집자는 안심하고 f에 대한 쓰기를 레지스터에 붙잡아 둔 채 bits 읽기를 앞당길 수 있고 — 결과는 “쓰기 전의 옛 비트”가 읽히는, 소스만 봐서는 설명되지 않는 유령이 된다. 더 고약한 점은 이 유령이 **최적화를 켜올 때만**, 그것도 컴파일러 판에 따라 나타났다 사라진다는 것이다(“디버그 빌드에서는 되는데 릴리스에서만 틀려요”의 전형적 정체다).

옳은 방법은 두 가지다 — memcpy로 바이트를 복사하거나(현대 컴파일러는 이 복사를 알아보고 공짜로 최적화한다), 다음 부에서 배울 공용체를 쓰는 것이다(39장에서 이 시연을 정식으로 한다). 요점은 기법이 아니라 관점 이다: **계약 위반은 “조금 위험한 코드”가 아니라, 컴파일러에게 사실이 아닌 것을 사실이라고 알리는 행위다**. 그래서 결과가 “조금 이상함”이 아니라 “무엇이든 가능함”이 된다 — 표준이 이런 상태에 붙인 이름이 **정의되지 않은 동작**(undefined behavior, UB)이고, 이 책의 42장이 그 정면 취급이다.

● 사라진 지우개 — 최적화가 지운 보안 코드

“겉보기만 같으면 속은 자유”가 실제로 문 사고가 있다. 보안 프로그램들은 비밀번호를 다 쓴 뒤 그 메모리를 0으로 덮어 지우는 관행이 있다 — 메모리 어딘가에 비밀이 남아 있지 않도록. 그런데 편집자의 눈에 이 지우기는 **아무도 다시 읽지 않는 쓰기**, 즉 죽은 일이다. 관찰 가능한 결과에 아무 기여가 없으므로 — 지워도 된다. 실제로 여러 컴파일러가 이 지우기를 조용히 삭제했고, 비밀번호가 메모리에 남은 채 배포된 소프트웨어들이 보안 권고의 대상이 됐다. 계약을 무시한 쪽은 컴파일러가 아니라, 계약서를 안 읽은 사람이었다 — 이후 표준과 플랫폼들은 “이 쓰기는 지우지 말라”고 명시하는 장치(memset_s 등)를 마련했다. 관찰 가능한 결과라는 조항 하나가 실무에서 얼마나 무거운지 보여주는 사건이다.

문 최적화가 이렇게 위험하면, 그냥 끄고 살면 안 되는가?

답 개발 중의 디버깅 빌드는 실제로 최적화를 낮춰서 소스와 실행의 대응을 살려 둔다. 그러나 배포에서 끄는 것은 답이 아니다 — 현대 소프트웨어의 속도는 상당 부분 편집자의 숨씨에서 나오고, 몇 배의 성능을 포기할 이유는 없다. 바른길은 하나다: 계약서를 알고, 계약 안에서 쓰는 것. “내 컴퓨터에서 돌았다”가 아니라 “추상 기계 위에서 옳다”가 기준이 되어야 한다 — 계약 밖의 코드에 무슨 일이 벌어지는지는 42장(정의되지 않은 동작)에서 정면으로 다룬다.

이제 재료가 다 모였다. 기억은 사다리이고(9장), 실행은 겹침과 짐작이며 (10장), 그 소용돌이 위에 편집자가 서서 걸보기만 보장한다(11장). 이 셋을 합치면 하나의 결론이 나온다 — C 프로그래머가 상대하는 것은 실제 기계가 아니라는 것. 다음 장에서 이 부의 논제를 완성하고, C가 결국 어느 기계의 언어인지 — CPU와 GPU의 대조까지 — 파노라마로 마무리한다.

12 그래서 C는 추상적인 언어다

이 장이 끝나면

제2부의 결론이다. 흩어져 있던 조각 — 사다리가 된 기억, 겹쳐 도는 실행, 편집자 노릇을 하는 컴파일러 — 을 모아 하나의 명제로 완성한다: C는 기계어의 별명이 아니라 추상 기계의 언어이고, 그래서 추상적으로 접근해야 한다. 요즘 더 엄격해지는 포인터 규칙(프로버넌스)이 왜 그 귀결인지 보고, 마지막으로 CPU와 GPU를 대조하며 C가 “어느 기계의” 언어인지로 부를 닫는다.

돌아보기 11장에서 “C 프로그래머가 상대하는 것은 실제 기계가 아니다”라고 했다. 그러면 2장에서 3장 내내 실제 기계 — 클록, 사물함, 워드 — 를 그렇게 들여다본 것은 헛수고였는가?

답 반대다. 그 그림들이 바로 추상 기계의 **뼈대**다. 표준의 추상 기계는 허공에서 지어낸 것이 아니라 실제 기계들의 공통 뼈대 — 바이트 열의 메모리, 주소, 순차 실행 — 를 추려 계약서에 옮겨 적은 것이다. 실제 기계를 알아야 추상 기계가 왜 그 모양인지 이해되고, 추상 기계를 알아야 실제 기계의 소용돌이(8-11장)에 흔들리지 않는다. 두 그림은 경쟁자가 아니라 한 벌이다.

12.1 논제의 완성 — 세 조각을 합치다

이 부가 쌓아 온 조각을 한 줄씩으로 줄이면 이렇다.

- 9장 — 기억은 한 덩어리가 아니라 레지스터에서 창고까지의 **사다리**이고, 내 변수가 지금 어느 층에 있는지 소스 코드에는 적혀 있지 않다.
- 10장 — 실행은 한 걸음씩이 아니라 **겹침과 짐작**이고, 명령들의 실제 진행 순서는 소스의 줄 순서와 다르다.
- 11장 — 그 위에 **편집자**가 서서, 겉보기 결과만 지키며 코드를 통째로 다시 쓴다.

그러면 소스 코드의 한 줄 한 줄이 “기계가 그대로 할 일”이라는 생각은 세 겹으로 무너진다. 남는 단단한 것은 하나 — **계약**이다. 표준이라는 계약서가 정의한 가상의 기계(추상 기계) 위에서 내 코드가 무엇을 뜻하는가. 그것만이 C 프로그램의 진짜 의미이고, 나머지 전부 — 캐시 적중, 파이프라인, 재배열, 레지스터 배치 — 는 그 의미를 빠르게 실현하는 속사정이다.

그래서 C는 추상적인 언어이고, 추상적으로 접근해야 한다. “이 코드를 기계가 어떻게 돌릴까”를 짐작하는 습관이 아니라, “이 코드가 계약 위에서 무엇을 뜻하는가”를 묻는 습관 — 이 책이 이후 모든 장에서 기르려는 것이 바로 그 습관이다.

△ “C는 하드웨어를 직접 조작하는 언어다”

절반의 역사와 절반의 오해가 섞인, C에 관한 가장 유명한 문장이다. 역사적으로는 사실에 가까웠고(2·3장), 지금도 C는 하드웨어에 **가장 가까운 곳까지 데려다주는** 언어다. 그러나 오늘의 C 코드와 하드웨어 사이에는 편집자(컴파일러)와 계약서(표준)가 서 있고, 순진한 “직접 조작” 감각으로 쓴 코드 — 이를테면 계약 밖의 지름길 — 는 편집자의 재량 앞에서 부서진다(11장의 사라진 지우개, 그리고 42장). 정확한 문장은 이렇다: C는 하드웨어를 직접 조작하는 언어가 아니라, **하드웨어를 추상 기계라는 계약으로 만나게 해 주는 언어**다.

12.2 요즘의 흐름 — 포인터에 붙는 출처

이 관점이 요즘 어디로 가고 있는지 보여 주는 대표적인 흐름이 포인터 규칙의 정밀화다.

3장에서 “주소도 수”라고 했다. 순진한 그림에서 포인터는 그냥 번호다 — 같은 번호면 같은 곳이고, 번호만 맞으면 어떻게 얻었든 그곳을 만질 수 있어야 한다. 그러나 편집자의 세계에서 이 그림은 유지될 수

없다. 컴파일러의 최적화는 “이 포인터는 어디에서 왔는가”라는 계보 추적 위에 서 있기 때문이다 — 이 변수에서 나온 포인터는 저 변수를 건드릴 수 없다는 전제(11장의 엄격한 앨리어싱도 그 한 갈래)가 있어야 재배열과 캐싱이 가능하다.

그래서 현대의 표준 논의는 포인터를 “번호”가 아니라 “번호 + 출처 딱지”로 다듬어 가고 있다. 같은 비트 패턴이라도 어디에서 유래했는가에 따라 쓸 수 있는 곳이 다르다는 것 — 이 출처 개념을 **프로버넌스**(provenance)라 부르고, C 표준 위원회는 이를 명문화하는 작업을 진행해 왔다. 방향은 일관된다: 포인터에 대한 규칙이 점점 더 명시적이고 엄격해지는 쪽, 즉 계약서의 조항이 더 정밀해지는 쪽이다. 입문 단계에서 조항의 세부까지 알 필요는 없다 — “포인터는 그냥 수가 아니다, 계보가 있다”는 감각만 여기서 심고, 실무 규칙은 32장에서, 계약 위반의 결과는 42장에서 만난다.

문 4장의 태그 포인터 — 주소의 빈 비트에 딱지를 끼우는 재주 — 도 이 규칙과 부딪히는가?

답 부딪힐 수 있는 지점이 정확히 그곳이다. 포인터를 수처럼 주물러 태그를 끼우고 떼는 일은 “번호 = 포인터” 그림에서는 자명했지만, 출처 딱지의 세계에서는 **계보를 잃지 않는 절차**를 지켜야 하는 일이 된다(정수로 변환해 다루는 합법적 경로가 마련되어 있다). 저수준 기법일수록 계약서를 정독해야 하는 시대 — 그것이 모던 C다.

12.3 마무리 파노라마 — C는 어느 기계의 언어인가

부를 닫기 전에, 시야를 한 번 넓힌다. “기계”라고 불러 온 것이 사실 한 종류가 아니기 때문이다.

같은 트랜지스터로 정반대의 기계를 지을 수 있다. 하나는 **한 가지 일을 최대한 빨리 끝내는** 기계 — 깊은 파이프라인, 큰 캐시, 정교한 분기 예측(8·10장)이 전부 이것, 즉 **지연시간**(latency)을 줄이는 장치다. 우리가 여태 그려 온 CPU다. 다른 하나는 **같은 일을 산더미에 한꺼번에 하는** 기계 — 예측도 깊은 파이프라인도 덜어 내고, 그 자리에 단순한 계산기를 수천 개 심는다. **처리량**(throughput)의 기계, GPU다. 화면의 수백만 픽셀에 같은 계산을 뿌리려고 태어났고, 그 체질이 행렬 계산 — 인공지능의 심장 — 과 맞아 떨어지며 시대의 주인공이 됐다.

식당으로 비유하면, CPU는 어떤 주문이든 즉시 쳐내는 달인 셰프 한두 명이고, GPU는 같은 메뉴 만 그릇을 동시에 만드는 급식 라인이다. 어느 쪽이 낫냐는 물음은 성립하지 않는다 — 주문이 다르다.

C와의 관계가 이 장의 마지막 문장이 된다. GPU의 세계조차 그 프로그래밍 언어(CUDA, OpenCL)는 C의 방언으로 만들어졌다 — 시스템 세계의 공용어가 C라는 1장의 이야기가 여기서도 반복된다. 그러나 정확히 말하면, **C의 추상 기계는 CPU의 상(像)이다**. 순차 실행, 하나의 메모리, 주소 — 계약서의 가상 기계는 CPU의 뼈대를 본떴고, GPU라는 다른 체질은 그 계약서의 방언과 확장으로 다룬다. C를 배운다는 것은 컴퓨팅의 여러 기계 가운데 가장 근본이 되는 한 상 — 폰 노이만 이래의 순차 기계 — 을 그 언어와 함께 배우는 일이다.

12.4 제2부를 닫으며

바닥 공사가 끝났다. 비트에서 시작해(2장) 사물함 복도를 걷고(3·4장), 정수와 소수와 글자와 흐름을 담는 법을 배웠고(5·8장), 기계가 복잡해진 사연과 그 위에 선 계약을 보았다(9·12장). 이 모든 것이 심화 문답으로 계속 소환될 밑천이다.

이제 정말로 시작할 수 있다. 다음 부에서 드디어 첫 프로그램을 쓴다 — 지면 위에 헬로 월드가 찍히고, 그 한 편을 읽어 내는 여정이 책의 나머지 전부다.

제3부 — 첫 프로그램

13 헬로 월드

이 장이 끝나면

드디어 첫 프로그램이다. 여섯 줄짜리 C 프로그램 한 편을 지면에서 실행하고, 한 줄 한 줄 소리 내어 읽는다. 아직 다 이해되지 않는 것이 정상이다 — 어디까지가 “주문”이고 언제 풀리는지를 여기서 정해 둔다.

돌아보기 7장에서 컴퓨터의 입출력은 “문자가 한 줄씩 흐르는 띠”(스트림)라 했고, 프로그램마다 표준 출력이라는 흐름이 기본으로 주어진다고 했다. 그러면 프로그램이 화면에 글자를 내보낸다는 것은 정확히 무엇을 하는 일인가?

답 표준 출력이라는 띠에 글자들을 흘려보내는 일이다. 화면은 그 띠의 끝에 붙어 있는 오늘날의 “종이”일 뿐이다(유리 텔레타이프의 후예라는 것을 기억하면 된다). 지금 만날 첫 프로그램이 하는 일이 정확히 이것 — 표준 출력에 인사말 한 줄을 흘려보내는 것 — 이다.

13.1 첫 프로그램

전통에 따라, 첫 프로그램은 세상에 인사를 건넨다.

```
examples/ch13/hello.c
#include <stdio.h>

int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

실행 결과

```
Hello, world!
```

위쪽 상자가 사람이 적는 소스 코드이고, 아래 검은 상자가 이 프로그램을 실제로 컴파일해 실행한 출력이다 — 이 책의 모든 실행 결과가 그렇듯, 장식이 아니라 기계가 실제로 낸 결과다.

13.2 한 줄씩 읽기

`#include <stdio.h>` — 준비물 선언이다. “표준 입출력(`st*an*d*ard i*input`)

14 일반적인 컴파일 과정

이 장이 끝나면

hello.c라는 텍스트가 실행 파일이 되기까지의 네 단계 릴레이 — 전처리, 컴파일, 어셈블, 링크 — 를 구경한다. 어느 플랫폼, 어느 컴파일러(gcc·clang)에서든 통하는 일반적인 그림이다. 13장의 “주문” 두 개(#include의 정체, printf의 출처)가 여기서 풀린다.

돌아보기 2장에서 추상화의 층계 — 트랜지스터에서 언어까지 겹겹이 쌓기 — 를 이야기했다. 그러면 컴파일러는 그 층계에서 몇 층부터 몇 층까지를 내려가는 번역인가?

답 “C 언어” 층에서 “기계 명령” 층까지다. 그런데 이 번역은 한 번에 건너뛰는 도약이 아니라, 중간 층계참을 밟아 내려가는 릴레이다 — 텍스트를 다듬는 층, C를 어셈블리라는 사람이 읽을 수 있는 기계어 표기로 바꾸는 층, 그것을 진짜 기계 명령으로 찍는 층, 그리고 조각들을 한 덩어리로 잇는 층. 이 장이 그 네 층계참의 견학이다.

14.1 겉보기 — 명령 한 줄

먼저 겉모습부터. gcc나 clang 같은 컴파일러에게 소스 파일을 주면 실행 파일이 나온다.

```
$ cc hello.c -o hello      (cc 자리는 gcc 또는 clang)
$ ./hello
안녕, 세상!
```

-o hello는 “결과물의 이름을 hello로 하라”는 뜻이다. 명령 한 줄로 끝나 보이지만, 이 한 줄 뒤에서 서로 다른 일꾼 네 명이 릴레이를 뛴다. 컴파일러에게 “중간 결과를 보여 달라”고 부탁하는 선택지들(-E, -S, -c)로 각 주자를 세워서 구경할 수 있다.

14.2 1주자 — 전처리: 텍스트를 짜깁기한다

전처리기(preprocessor)는 아직 C를 모른다. 그저 텍스트를 다루는 가위와 풀이다. #으로 시작하는 줄들 — 13장의 주문 — 이 전처리기에게 내리는 지시다.

#include <stdio.h>의 정체가 여기서 풀린다. 지시의 뜻은 놀랄 만큼 우직하다 — “**stdio.h라는 파일을 찾아서, 그 내용물을 이 자리에 통째로 붙여 넣어라.**” 그게 전부다. 실제로 전처리만 시켜 보면(cc -E hello.c) 여섯 줄짜리 hello.c가 수만 줄로 불어난 결과가 나온다 — printf를 비롯한 표준 도구들의 선언(이런 이름의 함수가 존재한다는 예고 — 정식으로서는 21장)이 줄줄이 붙어 들어온 것이다.

전처리기는 이 밖에도 글자 바꿔치기(매크로), 조건에 따라 코드 일부를 넣고 빼기 같은 텍스트 작업을 한다. 6장에서 만난 삼중자를 풀어 주던 것도(그 시절에는) 이 단계였다 — 전처리기는 C 컴파일의 첫 주자이자, 역사의 흥터가 모이는 곳이기도 하다.

14.3 2주자 — 컴파일: C를 어셈블리로

이제부터가 본편이다. 컴파일러 본체가 전처리된 C 코드를 읽고 — 문법을 검사하고, 뜻을 해석하고, 11장의 편집(최적화)을 가한 뒤 — 어셈블리라는 표기로 옮긴다. 어셈블리는 2장에서 스친 그 낮은 층의 언어다: 기계 명령과 거의 일대일이되, 사람이 읽을 수 있게 이름이 붙어 있다. cc -S hello.c로 이 중간 결과(hello.s)를 구경할 수 있는데, 대략 이런 모습의 조각이 들어 있다(기계와 컴파일러마다 다르다):

```
lea    rdi, [rip + .L.str]    ; 인사말의 주소를 준비하고
call   printf                 ; printf를 부른다
xor     eax, eax               ; 0을 만들어
ret                                ; 돌려주며 퇴장
```

우리가 쓴 네 문장의 골격이 그대로 비친다 — 그러나 11장에서 배웠듯, 이 대응이 언제나 이렇게 얹힌 것만 남는 것은 아니다(최적화를 켜면 문장 대 문장 대응은 무너진다).

14.4 3주자 — 어셈블: 기계 명령으로 찍어 내다

어셈블러가 어셈블리 텍스트를 진짜 기계 명령 — 비트들 — 로 찍어 낸다. 결과물이 **목적 파일**(object file, cc -c로 만들면 hello.o)이다. 이제 내용물은 사람이 읽는 텍스트가 아니라 2장의 세계, 기계 명령의 비트 열이다.

그런데 목적 파일은 아직 실행할 수 없다. **구멍이 뚫려 있기 때문이다** — call printf라고 찍기는 했는데, 정작 printf라는 것이 어디에 있는지 이 파일은 모른다. 우리 소스에는 printf의 선언(존재 예고)만 들어왔지, 그 몸통(실제 코드)은 들어온 적이 없다.

14.5 4주자 — 링크: 조각을 잇고 구멍을 메운다

마지막 주자 **링커**(linker)가 그 구멍을 메운다. printf의 몸통은 **표준 라이브러리**라는, 시스템에 미리 컴파일되어 놓여 있는 목적 파일들의 꾸러미 안에 있다. 링커는 우리의 목적 파일과 그 꾸러미를 이어 붙이고, “printf는 여기”라고 주소를 채워 넣어, 비로소 **실행 파일** 하나를 완성한다. 13장의 두 번째 주문 — printf는 어디서 오는가 — 의 답이 이것이다: 내가 쓴 적 없는 코드가, 링크 단계에서 내 프로그램에 이어져 들어온다.

● 가장 낮은 오류 — undefined reference

초보 시절 가장 어리둥절한 오류가 링크 단계에서 나온다. 함수 이름의 철자를 틀리거나 라이브러리를 빠뜨리면, 컴파일러는 멀쩡히 통과하고 마지막에 undefined reference to ... (정의를 찾을 수 없음) 같은 메시지가 나온다. “컴파일러는 통과시켰는데 왜?”가 어리둥절함의 정체인데 — 이제 답할 수 있다. 컴파일 단계는 **선언**(예고)만 있으면 만족하고, 몸통을 실제로 찾아 있는 것은 링커의 일이라서다. 오류를 낸 일꾼이 다르면 오류의 종류도 다르다 — 메시지를 읽을 때 “네 주자 중 누가 넘어졌는가”부터 가리는 습관이 문제 해결의 절반이다(정식은 43장).

△ “컴파일러가 소스를 실행 파일로 한 번에 번역한다”

겉보기(명령 한 줄)가 심어 주는 자연스러운 그림이지만, 실제로는 성격이 전혀 다른 네 도구의 릴레이이다 — 텍스트 짜깁기(전처리), 진짜 번역과 편집(컴파일), 비트로 찍기(어셈블), 조각 잇기(링크). 이 구별이 실무 감각의 뿌리가 된다: 오류가 어느 단계의 것인지 가려 읽게 되고, 여러 소스 파일을 각각 목적 파일로 만들어 두었다가 링크만 다시 하는 큰 프로젝트의 빌드 방식(43장)이 자연스러워지고, “선언과 정의”(21장)라는 언어 규칙이 왜 존재하는지 — 컴파일러는 예고만 보고 일하고 링커가 실물을 잇기 때문 — 가 이해된다.

문 네 단계를 매번 직접 시켜야 하는가?

답 아니다 — 보통은 cc hello.c -o hello 한 줄이면 컴파일러가 네 주자를 알아서 차례로 뛰게 한다. 단계 선택지(-E, -S, -c)는 학습과 진단용 도구다. 다만 큰 프로젝트에서는 3주자까지만 파일별로 뛰게 해 두고(각각 .o로) 마지막 링크만 다시 하는 방식이 표준 관행이 된다 — 파일 하나 고쳤다고 전부를 다시 번역할 이유가 없기 때문이다. 그 세계는 43장에서 열린다.

이제 소스에서 실행까지의 지도가 생겼다. 남은 것은 이 릴레이를 **내 컴퓨터에서** 돌릴 수 있게 도구를 갖추는 일이다 — 다음 장에서 컴파일러를 설치하고, 버그를 그물처럼 걸러 주는 현대적 보조 도구(새니타이저)까지 장만한다.

15 개발환경 구축

이 장이 끝나면

14장의 릴레이를 내 컴퓨터에서 돌릴 도구를 갖춘다. 어느 플랫폼에서든 통하는 일반론이 본문이고, 특정 운영체제에 묶이는 구체적 설치법은 “플랫폼 노트” 상자로 격리해 두었다 — 상자는 이 책의 예시 플랫폼인 Windows 기준이며, 건너뛰어도 본문은 이어진다. 컴파일러와 편집기에 이어, 프로그램을 멈춰 세워 들여다보는 디버거와 그 맹점, 그리고 버그를 실행 중에 잡아 주는 그물(새니타이저)까지 장만하면 이 부가 끝난다.

돌아보기 14장에서 컴파일러에게 `-E`, `-S` 같은 부탁을 하며 명령줄에서 일했다. 요즘은 클릭으로 다 되는 세상인데, 왜 명령줄인가?

답 일반론이기 때문이다. 그래픽 개발 도구는 저마다 생김새가 다르고 몇 해 마다 바뀌지만, `cc hello.c -o hello`라는 문장은 반세기째 모든 플랫폼에서 통하는 공용어다. 그리고 어떤 그래픽 도구를 쓰든 그 “빌드” 버튼의 밑바닥에서 실제로 도는 것이 바로 이 명령들이다 — 밑바닥을 아는 사람은 도구를 갈아타도 길을 잃지 않는다. 이 책이 명령줄을 기준으로 삼는 이유다.

15.1 필요한 것은 셋뿐이다

개발환경이라는 말이 거창하지만, C 프로그래밍에 필요한 것은 셋뿐이다.

- **컴파일러** — 14장의 네 주자 일체. 이 책은 gcc와 clang 두 계열을 기준으로 한다(둘 다 무료이고 모든 주요 플랫폼에 있다).
- **텍스트 편집기** — 소스 코드는 그냥 텍스트 파일이므로(6장), 글자를 적을 수 있는 도구면 무엇이든 된다. 어떤 편집기를 쓰든가는 취향의 영역이라 이 책은 정하지 않는다.
- **터미널** — 명령을 적어 넣는 창. 모든 운영체제에 기본으로 있다.

일과의 모양도 단순하다 — **편집하고, 컴파일하고, 실행한다**. 이 세 박자의 반복이 프로그래밍의 기본 사이클이고, 이 책의 모든 예제가 이 사이클 위에서 돈다(물론 독자는 지면으로 읽기만 해도 된다 — 1장의 약속대로, 이 장의 설치조차 시연이지 숙제가 아니다).

▣ 플랫폼 노트 — Windows에서 컴파일러 설치하기

이 책의 예시 플랫폼은 Windows다. 두 갈래 중 하나(또는 둘 다)를 고르면 된다 — 이 책의 예제는 어느 쪽으로도 동일하게 빌드된다.

갈래 1 — LLVM clang (권장). LLVM 프로젝트의 공식 Windows 배포판을 설치한다. 터미널(PowerShell)에서 한 줄이면 된다:

```
> winget install LLVM.LLVM
```

(또는 llvm.org의 릴리스 페이지에서 설치 프로그램을 받아도 같다.) 설치 후 새 터미널에서 `clang --version`이 판 번호를 답하면 준비 끝 이다. 이후 이 책의 `cc` 자리에 `clang`을 쓰면 된다:

```
> clang hello.c -o hello.exe
> .\hello.exe
안녕, 세상!
```

갈래 2 — MinGW-w64 gcc. gcc의 Windows 이식판이다. MSYS2라는 꾸러미 관리 환경(msys2.org)을 설치한 뒤, 그 터미널에서:

```
$ pacman -S mingw-w64-ucrt-x86_64-gcc
$ gcc --version
```

셋째 길 — Visual Studio(MSVC)에 관하여. Microsoft의 자체 컴파일러와 통합 환경도 훌륭한 선택지이지만, 이 책의 명령·선택지 표기(gcc/clang 계열)와 달라서 여기서 다루지는 않는다. 관심 있는 독자는 Microsoft의 공식 문서(learn.microsoft.com/cpp)가 설치부터 첫 프로그램까지 자세히 안내하므로 그쪽을 따르면 된다.

문 컴파일러가 두 계열이나 되는 이유가 있는가? 하나만 있으면 헛갈릴 일도 없을 텐데.

답 경쟁이 곧 품질이기 때문이다 — 그리고 프로그래머에게는 공짜 검증 도구이기 때문이다. 같은 코드를 gcc와 clang 양쪽으로 컴파일해 보면, 한쪽이 놓친 경고를 다른 쪽이 잡아 주는 일이 흔하다. 두 독립적인 구현이 모두 통과시키는 코드는 표준(계약서)에 맞게 쓰였을 가능성이 훨씬 높다 — 10장의 표현을 빌리면, 방언이 아니라 표준어로 말하고 있다는 교차 증거다. 실제로 이 책의 수록 예제들도 두 컴파일러로 교차 검증한다.

15.2 경고 — 공짜 검토를 켜 두어라

설치 직후에 들일 습관이 하나 있다. 컴파일러에게 **경고**를 넉넉히 켜 달라고 부탁하는 것이다:

```
$ cc -Wall -Wextra hello.c -o hello
```

-Wall -Wextra는 “수상한 구석을 최대한 알려 달라”는 뜻이다(이름과 달리 -Wall이 전부는 아니라서 관행상 둘을 함께 쓴다). 경고는 오류가 아니라 **컴파일러의 무료 코드 검토**다 — 계약(표준) 위반까지는 아니어도 사고 나기 좋은 무늬를 짚어 준다. 이 책의 모든 예제는 이 두 선택지를 켜 채, 경고 0개로 검증된다.

15.3 한글이 깨질 때 — 인코딩의 사슬

여기서 한국어권 독자가 거의 반드시 겪는 문제를 미리 짚어 둔다. 이 책의 예제가 출력 문자열을 대부분 영어로 쓰는 것도 이 사정 때문이다 — 한글을 찍는 순간, 프로그램 하나가 아니라 **네 접의 인코딩 설정이 전부 맞아야** 글자가 제대로 나온다.

사슬의 네 고리는 이렇다.

- ① **소스 파일의 인코딩** — 소스는 결국 바이트 열이다(7장). 편집기가 UTF-8로 저장했는지, 아니면 옛 한국어 윈도우의 기본이던 CP949(EUC-KR 계열)로 저장했는지에 따라 같은 “안녕”이 다른 바이트가 된다.
- ② **컴파일러가 읽는 인코딩** — 컴파일러는 소스가 어떤 인코딩인지 **추측하거나 기본값을 가정한다**. 소스는 UTF-8인데 컴파일러가 CP949로 읽으면, 문자열 리터럴의 바이트가 그 자리에서 어그러진다.
- ③ **실행 문자 집합** — 컴파일러가 실행 파일에 심는 바이트가 무엇인가 (7장의 5단계, 44장의 번역 단계에서 본 그 변환이다).
- ④ **터미널이 해석하는 인코딩** — 프로그램이 뱉은 바이트를 창이 어떤 표로 읽는가. 여기가 어긋나면 앞의 셋이 완벽해도 화면에는 깨진 글자가 뜬다.

네 고리 중 하나만 어긋나도 증상은 똑같이 “글자가 깨진다”로 나타나므로, 초보자가 원인을 짚기가 유난히 어렵다. 진단의 요령은 **어느 고리에서 깨졌는지 좁히는 것**이다 — 소스를 16진으로 열어 보면(①②를 가름), 파일로 출력을 돌려 보면(④를 가름) 범인이 드러난다.

▣ 플랫폼 노트 — Windows에서 한글이 깨질 때

Windows가 이 문제의 최대 격전지다. 오랫동안 한국어 Windows의 기본 코드 페이지가 CP949였고, 그 위에 UTF-8 세계가 없으면서 고리마다 설정이 깔렸기 때문이다. 실전 처방을 도구별로 적어 둔다.

공통 — 소스는 UTF-8로. 편집기에서 “UTF-8”로 저장한다. 다만 Windows 도구들과의 호환을 위해 **BOM이 붙은 UTF-8**(UTF-8 with signature)을 원하는 경우가 있다 — MSVC는 BOM이 있으면 UTF-8임을 확신하고 읽는다. gcc·clang은 BOM 없는 UTF-8을 기본으로 잘 읽는다.

Visual Studio(MSVC) — 파일을 “인코딩을 지정하여 저장”으로 UTF-8(BOM 포함)로 두거나, 컴파일 옵션 `/utf-8`을 주어 소스와 실행 문자 집합을 모두 UTF-8로 못박는 것이 가장 확실하다. 자세한 안내는 Microsoft 공식 문서에 있다.

VS Code — 오른쪽 아래 상태 표시줄에 현재 파일의 인코딩이 뜬다. 거기서 “Save with Encoding → UTF-8”로 바꾸고, 설정에서 `"files.encoding": "utf8"`을 기본으로 두면 ①이 고정된다. 자동 감지(`files.autoGuessEncoding`)는 편해 보이지만 추측이 틀리면 오히려 파일을 망칠 수 있으니, 팀 작업에서는 **끄고 명시하는 쪽이 안전하다.**

터미널(④) — Windows 콘솔은 현재 코드 페이지로 바이트를 해석한다. `chcp 65001`로 UTF-8 코드 페이지로 바꾸면 UTF-8 출력이 제대로 보인다. Windows Terminal은 UTF-8을 기본에 가깝게 다루므로 옛 콘솔 창보다 사고가 적다. 반대로 CP949로 맞춰 둔 콘솔에서 UTF-8 프로그램을 돌리면 자모가 깨져 보이는데 — 프로그램이 틀린 것이 아니라 **읽는 표가 다른 것이다** (7장의 그 이야기가 화면에서 재현되는 것이다).

MinGW·MSYS2 / WSL — 이쪽은 대개 UTF-8이 기본이라 사고가 적다. 다만 MinGW로 만든 프로그램을 옛 Windows 콘솔에서 돌리면 ④가 어긋날 수 있다는 것은 같다.

문 그러면 예제에서 한글 출력을 아예 피하는 것이 정답인가?

답 학습 단계에서는 그렇게 하는 편이 이롭다 — 이 책의 예제가 그런 선택을 했다. 배우려는 것이 포인터인데 인코딩 설정 때문에 막히면, 정작 배울 것에서 멀어지기 때문이다. 다만 **피하는 것이 해결은 아니다.** 실무의 프로그램은 결국 한글을 다루게 되고, 그때는 이 사슬을 하나씩 고정하는 것이 정공법이다 — 소스 UTF-8, 컴파일러에 UTF-8 명시, 터미널 UTF-8. 세 곳을 못박아 두면 그 뒤로는 조용하다. 문자열을 다루는 쪽의 규칙 (글자 수 ≠ 바이트 수, 경계에서 자르지 않기)은 7장과 34장에서 이미 배운 그대로다.

15.4 디버거 — 멈춰 세워 들여다보기

프로그램이 틀린 답을 낼 때 가장 먼저 하는 일은 대개 `printf`를 심는 것이다. 나쁜 방법이 아니다 — 어디서나 되고, 준비가 필요 없다. 다만 찍을 자리를 **미리 알아야** 하고, 한 번 찍을 때마다 다시 컴파일해야 하며, 찍히는 것은 그 순간 그 변수뿐이다.

디버거(debugger)는 그 제약을 걷어낸다. 프로그램을 원하는 지점에서 멈춰 세우고, 그 순간의 모든 변수를 들여다보고, 한 줄씩 걸어 보고, 어떻게 여기까지 왔는지 호출의 사슬을 되짚는다. 표준 도구는 GNU의 `gdb`와 LLVM의 `lldb`이고, 명령 이름만 다를 뿐 하는 일은 같다.

하고 싶은 일	gdb 명령	뜻
여기서 멈춰라	<code>break sum_all</code>	중단점(breakpoint)
시작	<code>run</code>	프로그램 실행

이 값이 뭐지	print n	변수·수식 평가
한 줄 진행	next / step	건너뛰기 / 함수 안으로
여기까지 어떻게 왔나	backtrace	호출 사슬(call stack)
이 변수가 바뀌면 멈춰라	watch s	감시점(watchpoint)
함수 끝날 때까지	finish	반환값까지 보여 준다

예제로 쓸 프로그램은 이렇다. 합이 150이어야 하는데 100이 나온다.

```
examples/ch15/bug.c
#include <stdio.h>

static int sum_all(const int *a, int n) {
    int s = 0;
    for (int i = 0; i < n - 1; i++) /* bug: stops one short */
        s += a[i];
    return s;
}

int main(void) {
    int data[5] = {10, 20, 30, 40, 50};
    printf("sum = %d (should be 150)\n", sum_all(data, 5));
    return 0;
}
```

실행 결과

```
sum = 100 (should be 150)
```

디버거로 붙잡으면 이렇게 보인다(gdb 17.2로 실제 실행한 것이다).

```
$ gcc -std=c23 -g -O0 bug.c -o bug
$ gdb -q ./bug
(gdb) break sum_all
Breakpoint 1 at 0x1154: file bug.c, line 4.
(gdb) run
Breakpoint 1, sum_all (a=0x7fff5302c8a0, n=5) at bug.c:4
4   int s = 0;
(gdb) print n
$1 = 5
(gdb) next
5   for (int i = 0; i < n - 1; i++) /* bug: stops one short */
(gdb) next
6       s += a[i];
(gdb) print a[4]
$3 = 50
(gdb) finish
Run till exit from sum_all (a=..., n=5) at bug.c:4
Value returned is $4 = 100
```

n은 5로 제대로 들어왔고, 배열의 마지막 칸 a[4]에는 50이 멀쩡히 있는데 반환값은 100이다. 재료는 옳고 계산이 틀렸다는 뜻이므로 남은 곳은 루프의 경계뿐이다 — $i < n - 1$. 이 좁혀 가는 과정 자체가 디버거의 쓸모다.

15.5 디버그 빌드의 옵션 — -g와 -O0은 다른 스위치다

디버거가 변수 이름과 줄 번호를 알려면 컴파일러가 그 정보를 실행 파일에 넣어 두어야 한다. 그것이 -g다.

- `-g` — **디버그 정보(debug info)**를 넣는다. 변수 이름, 타입, 줄 번호, 함수의 경계가 실행 파일 안에 함께 실린다.
- `-O0` — 최적화를 끈다. 소스의 문장과 기계 명령이 거의 일대일로 대응 하므로, 한 줄씩 걷기와 변수 보기가 정확하게 동작한다.
- `-Og` — “디버깅하기 좋을 만큼만” 최적화한다. `-O0`보다 빠르면서 `-O2`보다 훨씬 잘 보인다. 디버깅용 기본값으로 좋다.
- `-g3` — 매크로 정의까지 넣는다. `print MAX_LEN` 같은 것이 된다.
- `-fno-omit-frame-pointer` — 호출 사슬을 되짚는 실마리를 남긴다. 프로파일러와 크래시 보고에도 도움이 된다.

`assert`(41장)와 관련한 스위치도 하나 있다. `-D NDEBUG`를 주면 모든 `assert`가 통째로 사라진다 — 릴리스 빌드가 대개 그렇게 만들어진다. 검사를 지운다는 것은 **디버그 빌드에서 걸리던 것이 릴리스에서는 그냥 지나간다는** 뜻이므로, 사라져서는 안 되는 검사는 `assert`가 아니라 보통 `if`로 써야 한다.

△ “`-g`를 붙이면 프로그램이 느려진다”

그렇듯하지만 아니다. `-g`는 **정보를 덧붙일 뿐** 생성되는 기계 명령을 바꾸지 않는다 — 실행 파일이 커질 뿐 속도는 같다. 느려지는 원인은 함께 쓰는 `-O0`이다. 둘은 서로 다른 스위치이고, `-O2 -g`처럼 함께 쓸 수 있다. 실무에서는 릴리스 빌드도 `-g`로 만들어 디버그 정보를 별도 파일로 떼어 보관하는 것이 정석이다 — 출하하는 실행 파일은 가볍게 유지하면서, 현장에서 온 크래시 보고를 그 정보로 해독할 수 있기 때문이다. 정보를 버리면 그 순간부터 사고 현장은 주소 숫자의 나열이 된다.

15.6 디버거의 맹점 — 최적화된 빌드

문제는 릴리스 빌드다. 앞의 같은 프로그램을 `-O2 -g`로 만들어 붙잡으면 이렇게 된다(역시 실제 캡처다).

```
$ gcc -std=c23 -g -O2 bug.c -o bug
$ gdb -q ./bug
(gdb) break sum_all
Function "sum_all" not defined.
(gdb) break main
Breakpoint 1 at 0x1040: file bug.c, line 12.
(gdb) run
Breakpoint 1, main () at bug.c:12
12     printf("sum = %d (should be 150)\n", sum_all(data, 5));
(gdb) info locals
data = <optimized out>
```

찾던 함수가 **없다**. 컴파일러가 `sum_all`을 호출 자리에 펼쳐 넣고 (인라인), 상수 배열의 합을 아예 컴파일 시간에 계산해 버려서, 실행 파일 안에 그런 함수가 남아 있지 않기 때문이다. 배열 `data`도 마찬가지로 “최적화되어 사라졌다”. 11장에서 배운 편집자의 권리가, 여기서는 관찰자의 시야를 가리는 형태로 나타난 것이다.

최적화 빌드에서 흔히 만나는 맹점을 모으면 이렇다.

- `<optimized out>` — 변수가 레지스터에만 있거나 아예 없어져 값을 볼 수 없다(9장의 레지스터가 여기서 돌아온다).
- **줄 번호가 튜다** — 한 줄씩 걸으면 `12 → 15 → 12`처럼 오간다. 컴파일러가 명령을 재배열했기 때문이고, 디버거는 거짓말을 하는 것이 아니라 뒤섞인 실제 순서를 보여 주는 것이다.
- **호출 사슬이 짧다** — 인라인된 함수와 꼬리 호출(tail call)은 스택에 프레임을 남기지 않는다.
- **중단점이 걸리지 않는다** — 지워진 코드에는 멈출 자리가 없다.

15.7 디버그에선 되는데 릴리스에서만 틀린다

가장 곤혹스러운 상황이고, 원인의 대부분은 하나로 모인다 — 프로그램은 이미 깨져 있었고, 디버그 빌드가 우연히 그것을 가려 주고 있었다. 최적화가 멸절한 코드를 망가뜨린 것이 아니라, 계약 밖의 코드(11장·42장)가 전제로 쓰이면서 비로소 증상이 드러난 것이다. 흔한 뿌리는 다음과 같다.

- 초기화하지 않은 지역 변수 — `-O0`에서는 스택의 그 자리가 마침 0이라 잘 돌아다, 배치가 바뀌면 쓰레기 값이 들어온다.
- 경계를 넘는 쓰기 — 넘어간 자리에 무엇이 놓였느냐가 빌드마다 다르다. 디버그 빌드에서는 여유 공간을 덮고 지나가던 것이, 릴리스에서는 하필 중요한 값을 덮는다.
- 계약 밖을 전제로 삼는 최적화 — 부호 있는 오버플로는 없다(5장), 엄격한 앨리어싱(11장)은 지켜진다, 널을 역참조한 뒤라면 널일 리 없다 — 이런 전제는 `-O2`에서만 실제 코드 변형으로 나타난다.
- `volatile`을 빠뜨린 폴링 루프 — 11장에서 본 그대로, `-O0`에서는 매번 읽던 것이 `-O2`에서 한 번만 읽히고 끝는다.
- 사라진 `assert` — `-DNDEBUG`로 검사가 통째로 빠지면서 잘못된 입력이 그대로 흘러 들어간다.
- 타이밍이 바뀌며 드러나는 경쟁 — 여러 갈래로 도는 프로그램(10장)에서 빨라진 코드가 순서를 바꾸어 놓는다. 디버거를 붙이면 느려져서 증상이 사라지는, 이른바 하이젠버그(Heisenbug)의 전형이다.
- 실수 계산의 미세한 차이 — `-ffast-math`류의 선택지는 결합 법칙을 가정해 순서를 바꾸므로 마지막 자리가 달라질 수 있다(6장).

● “우리 코드는 디버그에서 잘 됩니다” — 커널이 플래그로 항복한 이야기

11장에서 본 엄격한 앨리어싱은 이 부류의 대표다. 손으로 짠 파서가 바이트 버퍼를 폭이 다른 포인터로 들여다보는 코드는 `-O0`에서 완벽하게 돌고, `-O2`에서 조용히 다른 답을 낸다. 리눅스 커널은 이 문제를 코드로 일일이 고치는 대신 빌드 전체를 `-fno-strict-aliasing`으로 컴파일하는 길을 택했다 — 규칙 하나 때문에 플래그 하나를 통째로 끈 것이다. 개인 프로젝트에서 흉내 낼 일은 아니지만, “릴리스에서만 나는 버그”가 얼마나 현실적인 문제인지를 보여 주는 가장 유명한 사례다.

15.8 디버거를 믿기 어려울 때의 도구들

디버거를 못 쓰거나 믿기 어려운 자리는 실제로 있다 — 최적화 빌드에서만 재현되는 버그, 붙이는 순간 사라지는 타이밍 버그, 디버거를 띄울 수 없는 운영 환경, 그리고 화면도 키보드도 없는 임베디드 보드. 그럴 때의 순서는 대략 이렇다.

1. `-Og -g`로 다시 세워 본다. 최적화를 완전히 끄지 않으면서 시야를 되찾는 첫 시도다. 여기서 증상이 남아 있다면 디버거가 다시 쓸 만해진다.
2. 새니타이저를 켜다. 다음 절의 도구들이다. 이들은 사고를 일어나는 순간에 잡으므로, 최적화 빌드에서만 나는 버그의 원인을 대개 여기서 찾는다. 릴리스와 같은 `-O2`에 얹어 쓸 수 있다는 점이 중요하다.
3. 가설을 플래그로 검증한다. `-fno-strict-aliasing`을 켜서 증상이 사라지면 원인이 앨리어싱 쪽임을 알 수 있다. 다만 이것은 진단이지 치료가 아니다 — 원인을 확인했으면 코드를 고친다.
4. 경고와 정적 분석을 최대한 올린다. `-Wall -Wextra`에 더해 `-fanalyzer`(GCC)나 `clang --analyze`는 실행 없이 잡아낸다.
5. 로그와 코어 덤프를 남긴다. 운영 환경에서는 이것이 유일한 창이다. 릴리스 빌드에 `-g`를 넣어 두었다면(앞의 오개념 상자) 사후에 그 덤프를 디버거로 열어 사고 지점을 소스 줄로 되짚을 수 있다.
6. 이분 탐색으로 좁힌다. 최적화 단계를 낮춰 가며(`-O2` → `-O1` → `-O0`) 어디서 갈리는지 보고, 버전 관리의 이력을 반으로 갈라 가며 언제 들어온 버그인지 찾는다(57장의 `git bisect`).
7. 최소 재현을 만든다. 문제를 재현하는 가장 작은 프로그램으로 줄이면 원인이 저절로 드러나는 일이 많고, 남에게 물어볼 수 있는 형태가 된다.

▣ 플랫폼 노트 — 디버거 설치와 Windows 사정

Linux·macOS — gdb는 배포판의 패키지 관리자로, lldb는 LLVM 패키지에 함께 온다. macOS는 lldb가 기본이다.

Windows(MSYS2·MinGW) — `pacman -S mingw-w64-ucrt-x86_64-gdb`로 받는다. MinGW로 만든 실행 파일은 이 gdb로 붙잡는다.

Windows(LLVM) — lldb가 함께 설치된다. 다만 MSVC 계열이 쓰는 PDB 형식의 디버그 정보와의 궁합은 도구 조합에 따라 달라진다.

Visual Studio — 통합 디버거의 완성도가 매우 높고, 조사식·메모리 창·시간 여행 디버깅(Time Travel Debugging, WinDbg)까지 갖추고 있다. 명령을 외울 필요 없이 F5·F10·F11로 앞의 표와 같은 일을 한다.

어느 쪽이든 개념은 같다 — 중단점, 한 줄 진행, 변수 보기, 호출 사슬. 도구가 바뀌어도 이 넷을 찾으면 된다.

문 그러면 printf 디버깅은 이제 그만두어야 하는가?

답 아니다. 둘은 경쟁 관계가 아니라 쓰는 자리가 다르다. 디버거는 **한 순간을 깊게** 보는 도구이고, 로그는 **긴 시간을 넓게** 보는 도구다. 타이밍 문제, 오래 도는 서버, 임베디드, 재현이 어려운 사고에 서는 로그가 유일한 창인 경우가 많다. 실무의 요령은 임시로 심었다 지우는 printf 대신 **켜고 끌 수 있는 로그**를 처음부터 코드에 두는 것이다 — 그러면 같은 창을 개발 중에도, 운영 중에도 쓸 수 있다.

15.9 새니타이저 — 실행 중에 치는 그물

현대 C 개발환경의 필수 장비가 하나 더 있다. 컴파일 때가 아니라 **실행 중에** 사고를 잡는 도구, **새니타이저(sanitizer)**다.

원리는 이렇다 — 컴파일할 때 특별한 선택지를 주면, 컴파일러가 프로그램 곳곳에 **감시 코드**를 심어 준다. 그 프로그램을 실행하면 감시 코드가 메모리 접근과 연산을 지켜보다가, 잘못이 **일어나는 순간** 현장을 짚으며 프로그램을 세운다. 대표 선수는 셋이다.

- **ASan(AddressSanitizer)** — 메모리 사고 담당. 경계를 넘는 접근, 해제 한 기억을 다시 만지기 같은, 제7부에서 만날 위험들을 현장에서 잡는다.
- **UBSan(UndefinedBehaviorSanitizer)** — 계약 위반 담당. 부호 있는 오버플로(5장), 폭 이상 시프트(5장) 같은 **정의되지 않은 동작**이 실행 중에 실제로 일어나면 그 자리에서 알린다. 42장의 주역이다.
- **TSan(ThreadSanitizer)** — 경쟁 사고 담당. 여러 코어(10장)가 같은 데이터를 다투는 문제를 잡는다. 이 책의 범위 밖이지만 이름은 알아 둔다.

쓰는 법은 컴파일 선택지 하나다:

```
$ cc -fsanitize=address,undefined -g hello.c -o hello
$ ./hello
```

(-g는 “사고 지점을 소스 몇 번째 줄인지로 알려 달라”는 부가 정보 선택지다.) 이렇게 빌드한 프로그램은 조금 느려지는 대신, 사고를 조용히 지나치지 않는다 — 5장에서 본 “오버플로는 조용하다”에 대한 현대의 응답이 바로 이 도구들이다. 이 책의 뒷부분(제7부, 제9부)에서 위험한 코드를 다룰 때, 새니타이저가 사고를 잡아내는 장면을 지면에서 여러 번 보게 된다.

▣ 플랫폼 노트 — Windows에서 새니타이저 쓰기

플랫폼마다 지원 폭이 달라서, Windows의 사정을 정직하게 적어 둔다.

- **ASan·UBSan**: 갈래 1의 **clang**으로 쓸 수 있다 — 위의 `-fsanitize=address,undefined` 선택지가 그대로 통한다. 갈래 2의 MinGW-w64 gcc는 Windows에서 새니타이저를 지원하지 않는다 — 새니타이저를 쓰려면 clang을 고르면 된다(두 갈래를 다 설치해 두고 평소엔 아무 쪽, 검사할 땐 clang을 쓰는 것도 좋은 배치다).
- **TSan**: Windows 자체를 지원하지 않는다(리눅스·macOS 전용). Windows 에서 굳이 쓰려면 WSL(Windows 안의 리눅스 환경, Microsoft 공식 기능)을 통하는 길이 있다 — 다만 이 책의 범위에서는 필요하지 않다.

참고로 MSVC도 자체 ASan을 지원한다(`/fsanitize=address`) — 상세는 앞서 언급한 Microsoft 공식 문서에 있다.

문 경고도 켜고 새니타이저도 있는데, 그래도 버그가 남는 이유는 무엇인가?

답 각 도구가 보는 범위가 다르기 때문이다. 경고는 **컴파일 시점**에 코드의 생김새를 보고, 새니타이저는 **실행 시점**에 실제로 일어난 일을 본다 — 즉 실행해 보지 않은 경로의 사고는 새니타이저도 모른다. 그래서 현대적 C 개발은 그물을 겹친다: 경고(항상) + 새니타이저(시험 실행) + 두 컴파일러 교차(습관) + 그리고 애초에 사고 나기 어려운 부품을 쓰는 것. 마지막 항목이 이 책 후반의 proven이 서는 자리다(35장) — 도구는 그물이고, 좋은 부품은 애초에 떨어지지 않는 발판이다.

15.10 제3부를 닫으며

첫 프로그램을 읽었고(13장), 그것이 실행 파일이 되는 릴레이를 구경했으며(14장), 그 릴레이를 내 책상에서 돌릴 도구와 그물까지 갖췄다(15장). 이제 준비는 끝났다.

다음 부부터 본격적인 언어 공부다 — 단, 이 책의 방식대로 간다. 새 문법을 나열하는 것이 아니라, 13장의 헬로 월드 **한 편을 완전히 읽어 내는 것**이 제4부의 목표다. 프로그램의 뼈대(문장과 블록)부터, 수식, 함수를 부르는 법, 그리고 출력까지 — 이미 본 여섯 줄이 전부 “아는 문장”이 될 때까지.

제4부 — 최소한의 도구 상자

16 프로그램의 구조

이 장이 끝나면

제4부의 목표는 하나다 — 13장의 헬로 월드 한 편을 **완전히** 읽어 내는 것. 그 첫걸음으로 프로그램의 뼈대를 익힌다: 주석, 문장, 블록, 그리고 `#include` 줄이 문장이 아닌 이유까지. 이 장이 끝나면 C 소스의 길모양이 더는 낯설지 않다.

돌아보기 2장에서 기계는 “메모리에서 다음 할 일을 가져와 그대로 하는” 반복 이라고 했다. 그러면 소스 코드 쪽에서 그 “할 일 하나”에 해당하는 것은 무엇인가?

답 **문장(statement)**이다. C 프로그램의 몸통은 문장들의 목록이고, 실행이란 그 목록을 위에서 아래로 한 문장씩 해치우는 것이다 — 2장의 계산 모델이 소스 코드에 그대로 비친 모습이다. 물론 11장에서 배웠듯 컴파일러가 속사정을 재배열할 수는 있지만, **겉보기 결과**는 언제나 “위에서 아래로 한 문장씩”과 같도록 보장된다. 그래서 독자는 안심하고 이 그림으로 읽으면 된다.

16.1 주석 — 사람만 읽는 글

뼈대를 보기 전에, 뼈대가 **아닌** 것부터 치운다. 소스 코드에는 컴파일러가 완전히 무시하는 글 — 주석(comment) — 을 적을 수 있다. 두 가지 표기가 있다:

```
// 이 표기는 여기부터 줄 끝까지가 주석이다
/* 이 표기는 여는 쪽과 닫는 쪽 사이가 주석이다 -
   여러 줄에 걸쳐 쓸 수 있다 */
```

주석은 기계에게는 없는 것이고, 사람에게는 문서다. 이 책의 예제에도 설명을 주석으로 달아 두었다 — 코드와 함께 읽으면 된다.

16.2 문장 — 한 걸음, 그리고 세미콜론

문장은 프로그램의 한 걸음이다. 13장에서 본 `printf(...)` 줄과 `return 0` 줄이 각각 문장 하나다. 그리고 C에서 문장의 끝은 **세미콜론** ;이 표시한다 — 우리글의 마침표다.

여기서 중요한 규칙 하나. **C에서 줄바꿈은 문장의 끝이 아니다**. 문장이 어디서 끝나는지는 오직 세미콜론이 정하고, 줄바꿈과 들여쓰기는 전부 사람 눈을 위한 정리 정돈일 뿐이다. 이런 언어를 **자유 형식**(free-form) 언어라 한다.

문 왜 줄 끝을 문장 끝으로 삼지 않았는가? 세미콜론을 매번 찍는 것은 번거로운데.

답 긴 문장 때문이다. 문장 하나가 길어져 여러 줄로 나눠 적고 싶을 때, “줄 끝 = 문장 끝”인 언어는 이어짐 표시 같은 예외 장치가 필요해진다. C는 반대쪽을 택했다 — 끝을 세미콜론으로 명시하고, 줄바꿈은 완전히 자유롭게 두는 것이다. 8장의 펀치카드를 기억하면 맛이 더 살아난다 — “한 장 = 한 줄”이라는 물리적 제약의 시대를 지나며, 언어는 줄이라는 형식에서 뜻을 독립시키는 쪽으로 진화했다.

△ “한 줄에 한 문장씩 쓰는 것이 C의 규칙이다”

그렇듯하다 — 잘 쓰인 코드가 대개 그렇게 생겼으니까. 그러나 그것은 **관행**이지 규칙이 아니다. 문법상으로는 한 줄에 문장 여럿을 몰아 적어도 되고, 문장 하나를 다섯 줄에 걸쳐 적어도 된다. 컴파일러에게는 전부 같은 프로그램이다. 그런데도 “한 줄에 한 문장, 블록 안은 들여쓰기”라는 관행이 반세

기를 살아남은 이유는 단순하다 — 코드는 기계보다 사람이 훨씬 자주 읽기 때문이다. 이 책의 예제 도 그 관행을 따른다: 문법이 허락하는 자유와, 관행이 원하는 절제를 구별하는 것 — 이것도 C 공부의 일부다.

16.3 블록 — 문장들의 묶음

문장 여럿을 중괄호 { }로 감싼 것이 **블록(block)**이다. 13장에서 본 `main` 뒤의 담장이 바로 블록이었다 — `main`이 할 일의 목록을 하나로 묶은 것이다. 블록은 “여기부터 여기까지가 한 묶음”이라는 문법적 괄호로, 앞으로 만날 거의 모든 구조(조건, 반복, 함수)가 블록을 부품으로 쓴다. 지금은 “문장 목록의 포장 단위” 하나로 충분하다.

16.4 문장이 아닌 줄 — #의 세계

이제 13장 첫 줄의 마지막 비밀이다. `#include <stdio.h>` 끝에는 세미콜론이 **없다**. 문장이라면 마침표가 있어야 할 텐데 — 없는 이유는 간단하다. **문장이 아니기 때문이다**.

14장의 릴레이를 떠올리면 된다. #으로 시작하는 줄은 첫 주자인 **전처리기**에게 하는 말이고, 전처리기는 C 문법이 아니라 **줄** 단위로 일하는 텍스트 도구다. 그래서 # 줄의 문법은 C의 문장 규칙과 아예 다르다 — 세미콜론 대신 줄 끝이 지시의 끝이고, 관례상 파일 맨 위에 모여 산다. 한 소스 파일 안에 사실 **두 개의 언어**(전처리 지시의 언어와 C)가 함께 있는 셈이다. 낯설지만, 역사를 아는 우리에게만 사연이 보이는 구조다 — 전처리기는 별도의 일꾼으로 태어나 지금까지 남았다.

16.5 종합 — 두 문장짜리 프로그램

이 장의 재료를 전부 넣은 변형을 하나 시연한다. 13장 헬로 월드에서 바뀐 것은 하나 — 문장이 둘이 됐다.

```
examples/ch16/two.c
// 한 문장씩, 위에서 아래로 차례로 실행된다.
#include <stdio.h>

int main(void)
{
    printf("Hello, "); /* 줄바꿈이 없다 - 다음 출력이 이어 붙는다 */
    printf("world!\n"); /* 줄바꿈은 여기, \n에서만 일어난다 */
    return 0;
}
```

실행 결과

Hello, world!

실행 결과가 한 줄인 것에 주목하면 된다. 문장은 둘, 줄음도 둘이지만 — 첫 문장에 `\n`이 없으므로 출력의 떠(8장) 위에서는 글자들이 그냥 이어 붙는다. **줄을 바꾸는 것은 문장이 아니라 \n이다** — 문장의 경계와 출력의 줄 경계는 서로 무관하다는 것, 이것이 이 시연의 교훈이다.

이제 헬로 월드 여섯 줄 중 빼대는 다 읽힌다 — `#include`(전처리 지시), `main`과 담장(블록), 문장과 세미콜론, 그리고 주석까지. 남은 것은 담장 **안쪽**의 알맹이다: `printf("...")` 라는 문장 안에서 정확히 무슨 일이 벌어지는가. 다음 장에서 그 안쪽의 세계 — 값이 되는 것들, **수식** — 로 들어간다.

17 수식 — 값이 되는 것

이 장이 끝나면

문장의 안쪽으로 들어간다. 소스 코드에 값을 적는 법(리터럴), 값을 계산으로 엮는 법(연산자와 수식), 그리고 계산의 순서(우선순위와 괄호) 까지 — 단, 이 책의 원칙대로 최소 집합만: 덧셈, 뺄셈, 곱셈, 괄호. 이것으로도 놀랄 만큼 멀리 간다.

돌아보기 5장과 6장에서 수를 기계에 담는 법(2의 보수, IEEE 754)을 배웠다. 그러면 소스 코드에 수를 적으면 — 이를테면 12라고 쓰면 — 그것은 언제 기계의 비트가 되는가?

답 컴파일 때다. 소스의 12는 그냥 글자 두 개(6장의 표현으로 문자 1과 2)이고, 컴파일러가 그것을 읽어 2의 보수 비트 패턴으로 번역해 실행 파일에 심는다. 소스에 적힌 값 표기를 **리터럴(literal)**이라 부른다 — “글자 그대로”라는 뜻이다. 글자의 세계와 비트의 세계를 잇는 다리가 컴파일러라는 것 — 14장의 릴레이가 여기서도 일하고 있다.

17.1 리터럴 — 소스에 적는 값

리터럴은 이미 여러 번 만났다. 헬로 월드의 0이 정수 리터럴이고, “안녕, 세상!\n”은 문자열 리터럴이다 — 따옴표 안의 글자들이 “글자 그대로” 값이 된다. 정수 리터럴에 소수점을 붙이면(3.5) 6장의 부동소수점 값이 된다.

문 그러면 소스에 0.1이라고 적으면, 기계에는 정확히 0.1이 담기는가?

답 아니다 — 6장에서 배운 그대로다. 컴파일러는 0.1이라는 리터럴을 **가장 가까운 이웃**(3FB9 9999 9999 999A)으로 번역한다. 리터럴은 “값을 그대로 적는 표기”이지만, 그 값이 표현 가능한 눈금 위에 없으면 번역 단계에서 이미 근사가 일어난다. 소스에 적었다고 정확한 것이 아니다 — 6장의 교훈이 소스 코드의 세계에서든 그대로 유효하다.

17.2 수식 — 계산되어 값이 되는 것

리터럴을 연산자로 엮으면 수식(expression)이 된다. $2 + 3 * 4$ 처럼. 수식의 정의는 한 문장이면 된다 — **계산되어(평가되어) 값이 되는 것**. 이 “값이 된다”는 성질이 수식의 전부이자, 앞으로 C를 읽는 눈의 절반이다. 코드의 어느 자리에 값이 필요하든, 그 자리에는 수식이 올 수 있다 — 리터럴 하나도 가장 단순한 수식이다.

이 장의 연산자는 넷뿐이다 — 더하기 +, 빼기 -, 곱하기 *, 그리고 괄호 (). 이 책의 나선형 원칙대로, 나머지 연산자들은 필요해지는 장에서 하나씩 합류한다(비교는 26장, 나눗셈은 24장 — 나눗셈이 뒤로 미뤄진 이유는 잠시 뒤에).

시연이다. 아래 예제의 %d는 “이 자리에 정수 값을 십진법으로 찍어라” 라는 표시로, 정식 설명은 19장에서 한다 — 지금은 계산 결과를 눈으로 확인하는 창으로만 쓴다.

```
examples/ch17/expr.c
#include <stdio.h>

int main(void)
{
    printf("%d\n", 2 + 3 * 4);    /* 곱셈이 먼저 계산된다 */
    printf("%d\n", (2 + 3) * 4); /* 괄호가 순서를 명시한다 */
}
```

```
    return 0;
}
```

실행 결과

14
20

17.3 순서 — 우선순위, 그리고 괄호라는 관행

위 시연의 첫 줄이 14인 것은 곱셈이 덧셈보다 **먼저** 계산됐기 때문이다. 수학 시간의 관례 그대로이고, C는 이런 “누가 먼저인가”의 서열 — **우선순위**(precedence) — 을 모든 연산자에 정해 두었다.

여기서 이 책의 권고를 미리 밝힌다. **우선순위 표를 외우지 말고, 괄호를 써라.** C의 연산자는 수십 개이고 서열표는 15줄이 넘는다 — 그것을 다 외운 사람들끼리도 헛갈려서 사고가 난다. 둘째 줄처럼 괄호로 순서를 명시하면 외울 것도, 오해할 것도 없다. 전체 서열표는 부록에 참고 자료로 두고, 본문은 “산술은 수학 관례대로, 그 밖에는 괄호”로 간다.

문 나눗셈은 왜 미룬 것인가? 사칙연산에서 하나만 빠지니 오히려 어색한데.

답 정수의 나눗셈이 **수학의 나눗셈과 다르기 때문이다.** C에서 $7 / 2$ 는 3.5가 아니라 3이다 — 정수끼리의 나눗셈은 소수점 아래를 버린 몫이고, 짝꿍 연산자 $\%$ (나머지)와 세트로 이해해야 뜻이 선다. 그 “버림”의 정확한 규칙(음수에서는 어느 쪽으로 버리나)까지 포함해서, 5장의 정수 세계 위에서 제대로 다룰 가치가 있는 주제라 24장에 자리를 잡아 두었다. 어중간하게 지금 소개해서 “ $7/2=3.5$ 인 줄 알았는데”라는 오개념을 만드는 것보다, 정색하고 한 번에 다루는 것이 낫다는 판단이다.

문 `printf("안녕")`도 값이 되는가? 수식의 자리에 함수 부르기가 적혀 있는데.

답 좋은 눈이다 — 된다. 함수를 부르는 것 자체가 수식이고, 따라서 값이 된다. 그 값이 무엇이고 어디로 가는지가 바로 다음 장의 주제다. 이 질문 하나로 다음 장의 문이 이미 반쯤 열렸다.

17.4 미리 심는 주의 하나 — 계산의 시간 순서는 다를 수 있다

우선순위에 관해, 지금은 씨앗만 심어 둘 주의가 하나 있다. 우선순위는 “누가 누구의 **재료**인가”라는 묵기의 규칙이지, 기계가 **시간상 무엇을 먼저 계산하는가**가 아니다 — 이를테면 덧셈의 왼쪽 재료와 오른쪽 재료 중 어느 쪽을 먼저 계산할지는 표준이 정해 두지 않았고, **컴파일러마다 다를 수 있다.** $2 + 3$ 같은 수식에서야 아무래도 좋지만, 재료의 계산이 출력 같은 흔적을 남기는 경우에는 그 흔적의 순서가 달라질 수 있다는 뜻이다.

다만 안심할 것 — **문장과 문장 사이의** 순서는 철석같이 보장된다. 16장의 “위에서 아래로 한 문장씩”은 계약이다. 흔들릴 수 있는 것은 한 문장 안쪽뿐이고, 그 안쪽의 정확한 규칙(부수효과, 시퀀스 포인트라는 개념)은 재료가 더 갖춰진 뒤 29장에서 정면으로 다룬다. 지금의 실전 수칙은 하나면 된다: **순서가 중요한 일들은 한 문장에 옥여넣지 말고 문장을 나눈다.**

정리하면 — 소스에 값을 적는 표기가 리터럴이고, 계산되어 값이 되는 것이 수식이며, 묵는 순서는 산술 관례와 괄호로 다스리되 한 문장 안의 계산 시간 순서는 달라질 수 있다. 다음 장에서는 값을 **소비하고 생산하는** 가장 중요한 장치 — 함수를 부르는 법 — 를 익힌다. 헬로 월드의 심장 `printf(...)`가 드디어 정면으로 다뤄진다.

18 함수 사용 — 부르는 법

이 장이 끝나면

함수를 만드는 법은 아직 멀었다(21장). 이 장은 부르는 법이다 — 호출, 인자, 반환값이라는 세 낱말을 익히고 나면, 헬로 월드의 심장 `printf("안녕, 세상!\n");` 이 완전한 문장으로 읽힌다. 남이 만든 일꾼을 부려 쓰는 것 — 프로그래밍의 절반은 사실 이것이다.

돌아보기 17장 끝에서 “함수를 부르는 것 자체가 수식이고, 따라서 값이 된다”고 했다. 그러면 `printf("%d\n", 2 + 3 * 4)` 안의 수식 `2 + 3 * 4`가 계산된 값 14는 — 정확히 어디로 가는 것인가?

답 함수의 입으로 들어간다. 함수를 부를 때 괄호 안에 적는 수식들의 값이 그 함수에게 **재료**로 전달되고, 함수는 그 재료로 제 일을 한 뒤 **결과 값**을 내놓는다. 재료를 인자, 결과를 반환값이라 부른다 — 이 두 낱말이 이 장의 전부다.

18.1 호출 — 이름을 부르면 일꾼이 된다

함수는 **이름 붙은 일꾼**이다. 어딘가에 일하는 방법(몸통)이 만들어져 있고, 우리는 이름만 부르면 된다. 부르는 문법은 이미 눈에 익었다:

이름(재료1, 재료2, ...)

이것이 **호출**(call)이다. 호출이 실행되는 순서는 이렇다 — ① 괄호 안의 수식들이 먼저 평가되어 값이 되고, ② 그 값들이 함수에게 전달되며, ③ 함수의 몸통이 실행되고, ④ 함수가 내놓은 **반환값**이 호출 자리의 값이 된다. 마지막 항목이 17장의 복선 회수다: **호출은 수식이다** — 호출이 적힌 자리는, 실행이 끝나면 반환값 하나로 바뀐다고 읽으면 된다.

호출이 수식이므로, 호출을 호출의 재료로 넣을 수도 있다. 시연한다 — `abs`는 표준 라이브러리의 일꾼으로, 정수 하나를 받아 그 절댓값을 돌려준다(`<stdlib.h>` 도구 상자에 산다 — 그래서 `#include`가 하나 더 붙었다).

```
examples/ch18/call.c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    printf("%d\n", abs(2 - 10)); /* 호출의 결과가 다시 호출의 재료가 된다 */
    return 0;
}
```

실행 결과

8

읽는 순서 그대로다 — 안쪽 수식 `2 - 10`이 먼저 `-8`이 되고, 그것이 `abs`의 재료로 들어가 반환값 8이 되고, 그 8이 다시 `printf`의 재료가 되어 찍혔다. **안에서 바깥으로, 값이 릴레이된다** — 수식과 호출이 겹겹이 쌓이는 C 코드를 읽는 기본 독법이다.

18.2 반환값 — 쓰거나, 버리거나

모든 호출은 값이 된다고 했다. 그러면 헬로 월드의 `printf("안녕, 세상!\n");` 에서 반환값은 어디로 갔는가.

버려졌다 — 그리고 그것은 합법이다. 수식 뒤에 세미콜론을 찍으면 “수식을 평가하고, 값은 버리는” 문장(수식문)이 된다. `printf`도 반환값이 있다(찍은 글자 수를 돌려준다). 다만 우리는 대개 찍는 **부수 효과**가 목적이고 글자 수는 관심 밖이라, 값을 버리는 것이다. “함수를 부르는 문장”이란 사실 “호출 수식 + 값 버리기”였던 셈이다.

문 버려도 되는 값이라면 애초에 왜 돌려주는가?

답 쓰는 사람이 따로 있기 때문이다. 같은 일꾼이라도 부르는 쪽의 사정에 따라 결과가 필요할 수도, 부수 효과만 필요할 수도 있다 — 함수는 일단 결과를 내놓고, 쓸지 버릴지는 부르는 쪽이 정한다는 것이 C의 분업이다. 다만 이 관대함에는 그림자가 있다 — 반환값이 “성공했는지”를 알리는 함수라면, 버리는 순간 실패를 놓친다. 오류를 값으로 알리는 C의 방식과 “버리면 안 되는 반환값”의 이야기는 41장의 주제다.

문 `abs`는 항상 양수를 돌려주는가? 절댓값이니 당연할 것 같은데.

답 거의 항상 — 그런데 정확히 한 곳에 함정이 있다. 5장에서 2의 보수의 유일한 비대칭을 배웠다: 가장 작은 음수(-128, 32비트라면 약 -21억)의 짝이 되는 양수가 **없다**. 그러면 그 수의 절댓값을 물으면 어떻게 되는가 — 답을 그릇이 없으므로, 계약 밖(정의되지 않은 동작)이다. “당연히 되겠지”의 딱 한 칸 바깥에 함정이 있는 것 — 5장의 비대칭이 실제 함수의 사용 설명서에까지 흔적을 남긴 사례다. 이런 경계 사냥은 42장의 주특기가 된다.

18.3 표준 라이브러리 — 미리 만들어진 일꾼들

`printf`도 `abs`도 우리가 만들지 않았다. C를 설치하면 따라오는 **표준 라이브러리** — 14장의 링크 단계에서 우리 프로그램에 이어지던 바로 그 꾸러미 — 의 일꾼들이다. 도구 상자(<stdio.h>, <stdlib.h> 등)마다 쓸 수 있는 일꾼의 명단(선언)이 들어 있고, `#include`로 명단을 가져오면 그 상자의 일꾼들을 부를 수 있게 된다.

표준 라이브러리에 어떤 상자가 있고 무엇이 들었는지는 43장에서 지도를 펴고 훑는다. 지금 챙길 것은 감각 하나다 — **프로그래밍의 절반은 남이 만든 함수를 부르는 일이다**. 잘 만들어진 일꾼을 알아보고 바르게 부르는 것이 코드의 절반을 결정하고, 그래서 이 책의 후반(35장)에서 “어떤 일꾼을 쓸 것인가”가 다시 큰 주제가 된다.

이제 헬로 월드에서 남은 칸은 둘뿐이다 — `printf`의 따옴표 안 서식이 정확히 어떻게 동작하는지(다음 장), 그리고 `int main(void)`와 `return 0`의 정체(제5부). 다음 장에서 출력을 정면으로 다루며 제4부의 목표에 바짝 다가선다.

19 출력

이 장이 끝나면

printf를 정면으로 다룬다 — 서식 문자열이라는 틀, %d와 %s라는 빈칸, 그리고 서식과 재료의 짝 맞추기까지. 이 장이 끝나면 제4부의 목표가 달성된다: 헬로 월드 한 편이 (두 줄의 외상만 남기고) 전부 아는 문장이 된다.

돌아보기 8장에서 프린터 터미널은 **행 버퍼링** — 종이에 찍기 전에 한 줄이 모일 때까지 기다리는 것 — 이 자연스러웠다고 했다. 오늘의 printf에도 그 버릇이 남아 있는가?

답 남아 있다 — 표준 출력이 터미널로 이어져 있을 때, 출력은 보통 \n을 만날 때까지 내부 창고(버퍼)에 모였다가 한 줄씩 흘러 나간다. 16장의 시연에서 printf("Hello, ")의 글자들이 두 번째 호출의 \n이 오고서야 화면에 나타났던 것이 바로 이것이다. 종이 타자기의 사정이 사라진 지 반세기 뒤에도, 그 리듬은 소프트웨어의 기본값으로 살아 있다 — 이 버릇이 실전에서 일으키는 소동(출력이 안 보여요!)과 다스리는 법은 입출력을 본격적으로 다룰 때 만난다.

19.1 printf — 서식이 있는 출력

printf라는 이름은 **print formatted** — “서식대로 찍어라” — 의 줄임이다. 첫 재료가 **서식 문자열** (format string)이고, 나머지 재료들이 서식의 빈칸에 끼워질 값들이다:

printf(서식 문자열, 값1, 값2, ...)

서식 문자열은 틀이다. 보통 글자는 그대로 흘러 나가고, %로 시작하는 자리 — **변환 지정** (conversion specification) — 만 빈칸으로 취급되어, 뒤따르는 값이 글자로 바뀌어 끼워진다. 이 책의 최소 집합은 셋이다:

- %d — 정수 값을 십진법 글자로 바꿔 끼운다 (decimal).
- %s — 문자열을 그대로 끼운다 (string).
- %% — 빈칸이 아니라, 퍼센트 글자 자체를 찍고 싶을 때.

시연한다. 빈칸 둘에 값 둘이 순서대로 끼워진다:

```
examples/ch19/fmt.c
#include <stdio.h>

int main(void)
{
    printf("The answer to %s is %d.\n", "life, the universe and everything", 6 * 7);
    return 0;
}
```

실행 결과

The answer to life, the universe and everything is 42.

%s 자리에 문자열 리터럴이, %d 자리에 수식 6 * 7의 값이 들어가 문장이 완성됐다. 서식은 틀, 재료는 순서대로 — 이것이 printf 읽기의 전부다. (변환 지정에는 자릿수 맞춤 같은 잔가지 선택지가 많지만, 필요할 때마다 하나씩 소개한다 — 전체 목록은 부록의 참조 자료다.)

문 화면에 찍힌 것이 왜 곧바로 안 보일 때가 있는가 — 프로그램이 멈춘 것처럼 보이다가 한꺼번에 쏟아지는 일이 있는데.

답 돌아보기에서 본 버퍼링 때문이다. 출력이 터미널로 갈 때는 줄 단위로 흘려보내지만, 파일이나 다른 프로그램으로 갈 때(8장의 리다이렉션)는 더 큰 덩어리가 찰 때까지 모았다가 한꺼번에 내보낸다 — 왕복 횟수를 줄이는 성능 요령이다. 그래서 진행 상황을 찍는 프로그램이 파이프를 이어지면 “멈춘 듯 보이다 몰아서 출력”되는 일이 생긴다. 급히 내보내야 할 때는 `fflush(stdout)`으로 창고를 비우고 지시할 수 있고, 프로그램이 정상 종료하면 남은 것은 자동으로 비워진다 — 다만 갑자기 죽으면 그 창고의 내용이 사라진다는 것이, 사고 직전의 로그가 안 남는 흔한 이유다.

19.2 짝 맞추기 — 서식과 재료의 계약

서식 문자열은 곧 **계약**이다. 빈칸의 개수와 종류가, 뒤따라야 할 재료의 개수와 종류를 약속한다. `%d` 두 개를 적었으면 정수 값 두 개가 따라와야 하고, `%s` 자리에는 문자열이 와야 한다.

약속을 어기면 — `%d` 자리에 문자열을 넣거나 재료 개수가 모자라면 — 계약 밖, 즉 정의되지 않은 동작이다(5장에서 만난 그 개념이 함수 사용 설명서에도 있는 것이다). 다행히 현대의 컴파일러는 15장에서 켜 둔 경고 (`-Wall`)로 이 어긋남을 대부분 잡아 준다 — `printf`의 서식 검사는 경고 기능 중에서도 가장 오래 되고 믿음직한 축이다.

● 서식 문자열이 무기가 된 사건 — format string 취약점

이 짝 맞추기 계약은 보안의 역사에도 등장한다. 사용자가 입력한 글을 그대로 찍고 싶을 때, `printf`(사용자입력)처럼 **입력을 서식 문자열 자리에** 넣은 프로그램들이 있었다. 입력이 평범한 글이면 문제가 없지만, 악의적인 사용자가 `%` 기호들을 섞어 보내면 — `printf`는 그것을 빈칸으로 해석하고, 있지도 않은 재료를 찾아 메모리를 뒤지기 시작한다. 이 원리로 프로그램의 내부를 훑쳐보거나 망가뜨리는 공격이 2000년대에 대유행했고, “format string 취약점”이라는 고유한 이름까지 얻었다. 올바른 코드는 `printf("%s", 사용자입력)` — **입력을 서식이 아니라 재료로 두는 것이다**. 데이터와 틀을 섞지 말 것 — 이 원칙은 보안 전반의 후렴구이기도 하다.

△ “printf는 화면에 찍는 함수다”

실용적으로는 대개 맞지만, 정확히는 아니다 — `printf`는 **표준 출력 스트림에 흘려보내는** 함수다(8장, 13장). 그 띠의 끝이 터미널이면 화면에 보이고, 운영체제의 기능으로 띠를 파일로 돌려 두면(리다이렉션) 같은 프로그램이 파일에 쓴다. 프로그램은 자신의 출력이 어디로 가는지 모르고, 알 필요도 없다 — 이 무심함이야말로 스트림 설계의 힘이다. 하나의 프로그램이 고치지 않고도 화면용·파일용·다른 프로그램의 입력용 으로 두루 쓰이는 것이 전부 이 덕분이다.

19.3 제4부를 닫으며 — 헬로 월드 재독

약속을 지킬 시간이다. 13장의 여섯 줄을 다시 실행한다.

```
#include <stdio.h>
```

```
int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

읽어 보면 — `#include`는 전처리 지시로 `stdio` 도구 상자의 명단을 붙여 넣고(14·16장), `main`의 블록이 문장 목록을 묶으며(16장), `printf` 호출이라는 수식문이 문자열 리터럴을 재료로 표준 출력에 글을 흘려보내고 (17·18·19장), `\n`이 줄을 바꾼다(8장). 처음 만났을 때 “주문”이던 것이 거의 전부 아는 문장이 됐다.

남은 외상은 정확히 두 줄이다 — `int main(void)`의 `int`와 `(void)`, 그리고 `return 0`. 이 둘은 선언의 세계, 즉 이름과 타입을 스스로 만드는 법을 알아야 풀리고, 그것이 바로 다음 부다. 다음 부의 끝에서 이 외상은 완전히 청산된다 — 그리고 그때, 드디어 입력도 받게 된다.

제5부 — 선언: 이름을 만드는 법

20 변수 선언

이 장이 끝나면

제5부의 주제는 “이름을 만드는 법”이다. 그 첫걸음으로 변수를 선언한다 — 타입과 이름을 약속하고, 값을 담고, 바꾸는 것까지. 연산자 하나가 새로 합류한다: 대입 `=`. 그리고 이 `=`가 수학의 등호가 **아**니라는 것이 이 장의 가장 중요한 문장이다.

돌아보기 3장에서 메모리는 번호(주소) 붙은 사물함들이라 했다. 그런데 지금까지 쓴 C 코드에는 주소가 한 번도 등장하지 않았다 — 그러면 프로그램은 사물함을 어떻게 쓰고 있었던 것인가?

답 이름은 통해서다. C에서 **변수**를 선언한다는 것은 “사물함 몇 칸을 잡고, 거기에 사람이 읽을 이름을 붙이는” 일이다. 번호(주소)는 컴파일러가 관리하고, 우리는 이름으로 부른다 — 이름은 주소의 사람용 별명이다. 이 장에서 그 이름을 만드는 법을 배우고, 별명 뒤의 진짜 주소를 다시 만나는 것은 제7부(30장)다.

20.1 선언 — 타입, 이름, 그리고 첫 값

변수 선언의 문법은 이미 여러 번 스쳐 본 모양이다:

```
int apples = 12;
```

세 부분으로 읽는다 — **타입**(int), **이름**(apples), 그리고 **초기화** (= 12). 뜻은: “정수를 담는 칸을 잡고, apples라 부르기로 하고, 첫 값으로 12를 담아라.”

타입은 두 가지를 약속한다. 첫째, **값의 집합** — int는 부호 있는 정수의 그릇으로, 흔한 기계에서 32비트, 즉 5장의 2의 보수 세계에서 약 ± 21 억 범위다. double이라 적으면 6장의 64비트 부동소수점 그릇이 된다. 둘째, **연산의 약속** — 그 이름에 무엇을 해도 되는지(정수 산술인지 근사 산술인지)가 타입에서 나온다. 3장에서 “덩어리를 아는 것은 읽는 쪽”이라 했는데, C에서는 **타입이 바로 그 읽는 눈**이다.

이름은 자유롭게 짓되 문법이 정한 틀 안에서다 — 글자·숫자·밑줄로 이루어지고 숫자로 시작할 수 없다. 좋은 이름은 문법이 아니라 사람을 위한 것이다: a보다 apples가 반년 뒤의 자신에게 친절하다.

초기화는 관행이 아니라 이 책의 규칙이다. **선언할 때 반드시 첫 값을 담아라**. 초기화 없이 선언만 한 변수의 칸에는 그 자리에 우연히 남아 있던 쓰레기 비트가 들어 있고(3장 — 칸은 언제나 무언가로 차 있다), 그것을 읽는 것은 대표적인 사고 경로다. 정식 취급은 35장에서 하되, 습관은 지금부터 들인다.

20.2 대입 — 상태를 바꾸는 부수효과

선언된 변수의 값은 **대입**(=)으로 바꾼다. 17장의 배분표에 예고됐던 연산자가 드디어 합류하는 것이다. 시연부터:

```
examples/ch20/var.c
#include <stdio.h>

int main(void)
{
    int apples = 12;           /* 선언과 동시에 초기화한다 */

    printf("apples: %d\n", apples);
    apples = apples + 3;       /* 새 값 = 지금 값 + 3 */
    printf("apples: %d\n", apples);
    return 0;
}
```

실행 결과

```
apples: 12
apples: 15
```

셋째 문장 `apples = apples + 3;`을 정확히 읽는 것이 이 장의 핵심이다. 읽는 순서는 — ① 오른쪽 수식 `apples + 3`을 평가한다(지금 값 12를 읽어 15를 얻는다), ② 그 값을 왼쪽 이름의 칸에 **담는다**(원래 있던 12는 덮여 사라진다). 즉 `=`는 “같다”가 아니라 **“오른쪽을 계산해 왼쪽에 담아라”**라는 지시다. 값이 바뀌는 것 — 이것이 17장에서 예고한, 출력 말고 두 번째로 만나는 **부수효과**다: 대입은 변수의 **상태**를 바꾼다.

△ “`x = x + 1`은 말이 안 되는 식이다 — 어떤 수도 자기 더하기 1과 같을 수 없으니까”

수학의 눈으로는 백번 옳다 — 등식으로 읽으면 해가 없는 식이다. 이 오개념의 뿌리는 `=` 기호의 재 활용이다. C의 `=`는 등호가 아니라 **담기 지시**라서, `x = x + 1`은 “`x`의 지금 값에 1을 더해, 그 결과를 `x`에 도로 담아라” — 즉 `x`를 1 키우라는 멀쩡한 명령이다. 오른쪽의 `x`는 **읽을 때의 값**(옛 값)이고 왼쪽의 `x`는 **담을 곳(칸)**이라는 것 — 같은 이름이 자리에 따라 값과 장소로 다르게 읽히는 이 구별이 C 읽기의 기본기다. 참고로 “같은가?”를 묻는 진짜 비교는 다른 기호(`==`, 26장)가 맡는다 — 기호를 나눠 쓴 이유가 바로 이 혼동 때문이다.

20.3 const — 바꾸지 않겠다는 약속

바꿀 일이 없는 값에는 선언에 `const`를 붙인다:

```
const int max_floor = 63; /* 이 이름의 값은 앞으로 바뀌지 않는다 */
```

`const`가 붙은 이름에 대입하려 하면 컴파일러가 오류로 막는다. 이것은 자물쇠라기보다 **문서**다 — “이 값은 변하지 않음”을 컴파일러와 다음 읽는 사람 모두에게 알리는 것이다. 바뀌지 않는 것이 많은 코드일 수록 읽기 쉽고 사고가 적다 — 그래서 현대적 관행은 “일단 `const`로 선언하고, 바뀌야 할 것만 뺀다”에 가깝다. 이 책의 예제도 그 감각을 따른다.

문 타입 이야기에서 `int`가 “약 ±21억”이라 했는데 — 정확한 크기가 기계마다 다르다는 뜻인가? 5장에서 배운 8비트, 16비트 이야기는 어디로 갔는가?

답 정확한 관찰이다. `int`의 크기는 표준이 “최소한 이만큼”만 정하고 구체 크기는 플랫폼에 맡긴 타입이다 — 지금 주류 환경에서는 32비트가 사실상의 표준이다. 5장의 8비트·16비트 그릇들, 그리고 “크기를 정확히 못박고 싶을 때”의 타입들(`int32_t` 같은)은 23장에서 정수 타입 가족 전체를 소개하며 정리한다. 이 부에서는 `int` 하나로 충분하다 — 나선행 원칙대로, 가족 상봉은 필요해질 때 한다.

이름을 만들었고, 값을 담았고, 바꾸는 법도 배웠다. 그런데 13장부터 끌고 온 외상 장부에는 아직 두 줄이 남아 있다 — `int main(void)`와 `return 0`. 둘 다 **함수**의 문법이다. 변수에 이름을 붙였으니, 다음 장에서는 **일**에 이름을 붙인다 — 함수를 스스로 만드는 법, 그리고 외상 장부의 완전한 청산이다.

21 함수 선언과 정의

이 장이 끝나면

18장에서 함수를 부르는 법을 배웠다 — 이제 만드는 법이다. 정의와 선언(원형)의 구별, 매개변수와 return, 스코프의 첫걸음까지. 그리고 이 장의 끝에서, 13장부터 끌고 온 외상 — `int main(void)`와 `return 0` — 을 완전히 청산한다.

돌아보기 14장에서 “컴파일 단계는 선언(예고)만 있으면 만족하고, 몸통을 찾아 있는 것은 링커”라고 했다. 그때의 “선언”과, 20장에서 배운 “변수 선언”은 같은 낱말을 쓴다 — 우연인가?

답 우연이 아니다. C에서 선언이란 언제나 같은 일 — “컴파일러에게 이름과 그 생김새(타입)를 미리 알리는 것” — 이다. 변수 선언은 값의 이름을, 함수 선언은 일의 이름을 알린다. 이 장에서 그 두 번째 종류를 만들고 나면, `#include`가 가져오던 것의 정체(선언들의 묶음)까지 한 줄로 꿰어진다.

21.1 정의 — 일꾼을 만드는 문법

18장의 `abs`나 `printf` 같은 일꾼을 이제 직접 만든다. 함수 정의의 문법은 이렇다:

반환타입 **이름**(매개변수 목록)
{
 몸통 - 문장들의 목록
}

시연부터 보면 빠르다. 정수를 받아 그 제곱을 돌려주는 일꾼이다:

```
examples/ch21/fn.c
#include <stdio.h>

int square(int n)    /* 정수 하나를 받아 정수 하나를 돌려주는 일꾼 */
{
    return n * n;
}

int main(void)
{
    printf("%d\n", square(12));
    return 0;
}
```

실행 결과

144

`square`의 정의를 부분별로 읽는다.

- **반환타입** `int` — 이 일꾼이 내놓는 결과가 정수라는 약속이다.
- **매개변수** `int n` — 재료를 받는 **변수 선언**이다(20장의 문법 그대로). 호출될 때, 재료의 값이 이 변수에 담긴 채 몸통이 시작된다. `square(12)`라 부르면 `n`에 12가 담기고 몸통이 돈다.
- **return 문** — 결과를 내놓고 **즉시 퇴장**한다. `return n * n;`은 수식을 평가해 그 값을 호출한 쪽에 돌려준다 — 18장에서 배운 “호출 자리가 반환값으로 바뀐다”의 공급자 쪽 모습이다.

이것으로 호출의 그림이 완성됐다. 18장은 소비자의 눈(부르기)이었고 이 장은 생산자의 눈(만들기)이다 — `printf("%d\n", square(12))`에서 값 12가 `n`으로, $n * n$ 의 144가 호출 자리로, 다시 `printf`의 재료로 흐르는 릴레이 전체가 이제 보인다.

21.2 선언 — 계약 서명만 미리

위 예제에서 `square`의 정의는 `main`보다 위에 있다. 우연이 아니다 — 컴파일러는 파일을 위에서 아래로 읽으므로(14장), `main` 안의 `square(12)`를 만나는 시점에 `square`가 무엇인지(재료와 결과의 타입)를 이미 알아야 호출이 맞는지 검사할 수 있다.

그런데 정의 전체를 미리 보여 줄 필요는 없다. **계약 서명만** 미리 알리면 된다 — 그것이 함수 선언, 관용적으로 **원형**(prototype)이다:

```
int square(int n); /* 선언: 이런 일꾼이 (어딘가에) 있다 */
```

몸통 없이 서명과 세미콜론뿐이다. 파일 위쪽에 원형을 적어 두면 정의는 아래 어디에 있어도 되고 — 더 중요하게는 — **다른 파일에** 있어도 된다. 14장의 수수께끼가 여기서 완전히 풀린다: `#include <stdio.h>`가 붙여 넣던 수만 줄의 정체가 바로 이런 **원형들의 묶음**이다. `printf`의 계약 서명을 컴파일러에게 미리 보여주는 것 — 그것이 헤더 파일의 일이고, 몸통은 링커가 있다(정의와 선언의 분업이 여러 파일 프로젝트로 자라는 이야기는 43장이다).

이 원형이라는 발명품의 나이도 이제 말할 수 있다 — 10장에서 본 대로, C89가 표준화하며 들여온 것이다. 그전의 C는 재료의 타입을 검사하지 않았고, 틀린 재료로 부른 호출이 소리 없이 통과해 사고가 났다. 원형은 “계약을 컴파일 시점에 검사한다”는, C가 계약의 언어로 자라는 첫걸음이었다.

● 원형 없는 호출이 남긴 상처 — 암묵 선언

원형이 없던 시절(10장)의 C에는 관대한 규칙이 하나 있었다. 선언한 적 없는 이름을 함수처럼 부르면, 컴파일러가 “정수를 돌려주는 함수겠거니” 하고 **멋대로 선언을 지어내** 통과시킨 것이다(암묵 선언). 편해 보이지만 결과는 참혹했다 — 실제로는 실수를 돌려주는 함수를 정수 함수로 오해하거나, 포인터를 받는 함수에 정수를 넘겨도 아무 경고 없이 컴파일 됐다. 특히 정수와 포인터의 크기가 다른 기계로 옮길 때 조용히 무너지는 코드가 쏟아졌다 — 32비트에서 64비트로 넘어가던 시기에 이 부류의 버그가 대거 드러났다. C99가 마침내 암묵 선언을 표준에서 제거했고, 오늘의 컴파일러는 이를 오류로 잡는다. “쓰기 전에 선언한다”는 규칙이 성가신 격식이 아니라 사고의 방벽이라는 것 — 반세기의 상처가 남긴 교훈이다.

21.3 스코프 — 이름이 보이는 범위

`square`의 `n`과 `main`의 어떤 이름이 부딪히지 않을까 — 걱정할 필요가 없다. **블록 안에서 선언된 이름은 그 블록 안에서만 보인다**. 이 보이는 범위를 **스코프**(scope)라 한다. `n`은 `square`의 몸통에서만 존재하고, `main`은 그 이름을 아예 모른다. 함수마다 자기만의 작업대가 있는 셈이라, 일꾼들이 서로의 도구를 건드릴 걱정 없이 이름을 자유로이 짓는다 — 프로그램이 커져도 무너지지 않는 비결의 절반이 이 칸막이다. (작업대의 더 깊은 사정 — 이름의 수명, 함수 사이에 값이 어떻게 오가는지의 정밀한 그림 — 은 29·36장에서 이 어진다.)

21.4 외상 청산 — `int main(void)`

이제 이 순간을 위해 아껴 둔 일을 한다. 13장의 헬로 월드에서 마지막까지 “주문”으로 남겨 둔 두 줄을, 오늘 배운 문법으로 다시 읽는다.

```
int main(void)
{
```

```
...
```

```
    return 0;
}
```

이것은 그냥 **함수 정의**다. 이름이 `main`이고, 매개변수 목록의 `void`는 “재료 없음”을 명시하는 표기이며, 반환타입은 `int` — 즉 “재료 없이 일하고 정수 하나를 내놓는 일꾼”이다. 특별한 것은 문법이 아니라 **호출자**다: `main`을 부르는 것은 우리 코드가 아니라 **운영체제**다. 프로그램 실행이란 운영체제가 `main`을 호출하는 일이고, `return 0;`은 그 호출자에게 결과값을 돌려주는 것이다 — 관례로 0이 “무사히 마쳤음”, 0 아닌 값이 “문제 있었음”이다. 그 값은 버려지지 않는다 — 운영체제와 셸이 받아서, 프로그램들을 이어 붙인 자동화가 성공·실패를 가리는 데 쓴다(8장의 스트림처럼, 이것도 프로그램을 부품으로 잇는 장치다).

13장의 외상 장부가 이것으로 **전부 청산되었다**. 헬로 월드의 여섯 줄에 이제 모르는 것이 없다 — 전처리 지시, 함수 정의, 블록, 호출 수식문, 문자열 리터럴, 서식, 반환까지. 입문의 첫 산 하나를 넘은 것이다.

문 `main`도 함수라면, 우리가 `main()`이라고 직접 불러도 되는가?

답 문법상 가능하지만 하지 않는 것이 관행이고, 할 이유도 없다 — `main`은 “운영체제가 부르는 입구”라는 역할이 본질이라서, 코드 안에서 다시 부르는 것은 건물의 정문을 방 안에 또 만드는 격이다. 재사용하고 싶은 일이 있다면 그 일을 별도 함수로 빼서 부르면 된다 — 사실 그것이 함수를 만드는 이유의 전부다: **이름 붙일 가치가 있는 일에 이름을 붙이는 것**.

이름 붙은 값(20장)과 이름 붙은 일(21장)을 다 갖췄다. 이제 19장에서 미뤄 둔 마지막 약속을 지킬 수 있다 — 값을 담을 곳이 생겼으니, 드디어 **입력**이다.

22 입력

이 장이 끝나면

19장의 예고를 완결한다 — 저장할 곳(변수)이 생겼으니 입력을 받는다. 이 책이 처음부터 가르치는 방식은 현대적 관행 그대로다: **한 줄을 통째로 읽고, 그 줄을 해석한다**. 읽기와 해석의 분리 — 이 두 단계 구조가 편리함과 안전함의 원천이다. 제5부가 여기서 완성된다.

돌아보기 8장에서 입력도 출력처럼 “문자가 한 줄씩 흐르는 때”라 했고, 사람의 입력은 엔터로 끝나는 줄 단위가 자연스럽다고 했다(행 버퍼링). 그러면 프로그램이 입력을 받는 자연스러운 단위도 — 줄인가?

답 바로 그렇다 — 그리고 그것이 이 장의 설계 원리다. 사람은 한 줄을 적고 엔터를 친다. 그러니 프로그램도 **한 줄을 통째로** 받아 온 다음, 그 안에서 필요한 값을 찾아 읽는 것이 흐름의 결을 따르는 방식이다. 글자의 띠에서 줄을 뜨는 일과, 뜬 줄을 해석하는 일 — 이 둘을 나누면 각각이 단순해진다.

22.1 두 단계 — 읽고, 해석한다

시연부터 본다. 표준 입력에서 정수 하나를 받아 제곱을 알려 주는 프로그램이다. (이 실행에서 표준 입력으로 준 내용을 가운데 상자에 보였다.)

```
examples/ch22/read.c
#include <stdio.h>

int main(void)
{
    char line[100]; /* 글자 100칸짜리 공간 - 정식 설명은 32장 */
    int n = 0;

    fgets(line, sizeof line, stdin); /* 1단계: 한 줄을 통째로 읽는다 */
    sscanf(line, "%d", &n);          /* 2단계: 그 줄에서 정수를 해석한다 */

    printf("%d squared is %d.\n", n, n * n);
    return 0;
}
```

표준 입력으로 준 것

7

실행 결과

7 squared is 49.

새 얼굴이 둘 있다 — 한 단계에 하나씩이다.

1단계, fgets — 한 줄을 통째로 읽는다. fgets(line, sizeof line, stdin)은 표준 입력(stdin이 그 이름이다)에서 한 줄을 읽어 line 이라는 공간에 담는다. 첫 줄의 char line[100];은 “글자 100칸짜리 공간을 잡고 line이라 부른다”는 선언인데 — 여러 칸짜리 공간(배열)의 정식 문법은 33장의 것이라, 지금은 “줄을 담는 그릇”이라고만 알아 두면 된다(의도된 외상이다). 둘째 재료 sizeof line은 “그릇의 칸 수” — fgets에게 그릇 크기를 알려서 **넘치게 담는 일이 없게** 하는 안전장치다 (sizeof 연산자의 정식 취급은 30장).

2단계, `sscanf` — **답아 온 줄을 해석한다**. `sscanf(line, "%d", &n)`은 19장 `printf`의 짝이다 — 같은 서식 언어를 **반대 방향**으로 쓴다. `printf`가 값을 글자로 바꿔 내보냈다면, `sscanf`는 글자(line 속 "7")에서 서식(`%d`)에 맞는 값을 찾아 **변수에 담는다**. 이름 앞의 `&`는 “값을 담을 변수의 **자리를** 알려 주는 표시”다 — 3장에서 배운 주소가 문법에 처음 얼굴을 내민 순간인데, 정식 취급은 30장에 있다 (이것이 제5부의 마지막 의도된 외상이다).

문 왜 두 단계로 나누는가? 입력에서 바로 `%d`를 읽어 주는 함수(`scanf`)도 있다고 들었는데.

답 있고, 많은 입문서가 그것부터 가르친다 — 이 책이 다른 길을 택한 이유는 두 가지다. 첫째, **실패의 뒤처리가 깨끗하다**. 해석에 실패해도 (숫자가 아닌 것이 들어와도) 줄은 이미 내 그릇에 있으므로, 입력의 띠가 어중간한 자리에서 멈춰 꼬이는 일이 없다 — 입력 띠에서 직접 읽다 실패하면 남은 글자들이 띠에 걸린 채 다음 읽기를 오염시키는데, 이것이 `scanf` 직접 사용의 고전적 골칫거리다. 둘째, **안전의 결이 같다**. 줄 읽기는 그릇 크기를 알려 주며 읽는 구조라 넘침이 원천 차단된다 — 그릇 크기를 말하지 않는 옛 방식들이 어떤 사고를 냈는지, 그리고 이 두 단계 관행이 어떻게 그 사고를 봉쇄하는지가 34장의 이야기다. 처음부터 안전한 결로 손에 익히는 것 — 그것이 이 책의 선택이다.

22.2 해석은 실패할 수 있다

출력과 달리, 입력에는 본질적인 새 문제가 하나 있다 — **상대가 무엇을 보낼지 모른다**. `r`을 기대했는데 일곱이 올 수 있다. 그래서 `sscanf`의 반환값은 **해석에 성공한 값의 개수**다 — 위 예제라면 성공 시 1, 실패 시 0이다.

지금의 우리는 이 반환값을 확인하고도 “성공이면 이 길, 실패면 저 길”로 갈라설 문법 — 분기 — 이 없다. 그래서 위 예제는 반환값을 버렸고, `n`을 0으로 초기화해 두는 것(20장의 습관)으로 실패 시의 바닥을 깔아 두었다. 갈림길은 다음 부(26장)에서 생기고, “실패를 값으로 알리고 반드시 확인한다”는 규율은 41장에서 정식으로 세운다 — 입력 검사는 그 규율의 첫 실전이 될 것이다.

△ “입력은 키보드에서 온다”

대개 그렇지만, 본질은 아니다 — 입력은 **표준 입력 스트림**에서 온다 (8장). 사실 방금의 시연이 그 증거다: 이 책의 예제 검증 기계에는 키보드를 두드리는 사람이 없다 — 위 실행의 입력 `r`은 파일에 담겨 표준 입력의 띠로 흘러 들어간 것이다(리다이렉션). 19장의 출력이 화면행이 아니듯 입력도 키보드행이 아니라서, 같은 프로그램이 사람과의 대화에도, 파일 처리에도, 프로그램들의 연결에도 고치지 않고 쓰인다. 스트림 설계의 힘이 입력 쪽에서도 반복되는 것이다.

22.3 제5부를 닫으며

이름을 만드는 법을 다 배웠다 — 값의 이름(20장), 일의 이름(21장), 그리고 바깥세상에서 값을 받아 이름에 담는 법(22장)까지. 13장의 외상 장부는 청산되었고, 새로 진 외상은 정확히 둘 — 줄 그릇(`char line[100]`, 33장에서)과 자리 표시(`&`, 30장에서) — 뿐이며, 둘 다 청산 기일이 적혀 있다.

무엇보다, 이제 프로그램이 **대화**할 수 있게 됐다. 받아서, 계산하고, 답한다 — 다음 부에서는 그 계산 자체를 버린다: 정수의 세계를 정면으로 다루고(23·24장), 비교하고 판단하고(25·27장), 반복하고(28장), 함수의 의미를 완성한다(29장). 값과 흐름의 부다.

제6부 — 값과 흐름

23 정수 — 유한한 수의 세계

이 장이 끝나면

제6부는 값을 버리고 흐름을 다스리는 부다. 첫 장은 C의 정수를 정면으로 다룬다 — 5장에서 배운 표현의 세계가 C의 타입 가족으로 어떻게 나타나는지, 오버플로가 실무에서 어떤 얼굴인지, 그리고 현대 C의 관행 (<stdint.h>)까지.

돌아보기 5장에서 부호 없는 정수의 넘침은 “정의된 감아 돌기”이고 부호 있는 넘침은 계약 밖이라 했다. 그런데 20장에서 만든 `int`는 부호가 있는가 없는가 — 우리가 여태 쓴 변수들은 어느 세계에 있었던 것인가?

답 `int`는 부호 있는 정수다 — 즉 여태 우리의 변수들은 “넘치면 계약 밖”의 세계에 있었다. 다행히 지금까지의 예제는 ± 21 억의 그릇을 넘볼 일이 없었지만, 이제 그 경계선을 정확히 그어 둘 때가 됐다. 그릇의 종류와 크기, 경계의 위치, 넘었을 때의 규칙 — 이 장의 내용이다.

23.1 정수 타입 가족

C의 정수 타입은 `int` 하나가 아니라 가족이다. 부호 있는 쪽의 기본 구성원은 크기 순으로 `char`(1바이트 — 2장의 그 “바이트=문자” 사연이 이름에 남았다), `short`, `int`, `long`, `long long`이고, 각각의 앞에 `unsigned`를 붙이면 부호 없는 짝이 된다(`unsigned int` 등). 표준은 각 타입의 **최소** 크기만 정한다 — `int`는 최소 16비트, `long`은 최소 32비트라는 식이라, 정확한 크기는 플랫폼의 몫이다(주류 환경에서 `int`는 32비트).

“기계마다 다르다”가 불안하게 들린다면 정확한 직감이다 — 그래서 현대 C의 관행은 크기가 이름에 박힌 타입을 쓰는 것이다. <stdint.h> 도구 상자의 `int32_t`(정확히 32비트, 부호 있음), `uint8_t`, `int64_t` 같은 타입들이다. 크기가 계약의 일부인 자리 — 파일 형식, 통신, 3장의 엔디안이 문제 되는 모든 곳 — 에서는 이쪽이 표준 관행이다. C23은 여기에 `_BitInt(N)` — 원하는 비트 폭을 직접 적는 정수 — 까지 보탬다. 이 책의 예제는 지면의 단순함을 위해 `int`를 기본으로 쓰되, 크기가 중요한 장면에서는 <stdint.h> 가족으로 갈아탄다.

23.2 경계를 눈으로 본다

그릇의 가장자리는 <limits.h> 도구 상자가 알려 준다. 그리고 부호 없는 세계의 감아 돌기는 — 5장에서 지면으로 배운 그것 — 이제 직접 실행해 보일 수 있다.

```
examples/ch23/wrap.c
#include <limits.h>
#include <stdio.h>

int main(void)
{
    printf("INT_MAX = %d\n", INT_MAX);
    printf("UINT_MAX = %u\n", UINT_MAX);
    printf("UINT_MAX + 1 = %u\n", UINT_MAX + 1u); /* 정의된 감아 돌기 */
    return 0;
}
```

실행 결과

```
INT_MAX = 2147483647
UINT_MAX = 4294967295
UINT_MAX + 1 = 0
```

서식 하나가 새로 합류했다 — 부호 없는 정수는 %d가 아니라 %u로 찍는다(19장의 서식 계약이 타입 가족을 따라 늘어나는 것이다). 그리고 셋째 줄이 5장의 약속 그대로다: 최댓값에 1을 더하니 0 — 시계가 한 바퀴 돈 것이고, 이것은 **정의된** 동작이다.

부호 있는 쪽은 사정이 다르다. `INT_MAX + 1`을 계산하는 코드는 계약 밖(정의되지 않은 동작)이라서, **이 책의 검증 파이프라인에 넣을 수조차 없다** — 15장에서 장만한 UBSan이 정확히 이런 코드를 실행 중에 잡아 낸다. 같은 “+1”인데 한쪽은 시연 가능하고 한쪽은 불가능하다는 것 자체가, 두 세계의 차이를 웅변한다.

△ “오버플로는 큰 수를 다루는 특수한 프로그램의 문제다”

그렇듯하지만, 실제 사고 목록은 반대를 말한다. 가장 흔한 오버플로는 평범한 곳에서 난다 — 두 인덱스의 평균을 $(a + b) / 2$ 로 구하다가 **더하는 중간 결과**가 넘치고(유명한 이진 탐색 버그: 표준 라이브러리 들에 20년 가까이 숨어 있었다), 밀리초 타이머가 49일 만에 감아 돌고 (Windows 95의 49.7일 멈춤 사고), 크기 계산 `개수 * 크기`가 넘쳐 터무니없이 작은 메모리를 잡는다(보안 사고의 고전). 공통 점은 **최종 값이 아니라 중간 계산**이 넘친다는 것이다 — 그릇 걱정은 결과가 아니라 수식의 모든 중간 단계에 대해 하는 것이고, 이것이 proven류의 검사 산술이 존재하는 이유다(35장).

문 char가 정수 타입 가족이라는 것이 이상하다 — 문자 아닌가?

답 C에서 문자와 작은 정수는 같은 것이다 — 6장에서 배운 대로 문자는 번호이고, char는 그 번호를 담는 1바이트 정수 그릇일 뿐이다. 'A'라는 문자 리터럴의 값은 그냥 65다. 한 가지 함정만 미리 — char가 부호 있는지 없는지는 **플랫폼마다 다르다**(둘 다 아닌 제3의 타입이다). 그래서 수치 계산에는 char를 쓰지 않고, 바이트가 필요하다면 `uint8_t`를 쓰는 것이 현대 관행이다. 문자열과 char의 본격적인 이야기는 34장에 있다.

정수의 그릇들을 정리했다. 다음 장은 그 그릇 위의 연산 — 17장에서 미뤄 둔 나눗셈의 진실과, 5장에서 배운 비트 세계의 연산자들이 C 문법으로 합류한다.

24 정수의 연산 — 나눗셈, 비트

이 장이 끝나면

17장에서 미뤄 둔 약속을 지킨다 — 정수 나눗셈 /와 나머지 %의 진실, 특히 음수에서의 규칙까지. 그리고 5장에서 개념으로 배운 비트 연산들이 C의 연산자로 합류한다. 형변환의 첫걸음도 여기서 뚫는다.

돌아보기 17장에서 “7 / 2는 3.5가 아니다”라고 예고만 했다. 5장의 지식으로 스스로 답해 보면 — 정수 그릇끼리의 나눗셈 결과가 3.5일 수 없는 이유는 무엇인가?

답 3.5를 담을 자리가 없기 때문이다. 정수 타입의 값 집합에는 3과 4만 있고 그 사이가 없다 — 그래서 정수끼리의 나눗셈은 결과도 정수여야 하고, 소수 부분을 어느 쪽으로든 **버려야** 한다. 문제는 “어느 쪽으로”다 — 그 규칙이 이 장의 첫 절이다.

24.1 나눗셈과 나머지 — 버림의 방향

시연부터 본다.

```
examples/ch24/divmod.c
#include <stdio.h>

int main(void)
{
    printf(" 7 / 2 = %d,  7 %% 2 = %d\n", 7 / 2, 7 % 2);
    printf("-7 / 2 = %d, -7 %% 2 = %d\n", -7 / 2, -7 % 2);
    printf("invariant: (7/2)*2 + 7%%2 = %d\n", (7 / 2) * 2 + 7 % 2);

    printf("5 & 3 = %d, 5 | 3 = %d, 5 ^ 3 = %d\n", 5 & 3, 5 | 3, 5 ^ 3);
    printf("1 << 4 = %d\n", 1 << 4);
    return 0;
}
```

실행 결과

```
7 / 2 = 3,  7 % 2 = 1
-7 / 2 = -3, -7 % 2 = -1
invariant: (7/2)*2 + 7%%2 = 7
5 & 3 = 1, 5 | 3 = 7, 5 ^ 3 = 6
1 << 4 = 16
```

양수는 직관대로다 — 7 / 2는 3, 나머지는 1. 규칙이 필요한 곳은 음수다. C는 0을 향해 버린다 (truncation toward zero) — -7 / 2는 -3.5에서 0 쪽으로 버린 -3이고, 나머지는 그에 맞춰 -1이다. “수학의 몫·나머지(나머지는 항상 0 이상)”와 다른 선택이라서, 음수 나머지를 쓰는 코드는 이 차이를 모르면 사고가 난다.

Σ 나눗셈-나머지 복원 불변식

방향이 무엇이든, C가 반드시 지키는 등식이 하나 있다:

$$\left(\frac{a}{b}\right) \times b + (a \% b) = a$$

몫과 나머지는 이 등식을 만족하도록 **한 쌍으로** 정의된다 — 시연의 마지막 줄이 이 등식의 확인이다. 0을 향한 버림을 택하면 나머지의 부호가 피제수(a)를 따르게 되는 것도 이 등식의 귀결이다. 덧붙여 역사 한 줄 — C89까지는 버림의 방향이 구현마다 달라도 됐고, C99가 “0을 향해”로 못박았다. 5장에서 본 “기계들이 갈리면 표준은 비워 둔다”가, 기계들이 수렴하자 약속으로 승격된 또 하나의 사례다.

문 0으로 나누면 어떻게 되는가?

답 계약 밖 — 정의되지 않은 동작이다. 수학에서 정의되지 않은 것이 C에서도 정의되지 않은 드문 일치 사례인데, 결과는 수학처럼 암전하지 않다: 많은 기계에서 프로그램이 그 자리에서 붕괴하고, 어떤 최적화 아래서는 더 기묘한 일도 벌어진다(42장). 나누기 전에 0인지 확인하는 것은 프로그래머의 몫이고 — 27장에서 분기를 배웠으니 이제 그 확인을 코드로 적을 수 있게 된다.

24.2 비트 연산 — 5장의 세계, C의 문법

5장에서 개념으로 배운 비트 다루기가 연산자로 합류한다 — AND $\&$, OR $\mid$, XOR $\wedge$, 반전 $\sim$, 그리고 시프트 $\ll$, $\gg$. 시연의 뒷부분이 그 맛보기다: $5 \& 3$ 은 $101 \& 011 = 001$ 이라 1, $5 \mid 3$ 은 111 이라 7, $1 \ll 4$ 는 5장의 약속대로 $2^4 = 16$ 이다.

실무의 기본 무늬는 5장에서 예고한 그대로 **시프트+마스킹**다 — 원하는 자리로 밀고($\ll$, $\gg$), 필요한 비트만 남긴다($\&$). 다만 두 가지 수칙을 함께 챙긴다. 첫째, **비트 연산은 부호 없는 타입에서 한다** — 부호 있는 수의 시프트에는 5장에서 본 함정들(폭 이상, 음수 왼쪽 시프트=계약 밖)이 있어서, `unsigned`나 `uint32_t` 위에서 노는 것이 안전 관행이다. 둘째, $\&$ (비트 AND)와 $\&\&$ (논리 AND, 다음 장)는 완전히 다른 연산자다 — 한 글자 차이가 값 전체를 바꾼다.

24.3 형변환 — 그릇 사이를 건너다

타입 가족(23장)이 생기니 새 질문이 따라온다 — 다른 그릇끼리 섞어 계산하면 무슨 일이 일어나는가. C의 답은 **암묵 변환**이다: 작은 그릇의 값은 계산 전에 자동으로 더 큰 그릇으로 넓혀지고(5장의 부호 확장이 바로 이때 일어나는 일이다), 정수와 실수가 만나면 정수가 실수로 승격된다. 대부분은 뜻대로 되지만, 자동이라는 것은 **안 보인다는** 뜻이기도 하다 — 특히 부호 있는 것과 없는 것이 섞이는 비교는 고전적 함정이라(음수가 거대한 양수로 둔갑한다), 컴파일러 경고(-Wall)가 지켜 주는 대표 지점이다.

변환을 명시하고 싶으면 캐스트 표기를 쓴다 — $(double)7 / 2$ 는 “7을 실수 그릇으로 옮긴 뒤 나눠라”이므로 3.5가 된다. 규칙의 전모(정수 승격, 통상 산술 변환)는 부록 참조 자료로 두고, 본문 수칙은 둘이다: **섞이는 계산은 의도를 캐스트로 명시하고, 부호 섞인 비교는 피한다.**

정수의 그릇(23장)과 연산(24장)을 갖췄다. 다음 장부터는 흐름이다 — 비교하고 판단하는 값, 불리언부터.

25 암묵적 변환 — 승격과 통상 산술 변환

이 장이 끝나면

C는 타입이 다른 값들을 만나게 하면 **말없이** 변환한다. 이 장은 그 보이지 않는 변환들을 한자리에 모은다 — 정수 승격, 통상 산술 변환, 그리고 가변 인자의 기본 진급까지. 흩어 놓으면 각각이 수수께끼지만, 모아 놓으면 하나의 규칙 체계다.

돌아보기 5장에서 부호 확장(8→16비트 넓히기)을 배웠고, 24장 끝에서 “작은 그릇의 값은 계산 전에 자동으로 넓혀진다”고 스쳤다. 그러면 `char + char`의 결과 타입은 `char`인가?

답 아니다 — `int`다. 그리고 이 사실이 이 장의 출발점이다. C의 산술 연산은 **작은 정수 타입 위에서 일어나지 않는다**. 계산 전에 모두 `int`(또는 더 큰 타입)로 넓힌 뒤 계산하고, 결과도 그 넓은 타입이다. 왜 그런지, 어디까지 그런지가 이 장의 내용이다.

25.1 규칙 1 — 정수 승격

정수 승격(integer promotion): `char`, `short`, `bool`, 비트 필드처럼 `int`보다 작은 정수 타입의 값이 산술 연산에 참여하면, 계산 전에 **`int`로 넓혀진다**(`int`가 그 값들을 다 담을 수 있으면 `int`로, 아니면 `unsigned int`).

```
examples/ch25/conv.c
#include <stdint.h>
#include <stdio.h>

int main(void)
{
    /* 정수 승격: char·short는 계산 전에 int로 넓혀진다 */
    signed char a = 100;
    signed char b = 100;
    printf("100 + 100 (as chars) = %d\n", a + b);    /* 200 - char에는 안 담기는 값 */

    /* 통상 산술 변환: 부호 없는 쪽이 이긴다 -
       -1을 unsigned의 눈으로 읽으면 거대한 양수가 된다 */
    int neg = -1;
    printf("(unsigned)(-1)  = %u\n", (unsigned)neg);
    printf("so -1 < 1u is false\n");

    /* 정수 나눗셈 vs 실수 나눗셈 - 캐스트로 의도를 밝힌다 */
    int total = 7, count = 2;
    printf("7 / 2          = %d\n", total / count);
    printf("(double)7/2    = %.1f\n", (double)total / count);

    /* 가변 인자의 기본 진급: float는 double로, char/short는 int로 */
    float f = 1.5f;
    printf("float 1.5f via %f = %f (promoted to double)\n", f);
    return 0;
}
```

실행 결과

```
100 + 100 (as chars) = 200
(unsigned)(-1)  = 4294967295
so -1 < 1u is false
7 / 2          = 3
```



```
(double)7/2 = 3.5
float 1.5f via %f = 1.500000 (promoted to double)
```

첫 줄이 그 확인이다 — signed char 100끼리 더했는데 결과가 200이다. char에는 담기지 않는 값인데도 넘치지 않은 이유는, 덧셈이 char의 세계가 아니라 int의 세계에서 일어났기 때문이다.

이유는 기계에 있다(9장). CPU의 계산 회로와 레지스터는 워드 크기 근처의 정수를 다루도록 만들어져 있어서, 작은 타입만을 위한 별도 산술을 두는 것이 오히려 비효율이다. C는 그 현실을 언어 규칙으로 받아들였다 — 작은 타입은 **저장의 단위**이지 계산의 단위가 아니다.

25.2 규칙 2 — 통상 산술 변환

승격 뒤에도 두 피연산자의 타입이 다르면, **통상 산술 변환**(usual arithmetic conversions)이 공통 타입 하나를 정한다. 실무에서 기억할 순서는 이렇다.

- 한쪽이 실수면 실수 쪽으로 (long double > double > float).
- 둘 다 정수면 — 더 넓은 쪽으로. 폭이 같으면 **부호 없는 쪽이 이긴다**.

마지막 줄이 함정의 원천이다. 시연의 둘째 부분이 그 실물이다 — -1을 unsigned의 눈으로 읽으면 42억이 넘는 거대한 양수가 된다(5장의 모듈러 세계 그대로다). 그래서 $-1 < 1u$ 같은 비교는 직관과 반대로 거짓이 된다. 배열 인덱스나 크기 계산(sizeof의 결과는 부호 없는 size_t다!) 에서 부호 있는 값과 섞이면 조용히 사고가 나는 이유가 이것이다.

다행히 이 함정은 컴파일러가 잘 지킨다 — 15장에서 켜 둔 경고가 부호 섞인 비교를 지목해 준다(이 책의 예제 하나도 그 경고에 걸려 다시 쓰였다). 수칙으로 줄이면: **부호 있는 것과 없는 것을 한 비교에 섞지 않는다**. 크기·인덱스는 size_t 계열로 일관되게 쓰거나, 명시적 캐스트로 의도를 드러낸다.

25.3 규칙 3 — 가변 인자의 기본 진급

세 번째 변환은 인자 개수가 정해지지 않은 함수 — printf 같은 **가변 인자 함수** — 에서 일어난다. 원형에 타입이 적혀 있지 않은 자리로 넘어 가는 인자에는 **기본 진급**(default argument promotions)이 적용된다: float는 double로, 작은 정수 타입은 int로 넓혀진다.

시연의 마지막 줄이 그 확인이다 — float 값을 %f로 찍었는데 잘 나온다. 서식 %f는 사실 double을 기대하는데, float 인자가 진급해 double이 되어 도착했기 때문이다(그래서 printf에는 float 전용 서식이 아예 없다). 19장에서 배운 “서식과 재료의 계약”에 이 진급 규칙이 숨은 조항으로 들어 있는 셈이다 — 이 계약의 전모와 가변 인자 함수를 직접 만드는 법은 45장에서 정면으로 다룬다.

● 변환 하나가 날린 로켓 — 아리안 5, 1996

암묵·명시 변환이 얼마나 무거울 수 있는지 보여 주는 사건이 있다. 1996년 유럽우주국의 아리안 5 로켓이 첫 발사 37초 만에 폭발했다. 조사 결과의 핵심은 한 줄의 변환이었다 — 관성 항법 장치가 수평 속도를 64비트 부동소수점으로 계산해 두고, 그 값을 **16비트 부호 있는 정수로** 옮겨 담는 코드가 있었다. 전작 아리안 4에서는 그 속도가 16비트 범위를 넘을 일이 없어 안전했지만, 더 빠른 아리안 5에서는 값이 그릇을 넘쳤다. 축소 변환의 실패(5장의 잘림)가 예외를 일으켰고, 그 예외가 처리되지 않은 채 항법 컴퓨터가 멈추자 로켓은 자세를 잃고 자폭했다. 손실은 수억 달러 규모였다. 재사용된 코드가 **새로운 값의 범위**를 만나는 순간 계약이 깨진 것 — “변환은 값을 바꾼다”는 이 장의 문장이 가장 비싸게 확인된 사례다.

△ “캐스트는 값을 바꾸지 않고 해석만 바꾼다”

포인터 캐스트(32장)에서는 대체로 맞는 말이지만, 산술 타입 사이의 캐스트는 값 자체를 바꾼다. (int)3.9는 3이 되고(소수부 버림), (char)300은 그릇에 안 들어가 잘리며(5장의 축소), (unsigned)-1은 거대한 양수가 된다. C의 캐스트는 “이 비트를 저 타입으로 읽어라”가 아니라 “이 값을 저 타입의 값으로 **변환하라**”는 뜻이다 — 비트를 그대로 두고 눈만 바꾸고 싶다면 39장의 공용체나 memcpy가 그 통로다. 두 요구를 같은 문법으로 쓰지 않는 것이 C의 몇 안 되는 친절이다.

문 이 규칙들을 다 외워야 하는가?

답 세 줄이면 충분하다 — 작은 정수는 int로 승격된다, 섞이면 넓은 쪽·부호 없는 쪽으로 간다, 가변 인자에서는 float가 double이 된다. 나머지 세부는 부록의 표에 두고, 실무에서는 두 습관이 규칙 암기를 대신한다: 경고를 켜 두는 것(15장), 그리고 의도가 있는 변환은 캐스트로 명시하는 것. 암묵 변환의 위험은 규칙의 복잡함이 아니라 그것이 보이지 않는다는 데 있으므로, 보이게 만드는 것이 최선의 방어다.

변환의 지도를 갖췄다. 다음 장부터 흐름의 도구다 — 판단을 값으로 만드는 불리언과 비교부터.

26 불리언과 비교

이 장이 끝나면

참과 거짓이라는 값의 세계다 — C23의 `bool`, 비교 연산자들, 그리고 논리 연산자와 그 특별한 성질 (단락 평가)까지. 다음 장의 분기가 물을 “조건”이 여기서 만들어진다.

돌아보기 20장에서 `=(담기)`와 `==(비교)`는 기호를 나눠 쓴다고 예고했다. 그러면 `3 < 5` 같은 비교의 결과는 무엇인가 — 수식은 값이 된다고 했는데 (17장), 비교도 수식이라면 무슨 값이 되는가?

답 참 또는 거짓 — 그 두 가지만을 담는 값이 된다. C23에서 그 타입의 이름이 `bool`이고, 값의 이름이 `true`와 `false`다. 비교는 “물음”이 아니라 **bool 값을 계산하는 수식**이라는 것 — 이 관점이 이 장의 중심이고, 다음 장에서 분기가 그 값을 소비한다.

26.1 `bool` — 두 값짜리 타입

`bool`은 값의 집합이 `{false, true}` 둘뿐인 타입이다. 23장의 정수 가족 끝에 있는 가장 작은 그릇인 셈이다. 역사를 한 줄 알아 두면 낯선 구석이 풀린다 — 초기 C에는 불리언 타입이 **없었고**, 0을 거짓으로 0 아닌 모든 값을 참으로 치는 관행이 그 자리를 대신했다. C99가 뒷문으로 (`<stdbool.h>` 상자를 통해) 들여왔고, C23에 이르러서야 `bool`, `true`, `false`가 상자 없이 쓰는 정식 키워드가 됐다. 반세기 만의 정식 입적이다 — 그리고 그 관행의 흔적으로, `bool`을 `%d`로 찍으면 1과 0으로 나온다(시연에서 확인된다).

26.2 비교 — `bool`을 만드는 수식

비교 연산자는 여섯이다 — `==(같다)`, `!=(다르다)`, `<`, `<=`, `>`, `>=`. 전부 두 값을 재료로 받아 `bool`을 내놓는 수식이다.

```
examples/ch26/cmp.c
#include <stdio.h>

int main(void)
{
    bool tall = 10 > 3;           /* 비교의 결과는 bool 값이다 */
    printf("%d\n", tall);
    printf("%d\n", 3 < 5 && 2 + 2 == 4);

    /* 단락 평가: 왼쪽이 0(거짓)이면 오른쪽은 아예 평가되지 않는다 */
    printf("%d\n", 0 && printf("this line is never printed\n"));
    return 0;
}
```

실행 결과

```
1
1
0
```

첫 줄이 그 확인이다 — `10 > 3`이라는 수식의 값(`true`)이 변수에 담기고, `%d`로 1이 찍혔다. 판단은 분기만의 것이 아니라 **값으로 저장하고 전달할 수 있는 것**이라는 감각 — 조건이 복잡해질수록 이 감각이 코드를 깨끗하게 만든다(`bool is_leap = ...`처럼 판단에 이름을 붙이는 관행).

△ “`x == 3` 대신 `x = 3`을 써도 어차피 비슷하게 동작할 것이다”

20장의 예고를 정면으로 회수한다 — 둘은 완전히 다른 수식이고, 최악의 점은 **둘 다 합법**이라는 것이다. $x = 3$ 은 담기 수식이라 x 를 3으로 바꿔 버리고, 그 수식의 값 3은 “0이 아니므로 참”으로 읽힌다 — 조건 자리에 실수로 들어가면 **항상 참이면서 변수까지 망가뜨리는** 이중 사고가 된다. 역사가 이 함정으로 쌓은 사고가 하도 많아, 컴파일러 경고(-Wall)는 조건 자리의 `=`를 특별히 지목해 알려 준다 — 15장에서 경고를 켜 두자고 한 이유의 대표 사례다.

26.3 논리 연산자 — 판단을 엮는다

판단 여럿을 엮는 연산자가 셋 있다 — AND `&&`(둘 다 참), OR `||`(하나라도 참), NOT `!`(뒤집기). 시연의 둘째 줄이 `&&`의 예다 — `3 < 5 && 2 + 2 == 4`, 두 판단이 모두 참이라 1.

그리고 이 연산자들에는 C에서 각별히 중요한 성질이 있다 — **단락 평가**(short-circuit)다. `&&`는 왼쪽이 거짓이면 오른쪽을 **아예 평가하지 않고** 거짓을 내놓는다(`||`는 왼쪽이 참이면 오른쪽을 건너뛴다). 시연의 셋째 줄이 그 증명이다 — 오른쪽의 `printf`는 부수효과(출력)를 가진 수식인데, 왼쪽이 0(거짓)이라 **호출 자체가 일어나지 않았다**.

이것은 최적화가 아니라 **보장**이다 — 그리고 17장에서 “평가 순서는 대부분 미지정”이라 했던 것의 귀한 예외다: `&&`와 `||`는 왼쪽을 먼저 평가하고, 오른쪽 평가 여부까지 계약으로 정한다. 그래서 C 프로그래머는 이 보장을 **문지기로** 쓴다 — “0이 아닐 때만 나뉘라”를 `b != 0 && a / b > 10` 처럼 왼쪽 판단이 오른쪽의 위험(24장의 0 나누기)을 막아서게 적는 것이 관용구다.

문 `&`(24장)와 `&&`가 정말로 어떻게 다른가 — 둘 다 AND라 불리는데.

답 층이 다르다. `&`는 **비트마다** AND를 하는 산술 연산자다 — `5 & 3`은 비트를 맞대 1을 얻는다. `&&`는 **판단 전체**의 AND다 — 값이 0인가 아닌가만 보고, 단락 평가의 보장이 있다. 실수로 바꿔 써도 우연히 답이 같은 경우가 많아 더 위험하다 — `2 && 4`는 1(둘 다 0 아님)이지만 `2 & 4`는 0(겹치는 비트 없음)이다. 판단에는 `&&`, 비트에는 `&` — 자리를 섞지 않는 것이 수칙이다.

조건을 만들 줄 알게 됐다. 다음 장에서 드디어 프로그램이 **갈라선다** — 조건의 값에 따라 다른 길을 가는 것, 분기다. 22장에서 미뤄 둔 “입력 검사”의 숙제도 그때 풀 수 있게 된다.

27 결정 — if와 switch

이 장이 끝나면

프로그램이 갈림길을 만난다. 조건에 따라 다른 길을 가는 if와, 여러 갈래를 한 번에 벌이는 switch — 두 분기 장치를 익힌다. 2장의 계산 모델(“결과가 0이면 건너뛰어라”)이 드디어 C 문법의 얼굴로 나타나는 장이다.

돌아보기 10장에서 “if 는 공짜가 아니다”라는 감각을 예고했다 — 분기 예측이 틀리면 파이프라인을 비우는 벌금을 묻는다. 그러면 프로그래머는 분기를 피해야 하는가?

답 아니다 — 분기는 프로그램의 뼈대이고, 예측기는 일상 코드에서 90% 넘게 맞힌다. 그 감각의 쓸모는 “분기를 쓰지 말라”가 아니라 **분기의 비용이 일정하지 않음을 아는 것이다** — 규칙적인 갈림길은 싸고, 무작위한 갈림길은 비싸다(정렬된 배열 사건). 성능이 문제 되는 극히 일부 코드에서만 꺼내는 지식이고, 오늘 배우는 분기는 마음껏 쓰면 된다.

27.1 if — 조건이 참이면

문법은 읽는 그대로다:

```
if (조건) {
    조건이 참일 때의 문장들
} else {
    거짓일 때의 문장들 /* else 부분은 없어도 된다 */
}
```

조건 자리에는 bool이 되는 수식(26장)이 오고, 갈래마다 블록(16장)이 온다. 갈래가 여럿이면 else if로 사다리를 만든다 — 시연이 그 모습이다. 22장의 입력 관행(fgets+sscanf)이 다시 나오는 것도 눈여겨보면 된다.

```
examples/ch27/grade.c
#include <stdio.h>

int main(void)
{
    char line[100];
    int score = 0;

    fgets(line, sizeof line, stdin);
    sscanf(line, "%d", &score);

    if (score >= 90) {
        printf("%d: excellent\n", score);
    } else if (score >= 80) {
        printf("%d: good\n", score);
    } else if (score >= 70) {
        printf("%d: fair\n", score);
    } else {
        printf("%d: needs work\n", score);
    }
    return 0;
}
```

표준 입력으로 준 것

실행 결과

87: good

사다리는 위에서부터 차례로 검사되고, 처음 참이 되는 갈래 하나만 실행된다 — 87점이 “우”에서 멈추고 아래로 내려가지 않는 것이 그 보장이자. 그래서 사다리의 순서가 곧 논리다: 좁은 조건을 위에, 넓은 조건을 아래에 두는 것이 정석이다.

문 갈래가 하나뿐이면 중괄호를 생략해도 된다고 들었다 — 써도 되는가?

답 문법상 허용된다 — 그리고 이 책은 쓰지 않기를 권한다. 중괄호 없는 분기는 “다음 문장 하나만” 갈래에 속하는데, 나중에 문장을 한 줄 보태며 중괄호 없음을 잊는 실수가 고전적 사고 경로다. 실제로 애플의 SSL 라이브러리에서 이 무늬로 검증 코드가 분기 밖으로 밀려나 보안 검사가 통째로 건너뛰어진 사건(“goto fail” 사고, 2014)이 있었다 — 들여쓰기는 사람 눈에만 보이고 컴파일러에게는 안 보인다는 것(16장)이 값비싸게 증명된 날이다. 갈래에는 언제나 중괄호 — 이 책의 전 예제가 따르는 수칙이다.

27.2 switch — 값으로 갈라서는 판

갈림길이 “어느 범위인가”가 아니라 “어느 값인가”로 갈릴 때는 전용 장치가 따로 있다 — switch다.

```
examples/ch27/season.c
#include <stdio.h>

int main(void)
{
    char line[100];
    int month = 0;

    fgets(line, sizeof line, stdin);
    sscanf(line, "%d", &month);

    switch (month) {
    case 12:
    case 1:
    case 2: /* 세 case가 하나의 답을 공유한다(폴스루) */
        printf("month %d: winter\n", month);
        break;
    case 3:
    case 4:
    case 5:
        printf("month %d: spring\n", month);
        break;
    case 6:
    case 7:
    case 8:
        printf("month %d: summer\n", month);
        break;
    case 9:
    case 10:
    case 11:
        printf("month %d: autumn\n", month);
```

```

        break;
    default:
        printf("month %d: no such month\n", month);
        break;
    }
    return 0;
}

```

표준 입력으로 준 것

12

실행 결과

month 12: winter

읽는 법 — switch (month)가 값을 하나 들고 판에 들어서면, 실행은 그 값과 일치하는 case 이름표로 건너 뛴다. 그리고 여기가 switch의 가장 중요한 성질이다: case는 담장이 아니라 **이름표**라서, 일단 건너뛴 뒤에는 다음 case를 지나쳐 **아래로 계속 흐른다**. 이 흐름을 폴스루(fall-through)라 부르고, 멈추고 싶은 자리에 break(판을 나가라)를 명시한다.

폴스루는 양날이다. 시연처럼 여러 값이 한 답을 공유하게 묶는 데는 제격이지만(12·1·2월이 겨울 하나로), break를 **잊으면** 의도치 않은 갈래까지 흘러 들어가는 고전적 사고가 된다 — 그래서 컴파일러 경고와 C23의 명시 표기([[fallthrough]] — “의도된 흐름임”을 적는 문서)가 이 자리를 지킨다. default는 어느 이름표에도 안 맞을 때의 갈래다 — “그런 달은 없다”처럼, **예상 밖의 값을 잡는 그물로** 항상 두는 것이 관행이다.

문 if의 사다리로도 다 되는데, switch가 굳이 필요한 이유는 무엇인가?

답 둘의 차이는 표현력이 아니라 **의도의 전달**이다. switch는 “하나의 값을 여러 상수와 맞춰 본다”는 의도를 문법으로 못박아서, 읽는 사람과 컴파일러 모두에게 구조가 보인다 — 컴파일러는 그 구조를 이용해 점프 표 같은 빠른 코드로 바꾸기도 하고(11장의 편집자가 좋아하는 모양이다), 경고 기능은 “열거된 값 중 빠진 case”를 짚어 줄 수 있다. 값 하나로 갈라서면 switch, 범위·복합 조건이면 if 사다리 — 도구를 의도에 맞추는 것이다. 그리고 다음 장에서, 이 폴스루라는 성질을 극한까지 악용(?)한 전설의 코드를 드디어 만난다.

갈림길을 배웠다 — 그러나 프로그램의 진짜 힘은 갈라서는 것이 아니라 **되풀이하는 것**에서 나온다. 2장에서 “단순한 걸음도 수십억 번이면”이라 했던 그 반복을, 다음 장에서 손에 넣는다.

28 반복 — 루프와 불변식

이 장이 끝나면

프로그램의 진짜 힘 — 되풀이 — 를 손에 넣는다. 루프 3형제(`while`, `for`, `do-while`)와 새 연산자들(`++`, `+=`), 루프를 옹계 읽고 쓰는 생각의 틀(불변식)까지. 그리고 10장에서 예약해 둔 재회 — Duff's device — 를 드디어 만난다.

돌아보기 2장에서 “단순한 걸음이라도 1초에 수십억 번이면 어떤 복잡한 일이든 지을 수 있다”고 했다. 그런데 27장까지의 우리 프로그램은 위에서 아래로 한 번 흐르고 끝난다 — 수십억 번의 걸음은 어디서 나오는 것인가?

답 같은 문장들을 다시 실행하는 장치에서 나온다. 조건이 참인 동안 블록으로 되돌아가는 것 — 루프다. 분기가 흐름을 가르는 장치였다면 루프는 흐름을 감아올리는 장치이고, 이 둘을 갖추는 순간 C는 계산 가능한 모든 것을 적을 수 있는 언어가 된다(이론적으로도 그렇다 — 2장의 계산 모델이 요구하는 전부가 “순차, 분기, 반복”이다).

28.1 루프 3형제

`while` — 가장 원초적인 형태다. “조건이 참인 동안, 블록을 반복하라”:

```
while (조건) {  
    반복할 문장들  
}
```

조건을 먼저 검사하므로, 처음부터 거짓이면 한 번도 돌지 않는다.

`for` — 반복의 살림살이(시작·조건·갱신)를 한 줄에 모은 형태다. 시연으로 본다 — 1부터 100까지의 합이다:

```
examples/ch28/sum.c  
  
#include <stdio.h>  
  
int main(void)  
{  
    int sum = 0;  
  
    for (int i = 1; i <= 100; i += 1) {  
        sum += i;          /* 불변식: sum == 1 + 2 + ... + (i - 1)... 갱신 후엔 ...+ i */  
    }  
    printf("sum of 1..100 = %d\n", sum);  
    return 0;  
}
```

실행 결과

```
sum of 1..100 = 5050
```

`for (int i = 1; i <= 100; i += 1)`을 세 칸으로 읽는다 — **시작**(`int i = 1`: 루프 변수를 만들고), **조건**(`i <= 100`: 돌기 전마다 검사), **갱신**(`i += 1`: 한 바퀴 끝날 때마다 실행). 새 연산자 `+=`는 20장 대입의 줄임말로 `i = i + 1`과 같고, 더 줄인 `i++`(증가 연산자)도 같은 일을 한다 — 관용적으로 `for (...; i++)` 꼴을 가장 많이 보게 된다. (`++`에는 앞에 붙는 꼴과 뒤에 붙는 꼴, 그리고 수식 안에서의 미묘한 사정이 있는데 — 다음 장의 시퀀스 포인트 이야기와 함께 다룬다.)

do-while — 블록을 **먼저** 한 번 실행하고 조건을 검사하는 형태다 (do { ... } while (조건);). “최소 한 번은 해야 하는 일”(메뉴를 먼저 보여 주고 반복 여부를 묻는 류)에 쓰이고, 셋 중 등장 빈도가 가장 낮다.

Σ 루프 불변식 — 반복을 증명하는 눈

루프를 “올게” 읽는 도구가 하나 있다 — **불변식(invariant)**: 매 바퀴의 같은 지점에서 항상 성립하는 명제다. 시연의 루프라면, 몸통에 들어설 때마다

$$\text{sum} = 1 + 2 + \dots + (i - 1)$$

이 성립한다(첫 바퀴에는 빈 합 = 0). 몸통이 `sum += i`를 하면 등식의 오른쪽이 $\dots + i$ 까지 자라고, 갱신이 i 를 올리면 다시 같은 꼴이 된다 — 불변식이 **유지**된다. 루프가 끝나는 순간은 $i = 101$ 이므로, 불변식에 대입하면 `sum = 1 +  $\dots$  + 100`. 즉 [불변식 + 종료 조건 = 루프의 정확성 증명]이다. 매번 이렇게 적진 않더라도, “이 루프에서 변하지 않는 사실이 무엇인가”를 묻는 습관은 루프 버그의 대부분 — 하나 모자라게/많게 도는 off-by-one — 을 잡아 주는 눈이 된다. 계약 으로서의 코드라는 관점(41장)의 첫 연습이기도 하다.

문 조건이 영영 거짓이 안 되면 어떻게 되는가?

답 무한 루프다 — 프로그램이 그 자리를 영원히 돈다. 대개는 갱신을 잊은 버그의 결과지만, 의도적인 무한 루프(`while (true)`)도 어엿한 관용구다 — 서버처럼 “끝나지 않는 것이 정상”인 프로그램의 뼈대가 그것이고, 안에서 조건을 보아 `break`(루프 탈출 — switch의 그 `break`와 같은 낱말이다)로 나온다. 위험한 것은 무한 루프 자체가 아니라 **의도치 않은** 무한 루프다.

28.2 재회 — Duff's device

10장에서 예약한 전설을 만날 시간이다. 이제 우리는 switch의 폴스루(27장)와 루프(이 장)를 모두 알고 있다.

1983년, 루카스필름의 Tom Duff는 데이터를 장치로 복사하는 루프가 너무 느려 고심하고 있었다. 한 바퀴마다 드는 살림 비용(조건 검사·갱신)을 줄이려 여덟 개씩 묶어 처리(언롤링)하되 — 개수가 8의 배수가 아닐 때의 나머지 처리를, 그는 아무도 상상 못 한 방법으로 해결했다:

```
switch (count % 8) {
case 0: do { *to = *from++;
case 7:      *to = *from++;
case 6:      *to = *from++;
case 5:      *to = *from++;
case 4:      *to = *from++;
case 3:      *to = *from++;
case 2:      *to = *from++;
case 1:      *to = *from++;
            } while ((count -= 8) > 0);
}
```

(*to, *from++ 부분은 30장의 포인터 문법이라 아직 겉모양만 보면 된다 — 핵심은 구조다.) 읽는 법: switch가 **루프의 한복판으로** 건너 뛴다. case는 이름표일 뿐이라는 27장의 사실을 기억하면 — 이름표가 do-while의 몸통 **안에** 찍혀 있어도 문법 위반이 아니다. 첫 진입은 나머지 개수만큼만 처리하는 지점으로 뛰어들고, 이후는 do-while이 여덟 줄짜리 몸통을 통째로 돈다. 나머지 처리와 언롤링이 한 몸이 된 것이다.

Duff 자신도 “발견하고 나서 자부심과 혐오감이 뒤섞였다”는 소감을 남긴 이 코드는, 합법과 기괴함의 경계에서 C 문법의 유연함(어쩌면 험거움)을 보여 주는 기념비다. 그리고 오늘의 교훈은 10장에서 예고한 그대로다 — **이 곡예는 이제 사람의 일이 아니다**. 현대 컴파일러는 평범하게 쓴 루프를 알아서 언롤링한

다(11장의 편집자). Duff's device는 박물관의 걸작으로 감상하고, 우리의 루프는 시연의 for처럼 평범하고 읽기 쉽게 쓰는 것 — 그것이 모던 C다.

반복까지 갖추며 흐름의 도구가 완성됐다 — 순차(16장), 분기(27장), 반복(28장). 다음 장은 제6부의 마무리로, 함수라는 장치의 **의미**를 파고든다 — 값은 어떻게 건너가는가, 부수효과란 정확히 무엇인가, 그리고 17장에서 심어 둔 씨앗(평가 순서)의 정식 답까지.

29 함수의 의미 — 값 복사와 부수효과

이 장이 끝나면

제6부의 마무리다. 함수 호출에서 값이 **어떻게** 건너가는지(복사), 부수효과와 평가 순서의 정식 규칙 (17장 씨앗의 회수), 재귀라는 새로운 반복, 그리고 조건 연산자 `?:`까지. 마지막에 심는 씨앗 — 계약이라는 관점 — 은 제9부에서 꽃핀다.

돌아보기 21장에서 매개변수는 “재료를 받는 변수 선언”이라 했다. 그러면 함수 안에서 그 매개변수를 **바꾸면** — 밖에서 재료로 넘긴 변수도 바뀌는가?

답 바뀌지 않는다 — 그리고 이것이 C 함수의 의미를 정하는 단 하나의 규칙이다: **값은 복사되어 건너간다**. 매개변수는 재료의 값을 복사받은 별개의 변수이고, 함수 안의 어떤 일도 원본에 닿지 않는다. 말보다 증거가 빠르다 — 바로 시연한다.

29.1 값 복사 — 원본은 안전하다

```
examples/ch29/copy.c
#include <stdio.h>

void try_change(int x)
{
    x = 999;                /* 복사본을 바꿀 뿐이다 */
    printf("inside function, x = %d\n", x);
}

int main(void)
{
    int n = 1;

    try_change(n);
    printf("after the call, n = %d\n", n); /* 원본은 그대로다 */
    return 0;
}
```

실행 결과

```
inside function, x = 999
after the call, n = 1
```

`try_change` 안에서 `x`를 999로 바꿨지만, 밖의 `n`은 1 그대로다. 호출 `try_change(n)`의 정확한 의미가 이것이다 — `n`의 **값** 1이 복사되어 `x`라는 **다른 변수**의 첫 값이 되고, 이후 둘은 남남이다. 21장의 스코프 칸막이에 이 복사 규칙까지 더하면, 함수는 완전한 격리실이 된다: 재료는 값으로 들어오고, 결과는 `return`의 값으로 나가며, 그 외의 통로는 없다.

△ “함수에 변수를 넘긴다”

일상어로는 자연스럽지만, C의 의미론으로는 틀린 문장이고 — 이 문장 대로 상상하는 순간 위 시연이 수수께끼가 된다. 넘어가는 것은 변수가 아니라 **변수의 값**(복사본)이다. “그러면 함수가 밖의 변수를 바꿔 주는 일은 아예 불가능한가?”라는 자연스러운 반문이 생기는데 — 가능하게 만드는 정공법이 따로 있다: 변수 대신 **변수의 주소**를 값으로 복사해 넘기는 것이다. 주소를 받은 함수는 그 주소로

찾아가 원본을 만진다. 22장에서 `sscanf(line, "%d", &n)`의 `&`가 정확히 이 일을 하고 있었다 — 값 복사 규칙은 그대로인 채(주소도 값이다 — 3장!), 효과는 “원본 수정”이 되는 것. 이 정공법의 문법이 30장의 포인터다.

29.2 부수효과와 평가 순서 — 씨앗의 회수

17장에서 “한 문장 안의 계산 시간 순서는 달라질 수 있다”는 씨앗만 심어 두었다. 이제 재료가 다 모였으니 정식으로 정리한다.

부수효과(side effect)란 수식 평가가 값을 내놓는 것 **외에** 세상의 상태를 바꾸는 모든 일이다 — 우리가 아는 것만 벌써 셋이다: 출력(19장), 대입과 `++`(20·28장), 그리고 입력(22장). 함수 호출은 몸통이 이런 일을 하면 부수효과를 품는다.

평가 순서의 규칙은 두 문장으로 요약된다. 첫째, 한 문장(정확히는 완전 수식) 안에서 부분 수식들의 평가 순서는 대부분 **미지정**이다 — 함수 호출 둘이 한 수식에 있으면 어느 쪽이 먼저 불릴지 계약에 없다(17장의 “호랑이/사자”). 둘째, **문장의 끝(세미콜론)에 도달하는 순간, 그 문장이 일으킨 부수효과는 전부 완료됨이 보장된다** — 이 보장 지점을 전통 용어로 **시퀀스 포인트(sequence point)**라 한다. 세미콜론 외에도 몇 곳이 더 있는데, 대표가 26장에서 만난 `&&||`(왼쪽 평가 완료 후 오른쪽)와 함수 호출의 경계(재로 평가 완료 후 몸통 진입)다. 현대 표준(C11 이후)은 같은 개념을 “앞서 차례 매겨짐”(sequenced before)이라는 관계로 더 정밀하게 다듬었지만, 고전 용어가 여전히 통용된다.

이 규칙에서 실전 수칙과 함정이 나온다. 수칙은 17장 그대로 — **순서가 중요한 부수효과들은 문장을 나눠라**. 함정은 새로 생겼다 — `++`를 배웠기 때문이다. 한 수식 안에서 **같은 변수를 두 번 바꾸거나, 바꾸면서 따로 읽는 코드**:

```
i = i++ + 1;    /* 같은 i를 두 곳에서 바꾼다 - 계약 밖 */
printf("%d %d\n", i, i++); /* 바꾸면서 따로 읽는다 - 계약 밖 */
```

이런 수식은 미지정(어느 순서든 하나가 일어남)을 넘어 아예 **정의되지 않은 동작**이다 — 표준은 결과를 전혀 보장하지 않는다(42장). 규칙을 외울 것 없이 무늬만 기억하면 된다: **한 문장에서 한 변수는 한 번만 바꾼다**. 15장의 UBSan과 컴파일러 경고가 이 무늬를 잘 잡아 주는 것도 든든한 뒷배다.

문 값 복사가 규칙이라면, 큰 구조체를 넘길 때마다 통째로 복사되는가 — 비용이 만만치 않을 텐데.

답 그렇다, 의미론상으로는 복사다 — 그리고 그 비용이 실무에서 포인터로 넘기는 관행의 이유가 된다(38장에서 구조체와 함께 다룬다). 다만 두 가지 단서가 있다. 첫째, **작은 값은 복사가 사실상 공짜다** — 9장에서 배운 대로 인자는 대개 레지스터로 건너가므로, 정수 몇 개를 넘기는 데는 메모리 왕복 조차 없다. 둘째, 11장의 편집자가 개입한다 — 관찰 가능한 결과가 같다면 실제 복사를 생략하기도 한다. 의미는 복사, 구현은 **컴파일러의 재량** — 이 구별이 성능을 걱정하기 전에 먼저 잡아야 할 관점이다.

29.3 재귀 — 자신을 부르는 함수

함수가 함수를 부를 수 있다면 — 자기 자신도 부를 수 있는가. 있다. **재귀(recursion)**다.

```
examples/ch29/fact.c
#include <stdio.h>

int fact(int n)
{
    if (n <= 1) {
```

```

    return 1;          /* 바닥: 더 내려가지 않는 경우 */
}
return n * fact(n - 1); /* 자신을 부르되, 더 작은 문제로 */
}

int main(void)
{
    printf("5! = %d\n", fact(5));
    return 0;
}

```

실행 결과

5! = 120

fact(5)는 $5 * \text{fact}(4)$ 이고, 그 안은 $4 * \text{fact}(3) \cdots$ 바닥($n \leq 1$)에 닿으면 1을 돌려주며 겹겹의 호출이 차례로 값을 물고 돌아온다. 이것이 가능한 이유는 이 장의 규칙 그 자체다 — 호출마다 매개변수 n 이 새로 복사되므로, 다섯 겹의 fact는 각자의 $n(5, 4, 3, 2, 1)$ 을 가진 다섯 개의 격리실이다. 값 복사와 스코프가 만드는 격리가 없다면 재귀는 성립 하지 않는다.

재귀는 “더 작은 같은 문제”로 쪼개지는 일에 자연스러운 표현이고(나무 구조 탐색이 대표 — 38장 이후), 모든 재귀는 루프로도 적을 수 있다(28장의 팩토리얼 루프판을 떠올리면 된다). 어느 쪽이 좋은가는 문제의 모양이 정한다 — 그리고 재귀에는 “겹겹의 호출”이 쌓이는 물리적 공간 문제가 따라오는데, 그 공간(스택)의 정제는 36장에서 만난다.

29.4 마지막 연산자, 그리고 계약이라는 씨앗

이 부의 배분표 마지막 항목 — **조건 연산자** $?:$ 다. 조건 $?$ 값1 : 값2는 “조건이 참이면 값1, 거짓이면 값2”가 되는 수식 이다. `int big = a > b ? a : b;`처럼 “골라 담기”를 한 줄로 적는다 — if는 문장이라 값이 될 수 없는 자리(초기화 등)에서 요긴하다. 남용하면 읽기 어려워지므로, 이 책은 “한 겹까지만”을 관행으로 삼는다. (비슷한 자리의 쉼표 연산자도 있으나 함정이 많아 부록으로 미룬다.)

끝으로, 다음을 위한 씨앗 하나. fact는 사실 **암묵의 약속** 위에 서 있다 — “ n 은 0 이상이어야 하고(음수면 바닥에 안 닿는다), 13 이상이면 int 그릇이 넘친다(23장).” 함수가 재료에 거는 조건(전제조건)과 결과에 관해 약속하는 것(후조건) — 이 관점으로 함수를 바라보기 시작하면, 함수 하나하나가 작은 **계약서**로 보인다. 이 관점이 오류 처리(41장)와 proven의 설계 철학(35장)의 뿌리이고, 제9부에서 정면으로 다룬다.

제6부가 끝났다 — 값을 버리고(23-26장), 흐름을 다스리고(26-28장), 함수의 의미까지 갖췄다(29장). 다음 부는 이 책의 두 번째 산, **기억**이다 — 3장의 사물함 복도로 돌아가, 이번에는 C의 문법으로 그 복도를 걷는다. 포인터가 기다린다.

제7부 — 기억

30 객체, 주소, 포인터

이 장이 끝나면

제7부는 이 책의 두 번째 산 — 기억이다. 3장의 사물함 복도를 이번에는 C 문법으로 걷는다. 첫 장에서 주소를 값으로 다루는 타입, **포인터**를 정면으로 배운다 — &와 *, 그리고 sizeof까지, 22장부터 미뤄 온 외상이 모두 청산된다.

돌아보기 29장에서 “함수가 밖의 변수를 바꾸는 정공법은 변수의 주소를 값으로 복사해 넘기는 것”이라 했다. 3장의 지식으로 풀어 보면 — 주소를 “값으로 복사”할 수 있는 근거는 무엇이었나?

답 주소도 그냥 수라는 것(3장) — 사물함 번호는 정수이고, 정수이므로 다른 사물함에 적어 둘 수도, 복사해 건넬 수도 있다. C에서 그 “주소를 담는 변수”의 타입이 포인터다. 오늘 배우는 것은 새 개념이 아니라, 3장의 착상에 문법 옷을 입히는 일이다.

30.1 세 가지 표기 — 선언, &, *

포인터 문법은 세 조각이 전부다.

선언 — `int *p;`는 “`int`의 주소를 담는 변수 `p`”를 선언한다. 읽는 요령은 뒤에서부터다: “`p`는 포인터인데 (`*p`), 가리키는 곳에 `int`가 있다”.

주소 얻기 — `&n`은 “`n`의 주소”라는 값이다. 22장의 `sscanf(..., &n)` 에서 스쳐 본 그 `&`가 드디어 정식 이름을 얻었다 — `n`이라는 사물함의 번호표를 떼어 주는 연산자다.

역참조 — `*p`는 “`p`에 적힌 주소로 찾아가서 그 칸”이라는 뜻이다. 읽는 자리에 있으면 그 칸의 값을 읽고, 대입의 왼쪽에 있으면 그 칸에 쓴다. 선언의 `*`와 수식의 `*`(역참조), 그리고 곱셈의 `*`가 같은 글자를 쓰는 것은 C의 악명 높은 절약 정신이다 — 자리로 구별하는 수밖에 없다.

시연으로 세 조각을 한 번에 본다. 29장의 숙제 — 함수가 원본을 바꾸는 정공법 — 까지 포함이다.

```
examples/ch30/ptr.c
#include <stdio.h>

void set_to(int *target, int value)
{
    *target = value;          /* 주소를 따라가 원본에 쓴다 */
}

int main(void)
{
    int n = 1;
    int *p = &n;              /* p는 n의 주소를 담는다 */

    printf("n at first: %d\n", n);
    printf("p == &n ? %d\n", p == &n);

    *p = 55;                  /* 역참조: p가 가리키는 곳(= n)에 쓴다 */
    printf("n after *p = 55: %d\n", n);

    set_to(&n, 99);            /* 28장의 숙제 - 함수가 원본을 바꾼다 */
    printf("n after set_to: %d\n", n);
    return 0;
}
```

실행 결과

```
n at first: 1
p == &n ? 1
n after *p = 55: 55
n after set_to: 99
```

set_to(&n, 99)의 동선을 정확히 읽어 두면 포인터의 절반이 끝난다 — ① &n(주소라는 값)이 매개변수 target에 복사되고(29장의 값 복사 규칙은 그대로다!), ② 함수 안에서 *target = value가 그 주소로 찾아가 원본 칸에 쓴다. 값 복사의 세계는 하나도 깨지지 않았다 — 복사된 것이 주소였을 뿐이다.

30.2 sizeof — 그릇의 크기를 묻다

미뤄 둔 마지막 외상 — sizeof는 타입이나 대상의 **바이트 크기**를 내놓는 연산자다(값은 size_t 타입 — 크기 전용 부호 없는 정수이고, 서식은 %zu다). 22장의 fgets(line, sizeof line, stdin)이 이제 완전히 읽힌다: “line이라는 그릇의 바이트 수를 fgets에게 알려 준다”. 함수가 아니라 연산자라서 대부분 컴파일 시점에 값이 정해지고, sizeof(int)처럼 타입에도, sizeof line처럼 대상에도 쓸 수 있다.

문 포인터 변수 자체의 크기는 얼마인가 — sizeof p는?

답 주소 하나를 담는 그릇이니, 3장의 답 그대로 **워드 크기**다 — 오늘의 64비트 환경에서 8바이트. 가리키는 대상이 무엇이든(int*든 double* 이든) 포인터의 크기는 같다 — 담는 것이 언제나 “주소라는 수” 하나이기 때문이다. 그러면 타입은 왜 구별하는가 — 역참조할 때 “그 주소에서 몇 바이트를 어떤 눈으로 읽을 것인가”를 타입이 정하기 때문이다. 3장의 후렴구(덩어리를 아는 것은 읽는 쪽)가 포인터 타입의 존재 이유다.

△ “포인터는 어렵고 위험한 고급 기능이다”

악명은 사실이지만 절반은 오해다. 개념 자체는 방금 본 대로 “번호를 적어 두고, 번호로 찾아간다” — 사물함 복도를 아는 사람에게는 평범한 이야기다. 악명의 진짜 출처는 개념이 아니라 **규칙 위반의 대가**다: 비어 있는 포인터를 따라가거나(다음 장), 남의 땅 번호를 만들거나(31-33장), 사라진 칸의 번호를 간직하는(35-37장) 코드는 조용히, 늦게, 크게 사고를 낸다. 그래서 이 부의 남은 장들이 전부 “규칙”의 장이다 — 개념은 오늘로 끝났고, 이제 안전 수칙을 배운다.

문 변수의 주소를 직접 찍어 보면 재미있을 것 같은데 — 왜 시연에서 주소 값을 인쇄하지 않았는가?

답 찍을 수 있다 — printf("%p", (void *)&n)이 그 서식이다. 시연에서 뺀 이유는 정직함 때문이다: 주소 값은 **실행할 때마다 다르다**. 현대 운영체제는 보안을 위해 프로그램을 매번 다른 자리에 배치한다(주소 공간 무작위화, ASLR — 공격자가 주소를 미리 알 수 없게 하는 방어다). 이 책의 실행 결과는 전부 실제 캡처인데, 매번 달라지는 값을 실으면 “같은 결과”를 보장할 수 없다. 주소의 값은 중요하지 않고, 주소들 사이의 **관계**(같다/다르다/이웃이다)가 중요하다 — 시연이 p == &n을 찍은 이유다.

포인터의 개념이 갖춰졌다. 다음 장은 이 부의 첫 안전 수칙 — “아무 데도 가리키지 않음”을 다루는 법이다. 4장에서 얼굴만 익힌 널 삼형제를, 이번에는 문법과 실무 규칙으로 정식 취급한다.

31 널 — 삼형제의 정식 취급

이 장이 끝나면

4장에서 얼굴만 익힌 널 삼형제 — 널 포인터, NULL/nullptr, NUL 문자 — 를 문법과 실무 규칙으로 정식 취급한다. “비어 있음”을 다루는 법은 포인터 안전 수칙의 첫 장이다.

돌아보기 4장에서 “널 포인터의 내부 표현은 0이 아닐 수 있다”는 역사(Prime 50, CDC Cyber 180)를 보았다. 그러면 소스 코드에서 `p == 0`이라는 비교는 — 그런 기계에서 깨지는가?

답 깨지지 않는다 — 그것이 표준의 약속이다. 소스의 정수 상수 0은 포인터 문맥에서 **널 포인터 상수**로 읽히고, 컴파일러가 그 기계의 실제 널 표현과 비교하도록 번역한다. 기호(소스의 0)와 표현(기계 속 비트)의 분리를 컴파일러가 지켜 주는 것이다. 깨지는 것은 다른 종류의 코드다 — “표현이 곧 0”이라고 **비트 차원에서** 가정하는 코드. 이 함정을 뒤에서 본다.

31.1 nullptr — 비어 있음의 이름

포인터 변수는 선언 즉시 무언가를 가리키게 하거나, 아직 대상이 없다면 **비어 있음**을 명시해야 한다 (20장의 초기화 규칙이 포인터에서는 더욱 사활적이다 — 쓰레기 주소를 따라가는 것이 최악의 사고이므로). 비어 있음의 표기는 두 가지가 통용된다 — 전통의 NULL 매크로와, C23이 들여온 키워드 nullptr. 이 책은 nullptr를 쓴다: NULL은 역사적으로 정의가 구현마다 달라 미묘한 함정(가변 인자 등)이 있었고, nullptr는 타입까지 명확한 현대적 표기다.

```
examples/ch31/null.c
#include <stdio.h>

int main(void)
{
    int *p = nullptr;          /* 아직 아무 데도 가리키지 않는다 */

    if (p == nullptr) {
        printf("p is empty for now\n");
    }

    int n = 7;
    p = &n;
    if (p != nullptr) {
        printf("now p points to %d\n", *p);
    }
    return 0;
}
```

실행 결과

```
p is empty for now
now p points to 7
```

수칙도 이 시연 그대로다 — **포인터를 쓰기 전에 비어 있는지 확인한다**. 비어 있는 포인터를 역참조하는 것(*p)은 계약 밖(정의되지 않은 동작) 이고, 4장에서 배운 대로 보호 모드 환경에서는 대개 그 자리에 서 프로그램이 붕괴한다 — 시끄럽게 죽는 것이 그나마 친절이라는 것도 4장의 이야기 그대로다.

● 십억 달러의 실수 — 널의 발명자가 남긴 사과

“비어 있음”이라는 값 자체가 논쟁의 역사를 가졌다. 널 참조를 1965년 ALGOL 계열 언어에 처음 도입한 토니 호어(Tony Hoare — 27장 퀵소트 계열 알고리즘의 그 호어다)는 2009년의 강연에서 그것을 “나의 십억 달러짜리 실수”라 불렀다 — “구현하기 쉽다는 유혹을 참지 못했고, 그 뒤 40년간 널 역참조가 일으킨 붕괴와 취약점의 비용이 십억 달러를 넘을 것”이라는 자기 고발이다. 현대 언어들(Rust, Swift, Kotlin)이 “비어 있을 수 있음”을 타입으로 강제 검사하게 진화한 것은 이 반성의 직계 후손이다. C에는 그 강제가 없다 — 검사는 문화와 규율의 몫이고, 그래서 이 책은 검사를 관용구로 못박는다.

31.2 함정 — 표현을 0으로 가정하는 코드

deepqa의 복선을 회수한다. 다음 두 코드는 뜻이 다르다:

```
int *arr[8];
for (int i = 0; i < 8; i += 1) { arr[i] = nullptr; } /* 올바르다 */
/* memset(arr, 0, sizeof arr);    - 비트를 0으로 채운다: 다른 뜻! */
```

첫 줄은 “널 포인터를 담아라”(기호의 세계 — 어떤 표현이든 컴파일러가 알아서). 주석의 memset은 “모든 비트를 0으로 채워라”(표현의 세계). 널 표현이 전부 0인 오늘의 주류 기계에서는 결과가 우연히 같지만, 4장의 역사가 보여 주듯 그 우연은 계약이 아니다. 실무에서 memset 방식을 흔히 보게 되는데(그리고 주류 플랫폼에서 실제로 동작하는데), **이식성의 눈**으로는 회색 지대임을 알고 쓰는 것과 모르고 쓰는 것이 다르다 — 이 책의 선택은 명시적 nullptr 대입이다.

문 NUL 문자(‘\0’) 쪽은 정식 취급이 언제인가 — 삼형제 중 하나가 아직 남았다.

답 세 장 뒤다. NUL 문자는 포인터의 세계가 아니라 **문자열의 세계**에서 일하는 물건이라(끝 표시), 문자열을 정식으로 다루는 34장이 그 무대다. 기다리는 동안 구별만 다시 — nullptr는 “가리킬 곳 없음”(포인터 값), ‘\0’은 “여기가 글의 끝”(문자 값, 크기 1바이트). 사는 세계가 다른 남남이다.

비어 있음을 다루는 법을 갖췄다. 다음 장은 포인터에 걸린 나머지 규칙들 — 4장의 정렬이 포인터 캐스트에 거는 제약과, 12장에서 예고한 프로버넌스 (출처 딱지)의 실무 감각이다.

32 포인터의 규칙 — 정렬과 프로버넌스

이 장이 끝나면

포인터에 걸린 두 가지 깊은 규칙을 배운다 — 4장의 정렬이 포인터 캐스트에 거는 제약(그리고 char*만의 특권), 12장에서 예고한 프로버넌스(출처 딱지)의 실무 감각까지. 짧지만, 이 부의 안전 수칙 중 가장 “계약”다운 장이다.

돌아보기 4장에서 “4바이트 짐은 4의 배수 번호에”라는 정렬 규칙을 배웠다. 그런데 포인터는 그냥 수를 담는 변수인데 — 아무 수나 담으면 왜 안 되는가?

답: 담는 것 자체는 막히지 않는 경우가 많다 — 사고는 **따라갈 때** 난다. int*로 역참조한다는 것은 “그 주소에서 4바이트를 int의 눈으로 읽겠다”는 뜻인데(30장), 그 주소가 int의 정렬(4의 배수)을 어기고 있으면 4장의 그 결과들 — 느려지거나, 기계에 따라 그 자리에서 버스 폴트 — 이 따라온다. 표준의 눈으로는 정렬이 맞지 않는 타입으로의 캐스트부터가 이미 계약 밖이다. 포인터의 타입은 눈이자 계약이다.

32.1 정렬과 캐스트 — char*의 특권

포인터 캐스트(타입 바꿔 보기)는 문법상 자유롭지만, 계약상으로는 좁다. 규칙을 실무 문장으로 줄이면 — **더 엄격한 정렬을 요구하는 타입으로의 캐스트는 위험하다**. char*(정렬 1)를 int*(정렬 4)로 바꿔 따라가는 것이 전형적 위반이다. 반대 방향은 안전하다 — 그리고 여기에 C의 의도된 특권이 있다: char*(그리고 unsigned char*)는 어떤 객체든 바이트 단위로 들여다볼 수 있다. 정렬 1이라 어디든 가리킬 수 있고, 표준이 명시적으로 “모든 객체의 표현은 바이트의 눈으로 읽어도 된다”고 허락해 둔 통로다(11장의 엄격한 앨리어싱에도 이 예외가 뚫려 있다). 3장에서 “칸 안은 그저 비트”라 했던 그 관점의 공식 통로가 unsigned char*인 셈이다.

각 타입의 정렬 요구는 alignof로 물을 수 있다(C23 키워드):

```
examples/ch32/align.c
#include <stdio.h>

int main(void)
{
    printf("alignof(char)   = %zu\n", alignof(char));
    printf("alignof(int)    = %zu\n", alignof(int));
    printf("alignof(double) = %zu\n", alignof(double));
    return 0;
}
```

실행 결과

```
alignof(char)   = 1
alignof(int)    = 4
alignof(double) = 8
```

32.2 프로버넌스 — 번호가 같아도 출처가 다르면

12장의 예고를 실무 감각으로 회수한다. 순진한 그림에서 포인터는 번호일 뿐이니, 번호만 맞으면 어떻게 만들었든 따라가도 될 것 같다 — 그러나 현대 C의 계약은 그렇지 않다. **포인터에는 출처(프로버넌스)가 있다**: 어떤 객체에서 유래했는가라는 보이지 않는 딱지다. 실무 규칙 세 줄로 줄인다.

- **포인터 산술은 태어난 객체 안에서만.** 어떤 객체의 주소에서 출발한 포인터를 밀고 당겨(33장) 다른 객체 위에 올려놓는 것은 — 설명 번호가 실제 그 객체와 일치해도 — 계약 밖이다.
- **한 칸 지나서까지는 허용.** 배열의 끝 “다음” 자리(one-past-the-end)를 가리키는 것은 합법이다(따라가지만 않으면). 루프의 끝 조건에 요긴해서 표준이 특별히 뚫어 둔 자리다(33장에서 실전).
- **수로 다루려면 공식 통로로.** 포인터를 정수로 바꿔 저장·연산하는 일(4장의 태그 포인터 같은)은 `uintptr_t` 라는 전용 타입을 경유하는 것이 계약이 마련한 길이다.

문 이런 규칙을 어긴 코드도 “내 컴퓨터에서는 잘 도는” 경우가 많지 않은가?

답 많다 — 그리고 그것이 이 규칙들이 위험한 이유다. 위반의 대가를 걷는 것은 기계가 아니라 **컴파일러의 최적화**다(11장): “포인터는 출처 밖을 가리키지 않는다”는 전제로 재배열·캐싱을 하므로, 위반 코드는 최적화 수준이나 컴파일러 판이 바뀔 때 **조용히** 다른 동작이 된다. “지금 돌았다”는 계약 준수의 증거가 아니라는 것 — 12장의 “추상 기계 위에서 옳은가”가 기준이라는 것 — 이 이 장의 결론이고, 42장에서 이 주제의 전모를 본다.

포인터의 규칙까지 갖췄다. 이제 이 도구들을 갖고 연속된 기억 — 배열 — 으로 간다. 22장부터 미뤄온 `line[100]`의 외상이 다음 장에서 청산된다.

33 배열

이 장이 끝나면

연속된 기억 — 배열이다. 선언과 순회, 배열과 포인터의 진짜 관계 (악명 높은 “무너짐”), 그리고 경계라는 생사의 규칙까지. 22장의 `char line[100]` 외상이 여기서 청산된다.

돌아보기 9장에서 캐시 라인은 “이웃 예수네 칸을 상자째” 나른다고 했고, 배열을 차례로 훑는 코드가 빠른 이유가 그것이라 했다. 그러면 “배열”이란 기억의 눈으로 정확히 무엇인가?

답 같은 타입의 칸들이 **빈틈없이 이웃하여** 늘어선 것 — 그것이 전부다. `int a[5]`는 `int` 다섯 개가 연속된 주소에 나란히 놓인 것이고, 그래서 첫 칸의 주소와 타입만 알면 나머지 위치가 전부 계산된다(i 번 원소 = 시작 주소 + $i \times$ 칸 크기). 이 “계산 가능한 이웃”이라는 성질이 배열의 힘(즉시 접근, 캐시 친화)이자 위험(경계 계산 착오)의 뿌리다.

33.1 선언, 접근, 순회

```
examples/ch33/arr.c
#include <stdio.h>

int main(void)
{
    int a[5] = {3, 1, 4, 1, 5};
    int sum = 0;

    for (int i = 0; i < 5; i += 1) {
        sum += a[i];
    }
    printf("sum: %d\n", sum);

    printf("a[2] = %d, *(a + 2) = %d\n", a[2], *(a + 2));

    int *p = a;          /* 배열 이름이 첫 원소의 주소로 무너진다 */
    printf("sizeof a = %zu, sizeof p = %zu\n", sizeof a, sizeof p);
    return 0;
}
```

실행 결과

```
sum: 14
a[2] = 4, *(a + 2) = 4
sizeof a = 20, sizeof p = 8
```

`int a[5] = {3, 1, 4, 1, 5};` — 타입, 이름, 칸 수, 그리고 중괄호 초기화. 접근은 `a[i]`이고 번호는 0부터다 (첫 원소가 `a[0]`, 마지막이 `a[4]`). 0부터 세는 이유는 위의 deepqa가 이미 답했다 — `a[i]`의 정체는 “시작에서 i 칸 떨어진 곳”이라서, 첫 원소는 떨어진 거리가 0이다.

33.2 배열과 포인터 — 무너짐의 진실

시연의 뒷부분이 이 장의 심장이다. C에서 가장 많은 혼란을 낳은 관계 — 배열과 포인터 — 를 정확히 정리한다.

규칙: 배열의 이름은 대부분의 문맥에서 “첫 원소의 주소”로 무너진다 (decay). `int *p = a;`가 합법인 이유다 — `a`가 `&a[0]`이라는 포인터 값으로 읽힌 것이다. 그리고 `a[i]`라는 표기 자체가 사실 `*(a + i)`의 당의정

이다 — 포인터에 정수를 더하면 “그 타입 크기만큼 i 칸 옆”의 주소가 되고(포인터 산술 — 32장의 규칙들이 여기 걸린다), 그것을 역참조한 것이 인덱스 접근이다. 시연의 `a[2] == *(a + 2)`가 그 확인이다.

△ “배열은 포인터다”

이 유명한 문장은 틀렸다 — 무너짐이 하도 자주 일어나서 생긴 착시다. 배열은 칸 다섯 개짜리 **기억 그 자체**이고, 포인터는 주소 하나를 담는 **다른 변수**다. 시연의 마지막 줄이 결정적 증거다: `sizeof a`는 20 (int 5칸의 총 크기), `sizeof p`는 8(주소 하나의 크기, 30장). 무너짐이 일어나지 않는 대표 문맥이 바로 `sizeof`라서 이 차이가 보인다 — 22장의 `fgets(line, sizeof line, stdin)`이 그릇 크기를 옳게 켜 것도 이 덕분 이다. 한 가지 파급이 중요하다: 배열을 함수에 “넘기면” 무너진 **주소만** 복사되므로(29장), 함수 쪽에서는 `sizeof`로 크기를 알 수 없다 — 그래서 C 함수는 배열과 **길이를 별도 인자로** 함께 받는 것이 관행이다. 7장에서 본 “길이를 데이터 곁에 두는” 표현 방식들의 문제의식이, C 함수 경계에서 이렇게 재현된다.

33.3 경계 — 생사의 규칙

배열의 안전 수칙은 단 하나이고, 타협이 없다 — **유효한 번호는 0부터 칸수-1까지다**. `a[5]`(다섯 칸짜리에서)를 읽거나 쓰는 것은 계약 밖이고, 그 칸은 **남의 기억**이다 — 이웃 변수일 수도, 35장에서 볼 함수 호출의 장부일 수도 있다. 읽으면 쓰레기이고, 쓰면 남의 데이터가 조용히 부서진다 — 배열 경계 침범(buffer overrun)은 C 역사상 가장 많은 사고와 보안 취약점을 낳은 단일 원인이다(다음 두 장이 그 실화들이다).

경계에 관해 표준이 특별히 허락한 자리가 딱 하나 있다 — 32장에서 예고한 **한 칸 지난 자리**(`a + 5`)의 “주소를 만드는 것”까지는 합법이다(따라 가지만 않으면). 순회 관용구가 그 위에 서 있다:

```
for (int *it = a; it != a + 5; it += 1) { /* *it 사용 */ }
```

문 경계를 어기면 컴파일러가 잡아 주지 않는가?

답 일부만이다. `a[7]`처럼 상수로 뻔한 위반은 요즘 컴파일러 경고가 잘 잡지만, 번호가 계산 결과일 때는 컴파일 시점에 알 수 없다 — 경계 검사를 실행 중에 상시로 하는 것은 C가 성능을 위해 **하지 않기로 한** 일이기 때문이다(이 선택의 대가와 보완이 이 부의 남은 주제다). 실행 중 그물은 15장의 ASan이다 — 경계 침범을 일으키는 코드를 ASan 빌드로 돌리면 침범의 순간 파일·줄 번호와 함께 잡아 준다. 그리고 애초에 경계 검사가 내장된 부품을 쓰는 길이 있다 — 35장의 proven이 그 길이다.

연속된 기억을 다루게 됐다. 다음 장은 배열의 가장 유명한 사용처 — 문자열이다. 6·7장의 배경지식(문자=번호, NUL 종단이라는 선택)이 드디어 C 문법과 완전히 만난다.

34 문자열

이 장이 끝나면

C의 문자열을 정면으로 다룬다 — char 배열 + NUL 종단이라는 정체, 문자열 리터럴의 특별한 사정, 길이 재기의 진짜 비용, 그리고 한글이 드러내는 “글자 수 ≠ 바이트 수”의 실전까지. 널 삼형제의 마지막(NUL 문자)이 여기서 정식 취급된다.

돌아보기 7장에서 C가 택한 문자열 표현은 “끝에 표지를 세우는” NUL 종단이라 했고, 그 대가 세 가지(길이는 세어야 한다, NUL을 내용에 못 담는다, 표지를 잃으면 폭주)를 예고했다. 이제 배열(33장)을 배웠으니 — C 문자열의 정확한 정체를 스스로 정의해 보면?

답 char 배열인데, 어딘가에 값 0인 바이트(NUL 문자, '\0')가 있고, “문자열”이란 시작부터 그 표지 직전까지를 뜻한다는 약속 — 이것이 전부다. 별도의 문자열 타입은 없다. 배열, 그리고 약속. 이 장은 그 약속의 사용법과 비용을 다룬다.

34.1 정체 — 배열, 그리고 약속

char greet[] = "안녕"; — 문자열 리터럴로 배열을 초기화하면, 컴파일러가 글자들의 바이트 뒤에 '\0'을 붙여 배열을 만든다. 시연으로 해부한다.

```
examples/ch34/str.c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char greet[] = "안녕";      /* 글자 2개 - 그러나 바이트는? */

    printf("strlen(\"안녕\") = %zu\n", strlen(greet));

    for (size_t i = 0; i < strlen(greet); i += 1) {
        printf("%02X ", (unsigned char)greet[i]);
    }
    printf("\n");

    printf("sizeof greet = %zu (NUL 종단 포함)\n", sizeof greet);
    return 0;
}
```

실행 결과

```
strlen("안녕") = 6
EC 95 88 EB 85 95
sizeof greet = 7 (NUL 종단 포함)
```

읽을거리가 겹겹이다. strlen(표준 <string.h>의 길이 재기)이 6을 답했다 — “안녕”은 글자 둘, 바이트 여섯이다(7장 UTF-8 표의 그대로: 한글 음절은 3바이트 구간). 바이트 덤프 EC 95 88 EB 85 95가 그 여섯 바이트의 민낯이고, sizeof greet는 7 — NUL 표지까지 담은 그릇의 크기다. 6장의 오개념(“한 글자=1바이트”)이 실행 결과로 반증되는 순간 이다.

strlen의 비용도 이제 정확히 말할 수 있다 — 7장의 예고대로, NUL 종단에는 길이가 적혀 있지 않으므로 strlen은 표지를 만날 때까지 한 칸씩 산책한다. 문자열이 길수록 오래 걸리고(칸 수에 비례), 루프 조

건에서 매번 strlen을 부르는 코드가 고전적인 성능 함정이 되는 이유다 (시연의 루프가 사실 그 무늬다 — 짧은 문자열이라 무해하지만, 긴 문자열이라면 길이를 변수에 받아 두는 것이 관행이다).

34.2 문자열 리터럴 — 읽기 전용의 땅

배열 초기화가 아니라 포인터에 리터럴을 담을 수도 있다 — 그리고 여기에 중요한 차이가 있다:

```
char buf[] = "고칠 수 있다"; /* 내 배열로 복사됨 - 수정 가능 */
const char *msg = "고치면 안 된다"; /* 리터럴 원본을 가리킴 */
```

문자열 리터럴 자체는 프로그램에 구워진 **읽기 전용 데이터**다 — 수정을 시도하는 것은 계약 밖이고, 현대 환경에서는 대개 그 자리에서 붕괴한다 (리터럴이 쓰기 금지 구역에 배치되므로 — 4장의 보호 구역이 또 하나의 친절을 베푸는 셈이다). 그래서 리터럴을 가리키는 포인터는 `const char*` 로 선언하는 것이 규칙이다 — 20장의 `const`가 “바꾸지 않겠다”는 문서였다면, 여기서는 “바꾸면 안 되는 것을 바꾸려는 실수”를 컴파일 시점에 막아 주는 자물쇠로 일한다.

● gets — 표준에서 퇴출당한 함수

NUL 종단의 세 번째 대가(표지를 잃으면 폭주)에 얽힌, C 표준 역사상 가장 유명한 장례식이 있다. 초창기 표준 라이브러리의 `gets`는 “한 줄을 읽어 배열에 담는” 함수였는데 — **그릇의 크기를 받지 않았다**. 입력이 그릇보다 길면 그대로 이웃 기억을 덮어쓰는, 경계 침범(33장)이 설계에 내장된 함수였던 것이다. 1988년 인터넷 최초의 대규모 웜(Morris worm)이 이 부류의 결함을 뚫고 퍼졌고, 수십 년의 사고 끝에 C11 표준이 `gets`를 **삭제**했다 — 표준이 자기 함수를 공식적으로 장례 치른 드문 사례다. 22장에서 이 책이 처음부터 `fgets`(그릇 크기를 받는 후계자)를 가르친 사연이 이것이고 — 다음 장에서 이 사고 부류 전체를 정면으로 본다.

문 한글 처리를 제대로 하려면 — 글자 단위로 다루려면 — 어떻게 해야 하는가?

답 두 층을 구별하는 것이 출발점이다. **바이트 층**(C 문자열의 세계)에서는 UTF-8을 그냥 바이트 열로 다루면 되고 — 복사·연결·저장·전송은 글자 경계를 몰라도 안전하다(UTF-8의 자기 동기화 설계 덕이다, 7장). **글자 층**이 필요한 일(글자 수 세기, 자르기, 대소문자)은 UTF-8 해독이 필요하고, 표준 라이브러리의 지원이 빈약한 지점이라 라이브러리를 쓰는 것이 실무다 — proven의 `u8str` 계열이 정확히 이 자리를 맡는다(다음 장에서 첫 만남, 46장에서 본격). 수칙 하나만 명심하면 사고의 대부분이 예방된다: **바이트 번호로 문자열을 자르지 말 것** — 한글 음절의 허리를 자르면 깨진 바이트 열이 된다.

문자열의 정체와 비용을 알았다. 다음 장은 이 부의 전환점이다 — 경계 침범이라는 사고 부류의 해부, 그리고 이 책의 기반 기술 proven의 등장이다. 22장에서 심은 “안전한 입력” 씨앗이 드디어 회수된다.

35 안전한 입력과 proven의 등장

이 장이 끝나면

22장에서 “안전한 입력은 왜 어려운가”를 예고했고, 33·34장에서 그 위험의 뿌리(경계, NUL 종단)를 배웠다. 이 장에서 사고의 해부를 완성하고 — 1장에서 약속한 손님, 이 책의 기반 라이브러리 **proven**이 드디어 등장한다. “필요가 스스로 증명된 시점에”라는 약속 그대로다.

돌아보기 34장의 gets는 그릇 크기를 안 받아서 퇴출됐고, 22장의 fgets는 크기를 받아서 안전하게 했다. 그러면 “크기를 받는다”는 것만으로 입력의 안전 문제가 다 풀리는가?

답 절반만이다. 크기 전달은 **넘침**을 막지만, 입력에는 두 번째 문제가 있다 — **내용을 믿을 수 없다**는 것(22장). 수를 기대했는데 글자가 오고, 짧은 줄을 기대했는데 긴 줄이 오고, 악의적인 상대는 일부러 경계를 노린 입력을 만든다(19장의 서식 문자열 공격이 그 맛보기였다). 안전한 입력이란 [넘침 차단 + 실패의 명시적 처리]의 합이고, 이 장의 두 기둥이 그 둘이다.

35.1 사고의 해부 — 경계 침범이라는 유행병

33장의 경계 규칙과 34장의 NUL 종단을 합치면, C 역사상 가장 값비싼 사고 무늬가 조립된다. 그릇보다 긴 입력이 이웃 기억을 덮어쓴다(경계 침범) — 덮어써진 곳이 하필 **함수 호출의 복귀 장부**(다음 장에서 정체를 본다)라면, 공격자는 입력만으로 프로그램의 실행 흐름을 탈취한다. 이것이 버퍼 오버플로 공격의 골자이고, Morris worm(1988, 34장)부터 오늘의 보안 공지까지 면면히 이어지는 유행병이다. 통계적으로도 압도적이다 — 주요 보안 기관들의 “가장 위험한 소프트웨어 결함” 목록에서 경계 침범 계열은 수십 년째 최상위 단골이다.

C가 실행 중 경계 검사를 “하지 않기로” 했기 때문에(33장), 방어는 층층이 쌓는 수밖에 없다 — 크기를 받는 함수(fgets), 경고와 새니타이저(15장), 그리고 **애초에 검사가 내장된 부품**. 마지막 층이 이 장의 손님이다.

35.2 proven — 검증된 기본기라는 부품

proven은 이 책의 저자가 만든 C 라이브러리로(1장의 정직 공개 그대로), 설계 사상은 한 문장이다 — 사고가 가장 잦은 기본기(문자열, 입력 해석, 수 변환)를, 실패가 값으로 드러나고 경계가 항상 검사되는 **형태로 제공 한다**. 7장에서 본 표현 방식의 언어로 말하면, NUL 종단의 세계 위에 길이·용량 명시 계열의 안전한 층을 세운 것이다.

첫 시연으로 22장의 숙제 — 입력 해석의 실패 처리 — 를 proven으로 풀어 본다. 성공 경로와 실패 경로를 둘 다 보인다:

```
examples/ch35/scan.c
#include <proven.h>
#include <stdio.h>

void try_parse(const char *text)
{
    proven_scan_t sc = proven_scan_init(proven_u8str_view_from_cstr(text));
    proven_result_i64_t r = proven_scan_i64(&sc);

    if (proven_is_ok(r.err)) {
        printf("%s" -> ok: %lld\n", text, (long long)r.val);
    } else {
        printf("%s" -> failed (not a number)\n", text);
    }
}
```

```

    }
}

int main(void)
{
    try_parse(" 42 and some text");
    try_parse("forty-two");
    return 0;
}

```

실행 결과

```

" 42 and some text" -> ok: 42
"forty-two" -> failed (not a number)

```

읽는 법 — proven_scan_init이 글자 열 위에 해석기(스캐너)를 세우고, proven_scan_i64가 정수 하나를 해석한다. 반환값이 이 설계의 심장이다: **값이 아니라 {err, val} 꾸러미를 돌려주고**, 쓰는 쪽은 proven_is_ok 로 성공을 확인한 뒤에만 값을 꺼낸다. sscanf(22장)와 비교하면 — “성공 개수”라는 관례적 신호 대신 **실패가 타입으로 명시**되고, 함수에는 [[nodiscard]](C23 — “이 반환값을 버리면 경고하라”는 표기)가 붙어 있어 **실패 확인을 잊는 실수를 컴파일러가 잡아 준다**. 41장에서 정식으로 다룰 “오류는 값이다” 규율의 실물이다.

문 표준 라이브러리로도 되는 일을 왜 라이브러리를 더해서 하는가 — 부품이 늘면 배울 것도 있는데.

답 이 책의 대답은 층의 구별이다. **개념은 표준 C로 온전히 배웠다** — 30-34장까지 proven 없이 온 것이 그 증거이고, proven을 걷어내도 이 책의 C 지식은 성립한다(1장의 약속). 그 위에서 **실무의 기본기**를 고를 때는, 표준 라이브러리의 역사적 함정(gets의 장래, scanf의 꼬임, 검사 없는 문자열 함수들)을 아는 채로 고르는 것이 현대적 선택이다 — Rust나 Zig가 언어 차원에서 내장한 안전을(1장), C에서는 부품 선택으로 얻는 것이다. proven은 그 선택지의 하나이자 이 책의 시연 재료이고, 같은 사상의 다른 라이브러리를 골라도 원리는 같다.

문 proven 함수들의 이름이 길다 — proven_scan_i64처럼. 왜 이렇게 짓는가?

답 C에는 이름 공간이 없기 때문이다(21장의 스코프는 블록의 것이고, 파일 경계를 넘는 이름들은 한 마당에서 산다 — 43장). 라이브러리들이 같은 이름을 쓰면 링크 단계(14장)에서 충돌하므로, C 라이브러리의 관행은 **접두어가 이름 공간을 대신하는 것**이다 — proven_이 그 답장이다. 길지만, 어느 부품의 것인지 이름만 봐도 드러난다는 실속이 있다.

(이 장은 첫 만남이라 필요한 만큼만 쓴다 — 라이브러리의 설계 전모와 사용법은 제11부에서 세 장에 걸쳐 다룬다. 이 책이 C 입문서이면서 동시에 proven의 안내서이기도 한 이유다.)

입력의 안전을 갖췄다. 그런데 이 장에서 “함수 호출의 복귀 장부”라는 말이 설명 없이 지나갔다 — 기억의 어디에 무엇이 살고, 언제 태어나고 죽는가. 다음 장이 그 지도 — 수명과 저장 기간 — 다.

36 수명과 저장 기간

이 장이 끝나면

기억의 지도를 그린다 — 변수는 어디에 살고, 언제 태어나 언제 죽는가. 자동 수명과 정적 수명, 스택이라는 장부, 그리고 이 부에서 가장 조심 해야 할 사고(사라진 것의 주소를 간직하기)까지.

돌아보기 21장에서 스코프 — 이름이 보이는 범위 — 를 배웠다. 그런데 35장에서 “함수 호출의 복귀 장부”라는 말이 나왔다. 이름이 안 보이게 되는 것과, 그 기억이 사라지는 것은 같은 일인가?

답 다른 일이고, 이 구별이 이 장의 뼈대다. **스코프**는 컴파일 시점의 개념(어디서 그 이름을 쓸 수 있는가)이고, **수명**은 실행 시점의 개념(그 기억이 언제부터 언제까지 유효한가)이다. 대개는 함께 가지만, 어긋나는 순간이 사고가 된다 — 이름은 사라졌는데 주소가 남아 있는 경우다(아래 오개념 블록).

36.1 두 가지 수명

C의 지역 변수는 기본적으로 **자동 저장 기간**(automatic)을 갖는다 — 블록에 들어설 때 태어나고 나갈 때 죽는다. 함수 호출마다 매개변수와 지역 변수가 새로 태어나는 것이 29장 재귀가 성립한 근거였다.

여기에 `static`을 붙이면 **정적 저장 기간**(static)이 된다 — 프로그램 시작 전에 한 번 태어나 끝날 때까지 산다(초기화도 딱 한 번). 시연으로 둘을 대조한다.

```
examples/ch36/life.c
#include <stdio.h>

int next_ticket(void)
{
    static int issued = 0;    /* 정적 수명: 호출 사이에도 살아남는다 */
    issued += 1;
    return issued;
}

int fresh_count(void)
{
    int n = 0;                /* 자동 수명: 호출마다 새로 태어난다 */
    n += 1;
    return n;
}

int main(void)
{
    /* 부수효과 있는 호출은 문장을 나눈다 - 28장의 수칙 그대로 */
    int t1 = next_ticket();
    int t2 = next_ticket();
    int t3 = next_ticket();
    printf("next_ticket: %d %d %d\n", t1, t2, t3);

    int f1 = fresh_count();
    int f2 = fresh_count();
    int f3 = fresh_count();
    printf("fresh_count: %d %d %d\n", f1, f2, f3);
    return 0;
}
```

실행 결과

```
next_ticket: 1 2 3
fresh_count: 1 1 1
```

next_ticket의 issued는 호출 사이에도 값을 간직해 1, 2, 3으로 자라고, fresh_count의 n은 호출마다 새로 태어나 언제나 1이다. 같은 “함수 안의 변수”인데 수명이 다른 것이다. (덧붙여, 시연이 호출을 문장으로 나눈 것에 주목하면 된다 — 부수효과 있는 호출을 한 수식에 몰면 평가 순서가 미지정이라는 29장의 수칙 그대로다.)

36.2 스택 — 호출의 장부

자동 변수들이 사는 곳에는 이름이 있다 — 스택(stack)이다. 함수를 부를 때마다 그 호출을 위한 칸 묶음(스택 프레임)이 얹히고, 함수가 끝나면 통째로 걷힌다. 프레임에는 지역 변수·매개변수와 함께 돌아갈 주소(복귀 주소)가 들어 있다 — 35장에서 이름만 스쳤던 “복귀 장부”의 정체다.

그림이 갖춰지면 두 가지가 한 번에 설명된다. 첫째, 29장의 재귀가 왜 겹겹이 쌓이는지 — 호출마다 프레임이 하나씩 얹히기 때문이다. 그리고 너무 깊이 재귀하면 스택 공간이 바닥나 프로그램이 붕괴한다(스택 오버플로 — 무한 재귀의 대표 증상이다). 둘째, 35장의 경계 침범 공격이 왜 그토록 위험한지 — 스택에 놓인 배열을 넘쳐 쓰면 같은 프레임의 복귀 주소를 덮어쓸 수 있고, 그러면 함수가 끝날 때 공격자가 정한 곳으로 “돌아간다”. 배열 하나의 경계가 프로그램의 통제권과 이어져 있는 것이다.

△ “함수가 끝나도 그 안의 변수를 가리키는 주소는 쓸 수 있다”

포인터를 배운 직후 가장 흔한 사고이고, 무서운 점은 **한동안 잘 도는 것처럼 보인다**는 것이다. 함수가 끝나면 프레임이 걷히지만 — 그 자리의 비트가 즉시 지워지는 것은 아니어서, 죽은 변수의 주소를 따라가도 잠시는 예전 값이 읽힌다. 그러다 다른 함수가 그 자리를 프레임으로 쓰는 순간 값이 뒤바뀐다 — 시간이 지나서, 엉뚱한 곳에서 터지는 버그가 되는 것이다. 이런 주소를 **댕글링 포인터(dangling pointer)**라 하고, 규칙은 하나다: **지역 변수의 주소는 그 함수보다 오래 살아서는 안 된다**. 함수가 만든 결과를 오래 살려 보내려면 정적 수명을 쓰거나, 다음 장의 동적 메모리를 쓰거나, 호출자가 준 그릇에 채워 주는(30장의 & 관용구) 셋 중 하나다. 15장의 ASan이 실행 중에 이 사고를 잡아 주는 대표 도구이기도 하다.

문 전역 변수 — 함수 밖에 선언한 변수 — 도 정적 수명인가?

답 그렇다. 함수 밖의 변수는 정적 저장 기간을 갖고 프로그램 내내 산다. 다만 수명과 별개로 **가시성**의 문제가 붙는데 — 여러 파일에서 보이게 할지 이 파일에만 둘지를 정하는 것이 43장의 주제(연결)다. 그리고 실무의 권고는 오래된 것이다: **전역 가변 상태는 최소로**. 어디서든 바뀔 수 있는 값은 추적이 어렵고, 10장의 멀티코어 세계에서는 사고의 근원이 된다. 함수 안의 static도 같은 성질을 작게 가진 것이라(시연의 issued), 편리하지만 남용하지 않는 것이 관행이다.

기억의 지도에서 두 자리 — 스택(자동)과 정적 영역 — 를 익혔다. 남은 한 자리가 이 부의 마지막이다: 크기를 실행 중에 정하고, 원하는 만큼 살려 두는 기억. 다음 장의 동적 메모리다.

37 동적 메모리

이 장이 끝나면

제7부의 마지막 자리 — 실행 중에 크기를 정하고, 원하는 때까지 살려 두는 기억이다. `malloc`과 `free`, 소유라는 개념, 그리고 이 세계의 세 가지 대표 사고(누수·이중 해제·해제 후 사용)까지. 기억의 지도가 여기서 완성된다.

돌아보기 36장에서 자동 수명(함수와 함께 죽는다)과 정적 수명(프로그램 내내 산다)을 배웠다. 그러면 “입력에 따라 크기가 정해지고, 필요한 동안만 살아야 하는” 기억은 — 둘 중 어디에 두는가?

답 어느 쪽도 맞지 않는다. 크기가 실행 중에 정해지고 수명이 프로그램의 논리에 따라 갈리는 기억이 필요하다 — 그래서 세 번째 자리가 있다: **동적 저장 공간**, 흔히 **힙(heap)**이라 부르는 창고다. 이 자리의 규칙은 앞의 둘과 결정적으로 다르다 — **태어남과 죽음을 프로그래머가 직접 지시한다**.

37.1 빌리고, 돌려준다

문법은 함수 두 개다. `malloc`(바이트 수)는 창고에서 그만큼의 연속된 칸을 빌려 그 시작 주소를 돌려주고, `free`(주소)는 빌린 것을 돌려준다.

```
examples/ch37/dyn.c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int count = 5;
    int *scores = malloc(count * sizeof *scores); /* 창고에서 빌린다 */

    if (scores == nullptr) { /* 빌리기는 실패할 수 있다 */
        printf("allocation failed\n");
        return 1;
    }

    int sum = 0;
    for (int i = 0; i < count; i += 1) {
        scores[i] = (i + 1) * 10;
        sum += scores[i];
    }
    printf("total: %d\n", sum);

    free(scores); /* 빌린 것은 돌려준다 */
    return 0;
}
```

실행 결과

```
total: 150
```

세 가지 관행이 시연에 박혀 있다.

크기 계산은 `sizeof *포인터`. `count * sizeof *scores`는 “원소 하나의 크기 × 개수”인데, 타입 이름 대신 **대상**으로 크기를 물었다 — 나중에 타입이 바뀌어도 이 줄은 고칠 필요가 없다.

빌리기는 실패할 수 있다. malloc은 창고가 부족하면 널 포인터를 돌려준다 — 그래서 쓰기 전에 확인한다(31장의 수칙이 여기서 실전이 된다). 확인 없이 쓰면 널 역참조로 붕괴다.

빌린 것은 돌려준다. free를 잊으면 그 칸은 프로그램이 끝날 때까지 묶여 있다 — 메모리 누수(leak)다. 짧은 프로그램에서는 티가 안 나지만, 오래 도는 서버에서는 조금씩 새어 나가 결국 기계를 잡아먹는다.

37.2 소유 — 누가 돌려줄 책임을 지는가

malloc이 준 주소는 여러 변수에 복사될 수 있고, 함수 사이를 오갈 수도 있다. 그런데 free는 정확히 한 번 불러야 한다 — 여기서 C 프로그래밍의 핵심 규율이 나온다: 어느 시점에든 그 기억을 해제할 책임을 지는 주체 하나를 정해 두는 것, 소유(ownership)라는 관념이다. C에는 소유를 강제하는 문법이 없다 — 그래서 소유는 주석과 이름과 규약으로 표현 된다(“이 함수는 반환한 포인터의 소유권을 넘긴다”처럼). 현대 언어들인 소유를 타입 체계로 끌어올린 것(Rust의 소유권, C++의 스마트 포인터)은 이 규율을 기계가 강제하게 만든 결과다 — 1장에서 본 이웃 언어들인 문제의식이 바로 이 자리에서 나왔다.

● 세 가지 대표 사고 — 누수, 이중 해제, 해제 후 사용

동적 메모리의 사고는 세 얼굴로 온다. 누수는 돌려주기를 잊는 것 — 느리게 죽는 병이다. 이중 해제(double free)는 같은 주소를 두 번 free하는 것으로, 창고 관리 장부를 망가뜨려 그 뒤의 모든 할당이 오염된다. 가장 위험한 것은 해제 후 사용(use-after-free) — 돌려준 칸의 주소를 계속 쓰는 것이다. 36장의 댕글링 포인터가 힙에서 재현되는 것인데, 창고는 그 칸을 곧 다른 요청에 내주므로 공격자가 그 자리에 자기 데이터를 놓을 수 있다. 브라우저와 커널의 심각한 취약점 목록에서 use-after-free는 오늘날도 최상위 단골이고, 그래서 현대 시스템 프로그래밍 전체가 “소유를 언어가 강제하게 하자”는 방향으로 움직여 왔다. C에서의 방어는 규율 + 도구다 — free 직후 포인터에 nullptr를 대입하는 관행, 그리고 15장의 ASan(이 세 사고를 모두 잡아내는 것이 ASan의 주특기다).

문 그러면 동적 메모리를 되도록 피하는 것이 상책인가?

답 실제로 그것이 첫 번째 전략이고, 이 책이 여기까지 malloc 없이 온 이유이기도 하다 — 크기를 아는 데이터는 자동 변수(스택)로 두는 것이 가장 빠르고 가장 안전하다(해제할 것이 없으니 세 사고가 원천 봉쇄된다). 임베디드나 안전 필수 분야에서는 아예 동적 할당을 금지하는 규약도 흔하다. 그러나 실행 중에 크기가 정해지는 데이터(사용자가 준 목록, 파일 내용)는 결국 창고가 필요하고 — 그때는 소유를 명확히 하고, 도구로 검사하고, 잘 만들어진 부품(35장의 proven처럼 할당과 경계를 함께 관리하는 계열)을 쓰는 것이 실무의 답이다.

37.3 제7부를 닫으며

기억의 지도가 완성됐다 — 레지스터와 캐시라는 사다리(9장) 위에, C의 세 저장 기간(자동·정적·동적)이 놓이고, 그 위를 포인터가 다닌다. 30장에서 개념을 얻고, 30-32장에서 규칙을, 32-34장에서 연속된 기억과 문자열을, 35장에서 안전한 부품을, 35-37장에서 수명과 창고를 배웠다. C의 힘과 위험이 함께 사는 자리를 한 바퀴 돈 것이다.

다음 부는 짧지만 오래 미뤄 온 곳이다 — 제5부에서 “타입을 만드는 선언은 메모리 모델을 갖춘 뒤”라며 미뤄 두었던 구조체와 공용체. 이제 그 조건이 갖춰졌다.

제8부 — 자료의 모양

38 구조체

이 장이 끝나면

제5부에서 “타입을 만드는 선언은 메모리 모델을 갖춘 뒤”라며 미뤄 둔 약속을 지킨다. 여러 값을 하나로 묶는 타입 — 구조체다. 선언과 초기화, 두 가지 접근 표기(.와 ->), 그리고 구조체가 값으로 오는 방식까지.

돌아보기 20장에서 “타입은 값의 집합 + 연산의 약속”이라 했고, 지금까지 쓴 타입은 전부 미리 있는 것들(int, double, char, 포인터)이었다. 그러면 프로그래머가 **새 타입을 만든다**는 것은 무슨 뜻인가?

답 기억의 모양을 새로 정하고 이름을 붙이는 일이다. “정수 둘이 나란히 있는 덩어리를 점(point)이라 부르겠다”고 선언하면, 그 순간부터 point는 변수를 만들고 함수에 넘기고 배열로 늘어놓을 수 있는 어엿한 타입이 된다. 33장의 배열이 **같은 타입의 반복**이었다면, 구조체는 **다른 타입의 묶음**이다 — 그리고 그 묶음에 이름이 붙는 순간, 프로그램의 어휘가 늘어난다.

38.1 선언, 초기화, 접근

```
examples/ch38/point.c
#include <stdio.h>

struct point {
    int x;
    int y;
};

struct point moved(struct point p, int dx, int dy)
{
    return (struct point){ .x = p.x + dx, .y = p.y + dy }; /* 복합 리터럴 */
}

int main(void)
{
    struct point a = { .x = 3, .y = 4 }; /* 지정 초기화 */
    struct point b = moved(a, 10, -1);

    printf("a = (%d, %d)\n", a.x, a.y); /* 값으로 접근: . */
    printf("b = (%d, %d)\n", b.x, b.y);

    struct point *p = &b;
    printf("p->x = %d\n", p->x); /* 포인터로 접근: -> */

    printf("sizeof(struct point) = %zu\n", sizeof(struct point));
    return 0;
}
```

실행 결과

```
a = (3, 4)
b = (13, 3)
p->x = 13
sizeof(struct point) = 8
```

선언은 `struct point { int x; int y; };` — 중괄호 안의 각 항목을 **멤버(member)**라 부른다. 이 선언 자체는 기억을 잡지 않는다. “이런 모양의 타입이 있다”는 정의일 뿐이고, `struct point a;`처럼 써야 변수가 생긴다.

초기화는 시연처럼 멤버 이름을 적는 **지정 초기화(designated initializer, C99)**를 권한다 — `{ .x = 3, .y = 4 }`. 순서에 기대는 `{3, 4}`보다 읽기 좋고, 멤버가 늘거나 순서가 바뀌어도 안전하다. 적지 않은 멤버는 0으로 채워진다.

접근은 두 표기다 — 값에는 점(`a.x`), 포인터에는 화살표(`p->x`). 화살표는 사실 `(*p).x`의 줄임이다(30장의 역참조 + 점). 포인터로 구조체를 다루는 일이 압도적으로 흔해서 전용 표기를 둔 것이다.

복합 리터럴 — 시연의 `(struct point){ .x = ..., .y = ... }`는 “그 자리에서 이름 없는 구조체 값을 하나 만드는” 표기다(C99). 반환값이나 인자로 구조체를 즉석에서 넘길 때 요긴하다.

38.2 구조체는 값이다

C에서 구조체는 **값처럼** 다뤄진다 — 대입하면 통째로 복사되고, 함수에 넘기면 29장의 규칙 그대로 복사되어 건너가며, `return`으로 통째로 돌려 줄 수도 있다. 시연의 `moved(a, 10, -1)`이 그 확인이다: `a`는 그대로이고 새 값 `b`가 나왔다.

이 복사는 멤버를 그대로 옮겨 적는 **얕은 복사(shallow copy)**다. 멤버가 전부 수라면 문제가 없지만, 멤버 중에 포인터가 있으면 주소가 그대로 복제되어 원본과 사본이 같은 곳을 가리키게 된다 — 이 사실이 뒤에서 자기를 가리키는 자료를 다룰 때 결정적인 함정이 된다.

다만 실무에서는 큰 구조체를 값으로 주고받는 대신 **포인터로 넘기는** 관행이 흔하다 — 복사 비용을 아끼기 위해서다(9장의 기억 사다리를 떠올리면 큰 덩어리의 복사가 공짜가 아님이 보인다). 읽기만 할 때는 `const struct`

`point *p`처럼 `const` 포인터로 받는 것이 관행이다 — 20장의 `const`가 “이 함수는 원본을 건드리지 않는다”는 계약 표시로 일하는 것이다.

문 `struct point`라고 매번 `struct`를 붙이는 것이 번거롭다 — 줄일 수 없는가?

답 전통적으로는 `typedef`로 별명을 만들어 왔다 — `typedef struct point`

`point_t;`처럼. 다만 취향과 유파가 갈리는 지점이라(별명이 “이것이 구조체”라는 정보를 감춘다는 반론이 있다), 이 책은 지면에서 정체가 보이도록 `struct`를 붙여 쓴다. 어느 쪽이든 일관성이 중요하다.

문 구조체 안에 자기 자신을 멤버로 넣을 수 있는가 — 리스트 같은 것을 만들려면 필요할 것 같은데.

답 자기 자신을 **값으로** 담을 수는 없다(무한한 크기가 되므로). 그러나 **자신을 가리키는 포인터**는 담을 수 있고, 바로 그것이 연결 자료구조의 씨앗이다: `struct node { int value; struct node *next; };` 이 한 줄에 30장의 포인터와 37장의 동적 메모리가 만나면 연결 리스트·트리 같은 구조가 열린다 — 이 책의 범위를 넘는 자료구조의 세계이지만, 문을 여는 열쇠가 여기 있다는 것은 알아 둘 만하다.

값들을 묶는 법을 얻었다. 다음 장은 같은 기억을 **다른 눈으로** 보는 장치 — 공용체와 표현의 세계다. 3장에서 예약해 둔 엔디언 관찰 시연이 드디어 열린다.

39 공용체와 표현

이 장이 끝나면

같은 기억을 다른 눈으로 보는 장치 — 공용체다. 그리고 이 장은 표현의 장이기도 하다: 3장에서 예약한 엔디안 관찰 시연을 실행하고, 구조체의 숨은 빈틈(패딩)을 눈으로 확인한다. 표현과 추상의 분리라는 이 책의 후렴구가 마지막으로 크게 울리는 자리다.

돌아보기 11장에서 “float의 비트를 uint32의 눈으로 읽는” 옛 기법이 엄격한 앨리어싱 위반이며, 옳은 방법은 memcpy나 공용체라고 했다. 그러면 공용체란 정확히 무엇이길래 그 자리에 서는가?

답 여러 멤버가 같은 기억을 공유하는 타입이다. 구조체가 멤버들을 나란히 놓는다면(38장), 공용체는 멤버들을 겹쳐 놓는다 — 크기는 가장 큰 멤버에 맞고, 어느 순간에든 실제로 담긴 것은 하나다. “같은 비트를 이 눈으로도 저 눈으로도 본다”는 3장의 관점이 문법이 된 것이다.

39.1 공용체 — 겹쳐 놓기

문법은 구조체와 쌍둥이다. struct를 union으로 바꾸면 된다:

```
union bits32 {
    uint32_t as_int;
    float    as_float;
};
```

멤버 접근도 같다(u.as_int). 다른 것은 기억의 배치뿐이다 — 두 멤버가 같은 4바이트를 공유하므로, as_float에 쓰고 as_int로 읽으면 같은 비트를 다른 해석으로 보게 된다. C 표준은 이 “쓴 멤버와 다른 멤버로 읽기”(타입 페닝)를 공용체에 한해 허용한다 — 11장의 포인터 캐스트 방식과 달리 계약 안이라는 점이 결정적 차이다(다만 읽은 값이 그 타입의 유효한 값이 아닐 수 있다는 주의는 남는다).

39.2 표현을 눈으로 — 엔디안과 패딩

3장에서 “이 확인을 C로 실제로 해 보이는 것이 이 장의 시연”이라 예약해 두었다. 이제 갓는다. 여기서는 공용체 대신 가장 이식성 좋은 방법 — 32장에서 배운 바이트의 눈(unsigned char)과 memcpy — 을 쓴다.

```
examples/ch39/endian.c

#include <stdio.h>
#include <stdint.h>
#include <string.h>

int main(void)
{
    uint32_t value = 0x12345678u;
    unsigned char bytes[4];

    memcpy(bytes, &value, sizeof value); /* 표현을 바이트로 들여다본다 */

    printf("value: 0x%08X\n", value);
    printf("memory order: %02X %02X %02X %02X\n",
           bytes[0], bytes[1], bytes[2], bytes[3]);
    printf("this machine is %s-endian\n",
           bytes[0] == 0x78 ? "little" : "big");

    struct padded {
        char tag; /* 1바이트 */
        int count; /* 4바이트 - 정렬 때문에 앞에 빈틈이 생긴다 */
    };
```

```
};
printf("1 + 4 = 5, but sizeof = %zu\n", sizeof(struct padded));
return 0;
}
```

실행 결과

```
value: 0x12345678
memory order: 78 56 34 12
this machine is little-endian
1 + 4 = 5, but sizeof = 8
```

첫 부분이 3장의 그림 그대로다. 0x12345678이 메모리에 78 56 34 12 순서로 놓였다 — 이 책의 검증 기계는 리틀 엔디안이라는 뜻이고, 3장의 도해가 실물로 확인된 것이다. (빅 엔디안 기계에서 이 예제를 돌리면 12 34 56 78이 찍히고 판정 문장도 바뀐다 — 코드는 그대로다.)

둘째 부분은 구조체의 숨은 사정이다. char 하나(1바이트)와 int 하나(4바이트)를 담은 구조체의 크기가 5가 아니라 8이다 — 4장의 정렬 규칙 때문에 char 뒤에 3바이트의 **패딩(padding, 빈틈)**이 끼워진 것이다. int 멤버가 4의 배수 자리에서 시작해야 기계가 한 움큼에 집을 수 있기 때문이다(4장). 그래서 실무 관행 하나가 나온다 — **큰 멤버부터 배치하면 빈틈이 줄어든다**. 구조체 수백만 개를 다루는 코드에서는 이 배치 하나가 메모리와 캐시 효율(9장)을 좌우한다.

● 빈틈이 흘린 비밀 — 커널의 패딩 정보 유출

패딩은 아무도 쓰지 않는 빈칸이라 무해해 보이지만, 보안의 눈으로 보면 **초기화되지 않은 기억**이다. 운영체제 커널이 구조체를 사용자 프로그램에게 통째로 복사해 줄 때 이 빈틈이 함께 건너가는데 — 멤버들은 전부 채워 넣어도 빈틈에는 그 자리에 남아 있던 **다른 데이터의 잔해**가 들어 있다. 공격자는 이 부스러기를 모아 커널 메모리의 내용이나 주소 배치를 엿볼 수 있고, 그것이 다음 공격의 발판이 된다. 리눅스를 비롯한 주요 커널들이 이 부류의 정보 유출 취약점을 수십 건 고쳐 왔고, 오늘의 대응은 단순하다 — 사용자에게 넘기는 구조체는 **멤버를 채우기 전에 통째로 0으로 지운다**. 20장에서 “선언과 동시에 초기화한다”고 한 규칙이, 보이지 않는 빈칸에까지 적용되는 순간이다.

△ “구조체를 그대로 파일에 쓰거나 네트워크로 보내면 된다”

솔깃한 생각이고, 실제로 많이 시도된 방법이다 — 구조체의 바이트를 통째로 저장하면 코드가 짧아지니까. 그러나 이 장이 방금 보인 두 사실이 그것을 가로막는다: 바이트 순서가 기계마다 다르고(엔디안), 빈틈의 크기와 위치도 컴파일러-플랫폼마다 다르다(패딩). 한 기계에서 쓴 파일이 다른 기계에서 깨지는 것이다 — 3장의 NUXI 사건이 파일 형식의 세계에서 재현되는 셈이다. 정답은 **직렬화(serialization)**다: 멤버를 하나씩, 약속된 크기와 바이트 순서로(네트워크 바이트 순서 — 3장) 적고 읽는 코드를 명시적으로 쓰는 것. 표현은 기계의 사정이고, 파일과 통신은 약속의 세계라는 것 — 두 세계를 잇는 다리는 손으로 놓아야 한다.

문 공용체는 그러면 언제 쓰는 것이 정석인가?

답 두 가지 자리다. 첫째, 방금 본 **표현 들여다보기**(타입 페닝) — 부동 소수점 비트를 검사하거나 하드웨어 레지스터를 여러 눈으로 보는 저수준 코드다. 둘째, 더 흔한 쓰임인 **태그된 공용체(tagged union)**: 구조체 안에 “지금 어느 멤버가 유효한가”를 알리는 표시(태그)와 공용체를 함께 넣어, “이 값은 정수이거나, 실수이거나, 문자열이다” 같은 선택형 데이터를 표현하는 것이다. 인터프리터의 값

표현, 설정 파일 파서 같은 곳의 기본 도구이고 — 4장에서 본 태그 포인터가 비트 차원의 같은 발상이었다. 현대 언어들의 열거형(Rust의 enum, Swift의 associated value)이 이 패턴을 언어 차원으로 끌어올린 것이다.

39.3 제8부를 닫으며

타입을 만드는 두 문법을 얻었다 — 나란히 놓는 구조체(38장)와 겹쳐 놓는 공용체(39장). 그리고 그 과정에서 표현의 현실(엔디언·패딩)을 눈으로 확인했다. 제2부의 배경지식이 문법으로 완전히 회수된 것이다.

다음 부는 정밀의 부다 — 6장에서 배운 근사의 수학이 C의 실수 타입으로 내려오고(40장), 29장에서 씨앗을 심은 계약의 관점이 오류 처리로 자라며 (41장), 이 책이 곳곳에서 예고해 온 “계약 밖”의 세계 — 정의되지 않은 동작 — 를 정면으로 만난다(42장).

제9부 — 정밀

40 실수 — 근사의 수학

이 장이 끝나면

6장에서 지면으로 배운 근사의 세계가 드디어 C 코드로 내려온다. `float` 와 `double`의 선택, 비교의 올바른 방법(엡실론 — 절대와 상대), 그리고 특수값(무한대·NaN)까지. 6장의 세 사건이 실행 결과로 확인된다.

돌아보기 6장에서 $0.1 + 0.2$ 와 0.3 은 마지막 비트 하나(1 ulp) 차이의 이웃이라 했고, 비교는 “충분히 가까운가”로 바뀌어야 한다고 했다. 그러면 그 “충분히”의 기준 — 엡실론 — 은 어떻게 정하는가?

답 값의 크기에 따라 달라져야 한다는 것이 6장 셋째 사건의 교훈이다. 0 근처에서는 눈금이 촘촘하니 작은 고정값이면 되지만, 10^{16} 쯤에서는 눈금 간격 자체가 1을 넘으므로 같은 기준이 무의미해진다. 그래서 실무는 두 가지를 갖춘다 — **절대 오차**(0 근처용)와 **상대 오차**(크기에 비례). 이 장의 시연이 둘을 나란히 보인다.

40.1 타입 선택과 비교

```
examples/ch40/eps.c

#include <math.h>
#include <stdio.h>

/* 절대 오차: 0 근처의 비교에 쓴다 */
bool near_abs(double a, double b, double eps)
{
    return fabs(a - b) < eps;
}

/* 상대 오차: 값이 커지면 눈금도 커지므로 크기에 비례한 허용치를 쓴다 */
bool near_rel(double a, double b, double rel)
{
    double scale = fabs(a) > fabs(b) ? fabs(a) : fabs(b);
    return fabs(a - b) <= rel * scale;
}

int main(void)
{
    double sum = 0.1 + 0.2;

    printf("0.1 + 0.2 == 0.3 ?      %d\n", sum == 0.3);
    printf("compare with absolute eps? %d\n", near_abs(sum, 0.3, 1e-9));
    printf("%.20f\n", sum);
    printf("%.20f\n", 0.3);

    double big = 1e16;
    printf("1e16 + 1 == 1e16 ?      %d (gap wider than 1)\n", big + 1 == big);
    printf("compare with relative eps? %d\n", near_rel(big + 1, big, 1e-12));
    return 0;
}
```

실행 결과

```
0.1 + 0.2 == 0.3 ?      0
compare with absolute eps? 1
0.30000000000000004441
```

```
0.29999999999999998890
1e16 + 1 == 1e16 ?      1 (gap wider than 1)
compare with relative eps? 1
```

첫 세 줄이 6장 첫 사건의 실행 확인이다 — `==`는 거짓이고, 20자리까지 찍어 보면 두 수가 마지막에서 갈린다. `near_abs`(절대 오차)를 쓰면 참이 된다.

뒷부분이 셋째 사건(흡수)의 확인이다 — 10^{16} 에 1을 더해도 값이 그대로 다. 그 크기에서는 표현 가능한 이웃 사이의 간격이 이미 1보다 넓기 때문이다(6장의 눈금 계산). 여기서는 절대 오차가 아니라 `near_rel`(상대 오차)이 옳은 도구다.

실무 수칙으로 줄이면 셋이다. **기본은 `double`** — 6장에서 본 대로 정밀도의 여유가 다르고, `float`는 메모리·대역폭이 아쉬운 곳에서만 고른다. **`==`는 쓰지 않는다** — 두 실수가 같은지 묻는 코드는 거의 언제나 의심 대상이다(정수와 비교할 때, 0과 비교할 때 등 예외는 있지만 의식 적으로 판단해야 한다). **허용치는 문제에서 나온다** — 계산의 성격과 값의 크기를 보고 정하는 것이지 마법 상수가 있는 것이 아니다.

40.2 특수값 — 무한대와 NaN

IEEE 754(6장)는 평범한 수 말고도 특별한 값을 정의해 두었다. **무한대** (양·음)는 넘침이나 0으로 나누기의 결과로 나오고, **NaN**(Not a Number)은 “수가 아님” — $\frac{0}{0}$ 이나 음수의 제곱근 같은 정의 불가 연산의 결과다. 정수의 0 나누기가 계약 밖(24장)인 것과 대조적으로, **부동소수점의 0 나누기는 정의된 동작** (무한대)이라는 점이 흥미롭다 — IEEE 754가 계산을 멈추는 대신 특별한 값으로 흘려보내는 설계를 택했기 때문이다.

NaN에는 유명한 성질이 하나 있다 — **자기 자신과도 같지 않다**. $x \neq x$ 가 참이면 x 는 NaN이라는 뜻이고, 이것이 NaN을 판별하는 고전적 관용구다(오늘은 `isnan()`을 쓴다). “같음”이라는 관계의 기본 성질이 깨지는 값이라, 정렬이나 검색 알고리즘에 NaN이 섞여 들면 기묘한 일이 벌어진다 — 실수 데이터를 다룰 때 NaN 검사를 경계에 두는 것이 관행인 이유다.

● 0.1초의 누적 — 패트리엇 미사일 사고

6장에서 배운 “작은 어긋남이 쌓인다”가 실제로 사람의 목숨과 이어진 사건이 있다. 1991년 걸프전에서 패트리엇 방공 시스템이 날아오는 미사일을 요격하지 못해 28명이 사망한 사고인데, 원인 분석의 핵심이 부동소수점 오차였다. 시스템은 시간을 0.1초 단위로 세었는데 — 6장에서 본 대로 0.1은 2진법으로 무한소수라 담을 때마다 미세한 오차가 생긴다. 그 시스템이 24비트 그릇을 쓴 탓에 오차가 상대적으로 컸고, 재부팅 없이 100시간을 연속 가동하자 누적 오차가 약 0.34초에 이르렀다. 그 0.34초 동안 목표는 500미터 넘게 이동하고, 추적 창은 엉뚱한 하늘을 보게 된 것이다. “군사는 성실하지만 무해하지는 않다” — 6장의 교훈이 가장 무겁게 확인된 사례다.

문 그러면 돈 계산 같은 곳에는 실수를 쓰면 안 되는가?

답 쓰지 않는 것이 정석이다 — 6장에서 배운 고정소수점의 자리가 바로 여기다. 금액을 원 단위 실수로 다루면 0.1원 단위의 어긋남이 쌓여 장부가 맞지 않게 되므로, **전 단위 정수로 계산하고 표시할 때만 소수점을 찍는 것이 금융 소프트웨어의 관행**이다. 규칙으로 줄이면 — **정확한 십진 값이 중요한 곳에는 정수(고정소수점), 물리량과 과학 계산에는 부동 소수점**. 도구를 문제에 맞추는 것이고, 그 판단의 근거가 6장과 이 장에서 배운 표현의 성격이다.

근사의 세계를 C에서 다루는 법을 익혔다. 다음 장은 이 부의 중심 주제 — 계산이 실패할 수 있다는 사실을 프로그램이 어떻게 다루는가, 오류와 계약의 이야기다.

41 오류와 계약

이 장이 끝나면

29장에서 심은 계약의 씨앗이 자란다 — 함수는 무엇을 요구하고 무엇을 약속하는가(전제조건·후조건), 실패를 어떻게 알리는가(C의 방식: 오류는 값이다), 그리고 그 값을 반드시 확인하게 만드는 장치들까지. 35장에서 본 proven의 설계 사상이 여기서 원리로 정리된다.

돌아보기 29장 끝에서 `fact` 함수가 “암묵의 약속”(n은 0 이상, 13 이상이면 넘침) 위에 서 있다고 했다. 그런데 그 약속은 코드 어디에도 적혀 있지 않았다 — 그러면 그것은 존재하는가?

답 존재하지만, **지켜지지 않는 상태로 존재한다** — 그것이 문제의 핵심이다. 모든 함수에는 암묵의 계약이 있다: 어떤 입력에 대해 제대로 동작하는가 (전제조건), 성공하면 무엇을 보장하는가(후조건). 그 계약이 문서와 코드 어디에도 없으면, 계약 위반은 조용히 지나가 나중에 엉뚱한 곳에서 터진다. 이 장은 계약을 드러내는 방법들의 이야기다.

41.1 오류는 값이다 — C의 방식

많은 현대 언어가 실패를 위한 별도 통로(예외)를 갖고 있지만, C에는 없다. C에서 실패는 **반환값으로 알린다** — 함수의 결과 자체가 “성공했는가”를 담고 나오는 것이다. 관행은 세 갈래다.

- 성공/실패 불리언 + 결과는 포인터로 받기(30장의 & 관용구).
- 특별한 값으로 실패 표시 — `malloc`의 널(37장), `fgets`의 널, 음수 반환 등.
- 오류 코드를 돌려주고 0을 성공으로 — 시스템 호출 계열의 전통.

첫 갈래를 시연으로 본다. 24장에서 배운 “0 나누기는 계약 밖”이라는 사실을, 함수 계약으로 다스리는 모습이다.

```
examples/ch41/errval.c

#include <stdio.h>
#include <stdlib.h>

/* 계약: divisor가 0이 아니어야 한다. 실패는 반환값으로 알린다. */
[[nodiscard]] bool safe_div(int a, int b, int *out)
{
    if (b == 0) {
        return false;          /* 실패 - out은 건드리지 않는다 */
    }
    *out = a / b;
    return true;
}

int main(void)
{
    int result = 0;

    if (safe_div(7, 2, &result)) {
        printf("7 / 2 = %d\n", result);
    } else {
        printf("7 / 2: failed\n");
    }

    if (safe_div(7, 0, &result)) {
        printf("7 / 0 = %d\n", result);
    }
}
```

```

    } else {
        printf("7 / 0: cannot divide by zero (reported as a value)\n");
    }
    return 0;
}

```

실행 결과

```

7 / 2 = 3
7 / 0: cannot divide by zero (reported as a value)

```

읽을 점 셋. **계약이 주석으로 적혀 있다** — 무엇을 요구하는지가 코드 곁에 있다. 실패해도 출력 인자를 건드리지 않는다 — “실패 시 아무것도 바꾸지 않는다”는 것도 계약의 일부다. 그리고 `[[nodiscard]]` — C23의 표기로, 이 반환값을 버리는 호출이 있으면 컴파일러가 경고한다. 18장에서 “반환값을 버리는 것은 합법”이라 했던 그 자유에, “이 값만은 버리지 말라”는 제동을 거는 장치다. 35장의 proven 함수들이 이 표기를 두른 이유가 바로 이것이다.

⚠ “오류 처리는 코드를 지저분하게 만드는 부수적인 일이다”

입문자가 흔히 갖는 인상이고, 실제로 C의 오류 처리는 눈에 띄게 상황하다 — 호출마다 if가 붙으니 까. 그러나 관점을 뒤집으면 정확히 반대다: **오류 경로가 곧 프로그램의 절반이다**. 파일이 없을 수 있고, 메모리가 부족할 수 있고, 입력이 엉터리일 수 있다는 것은 예외적 상황이 아니라 정상적인 현실이다. 실제 사고 분석에서 압도적으로 흔한 원인이 “반환값을 확인하지 않았다”인 것이 그 증거다 — 성공 경로만 적힌 코드는 절반만 쓰인 코드다. 상황함을 줄이는 법(공통 정리 지점으로 모으기, 실패를 감싸는 타입 쓰기)은 기술의 문제이고, **확인한다**는 원칙은 타협의 문제가 아니다.

41.2 계약을 코드로 — assert와 방어

전제조건을 문서가 아니라 코드로 적는 도구가 있다 — `<assert.h>`의 `assert(조건)`다. 조건이 거짓이면 프로그램을 즉시 세우고 위치를 알린다. 용도를 정확히 구별하는 것이 중요하다:

- **assert는 프로그래머의 실수를 잡는 것이다** — “여기까지 왔다면 이 조건은 반드시 참이어야 한다”는 내부 불변식(28장의 그 불변식과 같은 낱말이다). 배포 빌드에서는 꺼지는 것이 관행이다(NDEBUG).
- **바깥에서 온 것은 assert가 아니라 검사로** — 사용자 입력, 파일 내용, 네트워크 데이터는 “틀릴 수 있는 것이 정상”이므로 실행 중에 늘 확인 하고 오류 값으로 처리해야 한다. 입력 검증을 assert로 하면, 배포판에서 검사가 통째로 사라지는 사고가 된다.

이 구별이 곧 계약의 두 면이다 — 안쪽(내가 지켜야 할 불변식)은 assert로, 바깥(상대가 지킬지 모르는 약속)은 검사와 오류 값으로.

41.3 const — 가장 값싼 계약

계약을 코드로 적는 도구가 하나 더 있다. 20장에서 “바꾸지 않겠다는 문서”로 소개하고, 38장에서 “이 함수는 원본을 건드리지 않는다”는 표시로 써 온 `const`다. 이 장의 관점에서 다시 보면, `const`는 **가장 값싸게 적을 수 있는 계약 조항**이다 — 함수 서명 한 곳에 한 낱말을 더하는 것으로, 호출하는 쪽에게 “당신의 데이터는 안전하다”를 약속하고 컴파일러에게 그 약속의 감시를 맡긴다.

효과는 세 층에 걸친다.

① **사람에게** — 읽는 부담이 준다. `void render(const struct scene *s)` 라는 서명을 보는 순간, 이 함수가 `scene`을 바꾸지 않는다는 것이 확정 된다. 함수 몸통을 읽지 않아도 되는 것 — 큰 코드에서 이보다 값진 절약은 드물다. 20장에서 “바꾸지 않는 것이 많은 코드일수록 읽기 쉽다”고 한 것의 실체다.

② **컴파일러에게 — 최적화의 근거가 된다.** 11장에서 편집자가 값을 레지스터에 붙잡아 두는 장면을 보았고, 그 판단의 관건이 “이 값이 그 사이에 바뀔 수 있는가”였다. `const`는 그 판단을 돕는 신호다 — 다만 정확히 말해야 한다: **`const` 자체가 마법의 최적화 스위치는 아니다.** 포인터를 통해 들어온 데이터는 다른 경로로 바뀔 여지가 남아 있어서 (앨리어싱), `const` 하나로 컴파일러가 모든 걸 확신하지는 못한다. 확실한 이득은 **`const`로 선언된 실제 객체**(전역 상수, `static const` 테이블) 쪽이다 — 컴파일러가 값을 코드에 직접 심거나(상수 전파), 읽기 전용 구역에 배치해 아예 수정 불가로 만든다(34장의 문자열 리터럴이 그 자리에 살았다).

③ **기억의 층에게 — 공유가 안전해진다.** 바뀌지 않는 데이터는 복사할 이유가 없다. 여러 곳에서 같은 것을 함께 읽어도 되고, 9장의 캐시 관점에서는 여러 코어가 같은 캐시 라인을 나눠 읽어도 아무 다툼이 없다 — 10장에서 본 false sharing은 쓰기가 있어야 생기는 사고이기 때문이다. 큰 데이터를 값 복사 대신 `const` 포인터로 넘기는 관행(38장)이 안전하면서 빠른 이유가 이것이다.

그래서 현대적 관행은 단순하다 — **기본을 `const`로 두고, 바뀌야 하는 것만 예외로 푼다.** 계약을 넓히는 데는 비용이 들지만, 좁히는 데는 낱말 하나면 된다.

문 35장의 proven은 이 원리를 어떻게 구현하고 있는가?

답 세 가지가 이 장의 원리 그대로다. 첫째, **실패가 타입에 드러난다** — `{err, val}` 꾸러미를 돌려주므로 “성공했는지 묻지 않고 값을 꺼내는” 코드를 쓰기가 오히려 어색해진다. 둘째, **`[[nodiscard]]`로 확인을 강제한다** — 버리면 컴파일러가 경고한다. 셋째, **경계와 크기를 계약의 일부로 받는다** — 33-34장에서 본 경계 침범의 뿌리를 API 차원에서 막는 것이다. 요약하면: 계약을 문서가 아니라 **타입과 시그니처**로 적는 설계다. 1장에서 본 Rust·Zig의 문제의식과 같은 방향을, C 안에서 부품으로 구현한 것이라 보면 된다.

계약과 오류를 다루는 법을 찾았다. 그런데 이 책이 곳곳에서 “계약 밖”, “정의되지 않은 동작”이라 부르며 미뤄 온 세계가 아직 남아 있다. 다음 장에서 그 세계를 정면으로 본다 — C에서 가장 오해가 많고, 가장 값비싼 주제다.

42 정의되지 않은 동작

이 장이 끝나면

이 책이 곳곳에서 “계약 밖”이라 부르며 미뤄 온 세계를 정면으로 다룬다. UB란 정확히 무엇이고 왜 존재하며, 왜 “조금 위험함”이 아니라 “무엇이든 가능함”인지 — 그리고 실무에서 어떻게 피하고 잡아내는지까지. 제2부의 계약 서사가 여기서 완결된다.

돌아보기 11장에서 엄격한 앨리어싱 위반이 “컴파일러에게 사실이 아닌 것을 사실 이라고 알리는 행위”라 했다. 그러면 표준이 어떤 동작을 “정의되지 않음”으로 두었다는 것은 — 구현에게 무엇을 허락한 것인가?

답 아무것도 요구하지 않기로 한 것이다. 표준의 문장은 냉정하다 — 정의 되지 않은 동작에 대해 표준은 “어떤 요구도 부과하지 않는다”. 즉 그런 프로그램에는 옳은 실행 결과라는 것이 아예 없다. 흔한 오해가 “위험한 동작이지만 대개 예상대로 돌아간다”인데, 계약의 눈으로는 그 프로그램 전체가 의미를 잃는다. 이 차이가 이 장의 전부다.

42.1 세 가지 회색지대 — UB, 미지정, 구현 정의

혼동되는 세 낱말을 먼저 갈라 둔다. 이 책이 여기까지 만난 사례들로 정리하면 이렇다.

종류	표준의 태도	이 책에서 만난 예
구현 정의 (implementation-defined)	구현이 정하고 문서화한다	int의 크기(23장), char의 부호
미지정(unspecified)	정해진 선택지 중 하나이나 문서화 의무 없음	한 수식 안 부분식의 평가 순서(29장)
정의되지 않음 (undefined behavior, UB)	아무 요구도 하지 않는다	부호 있는 오버플로(5장), 경계 침범(33장), 널 역참조(31장)

앞의 둘은 “답이 여럿이지만 어쨌든 답은 있는” 세계다. UB만이 답 자체가 없는 세계다.

42.2 왜 존재하는가

계약 밖을 남겨 둔 이유는 두 가지다. 첫째, **기계들이 서로 다르기 때문이다** — 5장의 폭 이상 시프트가 대표다. x86과 옛 ARM이 다르게 대응하는데 표준이 한쪽을 택하면 다른 쪽 기계는 매번 보정 코드를 넣어야 한다. 어느 편도 들지 않는 대신 “그 코드를 쓰지 말라”고 한 것이다. 둘째, **최적화의 전제를 얻기 위해서다** — 5장에서 본 대로 “부호 있는 정수는 넘치지 않는다”는 전제가 있어야 컴파일러가 루프를 분석하고 재배열할 수 있다(11장). C가 반세기 동안 속도의 언어로 남은 대가가 이 조항들이다.

42.3 “무엇이든 가능함”의 실제 얼굴

UB의 결과는 붕괴만이 아니다. 11장에서 본 무늬가 더 무섭다 — **컴파일러가 UB를 “일어날 리 없는 일”로 읽고 코드를 지워 버리는 것이다**. 널 검사가 통째로 사라지거나(포인터를 이미 역참조한 뒤라면 “널 일 리 없다”고 추론한다), 오버플로 검사가 사라지거나(부호 있는 넘침은 없다고 전제 하므로), 루프가 무한이 되거나 아예 없어진다. 그래서 UB의 대표적 증상은 “그 자리에서 죽는 것”이 아니라 **영똥한 곳에서, 최적화 수준을 바꾸면 사라지는, 재현이 어려운 버그다**.

● 사라진 널 검사 — 리눅스 커널 CVE-2009-1897

이 무늬가 커널에서 실제로 터진 사건이 있다. 코드는 대략 이랬다 — 포인터 `tun`을 먼저 역참조해 값을 꺼내고, 그 아래에서 `if (!tun)`

`return ...;`으로 널을 검사했다. 순서가 뒤집힌 실수이지만, 사람 눈에는 “그래도 검사는 있으니 널이면 걸리겠지”로 보인다. 컴파일러의 추론은 달랐다: 이미 역참조했다 → 널이었다면 그 순간 UB → UB는 일어나지 않는다고 전제 → 따라서 `tun`은 널이 아니다 → 아래의 널 검사는 죽은 코드다. 검사가 통째로 최적화로 제거됐고, 널 페이지를 매핑할 수 있던 당시 환경과 결합해 권한 상승 취약점이 됐다. 컴파일러는 규칙대로 일했고, 무너진 것은 계약이었다.

42.4 피하는 법 — 규율, 도구, 그리고 부품

실무의 방어는 세 겹이다.

규율 — 이 책이 지나온 수칙들이 그 목록이다: 초기화하고 쓴다(20장), 경계를 지킨다(33장), 널을 확인한다(31장), 한 문장에서 한 변수는 한 번만 바꾼다(29장), 계약 밖의 지름길(포인터 캐스트, 표현 가정)을 쓰지 않는다(11·32장).

도구 — 15장에서 장만한 그물이다. 컴파일러 경고는 컴파일 시점에, UBSan·ASan은 실행 시점에 잡는다. C23이 들여온 **검사 산술**도 이 층의 도구다 — 넘침을 UB로 만들지 않고 값으로 알려 주는 함수들이다:

```
examples/ch42/checked.c
#include <stdckdint.h>
#include <stdio.h>

/* C23의 검사 산술: 넘치면 true를 돌려주고, 결과는 감아 둔 값이 담긴다. */
int main(void)
{
    int a = 2000000000;
    int b = 2000000000;
    int sum = 0;

    if (ckd_add(&sum, a, b)) {
        printf("%d + %d overflows int (we stayed inside the contract)\n", a, b);
    } else {
        printf("sum: %d\n", sum);
    }

    int small = 0;
    if (ckd_add(&small, 20, 22)) {
        printf("overflow\n");
    } else {
        printf("20 + 22 = %d\n", small);
    }
    return 0;
}
```

실행 결과

```
2000000000 + 2000000000 overflows int (we stayed inside the contract)
20 + 22 = 42
```

`ckd_add`는 “넘쳤는가”를 반환값으로 돌려준다 — 5장에서 배운 “부호 있는 넘침은 계약 밖”이라는 함정을, 41장에서 배운 “오류는 값이다”라는 규율로 다스리는 표준의 답이다.

부품 — 애초에 계약 위반이 어려운 API를 쓰는 것이다(35장의 proven이 그 층이다). 도구가 그물이 라면 좋은 부품은 발판이라는 15장의 비유가 여기서 완성된다.

문 UB를 전부 외워야 하는가? 표준에 수백 가지가 있다고 들었는데.

답 외우는 것이 목표가 아니다 — 목록은 방대하고 계속 다듬어진다. 실무에서 통하는 것은 **감각**이다: “이 코드가 옳다고 말하는 근거가 계약서에 있는가, 아니면 ‘내 컴퓨터에서 돌아서’인가”를 묻는 습관. 그리고 그 감각을 도구로 뒷받침하는 것 — 경고를 켜고, 새니타이저로 시험 실행 하고, 두 컵 파일러로 교차 검증하는 것(15장). 이 책이 제2부부터 “추상 기계 위에서 옳은가”를 되풀이한 이유가 바로 이 감각을 심기 위해서다.

42.5 제9부를 닫으며

정밀의 부가 끝났다 — 근사를 다루는 법(40장), 실패를 다루는 법(41장), 그리고 계약 밖의 세계를 아는 법(42장). 세 장의 공통 주제는 하나다: **C는 프로그래머에게 많은 것을 맡기는 언어이고, 맡겨진 것을 아는 사람이 안전한 코드를 쓴다.**

마지막 부가 남았다. 지금까지의 프로그램은 파일 하나짜리였다 — 이제 여러 파일로 자라고(43장), 전 처리와 번역의 층을 정면으로 보고(44장), 표준 라이브러리의 지형을 익히고(46장), proven을 정면으로 다루며(제11부), 모던 C의 관행으로 책을 닫는다(58장).

제10부 — 구성

43 여러 파일 — 분할과 링크

이 장이 끝나면

지금까지의 프로그램은 파일 하나였다. 이제 여러 파일로 자란다 — 헤더와 소스의 분업, 번역 단위라는 개념, 이름의 연결(외부·내부), 그리고 링크 오류를 읽는 법까지. 14장의 릴레이와 21장의 선언/정의 구별이 여기서 하나로 합쳐진다.

돌아보기 21장에서 “원형을 파일 위쪽에 적어 두면 정의는 아래 어디에 있어도 되고, 더 중요하게는 다른 파일에 있어도 된다”고 했다. 그 말이 성립하는 근거를 14장의 릴레이로 설명하면?

답 컴파일 단계는 **선언만** 보고 일하고, 몸통을 찾아 잇는 것은 링커이기 때문이다(14장). 그래서 파일 하나를 컴파일할 때 다른 파일의 내용은 전혀 필요하지 않다 — 필요한 것은 “이런 이름의 함수가 이런 서명으로 어딘가에 있다”는 약속뿐이고, 그 약속을 담아 나르는 그릇이 헤더 파일 이다. 이 장은 그 분업을 실물로 본다.

43.1 번역 단위와 헤더

전처리가 끝난 소스 파일 하나 — `#include`가 전부 펼쳐진 결과 — 를 **번역 단위**(translation unit)라 부른다. 컴파일러는 언제나 번역 단위 하나만 본다. 여러 파일 프로그램이란 번역 단위 여럿을 각각 목적 파일로 만든 뒤, 링커가 하나로 잇는 것이다.

세 파일짜리 예제로 분업을 본다.

```
examples/ch43/main.c
/* 여러 파일 예제 - 이 파일은 greet의 선언만 보고 컴파일된다. */
#include "greet.h"

int main(void)
{
    greet("world");
    printf_count();
    return 0;
}
```

실행 결과

```
Hello, world!
greet was called 1 times
```

`greet.h`(헤더)에는 **선언만** 있다 — 계약 서명. `greet.c`에는 **정의**가 있고, `main.c`는 헤더만 포함해 계약을 알고 호출한다. 컴파일은 각각 따로 되고, 링커가 `greet` 호출과 정의를 이어 준다.

헤더의 `#ifndef GREET_H ... #define ... #endif`는 **포함 보호**(include guard)다. 헤더가 여러 경로로 두 번 붙어도 내용이 한 번만 펼쳐지게 하는 관용구로, 두 번 선언되면 오류가 나는 것들(구조체 정의 등)을 막는다. 실무에서는 `#pragma once` 한 줄로 대신하는 경우도 많다(표준은 아니지만 주요 컴파일러가 전부 지원한다).

43.2 연결 — 이름이 파일 경계를 넘는다

이름에는 스코프(21장) 말고 또 하나의 성질이 있다 — **연결**(linkage), 즉 다른 번역 단위에서 그 이름을 볼 수 있는가다.

- **외부 연결**: 파일 밖에서도 보인다. 함수와 전역 변수의 기본값이다.

- **내부 연결:** 이 번역 단위 안에서만 보인다. 파일 수준 선언에 `static`을 붙이면 된다 — 예제의 `static int calls`가 그 예다.

`static`이라는 낱말이 36장(정적 수명)과 여기(내부 연결)에서 다른 뜻으로 쓰이는 것은 C의 유명한 낱말 재활용이다 — **함수 안의** `static`은 수명을, **파일 수준의** `static`은 연결을 뜻한다.

내부 연결은 실무의 기본 무기다. 35장에서 “C에는 이름 공간이 없어서 접두어가 담장을 대신한다”고 했는데, 파일 안에서만 쓰는 도우미 함수·변수에 `static`을 붙이면 그 이름은 아예 바깥 마당에 나가지 않는다 — 충돌도 없고, 컴파일러가 더 공격적으로 최적화할 수 있다(그 이름이 다른 파일에서 볼릴 리 없음을 아니까 — 12장의 편집자에게 주는 정직한 신호다).

△ “헤더 파일에 함수 정의를 넣어도 편하니 괜찮다”

포함되는 파일마다 정의가 복사되므로, 두 파일이 그 헤더를 포함하면 같은 함수의 정의가 둘이 되어 링커가 **중복 정의** 오류를 낸다(“multiple definition of ...”). 헤더의 역할은 계약을 나르는 것이지 구현을 나르는 것이 아니다 — 헤더에는 선언, 소스에는 정의가 기본형이다. (예외가 없지는 않다: `static inline` 함수나 매크로는 헤더에 두는 것이 관행 이고, C23의 `constexpr` 상수도 그렇다. 예외를 알고 쓰는 것과 규칙을 모르고 어기는 것은 다르다.)

문 링크 오류 메시지는 어떻게 읽는가?

답 두 문장만 알면 대부분 풀린다. “**undefined reference to X**” — 선언은 보았으나 정의를 못 찾았다는 뜻이다(14장). 소스 파일을 빌드에서 빠뜨렸거나, 라이브러리를 안 이어 줬거나, 철자가 다르다. “**multiple definition of X**” — 정의가 둘 이상이다. 헤더에 정의를 넣었거나, 전역 변수를 헤더에서 선언하며 초기화했을 때 흔하다. 두 메시지 모두 **컴파일러가 아니라 링커가** 내는 것이라는 점이 진단의 출발점이다 — 문법은 멀쩡했다는 뜻이니까.

파일 경계를 넘는 법을 배웠다. 그런데 이 장에서 `#include`와 `#ifndef`가 또 나왔다 — 14장에서 “전처리기는 C를 모르는 텍스트 도구”라 하고 지나친 그 층이다. 다음 장에서 그 층을 정면으로 열고, 소스 코드가 프로그램이 되기까지의 정식 단계까지 본다.

44 전처리기와 번역 단계

이 장이 끝나면

14장에서 “C를 모르는 텍스트 도구”라 부르고 지나친 층을 정면으로 연다. 매크로의 정체, #과 ##이라는 두 연산자의 원리와 관용구, 그 함정들, 그리고 소스가 프로그램이 되기까지 표준이 못박은 **번역 단계**의 전모 까지. C 소스 파일 안에 사는 “두 번째 언어”의 문법서다.

돌아보기 16장에서 #으로 시작하는 줄은 문장이 아니고 세미콜론도 없다고 했고, 그 이유가 “전처리기는 C 문법이 아니라 줄 단위로 일하는 텍스트 도구라 서”라고 했다. 그러면 전처리는 C의 무엇을 아는가?

답 거의 아무것도 모른다 — 타입도, 스코프도, 함수도 모른다. 아는 것은 **토큰**(더 이상 쪼갤 수 없는 낱말 조각: 이름, 숫자, 연산자, 문자열) 뿐이고, 하는 일은 토큰을 지우고 끼우고 이어 붙이는 것이다. 이 무지가 전처리의 힘이자(어떤 코드 조각이든 만들어 낼 수 있다) 위험이다 (문법과 의미를 지켜 주지 않는다). 이 장은 그 힘과 위험을 함께 본다.

44.1 매크로 — 이름을 토큰 열로 바꾸기

#define NAME 내용은 “이후 NAME이라는 토큰을 만나면 내용으로 바꿔 치우라”는 지시다. 인자를 받는 꼴(#define MAX(a,b) ...)도 있는데, 함수처럼 보이지만 함수가 아니다 — **호출이 아니라 치환**이고, 타입 검사도 평가 순서 보장도 없다.

두 가지 관용구가 이 차이에서 나온다. 첫째, **인자와 전체를 괄호로 감싼다** — #define SQ(x) ((x) * (x))처럼. 괄호가 없으면 sq(1+2)가 1+2*1+2로 펼쳐져 5가 된다(17장의 우선순위가 치환된 토큰 열에 그대로 적용되기 때문이다). 둘째, **인자를 두 번 쓰는 매크로를 조심한다** — sq(i++)는 i를 두 번 증가시킨다. 29장에서 배운 “한 문장에서 한 변수는 한 번만”이 매크로 때문에 몰래 깨지는 것이다.

44.2 # — 토큰을 문자열로 (stringize)

매크로 인자 앞에 #을 붙이면, 그 인자가 글자 그대로 문자열 리터럴이 된다. 값이 아니라 **적힌 모양**이 문자열이 되는 것이 핵심이다.

```
examples/ch44/macro.c
#include <stdio.h>

/* # : 인자를 "글자 그대로" 문자열로 만든다 (stringize) */
#define SHOW(expr) printf("#expr " = %d\n", (expr))

/* ## : 두 토큰을 하나로 붙인다 (token paste) */
#define MAKE_NAME(prefix, n) prefix##n

/* 한 겹으로는 매크로가 안 풀린다 - 두 겹 우회가 정석이다 */
#define STR_RAW(x) #x
#define STR(x) STR_RAW(x)
#define WIDTH 80

int MAKE_NAME(value_, 1) = 11; /* -> int value_1 = 11; */
int MAKE_NAME(value_, 2) = 22;

int main(void)
{
    SHOW(2 + 3 * 4);
}
```



```

SHOW(value_1 + value_2);

printf("STR_RAW(WIDTH) = %s\n", STR_RAW(WIDTH)); /* 안 풀린다 */
printf("STR(WIDTH)      = %s\n", STR(WIDTH));      /* 풀린 뒤 문자열이 된다 */
return 0;
}

```

실행 결과

```

2 + 3 * 4 = 14
value_1 + value_2 = 33
STR_RAW(WIDTH) = WIDTH
STR(WIDTH)      = 80

```

시연의 `SHOW(2 + 3 * 4)`가 그 실물이다 — `#expr`이 `"2 + 3 * 4"`라는 문자열이 되고, 옆의 `(expr)`은 계산되어 14가 된다. 하나의 인자가 글자 **로도**, 값**으로도** 쓰인 것이다. 이 무늬가 디버깅·검사 매크로의 기본형이다: `assert(x > 0)`가 실패했을 때 `"x > 0"`이라는 조건을 그대로 찍어 주는 것이 바로 이 원리다(41장에서 만난 그 `assert`의 속살이다).

44.3 ## — 토큰 두 개를 하나로 (token paste)

`##`은 양옆의 토큰을 붙여서 하나의 새 토큰으로 만든다. 시연의 `MAKE_NAME(value_, 1)`이 `value_1`이라는 식별자가 되는 것이 그 예다 — 문자열이 아니라 진짜 이름이라, 변수 선언에 쓸 수 있다.

용도는 이름을 조립하는 코드 생성이다. 같은 모양의 함수·구조체를 타입마다 찍어 내야 할 때(C에는 제네릭이 없으므로), 접두어와 타입 이름을 붙여 `stack_int_push`, `stack_double_push` 같은 이름들을 매크로 하나로 만들어 내는 것이 고전적 기법이다.

44.4 두 겹 우회 — 왜 한 겹으로는 안 풀리는가

`#`과 `##`을 쓸 때 반드시 만나는 규칙이 하나 있다. `#`이나 `##`의 피연산자가 되는 매크로 인자는 먼저 펼쳐지지 않는다. 시연의 마지막 두 줄이 그 대조다 — `STR_RAW(WIDTH)`는 `"WIDTH"`가 되고, `STR(WIDTH)`는 `"80"`이 된다.

이유는 치환 순서에 있다. 매크로 인자는 보통 먼저 완전히 펼쳐진 뒤 본문에 끼워지지만, `###`의 대상이면 펼쳐지지 않은 원래 토큰이 쓰인다. 그래서 값을 원할 때는 한 겹을 더 두른다 — 바깥 매크로(`STR`)가 인자를 평범하게 받아 펼친 뒤, 안쪽 매크로(`STR_RAW`)에 넘겨 거기서 문자열로 만드는 것이다. `##`도 같은 사정이라 `PASTE/PASTE_RAW` 짝을 두는 것이 관용구다. 전처리기의 함정 중 가장 자주 사람을 붙잡는 규칙 이고, 알고 나면 두 줄로 해결되는 문제이기도 하다.

● 매크로 하나가 만든 언어 — X 매크로

`#`과 `##`의 힘을 보여 주는 고전 기법이 있다. 목록을 한 곳에만 적어 두고 여러 형태로 펼치는 **X 매크로**다. 이를테면 오류 코드 목록을

```

#define ERROR_LIST \
    X(OK,          "성공") \
    X(TIMEOUT,     "시간 초과") \
    X(DENIED,      "권한 없음")

```

처럼 한 번 적어 두고, `x`를 그때그때 다르게 정의해 열거형도 만들고 (`X(name, msg) ERR_##name,`), 이름 문자열 표도 만들고(`#name`), 메시지 표도 만든다. 목록을 한 곳에서만 고치면 파생물이 전부 따라오니 — 항목을 추가하며 어딘가를 빠뜨리는 고전적 버그가 원천 봉쇄된다. C에 코드 생성 기능이 없다는 결핍을, 전처리기라는 텍스트 도구로 메운 민중의 발명품인 셈이다. 오늘날에는 가독성 때문에 신중히 쓰이지만, 커널과 임베디드 코드에서는 여전히 현역이다.

△ “매크로는 함수의 빠른 버전이다”

옛 시절에는 반쯤 사실이였다 — 함수 호출 비용을 아끼려고 짧은 함수를 매크로로 적곤 했다. 그러나 오늘의 컴파일러는 작은 함수를 알아서 인라인으로 펼치므로(12장의 편집자), 속도를 위해 매크로를 쓸 이유는 거의 사라졌다. 남은 것은 대가뿐이다 — 타입 검사 없음, 인자 다중 평가, 디버거에서 안 보임, 오류 메시지가 펼쳐진 코드 기준으로 나와 읽기 어려움. 현대적 지침은 분명하다: 값 계산은 함수(필요하면 `static inline`), 상수는 `enum`이나 C23의 `constexpr`, 매크로는 함수로 안 되는 일 — 조건부 컴파일, 토큰 조립, 소스 위치 포착 — 에만.

44.5 번역 단계 — 표준이 못박은 여덟 걸음

전처리가 “먼저” 일어난다는 것은 알았는데, 정확히 무엇이 어떤 순서로 일어나는가. 표준은 이것을 **번역 단계**(translation phases)라는 이름으로 여덟 걸음으로 못박아 두었다. 실제 컴파일러가 이 단계를 물리적으로 나눠 수행할 필요는 없지만, **마치 이 순서로 한 것처럼** 동작해야 한다(12장의 “겉보기만 같으면”이 여기에도 적용된다).

단계	하는 일
1	소스 문자를 읽어 내부 문자 집합으로 옮긴다(7장의 인코딩 세계). 옛날에는 삼중자 치환도 여기였다 — C23이 제거했다
2	줄 끝의 역슬래시로 이어진 줄들을 한 줄로 합친다
3	주석을 공백 하나로 바꾸고, 소스를 토큰 으로 쪼갬다
4	전처리 지시를 실행한다 — <code>#include</code> (재귀적으로 1~4단계 반복), <code>#define</code> 치환, 조건부 컴파일
5	문자·문자열 리터럴의 문자를 실행 문자 집합으로 변환한다
6	이웃한 문자열 리터럴들을 하나로 잇는다(“가” “나” → “가나”)
7	토큰을 C 문법으로 해석하고 번역한다 — 여기가 “컴파일”이다
8	필요한 정의들을 이어 실행 이미지를 만든다 — 링크

이 표가 여러 수수께끼를 한꺼번에 푼다. 주석이 3단계에서 공백이 되므로 `a/**/b`는 `ab`가 아니라 `a b`다. `#include`가 4단계에서 재귀적으로 처리되므로 헤더 안의 헤더가 자연스럽게 펼쳐진다. 문자열 이어 붙이기가 6단계에 있으므로 긴 문자열을 여러 줄로 나눠 적는 관용구가 성립하고, 매크로가 만든 문자열 조각(`#` 연산자의 결과)도 옆 리터럴과 붙는다. 그리고 **문법 해석이 7단계**라는 사실이 이 장의 핵심을 다시 말해 준다 — 전처리가 일할 때 C 문법은 아직 존재하지 않는다.

문 그러면 매크로 안에서 문법 오류를 내면 어디서 잡히는가?

답 7단계에서, **펼쳐진 뒤의 코드**를 기준으로 잡힌다 — 그래서 오류 메시지가 내가 적은 줄과 동떨어져 보이는 것이다. 진단의 정석은 14장에서 배운 `-E`(전처리만 하기)다: 문제의 파일을 전처리해 실제로 펼쳐진 모습을 눈으로 보면, 수수께끼 같던 오류가 대개 즉시 풀린다. 현대 컴파일러는 “매크로 NAME에서 펼쳐짐”이라는 추적 정보를 함께 보여 주어 이 일을 거들기도 한다.

소스가 프로그램이 되는 전 과정을 이제 층별로 알게 됐다. 다음 장은 `printf`가 인자를 몇 개든 받는 비밀 — 가변 인자 함수다.

45 가변 인자 함수

이 장이 끝나면

printf가 인자를 몇 개든 받을 수 있었던 비밀을 연다. <stdarg.h>의 네 도구, 이 장치가 왜 타입 안전하지 않은지, 그 역사(K&R의 varargs 에서 C89의 stdarg, C23의 정리까지), 그리고 오늘의 대안까지. 19장의 “서식은 계약”이 왜 그토록 엄격한 계약이었는지가 여기서 밝혀진다.

돌아보기 25장에서 가변 인자로 넘어가는 값에는 기본 진급(float→double, 작은 정수→int)이 적용된다고 배웠다. 그런데 왜 하필 그 자리에만 특별한 진급 규칙이 있는가?

답 받는 쪽이 **타입을 모르기 때문이다**. 보통의 함수는 원형(21장)에 인자 타입이 적혀 있어 컴파일러가 양쪽을 맞춰 준다. 그러나 가변 인자 자리는 원형에 ...뿐이라 맞출 상대가 없다 — 그래서 “일단 다들 큰 타입으로 통일해서 보내자”는 약속으로 혼란을 줄인 것이다. 이 장은 그 “타입을 모르는 자리”에서 어떻게 값을 꺼내는지, 그리고 그 대가가 무엇인지의 이야기다.

45.1 네 개의 도구

<stdarg.h>가 주는 것은 타입 하나와 매크로 셋이다 — va_list(읽개), va_start(시작), va_arg(하나 꺼내기), va_end(닫기). 시연으로 본다.

```
examples/ch45/va.c

#include <stdarg.h>
#include <stdio.h>

/* 개수를 첫 인자로 받는 방식 - 스스로는 개수를 알 길이 없다 */
int sum_n(int count, ...)
{
    va_list ap;
    int total = 0;

    va_start(ap, count);          /* count 다음부터 읽겠다 */
    for (int i = 0; i < count; i += 1) {
        total += va_arg(ap, int); /* 타입을 '내가' 알려 준다 */
    }
    va_end(ap);                  /* 반드시 닫는다 */
    return total;
}

/* 끝 표지(sentinel)로 개수를 대신하는 방식 */
int sum_until_zero(int first, ...)
{
    va_list ap;
    int total = first;

    va_start(ap, first);
    for (;;) {
        int v = va_arg(ap, int);
        if (v == 0) { break; }
        total += v;
    }
    va_end(ap);
    return total;
}
```

```
int main(void)
{
    printf("sum_n(4, 1,2,3,4)      = %d\n", sum_n(4, 1, 2, 3, 4));
    printf("sum_until_zero(1,2,3,0) = %d\n", sum_until_zero(1, 2, 3, 0));
    return 0;
}
```

실행 결과

```
sum_n(4, 1,2,3,4)      = 10
sum_until_zero(1,2,3,0) = 6
```

핵심은 `va_arg(ap, int)`의 둘째 인자다 — **꺼낼 값의 타입을 프로그래머가 알려 준다**. 함수는 무엇이 왔는지 모르므로, “지금 꺼낼 것은 `int`다”라고 말해 주는 사람이 필요하고 그 사람이 호출자와 약속을 공유한 작성자다. 이 약속이 어긋나면 — 실제로는 `double`이 왔는데 `int`로 꺼내면 — 계약 밖이다(42장). 기억의 잘못된 자리를 읽는 것이니 값이 쓰레기가 되거나 그 뒤의 꺼내기가 전부 어긋난다.

그리고 함수는 **인자가 몇 개인지도** 모른다. 그래서 개수를 알아내는 길을 호출 규약으로 만들어야 하는데, 시연이 두 관행을 보인다 — 첫 인자로 개수를 받거나(`sum_n`), 끝을 알리는 표지 값을 약속하거나(`sum_until_zero`). `printf`는 세 번째 길을 쓴다: **서식 문자열이 개수와 타입을 동시에 알려 주는 명세인 것이다**. 19장에서 “서식은 계약”이라 한 말의 정확한 무게가 이것이다 — 그 계약이 깨지면 `printf`는 있지도 않은 인자를 꺼내며 메모리를 뒤킨다(19장의 서식 문자열 취약점이 바로 이 메커니즘이었다).

45.2 역사 — `varargs`에서 `stdarg`로

이 장치의 역사는 C가 계약의 언어로 자란 과정의 축소판이다.

표준 이전(K&R 시절). 원형이 없던 시절(10장)의 C에서는 사실상 모든 함수가 인자 개수를 검사받지 않았다 — `printf` 같은 함수가 특별할 것도 없었다. 인자를 꺼내는 방법도 이식성이 없었다: 첫 인자의 주소에서 스택을 따라 훑는 식의 요령을 각자 썼는데, 인자를 스택이 아니라 레지스터로 넘기는 기체가 나오면서 이 요령이 깨졌다. 그래서 나온 것이 Unix 계열의 `<varargs.h>`였다 — 요령을 매크로로 감싸 기계 차이를 숨긴 첫 시도다.

C89 — `<stdarg.h>`. 표준화 위원회가 원형을 도입하면서(10·21장) 가변 인자도 정식 문법을 얻었다: 원형에 ...을 적고, 값 꺼내기는 `va_*` 매크로로 한다. 옛 `varargs`와의 결정적 차이는 **이름 있는 인자가 최소 하나 필요하다는 것**이다(`va_start`가 그 인자를 기준점으로 삼는다) — 그래서 `printf(const char *fmt, ...)`처럼 서식이 앞에 오는 서명이 표준이 됐다.

C23 — 다듬기. 기준점 인자가 필요하다는 제약이 완화되어(`va_start`의 둘째 인자가 선택적) 인자 없는 가변 함수도 적을 수 있게 됐고, 옛 `varargs.h`는 역사 속으로 완전히 사라졌다. 반세기에 걸쳐 “이식성 없는 요령 → 표준 매크로 → 군더더기 제거”로 다듬어 온 것이다.

△ “가변 인자 함수는 편리하니 자주 쓰면 좋다”

편리해 보이지만, C에서 가변 인자는 **타입 안전이 사라지는 자리**다. 컴파일러는 ... 자리의 인자를 검사하지 않고, 받는 쪽은 타입을 추측할 방법이 없다. `printf` 계열이 그나마 안전한 것은 컴파일러가 특별히 서식 문자열을 읽어 검사해 주기 때문이지(15장의 경고), 언어의 일반 규칙 덕분이 아니다 — 직접 만든 가변 인자 함수에는 그 그물이 없다. 실무 권고는 분명하다: **꼭 필요하지 않으면 만들지 않는다**. 대안이 대개 더 낫다 — 개수가 정해졌으면 그냥 인자를 나열하고, 여럿 이면 배열과 길이를 받고(33장의 관행), 종류가 다양하면 39장의 태그된 공용체로 명시적인 값 목록을 받는다. C 라이브러리 설계의 현대적 취향은 “가변 인자는 로깅·서식처럼 도구의 검사가 붙는 자리에만”이다.

문 `printf` 같은 서식 함수를 감싸는 내 함수를 만들려면 어떻게 하는가 — 예컨대 로그 함수?

답 그 자리를 위해 `v` 붙은 짝이 표준에 준비되어 있다 — `vprintf`, `vfprintf`, `vsprintf`처럼 `va_list`를 **그대로 받는** 판들이다. 내 함수가 ...으로 받아 `va_start`로 읽개를 만든 뒤, 그 읽개를 통째로 넘겨주면 된다. 서식 검사 경고를 내 함수에서도 받고 싶다면 컴파일러 확장 표기(gcc·clang의 `format` 속성)를 붙이는 것이 관행이다 — 언어가 못 주는 안전을 도구가 메우는, 이 책에 여러 번 나온 무늬다.

가변 인자의 정체를 알았다. 다음 장은 지금까지 쓰기만 해 온 표준 라이브러리의 지형을 펼친다 — 어떤 도구 상자가 있고, 무엇을 믿고 무엇을 조심할 것인가.

46 표준 라이브러리의 지형

이 장이 끝나면

지금까지 쓰기만 해 온 표준 라이브러리의 지도를 편다 — 어떤 도구 상자가 있고, 무엇을 믿고 무엇을 조심할 것인가. 그리고 이 라이브러리가 왜 “얇고 오래된” 모습인지, 그 역사적 이유까지.

돌아보기 14장에서 링커가 `printf`의 몸통을 표준 라이브러리에서 찾아 이어 준다고 했고, 35장에서 `gets`는 표준에서 삭제됐다고 했다. 표준 라이브러리는 누가 정하고 어떻게 바뀌는가?

답 언어 표준이 함께 정한다 — C 표준 문서는 언어 문법만이 아니라 **제공 되어야 할 라이브러리 목록과 각 함수의 계약**까지 못박는다(그래서 “C 표준 라이브러리”). 바뀌는 속도는 언어만큼 느리고, 무언가를 빼는 일은 훨씬 더 느리다 — `gets`의 장례가 수십 년 걸린 이유이자, 이 라이브러리의 성격을 설명하는 열쇠다.

46.1 지도 — 도구 상자들

자주 만나는 헤더를 갈래로 묶으면 지형이 한눈에 들어온다.

갈래	대표 헤더와 내용
입출력	<stdio.h> — 스트림, <code>printf/scanf</code> 계열, 파일
문자열·기억	<string.h> — 복사·비교·검색, <ctype.h> — 문자 분류
수	<stdlib.h> — 변환·난수, <math.h> — 수학 함수, <stdint.h>·<limits.h>·<float.h> — 타입의 한계
기억 관리	<stdlib.h> — <code>malloc/free</code> 계열
시간·환경	<time.h>, <stdlib.h>의 환경 접근
계약·진단	<assert.h>, <errno.h>
현대의 보탬	<stdbool.h>(C99, C23에서 불필요해짐), <stdatomic.h>·<threads.h>(C11), <stdckdint.h>(C23 검사 산술)

이 목록에서 두 가지가 눈에 띈다. 첫째, **얇다** — 자료구조(목록·해시 표), 정규식, 네트워크, 그래픽이 없다. 둘째, **오래됐다** — 대부분이 C89에 있던 것들이고, 새로 들어온 것은 손에 꼽는다.

46.2 왜 얇은가 — 설계와 역사

이유는 C가 자란 자리에 있다. C는 운영체제를 만드는 언어로 태어났고 (2장), 운영체제 **위에서** 도는 프로그램만이 아니라 운영체제 **자신**과 임베디드 펌웨어까지 같은 언어로 짜야 했다. 그런 곳에는 파일 시스템도, 동적 할당도, 심지어 운영체제도 없을 수 있다 — 그래서 표준 라이브러리는 “어디서나 있을 수 있는 최소한”으로 좁혀졌다(표준이 프리스탠딩 환경을 따로 정의해 두는 이유다). 풍부함을 포기한 대신 이식성을 얻은 것이다.

그리고 얇음에는 대가가 따랐다 — 세상은 결국 필요한 것을 만들었고, 그 결과가 플랫폼마다 다른 API들과 서드파티 라이브러리들의 숲이다. “C는 라이브러리를 고르는 일이 절반”이라는 말이 나온 배경이고, 이 책이 35장에서 `proven`을 고른 것도 그 선택의 한 예다.

46.3 출력 서식 문자열의 해부

가장 많이 쓰는 함수를 정면으로 뜯어볼 자리다. 19장에서는 최소 집합 (`%d`, `%s`, `%f`)만 익혔지만, `printf`의 서식은 사실 다섯 조각으로 된 작은 언어다.

% [플래그] [폭] [.정밀도] [길이] 변환

뒤에서부터 읽는 것이 이해가 빠르다. **변환**(conversion)이 “무엇으로 찍을 것인가”를 정하고 — 이것만 필수다 — 나머지는 그 위에 얹는 장식이다.

- **플래그(flag)**: - 왼쪽 정렬, 0 남는 자리를 0으로, + 양수에도 부호, 빈칸 양수 앞에 빈칸, # 대체 형식(%#x는 0x를 붙인다).
- **폭(width)**: 최소 글자 수. 모자라면 채우고, 넘치면 **자르지 않는다** — 폭은 하한이지 상한이 아니다.
- **정밀도(precision)**: .으로 시작한다. 실수에는 소수 자릿수, 문자열 에는 **최대** 길이, 정수에는 최소 자릿수다.
- **길이 수식어(length modifier)**: 인자의 폭을 알려 준다. l(long), ll(long long), z(size_t), h(short로 줄여 해석).

폭과 정밀도 자리에 *를 쓰면 그 값을 인자로 넘길 수 있다. 8장의 뷰처럼 “포인터와 길이”로 다니는 문자열을 찍을 때 %.*s가 요긴하다.

```
examples/ch46/fmtspec.c
#include <stdio.h>

int main(void) {
    int n = 42;
    unsigned u = 255;
    double x = 3.14159265;
    const char *s = "proven";
    size_t sz = 1234;
    long long big = 9000000000LL;

    /* 플래그와 폭: 오른쪽 정렬이 기본, - 는 왼쪽, 0 은 빈칸 대신 0 */
    printf("[%d] [%6d] [%-6d] [%06d] [%+d]\n", n, n, n, n, n);

    /* 정밀도: 실수는 소수 자릿수, 문자열은 최대 길이 */
    printf("[%f] [%.2f] [%10.3f] [%e] [%g]\n", x, x, x, x, x);
    printf("[%s] [%10s] [%-10s] [%.3s]\n", s, s, s, s);

    /* 진법과 문자 */
    printf("[%c] [%x] [%X] [%#x] [%o] [%u]\n", 'A', u, u, u, u, u);

    /* 길이 수식어: 인자의 폭을 서식에 알려 준다 */
    printf("[%zu] [%lld]\n", sz, big);

    /* 폭·정밀도를 인자로 넘기기 */
    printf("[%.*s] [%*d]\n", 4, s, 6, n);

    /* 퍼센트 글자 자체 */
    printf("100%\n");
    return 0;
}
```

실행 결과

```
[42] [ 42] [42] ] [000042] [+42]
[3.141593] [3.14] [ 3.142] [3.141593e+00] [3.14159]
[proven] [ proven] [proven] ] [pro]
[A] [ff] [FF] [0xff] [377] [255]
[1234] [9000000000]
[prov] [ 42]
100%
```


읽을거리가 몇 개 있다. %06d가 000042가 되는 것은 0 플래그가 빈칸 대신 0으로 채우기 때문이고, %.3s가 pro에서 멈추는 것은 문자열에서 정밀도가 **최대 길이**이기 때문이다. %#x의 0x는 #가 붙인 것이다. %g가 3.14159로 짧아진 것은 이 변환이 %e와 %f 중 짧은 쪽을 자동으로 고르기 때문이다.

길이 수식어는 장식이 아니라 **계약**이다. 45장에서 본 대로 가변 인자에는 타입 정보가 실려 가지 않으므로, printf는 서식이 말한 폭 그대로 스택을 읽는다. size_t를 %d로 찍는 코드가 64비트에서 조용히 어긋나는 이유가 이것이다 — 8바이트를 넣었는데 4바이트만 꺼내 읽는다.

데이터형	출력 서식	메모
int	%d %i	기본형
unsigned int	%u %x %o	비트를 볼 때는 16진
short	%d	승격되므로 %hd는 선택
long	%ld	
long long	%lld	
size_t	%zu	★ %d로 찍으면 어긋난다
ptrdiff_t	%td	
double	%f %e %g	float도 승격되어 같다
long double	%Lf	
char(문자로)	%c	인자는 int로 승격된다
char *	%s	NUL 종단이어야 한다
void *	%p	형식은 구현 정의(implementation-defined)
bool	%d	0 또는 1로 찍힌다

⚠ “float을 찍을 때는 %f, double은 %lf”

절반만 맞다. **출력**에서는 float이 가변 인자 기본 진급(25장)으로 double이 되어 넘어가므로 둘 다 %f다 — %lf도 표준이 허용하지만 같은 뜻이다. 헛갈림의 진짜 뿌리는 **입력**에 있다. scanf는 주소를 받으므로 진급이 일어나지 않고, float *와 double *를 반드시 구별 해야 한다 — %f는 float *, %lf는 double *다. 같은 글자가 방향에 따라 다른 뜻이 되는, 이 두 함수의 가장 유명한 비대칭이다.

46.4 입력 서식 문자열 — 무엇이 다른가

scanf 계열의 서식은 출력과 글자가 겹쳐서 같은 언어처럼 보이지만, 하는 일은 반대이고 규칙도 다르다. 다른 점을 다섯으로 정리한다.

첫째, 인자는 값이 아니라 답을 곳의 주소다. &n을 빠뜨리면 정수를 주소로 착각해 그 번지에 쓰려 든다 — 22장에서 만난 그 &가 여기서 필수인 이유다.

둘째, 서식의 공백은 “공백 몇 개든(없어도 좋다)”이다. 출력에서 공백은 그저 공백 한 칸이지만, 입력에서는 건너뛰기 지시자. 대부분의 변환은 앞선 공백을 알아서 건너뛴다 — 예외가 %c와 %[로, 이 둘은 공백도 글자로 읽는다.

셋째, 서식 안의 보통 글자는 입력과 그대로 맞아야 한다. “x=%d”는 입력이 x로 시작할 것을 요구한다. 맞지 않으면 **매칭 실패**로 멈춘다.

넷째, 반환값은 찍은 글자 수가 아니라 성공한 항목 수다. 그래서 셋을 읽으려 했는데 2가 돌아오면 하나는 채워지지 않았다는 뜻이고, 그 인자는 **손대지 않은 채로 남는다**. 입력이 아예 없으면 EOF(음수)가 돌아온다 — 0과 구별해야 한다.

다섯째, %s에는 반드시 최대 폭을 준다. 폭 없는 %s는 목적지 크기를 모르는 채로 쓰는 것이라 35장의 gets와 같은 위험이다.

```
examples/ch46/scanspec.c
#include <stdio.h>

int main(void) {
    int a = 0, b = 0, k;
    double d = 0.0;
    char word[8] = {0};
    char rest[16] = {0};

    /* 서식의 공백 하나가 "공백 몇 개든" 을 뜻한다 */
    k = sscanf(" 12 34", "%d %d", &a, &b);
    printf("ints    : k=%d a=%d b=%d\n", k, a, b);

    /* %s 는 공백에서 멈춘다. 폭을 주어 버퍼를 지킨다 */
    k = sscanf("hello world", "%7s", word);
    printf("bounded : k=%d word=[%s]\n", k, word);

    /* 서식 안의 보통 글자는 입력과 그대로 맞아야 한다 */
    k = sscanf("x=5", "x=%d", &a);
    printf("literal : k=%d a=%d\n", k, a);

    /* 맞지 않으면 매칭 실패 - 0 을 돌려주고 인자는 건드리지 않는다 */
    k = sscanf("y=5", "x=%d", &a);
    printf("mismatch: k=%d (a stays %d)\n", k, a);

    /* 변환 실패도 0 이다 - 숫자가 아니면 아무것도 읽지 않는다 */
    k = sscanf("abc", "%d", &b);
    printf("nonnum  : k=%d (b stays %d)\n", k, b);

    /* 입력이 비면 EOF(음수)다 - 0 과 구별해야 한다 */
    k = sscanf("", "%d", &b);
    printf("empty   : k=%d\n", k);

    /* 부분 성공: 앞은 읽히고 뒤에서 멈춘다 */
    k = sscanf("7 oops", "%d %lf", &a, &d);
    printf("partial : k=%d a=%d\n", k, a);

    /* 집합 지정자: 심표가 아닌 글자를 모은다 */
    k = sscanf("name,42", "%15[^,],%d", rest, &b);
    printf("set      : k=%d rest=[%s] b=%d\n", k, rest, b);

    /* 실수와 길이 수식어: double 은 %lf 다 */
    k = sscanf("3.5", "%lf", &d);
    printf("double  : k=%d d=%.2f\n", k, d);
    return 0;
}
```

실행 결과

```
ints    : k=2 a=12 b=34
bounded : k=1 word=[hello]
literal : k=1 a=5
mismatch: k=0 (a stays 5)
nonnum  : k=0 (b stays 34)
empty   : k=-1
partial : k=1 a=7
```

```
set      : k=2 rest=[name] b=42
double   : k=1 d=3.50
```

출력을 한 줄씩 쪼이면 규칙이 눈에 보인다. mismatch와 nonnum은 둘 다 k=0인데, 인자가 그대로 남아 있다는 점이 중요하다 — 값을 확인하지 않고 쓰면 **이전 값**을 새 입력으로 착각한다. empty의 -1은 “입력이 끝났다” 이지 “형식이 틀렸다”가 아니다. partial은 앞의 정수만 읽히고 뒤에서 멈춘 경우다. set의 %15[^,]는 “심표가 아닌 글자를 최대 15개”라는 뜻으로, 구분자가 있는 줄을 자를 때 쓴다.

데이터형	입력 서식	인자
int	%d	int *
unsigned	%u %x	unsigned *
long	%ld	long *
long long	%lld	long long *
size_t	%zu	size_t *
float	%f	★ float *
double	%lf	★ double *
long double	%Lf	long double *
문자 하나	%c	char *(공백도 읽는다)
단어	%99s	char[100] — 폭 필수
글자 집합	%15[^,]	char[16]
건너뛰기	%*d	읽되 저장하지 않는다

● 서식 하나가 무너뜨린 것 — 형식 문자열 취약점

19장에서 이름만 스쳤던 사고를 여기서 완성한다. 사용자가 준 문자열을 서식 자리에 그대로 넣은 코드(`printf(user)`)는 공격자가 %s나 %x를 입력해 스택을 읽게 만들 수 있고, 옛 %n(찍은 글자 수를 인자가 가리키는 곳에 쓴다)까지 살아 있던 시절에는 임의의 메모리 쓰기로까지 이어졌다. 1999년 무렵 이 부류가 대규모로 발견되면서 wu-ftpd를 비롯한 여러 서버가 원격 침해됐다. 교훈은 한 줄이다 — **서식 문자열은 언제나 프로그램이 쓴 상수여야 하고, 사용자 입력은 인자로만 들어간다** (`printf("%s", user)`).

문 그러면 입력 해석은 늘 `sscanf`로 하면 되는가?

답 간단한 형식에는 충분하다. 다만 `sscanf`는 **어디에서 왜 실패했는지**를 알려 주지 않는다 — 돌려주는 것은 성공한 항목 수뿐이라, “세 번째 필드가 숫자가 아니었다”와 “줄이 일찍 끝났다”를 구별할 수 없다. 게다가 정수 오버플로를 감지하지 못하고(%d에 9999999999를 주면 계약 밖이다), 실패한 자리까지 몇 글자를 소비했는지도 알 수 없다. 형식이 복잡해지고 입력이 남이 준 것이라면, 실패가 값으로 드러나고 남은 입력을 손에 쥌 수 있는 도구가 필요하다 — 제11부의 스캐너가 그 답이다.

46.5 조심할 자리들

이 책이 지나오며 만난 함정들을 라이브러리 관점에서 모으면 이렇다.

- **크기를 안 받는 함수들** — `strcpy`, `strcat`, `sprintf` 계열은 목적지 그릇의 크기를 모른다(35장의 `gets`가 극단이었다). 대안은 크기를 받는 판(`snprintf`)이나 경계를 관리하는 부품이다.
- **반환값이 실패를 알리는 함수들** — `malloc`, `fopen`, `fgets`는 실패 시 널을 준다(41장의 규율: 확인 없이 쓰지 않는다).

- **errno**라는 전역 상태 — 많은 함수가 실패의 이유를 전역 변수에 남긴다. 성공해도 지우지 않는 함수가 있어 “호출 직전에 0으로 초기화하고 직후에 읽는” 규약을 지켜야 한다 — 전역 가변 상태의 불편함을 보여 주는 교과서 사례다(36장).
- **지역/문화 의존 함수들** — `toupper` 같은 문자 함수와 `strtod`의 소수점 해석은 로케일 설정에 따라 동작이 달라진다. 데이터 형식을 다룰 때는 로케일 독립적인 처리를 쓰는 것이 안전하다(7장의 “인코딩을 추측하지 말라”와 같은 결).
- **정적 버퍼를 돌려주는 함수들** — `asctime`, `strtok` 계열은 내부의 고정된 자리를 재사용해서, 다음 호출이 이전 결과를 덮는다. 여러 갈래로 도는 프로그램(10장의 멀티코어)에서는 특히 위험하다.

문 그러면 표준 라이브러리는 낡아서 안 쓰는 것이 좋은가?

답 전혀 아니다 — **어디에나 있다**는 것이 압도적인 미덕이다. 어떤 플랫폼, 어떤 컴파일러에서도 같은 계약으로 존재하는 부품은 이것뿐이고, 이 책의 예제 대부분도 표준 라이브러리만으로 돌았다. 옳은 태도는 “쓰지 않기”가 아니라 **어느 함수가 어떤 계약인지 알고 고르기**다 — 크기를 받는 판을 고르고, 반환값을 확인하고, 전역 상태에 기대는 함수는 규약을 지켜 쓰는 것. 그리고 반복되는 위험 지대(문자열 조립, 입력 해석)에는 검사가 내장된 부품을 엮는 것 — 다음 장이 그 실천이다.

지도를 펴으니, 이제 그 위에 엮을 층을 본다. 다음 부(제11부)는 이 책의 기반 라이브러리 `proven`을 세 장에 걸쳐 다룬다 — 설계의 뼈대인 다섯 계약, 문자열과 텍스트, 그리고 할당과 컨테이너까지. C 입문서로 읽어 온 이 책이 그 대목에서는 `proven`의 안내서가 된다.

제11부 — proven — 검증된 기본기

1장에서 밝힌 대로, 이 책에는 저자가 만든 라이브러리를 알리려는 목적도 있다. 그 약속을 지키는 자리가 여기다 — 다만 순서는 처음에 말한 그대로다. 앞의 마흔여섯 장은 표준 C만으로 왔고, proven은 필요가 스스로 증명된 자리(35장)에서 한 번 얼굴을 비쳤을 뿐이다.

그래서 이 부도 도구가 아니라 **문제**로 시작한다. 47장이 C가 반세기 동안 같은 자리에서 미끄러져 온 다섯 부류의 결함을 실제로 도는 코드로 확인하고, 이어지는 아홉 장이 그 각각에 대한 답을 짚는다 — 설치와 첫 프로그램(48장), 값으로 오는 에러(49장), 바이트와 뷰(50장), 할당자(51장), 문자열(52장), 형식화와 파싱(53장), 컨테이너와 알고리즘(54장), 파일·시간·난수(55장), 그리고 겹쳐 돌리기와 OS가 없는 환경(56장)이다.

읽는 목적은 두 가지로 잡으면 좋다. proven을 쓸 생각이라면 이 부가 시작점이고, 쓰지 않더라도 — **이 책이 가르친 위험들이 API 설계로 어떻게 막히는지** 끝까지 따라가 본 사례로 읽으면 된다. 어느 라이브러리를 고르든 그 눈은 그대로 쓰인다. 각 절이 “그래서 대가는 무엇인가”를 빠뜨리지 않고 적는 것도 같은 이유다 — 공짜인 선택은 없다.

47 50년째 출하되는 다섯 가지 버그

이 장이 끝나면

이 부의 문제 제기다. C가 반세기 동안 같은 다섯 부류의 버그를 계속 출하해 온 이유를 — 프로그래머의 부주의가 아니라 **API의 모양** 때문임을 — 실제로 도는 코드로 확인한다. 다섯을 다 본 뒤에야 proven이라는 이름이 다시 나온다. 도구를 먼저 소개하지 않는 것이 이 부의 원칙이다.

돌아보기 34장에서 문자열을 다루는 함수들이 “그릇의 크기를 모른다”고 했고, 41장에서는 “실패를 확인하지 않으면 없던 일이 된다”고 했다. 그러면 이런 문제들은 왜 아직도 남아 있는가 — 반세기면 고치고도 남을 시간이 아닌가?

답 고칠 수 없어서가 아니라, 고치면 **이미 쓰인 코드가 전부 깨지기 때문이다**. strcpy의 시그니처에 크기 매개변수를 하나 더 넣는 순간 세상의 모든 C 코드가 컴파일되지 않는다. 표준은 기존 코드를 지켜야 하는 기관이라(46장의 “얇고 오래된” 이유가 여기서도 나온다), 위험한 함수를 없애는 대신 **더 나은 함수를 옆에 두는 길**을 택해 왔다. 그래서 선택은 프로그래머에게 넘어온다 — 무엇이 위험한지 알고 고르는 일이 곧 실력이 되는 언어인 셈이다.

문 그러면 이 다섯 가지는 초보자가 저지르는 실수 목록인가?

답 아니다. 이것들은 **숙련자도 계속 저지르는 실수**이고, 그 점이 중요하다. 문법을 다 아는 사람도 같은 자리에서 미끄러진다면 원인은 사람이 아니라 **도구의 모양**이다. 크기를 받지 않는 함수는 크기를 검사할 방법이 없고, 반환값을 버려도 아무 일 없는 함수는 언젠가 버려진다. 이 장은 그 “모양”을 하나씩 뜯어보는 장이다.

47.1 하나 — 문자열 함수는 그릇의 크기를 모른다

가장 오래되고 가장 많이 악용된 부류다. 34장에서 본 그대로다.

```
char buf[64];
strcpy(buf, name);           /* name 이 얼마나 긴가? strcpy 는 묻지 않는다 */
strcat(buf, ", welcome!");   /* 그리고 이제 남은 자리는 얼마인가? */
```

strcpy의 시그니처에는 목적지의 크기가 없다. 없는 정보는 검사할 수 없으므로, 이 함수는 64바이트 그릇의 200번째 바이트에도 기꺼이 쓴다. 무엇이 망가지는가는 컴파일러가 그 뒤에 무엇을 놓았느냐에 달렸고, 흔히 그것은 함수의 반환 주소다(36장의 스택 그림을 떠올리면 된다).

현대의 처방은 크기를 받는 판을 쓰는 것이다 — sprintf가 대표다. 그런데 여기에 두 번째 함정이 있다. 크기를 받는 함수는 넘칠 때 **조용히 자른다**.

```
examples/ch47/truncate.c

#include <stdio.h>
#include <string.h>

/* 흔한 방식: 고정 버퍼에 경로를 조립한다.
   컴파일러는 이 함수 안에서 dir/name 의 길이를 알 수 없다. */
static int build_path(char *out, size_t cap, const char *dir, const char *name) {
    return sprintf(out, cap, "%s/%s", dir, name); /* 넘치면 조용히 자른다 */
}

int main(void) {
```

```

char path[24];

build_path(path, sizeof path, "/var/log", "app.log");
printf("fits      : %s\n", path);

build_path(path, sizeof path, "/var/log/service/http", "access.log");
printf("truncated : %s\n", path);
printf("          length=%zu, buffer=%zu\n", strlen(path), sizeof path);

/* 잘렸는지 알아내려면 반환값을 보아야 한다 */
int need = build_path(path, sizeof path, "/var/log/service/http", "access.log");
if (need >= (int)sizeof path)
    printf("detected : needed %d bytes, had %zu\n", need + 1, sizeof path);
return 0;
}

```

실행 결과

```

fits      : /var/log/app.log
truncated : /var/log/service/http/a
          length=23, buffer=24
detected  : needed 33 bytes, had 24

```

세 번째 줄을 보라. 만들려던 것은 /var/log/service/http/access.log인데 손에 남은 것은 /var/log/service/http/a다. 프로그램은 멈추지도, 경고하지도 않았다. 이 문자열이 파일 경로라면 엉뚱한 파일을 열고, 명령이라면 다른 명령이 되며, 로그라면 사고 기록이 소리 없이 잘린다. **잘린 경로는 짧은 경로가 아니라 틀린 경로다.**

✗ 잘림을 성공으로 취급하기

```

snprintf(path, sizeof path, "%s/%s", dir, name);
open_file(path);          /* 잘렸는지 아무도 묻지 않았다 */

```

snprintf는 사실 답을 준다 — **필요했던 길이**를 반환한다. 그 값이 그릇 크기 이상이면 잘린 것이다(예제의 마지막 줄이 그 검사다). 문제는 이 검사가 **선택 사항**이라는 데 있다. 반환값을 버려도 컴파일러는 아무 말도 하지 않는다. 이것이 곧바로 두 번째 버그로 이어진다.

● 컴파일러는 자기가 볼 수 있는 것만 잡는다

이 예제를 만드는 동안 실제로 겪은 일이다. 처음에는 snprintf를 main 안에서 리터럴 인자로 직접 불렀는데, gcc가 그것을 잡아냈다.

```

error: '%s' directive output truncated writing 10 bytes
      into a region of size 2 [-Werror=format-truncation=]
note: 'snprintf' output 33 bytes into a destination of size 24

```

훌륭한 진단이다. 그런데 같은 호출을 build_path라는 함수 안으로 옮기자 경고가 **사라졌다**. 함수 경계를 넘는 순간 컴파일러는 dir과 name의 실제 길이를 알 수 없기 때문이다. 실제 프로그램에서 문자열은 파일이나 네트워크에서 오므로 컴파일러가 도울 수 있는 경우가 오히려 드물다. 경고는 공짜 검토이지 보증이 아니다(15장).

47.2 둘 — 실패를 확인하게 만드는 장치가 없다

```

char *p = malloc(n);
p[0] = 'x';          /* malloc 은 실패하면 널을 준다 */

```

C의 실패 통보 방식은 두 가지다. **센티널 값**(널, -1, EOF)을 돌려주거나, 전역 변수 `errno`에 이유를 남기거나. 둘 다 확인을 강제하지 못한다. 반환값을 버리는 코드는 완벽하게 합법이고, `errno`는 다음 호출이 덮어쓰기 전에 정확한 순간에 읽어야 하는 전역 상태다(46장).

```
examples/ch47/unchecked.c

#include <stdio.h>
#include <stdlib.h>

/* 설정 줄에서 포트 번호를 읽는다 - 실패를 확인하지 않는 판 */
static int read_port_careless(const char *line) {
    int port = 8080;          /* 기본값 */
    sscanf(line, "port=%d", &port); /* 반환값을 보지 않는다 */
    return port;
}

/* 같은 일 - 실패를 확인하는 판 */
static int read_port_checked(const char *line, int fallback) {
    int port;
    if (sscanf(line, "port=%d", &port) != 1) {
        printf(" (parse failed, keeping %d)\n", fallback);
        return fallback;
    }
    return port;
}

int main(void) {
    const char *good = "port=9000";
    const char *typo = "prot=9000"; /* 오타 */

    printf("careless good: %d\n", read_port_careless(good));
    printf("careless typo: %d\n", read_port_careless(typo));
    printf("checked  typo: %d\n", read_port_checked(typo, 8080));

    /* strtol 도 마찬가지다: 실패를 값으로 알리지만 아무도 강제하지 않는다 */
    long n = strtol("abc", nullptr, 10);
    printf("strtol(\"abc\") = %ld\n", n);
    return 0;
}
```

실행 결과

```
careless good: 9000
careless typo: 8080
 (parse failed, keeping 8080)
checked  typo: 8080
strtol("abc") = 0
```

`careless typo`가 8080을 돌려준 것이 이 절의 핵심이다. 설정에 오타가 있었는데 프로그램은 **기본값으로 조용히 돌아갔다**. 겉으로는 아무 일도 없었고, 몇 달 뒤 “왜 설정이 안 먹지?”라는 물음만 남는다. `strtol("abc")`가 0을 주는 것도 같은 무늬다 — 실패와 “진짜 0”이 같은 값으로 돌아온다.

⚠ “실패는 예외적인 일이니 나중에 처리하면 된다”

실패가 드물다는 전제부터 틀렸다. 파일은 없을 수 있고, 입력에는 오타가 섞이며, 디스크는 차고, 네트워크는 끊긴다 — 프로그램이 바깥 세계와 닿는 모든 자리가 실패 지점이다. 그리고 “나중에”가 위험한 진짜 이유는 따로 있다. 실패를 무시한 코드는 **멈추지 않고 계속 달린다**. 잘못된 값이 다음 계산

으로, 그다음 함수로 흘러 들어가서, 마침내 문제가 드러날 무렵에는 원인에서 멀리 떨어진 자리에서 터진다. 41장의 “일찍 실패하라”가 여기서 되돌아온다.

47.3 셋 — printf는 여러분이 말하는 것을 그대로 믿는다

46장에서 서식 문자열의 문법을 뜯어보았다. 그 문법에는 구조적 약점이 하나 있다 — **타입을 두 번 적는다는 것이다**. 한 번은 서식에서(%d), 한 번은 인자에서(변수의 타입). 둘이 어긋나도 언어는 막지 못한다. 45장에서 본 대로 가변 인자에는 타입 정보가 실려 가지 않기 때문이다.

요즘 컴파일러는 이것을 잡아 준다. 실제 메시지는 이렇다.

```
warning: format '%d' expects argument of type 'int',
      but argument 2 has type 'double' [-Wformat=]
3 |     printf("%d\n", 3.0);
  |               ^~   ~~~
  |               |   |
  |               int double
```

다만 여기까지만. 서식이 **변수**가 되는 순간 — 다국어 메시지를 표에서 꺼내 쓰거나 로그 형식을 설정에서 받는 순간 — 컴파일러는 더 이상 볼 것이 없다.

x 서식을 변수로 받는 코드

```
const char *fmt = load_message("greeting"); /* 표에서 꺼낸 서식 */
printf(fmt, count); /* 경고 없음. 검사도 없음 */
```

이 코드에는 어떤 경고도 나오지 않는다. 서식과 인자가 맞는지 아는 방법이 컴파일 시간에 존재하지 않기 때문이다. 최악은 서식이 **사용자 입력**인 경우로, 46장에서 본 형식 문자열 취약점이 된다.

47.4 넷 — 이걸 누가 해제하나

```
char *s = build_message(); /* 이걸 free 해야 하나? 타입은 말이 없다 */
```

37장에서 배운 대로 동적으로 잡은 기억은 누군가 정확히 한 번 놓아 주어야 한다. 그런데 함수가 돌려준 char *는 네 가지 중 무엇이든 될 수 있다.

- 방금 할당한 것 — 해제해야 한다.
- 호출자가 준 버퍼를 가리키는 것 — 해제하면 안 된다.
- 읽기 전용 자리에 있는 문자열 리터럴 — 해제하면 사고다.
- 다음 호출이 덮어쓸 정적 버퍼 — 해제해서도 안 되고 오래 들고 있어도 안 된다(46장의 strtok이 그랬다).

네 경우의 타입이 **전부 같다**. 답은 문서에 있고, 문서는 코드와 어긋나기 마련이다. 여기서 37장의 사고셋이 나온다 — 누수(아무도 해제하지 않음), 이중 해제(둘 다 해제함), 사용 후 해제(해제한 뒤에도 누군가 가리키고 있음).

x 타입이 소유권을 말하지 않는 API

```
const char *lookup(int code); /* 리터럴? 할당? 정적 버퍼? */
char *format_time(time_t t); /* free 해야 하나? */
```

이름과 타입만 보고는 알 수 없다. 이 API를 쓰는 **모든 곳**이 문서를 기억해야 하고, 한 곳이라도 잊으면 앞의 사고 셋 중 하나가 된다. 규율을 사람의 기억에 맡기는 설계라는 점이 문제의 본질이다.

47.5 다섯 — 아무도 타입을 검사해 줄 수 없는 콜백

```
qsort(a, n, sizeof *a, cmp); /* cmp 는 const void* 를 받는다 */
```


qsort는 어떤 타입이든 정렬하기 위해 비교 함수를 void * 인터페이스로 받는다. 제네릭이 없는 언어에서 이것은 거의 유일한 방법이지만, 대가는 **타입 검사의 완전한 포기**다. 비교자 안에서 무엇으로 캐스트하든 컴파일러는 믿는다.

```
examples/ch47/cmp_bad.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* 반례: 첫 글자만 비교한다. 타입은 완벽하게 맞고, 경고도 없다 */
static int cmp_first_char(const void *a, const void *b) {
    const char *const x = a;
    const char *const y = b;
    return (*x)[0] - (*y)[0];
}

/* 정례: 문자열 전체를 비교한다 */
static int cmp_full(const void *a, const void *b) {
    const char *const x = a;
    const char *const y = b;
    return strcmp(x, y);
}

static void show(const char *label, const char **v, size_t n) {
    printf("%s:", label);
    for (size_t i = 0; i < n; i++) printf(" %s", v[i]);
    printf("\n");
}

int main(void) {
    const char *src[] = {"pear", "apple", "peach", "apricot"};
    const char *v[4];
    size_t n = sizeof src / sizeof src[0];

    memcpy(v, src, sizeof src);
    qsort(v, n, sizeof v[0], cmp_first_char);
    show("first-char", v, n);

    memcpy(v, src, sizeof src);
    qsort(v, n, sizeof v[0], cmp_full);
    show("full", v, n);
    return 0;
}
```

실행 결과

```
first-char: apple apricot pear peach
full      : apple apricot peach pear
```

first-char 비교자는 타입이 완벽하게 맞고, 경고 하나 없이 컴파일되며, 죽지도 않는다. 그저 peach와 pear의 순서가 틀렸을 뿐이다 — 첫 글자만 보았으니 둘은 “같다”고 판정됐고 나머지는 운에 맡겨졌다. 이 부류의 버그는 **조용히 잘못된 답**을 내놓기 때문에 가장 늦게 발견된다.

● 자료구조의 최악을 노리는 공격

같은 자리에 성능의 함정도 있다. 널리 쓰인 정렬·해시 구현들은 평균적 으로는 빠르지만 특정 입력에서 급격히 느려지는데, 공격자가 그런 입력을 일부러 만들어 서버를 마비시키는 수법(algorithmic

complexity attack)이 2003년 논문으로 정리되며 실제 공격에 쓰였다. 해시 충돌을 노린 같은 계열의 공격은 2011년 여러 웹 프레임워크를 한꺼번에 주저앉혔다. 자료구조의 **최악 케이스**가 곧 보안 문제라는 것을 보여 준 사건들이고, 그래서 오늘의 라이브러리는 최악에도 보장이 있는 정렬(introsort)과 무작위 씨앗을 쓰는 해시를 기본값으로 삼는다 — 54장에서 실물을 본다.

47.6 여섯 번째 — 바이트에는 타입이 있다

다섯이라 했지만 하나를 더 붙여야 공평하다. 11장에서 본 엄격한 앨리어싱 이다. 바이트 버퍼를 폭이 다른 포인터로 들여다보는 손수 짠 파서는 -00에서 완벽히 돌고 -02에서 조용히 다른 답을 낸다. 15장에서 본 “릴리스에서만 나는 버그”의 대표이자, 리눅스 커널이 플래그 하나로 행복한 그 조항이다.

여섯을 한 표로 모으면 이 부의 지도가 된다.

≡ 복습 정리

문제	C가 주는 것	필요한 것
버퍼 넘침·잘림	크기를 모르는 문자열 함수	길이를 지니고 다니는 문자열, 자르지 않는 쓰기
확인되지 않은 실패	센티널 값과 errno	값으로 오는 에러, 버리면 컴파일 거부
서식 불일치	서식 문자열을 믿는 printf	타입을 인자에서 가져오는 자리표시자
불분명한 소유권	네 가지를 뜻하는 char *	소유와 빌림이 다른 타입
검사되지 않는 콜백	void * 인터페이스	문서화된 계약과 최악 보장 알고리즘
바이트의 숨은 타입	앨리어싱 규칙 위반의 UB	규칙이 면제하는 바이트 타입

문 이런 문제를 피하려고 아예 다른 언어를 쓰는 것이 낫지 않은가?

답 그것도 하나의 답이고, 실제로 많은 곳이 그 길을 갔다(1장). 다만 C를 써야 하는 자리는 여전히 남아 있다 — 운영체제, 펌웨어, 다른 언어들이 기대는 바닥층, 그리고 이미 수십 년치 코드가 쌓인 프로젝트들. 그런 자리에서 할 수 있는 일은 **언어를 바꾸는 것이 아니라 API의 모양을 바꾸는 것이다**. 위 표의 오른쪽 열은 새 언어가 필요한 항목이 아니라 C 안에서 설계로 만들 수 있는 것들이다. 다음 장부터 그 설계를 본다.

표의 오른쪽 열을 그대로 구현한 라이브러리가 이 책이 기대 온 proven이다. 35장에서 처음 만났고 그 뒤로 몇 번 이름이 나왔지만, 정면으로 다루는 것은 지금부터다. 다음 장은 설치와 첫 프로그램 — 그리고 이 라이브러리가 왜 “설치할 것이 없는” 모양을 하고 있는지다.

48 시작하기 — 설치할 것이 없다

이 장이 끝나면

proven을 실제로 쓰는 첫 장이다. 이 라이브러리에 configure도, 패키지 관리자도, 링크할 공유 라이브러리도 없는 이유 — 그리고 그 선택이 무엇을 주고 무엇을 앗아가는지 — 를 먼저 보고, 첫 프로그램을 돌린다. 47장에서 본 세 번째 버그(서식 불일치)가 이 첫 프로그램에서 이미 사라진다.

돌아보기 43장에서 여러 파일 프로그램을 만들며 헤더와 목적 파일, 링크를 배웠고, 14장에서는 컴파일 릴레이의 네 주자를 보았다. 그러면 “라이브러리를 쓴다”는 것은 그 그림에서 정확히 무엇을 하는 일인가?

답 둘 중 하나다. **함께 컴파일하거나, 따로 컴파일된 것을 링크하거나**. 앞의 길은 남의 소스를 내 소스와 같이 컴파일러에 넘기는 것이고, 뒤의 길은 이미 목적 코드가 된 덩어리(정적 .a나 공유 .so/.dll)를 링커에게 넘기는 것이다. 어느 쪽이든 컴파일러는 **선언(헤더)**을 보고 호출을 만들고, 링커가 **정의**를 찾아 잇는다 — 43장의 그림 그대로다. proven은 앞의 길을 택했고, 다음 절이 그 이유다.

48.1 설치할 것이 없다는 선택

proven에는 설치 절차가 없다. 받아 둔 소스를 프로그램과 함께 컴파일하면 그만이다. 중요한 디렉터리는 둘뿐이다.

- `src/proven/` — 이식 가능한 본체. 여기서는 운영체제를 부르지 않는다.
- `platform/` — 시스템 호출을 하는 얇은 층. 새 기계로 옮길 때 바뀌어야 할 유일한 부분이다.

운영체제가 없는 환경(임베디드)에서는 `platform/`을 빼고 빌드한다. 이 분리가 라이브러리 전체의 모양을 결정했다 — “어디서나 돌아야 한다”는 요구가 곧 “숨은 할당도, 숨은 전역 상태도 두지 않는다”는 규율이 된다.

문 왜 패키지로 배포하지 않는가? 설치가 더 편할 텐데.

답 대가와 이득을 맞바꾼 것이다. 잃는 것은 편의다 — 시스템 패키지로 받을 수 없고, 갱신은 “버전을 올리는 일”이 아니라 “새 소스를 받아오는 일”이 된다. 얻는 것은 통제권이다. 라이브러리가 지금 보고 있는 소스와 다를 수 없고, 내가 고르지 않은 컴파일 선택지가 딸려 오지 않으며, 배포판이 다른 설정으로 빌드했다는 이유로 링크가 깨지는 일이 없다. 무엇보다 **호스트 환경과 베어메탈 양쪽에서 통하는 유일한 모델**이다 — 임베디드에는 애초에 패키지 관리자가 없다.

48.2 이 책의 예제는 어떻게 빌드되는가

정직하게 밝혀 두면, 이 책의 proven 예제들은 다음과 같이 컴파일된다. 라이브러리 소스를 한 번 목적 파일로 만들어 두고, 예제와 함께 링크한다.

```
$ cc -std=c23 -O1 -Ivendor/proven/include -c vendor/proven/src/proven/*.c
$ cc -std=c23 -Wall -Wextra -Werror -Ivendor/proven/include \
    hello.c vendor-obj/*.o -lm -o hello
```

첫 줄이 본체를, 둘째 줄이 내 프로그램을 다룬다. `-I`는 헤더를 찾을 자리를 알려 주는 것(43장), `-lm`은 수학 함수를 잇는 것이다. 이 두 줄이 이 책의 검증 스크립트가 실제로 매번 돌리는 명령이고, 지면에 인쇄된 모든 실행 결과는 그렇게 만들어진 프로그램의 출력이다.

48.3 첫 프로그램

#include <proven.h> 하나면 라이브러리 전체가 열린다.

```
examples/ch48/hello.c

#include <proven.h>

int main(void)
{
    const char *name = "world";
    int         count = 3;
    double      ratio = 0.75;
    bool        ready = true;
    char        grade = 'A';

    /* {} 는 자리표시자일 뿐 타입이 없다. 타입은 인자에서 온다 */
    proven_println("Hello, {}! You have {} messages.",
                  PROVEN_ARG(name), PROVEN_ARG(count));

    /* 어떤 타입이든 같은 자리표시자로 간다 */
    proven_println("ratio={} ready={} grade={}",
                  PROVEN_ARG(ratio), PROVEN_ARG(ready), PROVEN_ARG(grade));

    /* 서식 지정은 콜론 뒤에 붙인다 - 폭, 정렬, 자릿수 */
    proven_println("|{:>8}|{:<8}|{: .3}|",
                  PROVEN_ARG(name), PROVEN_ARG(name), PROVEN_ARG(ratio));

    return 0;
}
```

실행 결과

```
Hello, world! You have 3 messages.
ratio=0.750000 ready=true grade=A
| world|world |0.750|
```

한 줄씩 읽는다. proven_println은 서식과 인자를 받아 표준 출력에 한 줄을 찍는다 — 여기까지는 printf와 같다. 다른 것은 자리표시자다.

- {}에는 타입이 들어 있지 않다. %d도 %s도 아니고 그냥 {}다.
- 타입은 **인자에서** 온다. PROVEN_ARG(x)가 x의 타입을 보고 알맞은 꼬리표를 붙여 값을 감싼다.
- 그래서 47장의 셋째 버그 — 서식과 인자의 불일치 — 가 **구조적으로** 일어날 수 없다. 타입을 두 번 적지 않으니 어긋날 자리가 없다.

{:>8}처럼 콜론 뒤에 적는 것은 46장에서 본 폭·정렬·정밀도에 해당한다. >는 오른쪽 정렬, <는 왼쪽 정렬, .3은 소수 셋째 자리까지다. 정렬 기호가 앞에 오는 것이 printf와 다른 점이다.

문 PROVEN_ARG는 어떻게 타입을 알아내는가? C에는 함수 오버로딩이 없지 않은가?

답 C11에 들어온 _Generic이라는 장치를 쓴다 — 표현식의 타입에 따라 여러 가지 중 하나를 **컴파일 시간에** 고르는 문법이다. PROVEN_ARG(x)는 x가 int면 정수 꼬리표를, double이면 실수 꼬리표를, const char *면 문자열 꼬리표를 붙인 작은 구조체를 만든다. 실행 시간에 타입을 판별하는 것이 아니라 **컴파일러가 이미 아는 것을 그대로 쓰는 것**이라 비용이 없다. 문법과 전체 서식 규칙은 53장에서 정면으로 다룬다.

⚠ “라이브러리를 쓰면 프로그램이 무거워진다”

자주 듣는 걱정이고, 언어와 라이브러리의 성격에 따라 다르다. C에서 소스로 함께 컴파일하는 라이브러리는 **쓰지 않은 것이 실행 파일에 남지 않는다** — 링커가 참조되지 않은 목적 파일을 아예 넣지 않기 때문이다 (14장의 링크 단계). 게다가 proven에는 시작할 때 도는 초기화 코드도, 등록되는 전역 상태도, 몰래 뜨는 스레드도 없다. 무거워지는 것은 라이브러리를 쓴 대가가 아니라, 프레임워크가 프로그램의 구조를 가져갈 때 생기는 일이다.

● 소스로 배포하는 관행 — 한 파일짜리 SQLite

이 배포 모델은 proven만의 별난 선택이 아니다. 세계에서 가장 널리 쓰이는 데이터베이스 엔진 SQLite는 수십 개 소스 파일을 하나의 거대한 .c 파일로 합친 **amalgamation**을 공식 배포 형태로 제공한다 — 받아서 프로그램과 함께 컴파일하면 끝이다. 이미지·폰트 처리로 유명한 stb 계열 라이브러리는 아예 헤더 파일 하나가 전부다. 이유는 모두 같다. 빌드 환경이 제각각인 세상에서 **가장 이식성 높은 배포 단위는 소스**이기 때문이다.

≡ 복습 정리

이 장의 요약.

무엇	어떻게
헤더	#include <proven.h> 하나
빌드	src/proven/*.c를 프로그램과 함께 컴파일
OS 의존	platform/에만 있다 (없으면 빼고 빌드)
출력	proven_println("... {} ...", PROVEN_ARG(x))
서식 지정	{:>8} {:<8} {:.3} — 콜론 뒤
대가	인자마다 PROVEN_ARG, 익숙한 %d와 다른 문법

첫 프로그램이 돌아왔다. 그런데 방금 쓴 proven_println도 사실은 실패할 수 있다 — 화면으로 가는 띠가 끊길 수 있기 때문이다(8장). 이 함수는 에러를 돌려주되 **확인을 강요하지는 않는데**, 그 선택 자체가 이 라이브러리의 에러 모델을 이해하는 좋은 입구다. 다음 장이 그것이다.

49 에러는 값이다

이 장이 끝나면

47장의 둘째 버그 — 확인되지 않는 실패 — 에 대한 답을 본다. 실패를 값으로 돌려주는 방식, 값과 에러를 함께 담는 꾸러미, 그리고 에러를 버리면 컴파일러가 항의하게 만드는 장치까지. 41장에서 세운 “오류는 값이다”라는 규율이 여기서 타입으로 굳는다.

돌아보기 41장에서 C의 오류 통보 방식이 반환값·전역 상태·프로그램 중단 셋뿐이라 했고, 그중 반환값이 가장 정직하다고 했다. 그러면 반환값 하나로 “실패 했다”와 “결과는 이것이다”를 동시에 말하려면 어떻게 해야 하는가?

답 방법은 셋이다. 결과 자리에 **불가능한 값**을 섞어 쓰거나(널, -1 — 47장에서 본 그 함정이다), 결과를 **출력 매개변수**로 빼고 반환값을 상태 전용으로 쓰거나, 둘을 **하나의 구조체에 담아** 함께 돌려주거나. proven은 둘째와 셋째를 함께 쓴다 — 그리고 셋째가 가능한 이유는 38장에서 배운 대로 C의 구조체가 값으로 반환될 수 있기 때문이다.

49.1 두 가지 반환 모양

규칙은 단순하다. 실패할 수 있는 함수는 반드시 실패를 값으로 돌려준다.

- 돌려줄 결과가 없으면 proven_err_t 하나를 돌려준다.
- 돌려줄 결과가 있으면 {err, value} 꾸러미를 돌려준다 — proven_result_u8str_t, proven_result_size_t처럼 타입마다 이름이 있다.

proven_err_t는 열거형이고 성공은 PROVEN_OK(0)다. 실패는 종류마다 이름이 있다 — PROVEN_ERR_NOMEM(기억 부족), PROVEN_ERR_OUT_OF_BOUNDS(자리 부족), PROVEN_ERR_NOT_FOUND, PROVEN_ERR_INVALID_ARG, PROVEN_ERR_IO, PROVEN_ERR_EOF 등. 확인은 언제나 proven_is_ok(err) 하나로 한다.

```
examples/ch49/errval.c

#include <proven.h>
#include <stdio.h>

/* 실패할 수 있는 일을 한다: 정해진 용량의 문자열에 두 조각을 붙인다.
   에러는 값으로 돌아오고, 호출자는 그것을 그대로 위로 전달한다. */
static proven_err_t make_greeting(proven_allocator_t alloc,
                                   const char *who,
                                   proven_size_t cap,
                                   proven_u8str_t *out)
{
    proven_result_u8str_t made = proven_u8str_create(alloc, cap);
    if (!proven_is_ok(made.err))
        return made.err;          /* 실패를 위로 넘긴다 */

    proven_u8str_t s = made.value; /* 확인한 뒤에야 value 를 쓴다 */

    proven_err_t e = proven_u8str_append(&s, proven_u8str_view_from_cstr("Hello, "));
    if (proven_is_ok(e))
        e = proven_u8str_append(&s, proven_u8str_view_from_cstr(who));
    if (!proven_is_ok(e)) {
        proven_u8str_destroy(alloc, &s); /* 실패했으면 뒷정리도 우리 몫 */
        return e;
    }
}
```

```

    *out = s;
    return PROVEN_OK;
}

static void try(proven_allocator_t alloc, const char *who, proven_size_t cap)
{
    proven_u8str_t s;
    proven_err_t e = make_greeting(alloc, who, cap, &s);

    if (!proven_is_ok(e)) {
        printf("cap=%2zu who=%-8s -> failed with error code %d\n",
            cap, who, (int)e);
        return;
    }
    proven_u8str_view_t v = proven_u8str_as_view(&s);
    printf("cap=%2zu who=%-8s -> \"%.*s\"\n",
        cap, who, (int)v.size, (const char *)v.ptr);
    proven_u8str_destroy(alloc, &s);
}

int main(void)
{
    proven_allocator_t alloc = proven_heap_allocator();

    try(alloc, "world", 32); /* 넉넉하다 */
    try(alloc, "world", 8); /* 자리가 모자란다 - 자르지 않고 거부한다 */

    printf("PROVEN_OK is %d, and every failure is non-zero\n", (int)PROVEN_OK);
    return 0;
}

```

실행 결과

```

cap=32 who=world    -> "Hello, world"
cap= 8 who=world    -> failed with error code 2
PROVEN_OK is 0, and every failure is non-zero

```

이 예제에 이 장의 문법이 다 들어 있다. `make_greeting`은 결과를 출력 매개변수(`out`)로 내보내고 반환값은 상태 전용으로 썼다 — 실패하면 위로 그대로 넘긴다. 그리고 `proven_u8str_create`는 꾸러미를 돌려주므로, **확인한 뒤에야** `made.value`를 꺼낸다. 순서가 뒤바뀌면 안 된다.

✗ 확인하기 전에 `value`를 꺼내기

```
proven_u8str_t s = proven_u8str_create(alloc, 64).value; /* 위험 */
```

한 줄로 끝나서 깔끔해 보이지만, 실패했을 때 손에 들어오는 것은 **의미 없는 값**이다. 꾸러미의 계약은 “`err`가 `PROVEN_OK`일 때만 `value`가 뜻을 가진다”이므로, 이 코드는 계약을 건너뛴 것이다. 실패해도 대개 0으로 채워진 구조체가 와서 당장은 죽지 않는다는 점이 오히려 위험하다 — 사고가 한참 뒤로 미뤄진다(47장의 그 무늬다).

두 번째 호출의 출력이 이 부의 주제를 압축한다. 용량 8바이트에 “Hello, world”를 넣으려 하자 라이브러리는 들어가는 만큼만 넣고 성공을 말하는 대신, 아무것도 쓰지 않고 실패를 돌려주었다. 47장에서 `snprintf`의 조용한 잘림과 정확히 반대되는 선택이다.

49.2 버리면 컴파일러가 항의한다

에러를 값으로 돌려주는 것만으로는 부족하다. 47장에서 본 대로 반환값은 **버릴 수 있기** 때문이다. 그래서 실패가 의미 있는 함수에는 C23의 `[[nodiscard]]`가 붙어 있다(41장에서 이름을 보았다). 결과를 버리면 컴파일러가 실제로 이렇게 말한다.

```
warning: ignoring return value of 'proven_u8str_append',
      declared with attribute 'nodiscard' [-Wunused-result]
   5 |     proven_u8str_append(&s, proven_u8str_view_from_cstr("hi"));
      |     ^~~~~~
note: declared here
 123 | [[nodiscard]] proven_err_t proven_u8str_append(...);
```

15장에서 권한 `-Werror`를 켜 두었다면 이것은 경고가 아니라 **빌드 실패**가 된다. “확인”은 선택”이던 것이 “확인하지 않으면 컴파일되지 않음”으로 바뀐 것이다.

물론 정말로 무시하고 싶을 때가 있다. 그때는 `(void)`를 앞에 붙인다.

```
(void)proven_u8str_append(&s, view); /* 알고도 무시한다 */
```

문 `(void)`로 무시할 수 있으면 강제가 아니지 않은가?

답 강제의 목적이 “무시를 막는 것”이 아니라 “**무시를 눈에 보이게 만드는 것**”이기 때문이다. 아무 표시 없이 버려진 에러는 코드 검토에서 보이지 않지만, `(void)`가 붙은 줄은 “이 실패는 의도적으로 무시한다”는 선언이 된다. 타이핑해야 한다는 사실 자체가 핵심이다 — 실수로는 못 하고, 일부러만 할 수 있다.

문 그런데 48장의 `proven_println`에는 그 표시가 없었다. 왜 화면 출력만 예외인가?

답 계약에도 등급이 있기 때문이다. 콘솔로 나가는 쓰기의 실패는 관례적으로 무시되어 왔고(`printf`의 반환값을 확인하는 코드를 본 적이 있는가?), 모든 출력 한 줄마다 `(void)`를 붙이게 만들면 코드가 소음으로 뒤덮인다. 그래서 이 라이브러리는 **에러는 돌려주되 확인을 강요하지는 않는** 등급을 출력에 두었다. 반대로 **입력** 쪽 함수들에는 표시가 붙어 있다 — 읽기의 실패를 무시하면 “읽지 않은 데이터”를 읽은 것처럼 다루게 되어, 47장의 둘째 버그가 그대로 재현되기 때문이다. 어디에 표시를 붙일지가 곧 설계 판단이다.

49.3 실패했을 때 남는 것 — 실패 원자성

에러를 돌려받은 뒤 자연스럽게 드는 물음이 있다. **실패한 함수가 건드리던 객체는 지금 어떤 상태인가?**

라이브러리의 답은 **실패 원자성**(failure atomicity)이다 — 문서에 달리 적지 않은 한, 실패한 연산은 대상을 손대기 전 상태로 남긴다. 배열을 늘리다 기억이 모자라면 기존 원소는 그대로 살아 있고, 문자열에 덧붙이다 자리가 모자라면 원래 내용이 그대로다. 방금 예제의 두 번째 호출이 그 예다 — 실패했지만 반쯤 쓰인 문자열이 남지는 않았다.

이것이 왜 중요한가. 실패 원자성이 없으면 호출자는 실패한 뒤 **객체를 버리는 것 말고는** 할 수 있는 일이 없다. 있으면 “이번 추가는 포기하고 지금까지 모은 것으로 계속 간다”가 가능해진다.

△ “실패하면 어차피 프로그램을 끝낼 텐데 상태가 무슨 상관인가”

짧게 도는 명령줄 도구라면 그럴 수 있다. 그러나 오래 도는 프로그램 — 서버, 편집기, 게임, 펌웨어 — 은 실패 하나로 죽으면 안 된다. 한 요청의 처리가 기억 부족으로 실패했다고 서버 전체가 내려간

다면 그것이 더 큰 사고다. 실패 원자성은 “이 요청만 버리고 다음 요청을 받는” 복구를 가능하게 하는 최소 조건이다.

49.4 돌려줄 상대가 없을 때 — 패닉

에러를 값으로 돌려주려면 **돌려줄 상대**가 있어야 한다. 그런데 계약 자체가 깨진 자리에서는 그 상대가 없다 — 예를 들어 널 포인터를 넘겨야 할 이유가 전혀 없는 자리에 널이 왔다면, 그것은 실패가 아니라 **프로그램의 논리가 이미 틀렸다는 뜻**이다.

이런 자리를 위해 라이브러리에는 패닉(panic) 경로가 있다. 41장의 `assert`와 같은 정신이고, 다른 점은 임베디드에서도 쓸 수 있도록 처리 방식을 갈아 끼울 수 있다는 것이다(56장). 구분은 이렇게 기억하면 된다.

≡ 복습 정리

상황	예	라이브러리의 처리
바깥 세계의 실패	기억 부족, 파일 없음, 자리 부족	에러를 값으로 반환
호출자의 계약 위반	있어서는 안 될 널, 파괴한 객체 재사용	패닉 (또는 미정의)
무시해도 좋은 실패	콘솔 출력 실패	에러를 반환하되 강요하지 않음

● 값으로 오는 에러를 택한 다른 언어들

이 설계는 C만의 발명이 아니라 최근 시스템 언어들의 공통된 흐름이다. Go는 함수가 결과와 에러를 나란히 돌려주고, Rust는 `Result` 타입 하나로 성공과 실패를 감싸며 무시하면 경고한다. 둘 다 예외(exception)를 쓰지 않기로 한 언어이고, 이유도 같다 — **에러 경로가 코드의 모양에 드러나야 한다**는 것. 예외는 편리하지만 “여기서 어떤 실패가 어디로 튈는가”를 시그니처에서 지운다. 47장 표의 둘째 줄에 대한 답을, 서로 다른 세 언어가 같은 방향에서 찾은 셈이다.

문 이 방식의 대가는 무엇인가?

답 `if`가 늘어난다. 예외처럼 깊은 곳의 실패를 한 번에 건어 올려 주는 장치가 없으므로, 실패를 만나는 자리마다 확인하고 위로 넘기는 코드가 붙는다. 방금 예제의 `make_greeting`이 스무 줄 남짓 되는 이유의 절반이 그것이다. 그 대신 얻는 것이 하나 있다 — **어디서 무엇이 실패할 수 있는지가 코드에 그대로 보인다**. 감춰진 실패가 없다는 것, 그것이 이 라이브러리가 파는 물건이다.

에러의 모양을 알았으니, 이제 그 에러들이 지키는 대상 — 기억 그 자체 — 으로 내려간다. 다음 장은 바이트와 뷰, 그리고 넘치지 않는 크기 계산이다.

50 기반 — 바이트, 뷰, 그리고 넘치지 않는 산술

이 장이 끝나면

라이브러리 전체가 닫고 선 네 가지 기본 어휘를 본다 — 원시 바이트를 가리키는 타입, 포인터와 길이를 하나로 묶은 뷰(view), 감기지 않는 크기 계산, 그리고 정렬. 47장의 여섯째 버그(바이트의 숨은 타입)와 33장의 경계 문제가 여기서 타입 차원의 답을 얻는다.

돌아보기 32장에서 `char*`(그리고 `unsigned char*`)만은 어떤 객체든 바이트 단위로 들여다볼 수 있는 특권을 가진다고 했고, 11장에서는 그 규칙을 어긴 코드가 최적화에서 조용히 무너지는 것을 보았다. 그러면 “안전하게 바이트를 다룬다”는 것은 구체적으로 무엇을 지키는 일인가?

답 두 가지다. 첫째, 들여다보는 타입을 규칙이 면제한 것으로 고정한다 — `unsigned char`다. 둘째, 들여다보는 범위를 잃어버리지 않는다 — 포인터 하나만 들고 다니면 어디까지가 내 땅인지 잊게 되고, 그것이 33장의 경계 침범이다. proven의 기본 어휘는 이 두 가지를 각각 타입으로 만든 것이다.

50.1 바이트에는 이름이 있다

`proven_byte_t`는 `unsigned char`의 별칭이다. 별것 아닌 것 같지만 선언 하나가 계약을 적는다 — “이 포인터는 표현(representation)을 보는 눈이지, 어떤 타입의 값을 가리키는 것이 아니다”.

이 구분이 왜 중요한지는 11장에서 이미 보았다. 같은 기억을 `uint32_t*`와 `uint16_t*`로 번갈아 보는 코드는 앨리어싱 규칙 위반이고, 컴파일러는 그 전제를 최적화에 쓴다. 반면 `unsigned char`로 보는 것은 표준이 명시적으로 허용한다. 라이브러리가 원시 메모리를 다룰 때 언제나 이 타입을 거치는 이유이고, 그래서 라이브러리를 통해서 그 버그를 쓸 수가 없다.

50.2 뷰 — 포인터와 길이를 하나로

읽기 전용 뷰는 이렇게 생겼다. 구조체 두 칸이 전부다.

```
typedef struct {
    const proven_byte_t *ptr;
    proven_size_t        size;
} proven_mem_view_t;
```

쓰기가 가능한 판(`proven_mem_mut_t`)은 `const`만 빠진 쌍둥이다. 이 작은 구조체가 하는 일은 하나다 — 포인터와 길이가 절대로 헤어지지 않게 하는 것. 34장에서 “문자열의 진짜 문제는 길이를 따로 들고 다니는 것”이라 했던 그 문제에 대한 답이다.

```
examples/ch50/view.c

#include <proven.h>
#include <stdio.h>

static void dump(const char *label, proven_mem_view_t v)
{
    printf("%-10s size=%zu bytes:", label, v.size);
    for (proven_size_t i = 0; i < v.size; i++) printf(" %02x", v.ptr[i]);
    printf("\n");
}

int main(void)
{
    /* 원시 바이트는 proven_byte_t 다 - unsigned char 의 별칭이라
       어떤 객체의 표현이든 들여다볼 수 있는 유일한 타입이다 */
```

```

proven_byte_t buf[8] = {0x10, 0x20, 0x30, 0x40, 0x50, 0x60, 0x70, 0x80};

/* view = 포인터 + 길이. 길이를 따로 들고 다니지 않는다 */
proven_mem_view_t all = { .ptr = buf, .size = sizeof buf };
dump("all", all);

/* 자르기: 경계를 넘으면 실패가 값으로 온다 */
proven_result_mem_view_t mid = proven_mem_view_slice_checked(all, 2, 3);
if (proven_is_ok(mid.err)) dump("slice 2+3", mid.value);

proven_result_mem_view_t over = proven_mem_view_slice_checked(all, 6, 4);
printf("%-10s err=%d (refused, not clamped)\n", "slice 6+4", (int)over.err);

/* 크기 계산도 감기지 않게 한다: 배열 원소 수 x 원소 크기 */
proven_size_t n = (proven_size_t)-1 / 2; /* 터무니없이 큰 개수 */
proven_size_t bytes;
if (PROVEN_CKD_MUL(&bytes, n, (proven_size_t)8))
    printf("%-10s %zu * 8 overflows - allocation refused\n", "size calc", n);
else
    printf("%-10s %zu bytes\n", "size calc", bytes);

/* 정렬 올림: 다음 경계로 밀어 올린다 */
printf("align_up(13, 8) = %zu\n", proven_mem_align_up(13, 8));
return 0;
}

```

실행 결과

```

all          size=8 bytes: 10 20 30 40 50 60 70 80
slice 2+3    size=3 bytes: 30 40 50
slice 6+4    err=2 (refused, not clamped)
size calc    9223372036854775807 * 8 overflows - allocation refused
align_up(13, 8) = 16

```

slice 6+4가 이 절의 핵심이다. 8바이트짜리 뷰에서 6번째부터 4바이트를 달라고 했으니 두 바이트가 모자란다. 라이브러리는 **있는 만큼 잘라 주지 않는다** — 에러를 돌려준다(2번은 `PROVEN_ERR_OUT_OF_BOUNDS`다). 있는 만큼 주는 편이 친절해 보이지만, 그러면 호출자는 자기가 받은 것이 요청한 것인지 알 수 없게 된다. 47장의 잘림 문제가 여기서 되풀이되지 않도록 막은 것이다.

문 자르기 함수가 `_checked`와 `_unchecked` 두 판으로 있던데, 후자는 왜 존재하는가?

답 경계를 **이미 확인한** 자리를 위해서다. 반복문 안에서 매 회 같은 검사를 다시 하는 것은 낭비이므로, 루프 앞에서 한 번 확인하고 안쪽에서는 검사 없는 판을 쓰는 식이다. 이름에 `_unchecked`가 붙어 있다는 점이 중요하다 — 위험한 선택은 **눈에 보이는 이름**으로만 할 수 있고, 기본값은 언제나 안전한 쪽이다. 41장에서 말한 “계약을 코드로 적는다”의 한 형태다.

x 뷰를 소유자보다 오래 들고 있기

```

proven_mem_view_t get_view(void) {
    proven_byte_t local[16] = {0};
    return (proven_mem_view_t){ .ptr = local, .size = sizeof local };
} /* local 은 여기서 죽는다 - 돌려준 뷰는 이미 유효하지 않다 */

```

뷰는 **빌린** 것이다. 소유자가 사라지면 그 순간 유효하지 않게 되고, 그 뒤의 사용은 36장에서 배운 죽은 자동 변수 접근이다. 뷰가 포인터보다 안전한 것은 **경계**에 대해서이지 **수명**에 대해서가 아니다 — 수명은 여전히 사람이 지켜야 한다. 그래서 다음 장의 할당자 이야기가 필요해진다.

50.3 넘치지 않는 크기 산술

23장에서 무부호 정수의 감김을 배웠고, 5장에서 그것이 왜 정의된 동작인지 보았다. 감김이 조용하다는 사실은 한 자리에서 특히 위험해진다 — **할당할 바이트 수를 계산할 때다**.

```
void *p = malloc(count * sizeof(item_t)); /* count 가 크면 감킨다 */
```

count가 충분히 크면 곱이 감겨서 아주 작은 수가 되고, malloc은 그 작은 크기로 성공한다. 그 뒤 프로그램은 원래 의도한 개수만큼 쓰기 시작한다 — 전형적인 힙 넘침이다. 23장에서 “크기 계산”을 오버플로의 실제 사고 사례로 든 것이 이 무늬였다.

라이브러리는 크기 계산에 C23의 검사 산술을 쓴다. 예제의 `PROVEN_CKD_MUL`이 그것으로, 곱이 넘치면 **참**을 돌려준다(계산 결과가 아니라 넘침 여부가 반환값이다). 넘치면 할당을 아예 시도하지 않는다.

△ “size_t는 64비트나 되니 넘칠 리 없다”

넘칠 수 있고, 실제로 넘긴다. 곱셈은 값을 **제공 규모로** 키우기 때문이다 — 40억 × 40억은 이미 64비트를 넘는다. 게다가 32비트 기계에서 size_t는 여전히 32비트이고, 파일 형식에서 읽어 온 32비트 필드 둘을 곱하는 코드는 그 자리에서 감킨다. 무엇보다 중요한 것은 **누가 그 수를 정하는가**다. 크기가 프로그램 안의 상수라면 안심해도 좋지만, 파일이나 네트워크에서 온 수라면 그것은 **공격자가 고른 피연산자**다.

● 곱셈 하나가 만든 취약점들

이 무늬는 CVE 목록의 단골이다. 이미지 디코더가 `width * height *`

`bytes_per_pixel`을 32비트로 계산하다 감킨 사례, 폰트 파서가 글리프 개수를 곱하다 감킨 사례, 압축 해제 코드가 원본 크기를 곱하다 감킨 사례가 반복해서 보고됐다. 공통점은 모두 **입력 파일이 크기를 정한다**는 것 — 즉 공격자가 곱셈의 피연산자를 고를 수 있다는 것이다. 그래서 요즘 언어와 라이브러리는 크기 계산을 검사 산술로 하거나 아예 넘칠 수 없는 타입으로 다룬다.

50.4 정렬 — 다음 경계로 밀어 올리기

4장에서 배운 정렬(alignment)이 여기서 실용적인 도구가 된다. `proven_mem_align_up(13, 8)`은 16을 돌려준다 — 13번지에서 시작하는 객체를 8바이트 경계에 맞추려면 16으로 밀어야 하기 때문이다. 다음 장의 아래나가 객체를 줄지어 놓을 때 매번 하는 계산이 바로 이것이다.

`PROVEN_MAX_ALIGN`은 “어떤 타입이든 담을 수 있는 가장 엄격한 정렬”이고, `proven_is_pow2`는 정렬 값이 2의 거듭제곱인지 확인한다(정렬은 언제나 2의 거듭제곱이어야 한다 — 그래야 비트 연산으로 올림을 계산할 수 있다).

≡ 복습 정리

이 장의 어휘.

이름	무엇	계약
<code>proven_byte_t</code>	unsigned char 별칭	표현을 보는 유일한 합법 창

proven_mem_view_t	읽기 전용 뷰(ptr+size)	빌린 것 — 소유자보다 오래 살 수 없다
proven_mem_mut_t	쓰기 가능 뷰	같음
..._slice_checked	부분 뷰 만들기	범위를 넘으면 에러, 자르지 않음
PROVEN_CKD_MUL/ADD	검사 산술	넘치면 참 — 계산은 버린다
proven_mem_align_up	정렬 올림	정렬은 2의 거듭제곱

기억을 **보는** 어휘를 갖췄다. 다음은 기억을 **얻는** 이야기다 — 그리고 그 자리에서 이 라이브러리의 가장 특징적인 결정을 만나게 된다.

51 할당은 매개변수다

이 장이 끝나면

47장의 넷째 버그 — 불분명한 소유권 — 에 대한 답이다. 기억의 출처를 값으로 다루는 **할당자** (allocator), 수명이 같은 것들을 한꺼번에 되돌리는 아레나(arena), 그리고 소유와 빌림을 타입으로 가르는 규칙까지. 37장에서 배운 힙의 위험이 여기서 설계로 정리된다.

돌아보기 37장에서 malloc으로 얻은 기억은 누군가 정확히 한 번 free해야 하고, 그 “누군가”를 정하는 것이 프로그램의 설계라고 했다. 그러면 함수 하나만 보고 “이 함수가 기억을 할당하는가”를 알 방법이 있는가?

답 표준 C에는 없다. malloc은 어디서든 부를 수 있으므로 어떤 함수든 몰래 할당할 수 있고, 시그니처는 그 사실을 말하지 않는다. proven의 답은 규칙 하나로 그 정보를 시그니처에 되돌리는 것이다 — **할당자를 인자로 받지 않는 함수는 할당하지 않는다**. 이 규칙이 지켜지면 시그니처를 읽는 것만 으로 “이 함수는 기억을 잡을 수 있다”를 알 수 있다.

51.1 할당자는 값이다

proven_allocator_t는 구조체 하나다 — 문맥 포인터와 함수 포인터 셋 (할당·재할당·해제). 특별한 전역도, 등록 절차도 없다. 그냥 값이라서 인자로 넘기고, 구조체에 담고, 함수에서 돌려줄 수 있다.

그래서 할당하는 함수는 모두 이런 모양이다.

```
proven_result_u8str_t proven_u8str_create(proven_allocator_t alloc,
                                          proven_size_t limit);
void proven_u8str_destroy(proven_allocator_t alloc,
                          proven_u8str_t *str);
```

만들 때 준 할당자로 파괴한다 — 이것이 소유권 규칙의 전부다.

```
examples/ch51/swap.c

#include <proven.h>
#include <stdio.h>

/* 이 함수는 어디서 기억을 얻는지 모른다 - 받은 할당자에게 물을 뿐이다.
   시그니처에 allocator 가 있다는 사실 자체가 "할당한다"는 선언이다. */
static proven_err_t build(proven_allocator_t alloc, const char *who)
{
    proven_result_u8str_t made = proven_u8str_create(alloc, 64);
    if (!proven_is_ok(made.err)) return made.err;

    proven_u8str_t s = made.value;
    proven_err_t e = proven_u8str_append(&s, proven_u8str_view_from_cstr("hi, "));
    if (proven_is_ok(e))
        e = proven_u8str_append(&s, proven_u8str_view_from_cstr(who));
    if (!proven_is_ok(e)) { proven_u8str_destroy(alloc, &s); return e; }

    proven_u8str_view_t v = proven_u8str_as_view(&s);
    printf(" -> \"%s\"\n", (int)v.size, (const char *)v.ptr);

    proven_u8str_destroy(alloc, &s); /* 준 곳으로 돌려준다 */
    return PROVEN_OK;
}
```

```

int main(void)
{
    /* ① 힙 - malloc/free 를 감싼 것 */
    printf("heap : \n");
    (void)build(proven_heap_allocator(), "heap");

    /* ② 아레나 - 정적 버퍼 위에서 잘라 쓴다. 힙이 아예 없어도 된다 */
    static proven_byte_t backing[512];
    proven_arena_t arena = proven_arena_create(
        (proven_mem_mut_t){ .ptr = backing, .size = sizeof backing });

    printf("arena : \n");
    (void)build(proven_arena_as_allocator(&arena), "arena");
    printf(" used %zu of %zu bytes\n", arena.offset, sizeof backing);

    /* 아레나는 개별 해제가 없다 - 한 번에 되돌린다 */
    proven_arena_reset(&arena);
    printf(" after reset: %zu bytes used\n", arena.offset);

    /* ③ 같은 아레나를 다시 쓴다 - 같은 코드, 다른 기억의 출처 */
    printf("arena : \n");
    (void)build(proven_arena_as_allocator(&arena), "arena again");
    return 0;
}

```

실행 결과

```

heap :
-> "hi, heap"
arena :
-> "hi, arena"
used 65 of 512 bytes
after reset: 0 bytes used
arena :
-> "hi, arena again"

```

build 함수를 보라. 이 함수는 자기가 쓰는 기억이 malloc에서 오는지 정적 배열에서 오는지 **알지 못한다**. 호출자가 정한다. 같은 코드가 힙 위에서 한 번, 아레나 위에서 두 번 돌았고 결과는 같다.

문 이게 왜 그렇게 중요한 성질인가? 어차피 대부분은 힙을 쓸 텐데.

답 세 가지가 한꺼번에 따라오기 때문이다. 첫째, **힙이 없는 환경에서도 같은 코드가 돈다** — 임베디드가 그렇고(56장), 커널 안이 그렇다. 둘째, **시험이 쉬워진다** — 일부러 실패하는 할당자를 끼워 넣으면 “기억이 모자랄 때 이 코드가 제대로 복구하는가”를 시험할 수 있다. 셋째, **성능을 호출자가 고를 수 있다** — 짧게 살다 한꺼번에 죽는 자료라면 아레나가 힙보다 훨씬 빠르다. 라이브러리가 malloc을 직접 부르는 순간 이 셋이 전부 사라진다.

51.2 아레나 — 수명이 같은 것들을 한 번에

37장에서 힙의 위험 셋(누수·이중 해제·사용 후 해제)을 배웠다. 그 셋은 모두 **개별 해제**에서 나온다. 그렇다면 개별 해제를 없애면 어떨까 — 그것이 아레나의 착상이다.

아레나는 큰 기억 덩어리 하나를 받아 두고, 요청이 올 때마다 앞에서부터 잘라 준다. 해제 함수는 있지만 아무 일도 하지 않는다. 대신 **리셋**이 있다 — 한 번에 전부 되돌린다.

```

examples/ch51/arena.c
#include <proven.h>
#include <stdio.h>

int main(void)
{
    /* 아레나: 뒤를 받쳐 줄 기억을 통째로 주고, 그 안에서 잘라 쓴다 */
    static unsigned char backing[1024];
    proven_arena_t arena = proven_arena_create(
        (proven_mem_mut_t){ .ptr = backing, .size = sizeof backing });

    proven_result_mem_mut_t a = proven_arena_alloc(&arena, 100);
    proven_result_mem_mut_t b = proven_arena_alloc(&arena, 200);

    if (!proven_is_ok(a.err) || !proven_is_ok(b.err)) {
        printf("arena allocation failed\n");
        return 1;
    }
    printf("carved two blocks: %zu, %zu bytes\n", a.value.size, b.value.size);

    /* 개별 해제는 없다 - 수명이 같은 것들을 한 번에 되돌린다 */
    proven_arena_reset(&arena);
    printf("one reset reclaimed everything\n");

    /* 그릇보다 큰 요청은 실패가 값으로 온다 (붕괴하지 않는다) */
    proven_result_mem_mut_t too_big = proven_arena_alloc(&arena, 100000);
    printf("oversized request: %s\n", proven_is_ok(too_big.err) ? "granted" : "refused");
    return 0;
}

```

실행 결과

```

carved two blocks: 100, 200 bytes
one reset reclaimed everything
oversized request: refused

```

이 모델이 맞는 자리가 실제로 많다. 한 요청을 처리하는 동안 만든 임시 자료들, 한 프레임 동안 쓰는 게임의 계산 결과, 한 파일을 파싱하는 동안 만든 구문 트리 — 전부 태어난 시점은 다르지만 죽는 시점이 같은 자료들이다. 그런 자료에 개별 해제는 비용이자 위험일 뿐이다.

≡ 복습 정리

세 가지 기억의 출처 비교.

	힙	아레나	풀(pool)
주는 방식	임의 크기	앞에서부터 자름	고정 크기 칸
개별 해제	있음	없음(리셋)	있음(칸 반납)
단편화	생길 수 있음	없음	없음
속도	보통	매우 빠름	매우 빠름
맞는 자리	수명이 제각각	수명이 같은 무리	같은 크기를 반복

풀(proven_pool_t)은 세 번째 갈래다. 같은 크기의 객체를 반복해서 만들고 없애는 자리 — 게임의 총알, 서버의 연결 객체, 파서의 노드 — 에서 쓴다. 칸의 크기가 고정이라 단편화가 없고, 반납된 칸을 곧바로 재사용한다. 풀도 proven_pool_as_allocator로 할당자가 되므로 앞 예제의 build 함수가 그대로 돈다.

✗ 다른 할당자로 파괴하기

```
proven_result_u8str_t r = proven_u8str_create(proven_heap_allocator(), 64);
...
proven_u8str_destroy(proven_arena_as_allocator(&arena), &r.value); /* 틀렸다 */
```

malloc으로 얻은 것을 아레나에게 돌려주는 셈이다. 이 규칙은 라이브러리가 강제할 수 없다 — 할당자는 그냥 값이고, 어떤 값을 넘길지는 호출자가 정하기 때문이다. 그래서 실무의 관행은 **할당자를 자료구조와 함께 들고 다니거나**, 한 모듈 안에서 하나만 쓰도록 범위를 좁히는 것이다.

⚠ “아레나를 쓰면 해제를 신경 쓸 필요가 없다”

개별 해제는 없어지지만 **수명은 그대로 남는다**. 아레나를 리셋하는 순간 그 위에서 잘라 준 모든 조각이 한꺼번에 무효가 되므로, 리셋 이후에도 살아 있어야 하는 자료를 아레나에 두면 안 된다. 50장에서 본 “뷰는 소유자보다 오래 살 수 없다”가 여기서 아레나 단위로 커진 것이다. 아레나가 없애는 것은 해제의 횟수이지 수명에 대한 사고가 아니다.

51.3 소유와 빌림, 그리고 자기를 가리키는 상태

이제 47장의 넷째 버그로 되돌아간다. `char *`가 네 가지를 뜻하던 문제는 타입을 둘로 가르는 것으로 풀린다.

- **소유하는 것(owned)** — `proven_u8str_t`, `proven_array_t` 같은 것들. `_create`로 얻고 `_destroy`로 놓는다.
- **빌린 것(borrowed)** — `proven_u8str_view_t`, `proven_mem_view_t`. 절대 파괴하지 않는다. 소유자가 사라지면 함께 무효가 된다.

시그니처만 보고 “이걸 해제해야 하나”를 답할 수 있다는 것, 그것이 이 구분의 전부이자 목적이다.

여기에 딸린 규칙이 하나 더 있다. **자기를 가리키는 상태는 복사하지 않는다**. 38장에서 구조체 대입이 얇은 복사(shallow copy)라는 것을 배웠다. 내부에 자기 버퍼를 가리키는 포인터를 둔 객체를 그렇게 복사하면 사본의 포인터가 여전히 원본을 가리켜 두 객체가 같은 기억을 공유하게 되고, 둘 다 파괴하면 이중 해제다. 그래서 이런 객체는 값으로 복사하지 않고 포인터로 넘긴다.

● 할당자를 인자로 받는 설계의 확산

이 방식은 proven의 발명이 아니라 시스템 프로그래밍의 최근 합의에 가깝다. Zig는 표준 라이브러리의 거의 모든 할당 API가 할당자를 인자로 받고 “숨은 할당 없음”을 언어의 표어로 삼는다. Rust에서도 컨테이너에 할당자를 붙이는 기능이 표준에 들어오는 중이고, C++의 `std::pmr`도 같은 문제의식에서 나왔다. 이유는 모두 같다 — 게임, 임베디드, 커널, 고성능 서버에서 **기억의 출처를 고르는 일**이 성능과 안전 양쪽의 핵심이기 때문이다.

문 그러면 대가는 무엇인가?

답 시그니처에 매개변수가 하나 더 붙는다. 그리고 그 할당자를 **어디까지 들고 다닐 것인가**라는 설계 문제가 새로 생긴다 — 함수마다 넘길 것인지, 구조체에 담아 둘 것인지. 작은 프로그램에서는 이것이 성가신 격식으로만 보일 수 있다. 값이 드러나는 것은 프로그램이 커진 뒤, 또는 힘을 쓸 수 없는 자리로 코드를 옮겨야 할 때다.

기억을 얻고 놓는 규칙을 세웠다. 다음 장부터는 그 위에 올라가는 실제 부품들이다 — 먼저, 이 책이 34장 내내 조심하라고 했던 문자열이다.

52 문자열과 텍스트

이 장이 끝나면

47장의 첫째 버그 — 그릇의 크기를 모르는 문자열 함수 — 에 대한 답이다. 길이를 지니고 다니는 문자열, 소유와 빌림의 구분, 그리고 이 라이브러리 에서 가장 논쟁적인 결정인 **거부하되 자르지 않는** 다까지. 7장의 인코딩 이야기와 34장의 경계 이야기가 여기서 하나로 모인다.

돌아보기 34장에서 C 문자열은 “NUL을 만날 때까지”가 곧 길이라, 길이를 알려면 매번 세어야 한다고 했다. 그러면 길이를 함께 들고 다니면 무엇이 좋아지는가 — 그리고 무엇을 잃는가?

답 얻는 것이 셋이다. 길이를 세는 비용이 사라지고($O(n)$ 이 $O(1)$ 이 된다), 중간에 0바이트가 들어 있는 자료도 다룰 수 있으며, 무엇보다 **경계를 검사할 수 있다** — 길이를 모르면 검사할 것 자체가 없다. 잃는 것은 둘이다. 문자열 하나가 한 워드가 아니라 두 워드가 되고, `printf("%s")` 처럼 NUL 종단을 기대하는 세상과 만날 때 변환이 필요해진다. 이 라이브러리는 그 변환 비용을 없애려고 내부적으로 NUL도 함께 유지한다 — 둘 다 가지는 절충이다.

52.1 두 개의 타입, 하나의 규칙

문자열 어휘는 둘뿐이다.

- `proven_u8str_t` — **소유**하는 문자열. 버퍼와 길이와 용량을 가진다.
- `proven_u8str_view_t` — **빌린** 문자열. 포인터와 길이뿐이다.

이름에 `view`가 있으면 빌린 것이고, 빌린 것은 파괴하지 않는다. 51장에서 세운 규칙이 문자열에 그대로 적용된 것이다. `u8`은 UTF-8을 뜻한다 — 7장에서 본 그 인코딩이 이 라이브러리의 기본 텍스트 표현이다.

소유하는 문자열을 얻는 길은 둘이다. 할당자에게 받거나(`_create`), 남의 버퍼를 빌려 쓰거나(`_borrow`).

```
examples/ch52/ops.c

#include <proven.h>
#include <stdio.h>

static void show(const char *label, proven_u8str_view_t v)
{
    printf("%-12s [%.*s] (%zu bytes)\n", label, (int)v.size, (const char *)v.ptr, v.size);
}

int main(void)
{
    /* ① 할당 없는 문자열: 스택 버퍼를 빌려 쓴다.
       allocator 가 없는 시그니처 = 할당하지 않는다 */
    proven_byte_t buf[16];
    proven_u8str_t s = proven_u8str_borrow(buf, sizeof buf);

    proven_err_t e = proven_u8str_append(&s, proven_u8str_view_from_cstr("hello"));
    printf("append 'hello' : %s\n", proven_is_ok(e) ? "ok" : "refused");
    show("content", proven_u8str_as_view(&s));

    /* ② 자리가 모자라면 거부한다 - 자르지 않는다 */
    e = proven_u8str_append(&s, proven_u8str_view_from_cstr(" world, and more"));
    printf("append long   : %s (err=%d)\n",
           proven_is_ok(e) ? "ok" : "refused", (int)e);
    show("unchanged", proven_u8str_as_view(&s)); /* 실패 원자성 */
}
```

```

/* ③ 찾기와 자르기 - 전부 view 위에서, 복사 없이 */
proven_u8str_view_t csv = proven_u8str_view_from_cstr("name,age,city");
proven_u8str_view_t comma = proven_u8str_view_from_cstr(",");

proven_size_t at = proven_u8str_view_find(csv, 0, comma);
printf("first comma at : %zu\n", at);
show("field 1", proven_u8str_view_slice(csv, 0, at));

/* ④ 구분자로 옮기: 못 찾으면 센티널(PROVEN_INDEX_NOT_FOUND)이 온다 */
proven_size_t start = 0;
int n = 0;
for (;;) {
    proven_size_t hit = proven_u8str_view_find(csv, start, comma);
    proven_size_t end = (hit == PROVEN_INDEX_NOT_FOUND) ? csv.size : hit;
    char label[24];
    snprintf(label, sizeof label, "field %d", ++n % 100);
    show(label, proven_u8str_view_slice(csv, start, end - start));
    if (hit == PROVEN_INDEX_NOT_FOUND) break;
    start = hit + 1;
}

/* ⑤ 비교와 접두사 검사 */
printf("eq 'name,age,city' : %d\n",
    proven_u8str_view_eq(csv, proven_u8str_view_from_cstr("name,age,city")));
printf("starts with 'name' : %d\n",
    proven_u8str_view_starts_with(csv, proven_u8str_view_from_cstr("name")));
return 0;
}

```

실행 결과

```

append 'hello' : ok
content      [hello] (5 bytes)
append long   : refused (err=2)
unchanged     [hello] (5 bytes)
first comma at : 4
field 1       [name] (4 bytes)
field 1       [name] (4 bytes)
field 2       [age] (3 bytes)
field 3       [city] (4 bytes)
eq 'name,age,city' : 1
starts with 'name' : 1

```

첫 줄이 51장의 규칙을 다시 보여 준다. proven_u8str_borrow에는 할당자가 없다 — 그러므로 할당하지 않는다. 스택에 잡은 16바이트 배열 위에서 문자열 연산을 하는 것이고, 임베디드에서 힙 없이 문자열을 다루는 방식이 정확히 이것이다.

52.2 거부하되, 자르지 않는다

두 번째 append가 이 부에서 가장 중요한 한 줄이다. 16바이트 그릇에 "hello" 다음으로 " world, and more"를 붙이려 하자 라이브러리는 아무것도 쓰지 않고 PROVEN_ERR_OUT_OF_BOUNDS를 돌려주었다. 그리고 그다음 줄이 보여 주듯 원래 내용 hello는 그대로다 — 49장에서 배운 실패 원자성이다.

47장에서 본 snprintf의 선택과 정반대다. 왜 자르지 않는가?

잘린 값은 짧은 값이 아니라 다른 값이기 때문이다. 잘린 경로는 다른 파일을 가리키고, 잘린 명령은 다른 명령이며, 잘린 사용자 이름은 다른 사람이다. 자르고 성공을 말하면 호출자는 자기가 받은 것이 요청

한 것인지 알 수 없다. 그래서 이 라이브러리는 **결정을 호출자에게 되돌려준다** — “자리가 모자랍니다. 어떻게 할까요?”

문 그래도 로그 한 줄처럼 잘려도 괜찮은 자리가 있지 않은가?

답 있다. 그래서 **명시적으로 부분만 쓰는** 함수가 따로 있다 — `proven_u8str_append_partial`은 들어간 바이트 수를 돌려준다. 이름이 길고 반환값이 다르다는 점이 핵심이다. 자르기는 여전히 가능하지만, 그것을 하려면 **그렇게 적어야** 한다. 기본값은 언제나 안전한 쪽이고 위험한 선택은 눈에 보이는 이름으로만 할 수 있다는 원칙(50장의 `_unchecked`와 같다)이 여기서도 지켜진다.

x 성장과 고정 용량을 헛갈리기

```
proven_u8str_t s = proven_u8str_borrow(buf, sizeof buf);
proven_err_t e = proven_u8str_append_grow(alloc, &s, view); /* 위험 */
```

`_append`는 용량 안에서만 쓰고, `_append_grow`는 모자라면 할당자에게 더 달라고 한다 — 그래서 후자에만 할당자 인자가 있다. 문제는 빌린 버퍼(`_borrow`) 위에서 성장을 요구하는 경우다. 스택 배열은 자랄 수 없으므로 이것은 설계 오류다. 시그니처를 읽는 습관이 여기서 그대로 방어가 된다 — **할당자가 있으면 자랄 수 있고, 없으면 자랄 수 없다.**

52.3 찾기와 자르기 — 복사 없는 텍스트 처리

예제의 ③·④가 뷰의 진짜 쓸모다. 부분 문자열을 얻는 데 복사가 필요 없다 — 원본을 가리키는 포인터와 길이를 새로 계산할 뿐이다. CSV 한 줄을 세 필드로 가르는 동안 할당은 한 번도 일어나지 않았다.

이 무늬는 파서에서 특히 강력하다. 22장에서 `sscanf`로 줄을 해석할 때는 결과를 담을 버퍼를 미리 준비해야 했지만, 뷰로 자르면 **원본 위에 표시만 남긴다**. 대신 지켜야 할 것이 하나 늘어난다 — 50장에서 본 대로 **원본이 살아 있는 동안만 유효하다**.

`find`가 못 찾았을 때 돌려주는 값에도 규약이 있다. 0이나 음수가 아니라 `PROVEN_INDEX_NOT_FOUND`라는 이름의 센티널이다. 47장에서 센티널 값의 위험을 이야기했는데, 여기서는 **이름이 붙어 있고 문서화되어 있다**는 점이 다르다 — 이름 없는 마법의 수와 이름 붙은 계약은 다른 물건이다.

△ “길이를 들고 다니면 글자 수를 아는 것이다”

아니다. 길이는 **바이트** 수다. 7장에서 배운 대로 UTF-8에서 한 글자는 1~4바이트이므로, 한글 “가”는 3바이트이고 이모지는 4바이트다. 예제의 출력이 굳이 “(5 bytes)”라고 밝히는 이유다. 그래서 “n번째 글자”를 다루는 일은 여전히 조심스럽고, **바이트 경계에서 아무 데나 자르면** 7장에서 본 깨진 글자가 나온다. 문자열 라이브러리가 해결해 주는 것은 경계 침범이지 인코딩의 본질적 난이도가 아니다.

52.4 두 세계의 경계 — NUL 종단과 UTF-16

바깥 세계와 만나는 자리마다 변환이 필요하다.

- `proven_u8str_as_cstr` — 소유 문자열에서 NUL 종단 포인터를 얻는다. 내부에 NUL을 유지해 두므로 복사도 할당도 없다. `printf("%s")`나 파일 API에 넘길 때 쓴다.
- `proven_u8str_view_to_cstr` — 뷰에서 NUL 종단 문자열을 만든다. 이쪽은 **할당자를 받는다** — 뷰는 원본 중간을 가리킬 수 있어 그 자리에 NUL을 쓸 수 없기 때문이다. 시그니처가 다시 한번 사실을 말해 준다.
- `proven_u16str_t` / `proven_u16str_view_t` — UTF-16 세계와의 다리다. Windows API가 이 인코딩을 쓰므로 변환이 필요하고, 변환은 실패할 수 있다(짜이 맞지 않는 서러게이트 같은 것). 그래서 결과는 꾸러미로 온다.

● “추측하지 말고 거부하라” — 인코딩 처리의 원칙

7장에서 본 텍스트 보안 문제들의 뿌리는 대개 **너그러운 디코더**다. 잘못된 UTF-8을 만났을 때 “비슷한 것으로 고쳐 읽는” 구현은 과거 여러 번 보안 사고의 통로가 됐다 — 특히 지나치게 긴 인코딩 (overlong)을 허용한 디코더들이 검사를 우회당했다. 그래서 오늘의 규범은 한 문장이다. **유효하지 않은 입력은 고쳐 읽지 말고 거부하라**. proven의 인코딩 변환이 `PROVEN_ERR_INVALID_ENCODING`을 돌려주며 멈추는 것이 그 규범의 구현이고, 이 원칙은 이 장 전체의 “자르지 않는다”와 정확히 같은 정신이다.

≡ 복습 정리

문자열 어휘 요약.

함수	하는 일	실패·소유
<code>u8str_create(alloc, cap)</code>	소유 문자열 생성	꾸러미 반환, <code>_destroy</code> 필요
<code>u8str_borrow(buf, cap)</code>	남의 버퍼 위에 문자열	할당 없음, 파괴 없음
<code>u8str_append(&s, v)</code>	용량 안에서 덧붙임	모자라면 거부(원본 보존)
<code>u8str_append_partial</code>	들어가는 만큼	넣은 바이트 수 반환
<code>u8str_append_grow(alloc,...)</code>	모자라면 늘려서	할당 실패 가능
<code>u8str_view_find</code>	부분 문자열 위치	없으면 <code>PROVEN_INDEX_NOT_FOUND</code>
<code>u8str_view_slice</code>	부분 뷰	복사 없음 — 원본 수명에 묶임
<code>u8str_as_cstr</code>	NUL 종단 포인터	복사 없음
<code>u8str_view_to_cstr</code>	뷰 → C 문자열	할당자 필요

문자열을 안전하게 담고 자를 수 있게 됐다. 그런데 아직 **만드는** 일이 남았다 — 수와 값을 글자로 바꾸고, 글자를 다시 수로 되돌리는 일. 47장의 셋째 버그가 기다리는 자리이자, 이 라이브러리에서 가장 눈에 띄게 다른 문법이 나오는 자리다.

53 형식화와 파싱 — 타입을 두 번 적지 않기

이 장이 끝나면

47장의 셋째 버그 — 서식과 인자의 불일치 — 에 대한 답이다. {}라는 타입 없는 자리표시자가 어떻게 타입 안전을 얻는지, 그 밑에서 _Generic이 무엇을 하는지, 그리고 반대 방향인 파싱에서 실패가 어떻게 값으로 드러나는지를 본다. 46장에서 뜯어본 printf·scanf의 대안이다.

돌아보기 45장에서 가변 인자에는 타입 정보가 실려 가지 않는다고 했고, 46장에서는 그래서 서식 문자열이 “스택을 어떻게 읽을지”를 혼자 정한다고 했다. 그러면 인자에서 타입을 가져오려면 무엇이 필요한가?

답 호출 자리에서 타입을 붙잡아 값과 함께 실어 보내면 된다. 즉 인자를 그대로 넘기지 않고 {타입 꼬리표, 값} 꾸러미로 감싸는 것이다. 남은 문제는 “타입 꼬리표를 어떻게 자동으로 붙이는가”인데, 그 답이 C11의 _Generic이다 — 표현식의 타입에 따라 컴파일 시간에 서로 다른 코드를 고르는 장치. 실행 시간 비용은 0이고, 25장에서 배운 암묵적 변환 규칙과 달리 여기서는 타입이 보존된다.

53.1 {} — 타입이 없는 자리표시자

규칙은 셋뿐이다.

- {} — 여기에 다음 인자를 넣는다. 타입은 적지 않는다.
- {...} — 콜론 뒤에 서식 지정(폭·정렬·자릿수)을 적는다.
- {{}} — 중괄호 글자 자체.

%d가 없으니 %d와 double이 어긋날 자리도 없다. 46장에서 본 불일치의 가능성이 문법 차원에서 제거된 것이다.

examples/ch53/fmt.c

```
#include <proven.h>
#include <stdio.h>

int main(void)
{
    /* 화면이 아니라 문자열로 형식화한다 - 버퍼는 스택에서 빌린다 */
    proven_byte_t buf[64];
    proven_u8str_t out = proven_u8str_borrow(buf, sizeof buf);

    int    port = 8080;
    double load = 0.4237;
    const char *host = "example.org";

    proven_fmt_result_t r = proven_u8str_append_fmt(&out, "{}: {} load={:.2}",
                                                    PROVEN_ARG(host), PROVEN_ARG(port),
                                                    PROVEN_ARG(load));

    if (proven_is_ok(r.err)) {
        proven_u8str_view_t v = proven_u8str_as_view(&out);
        printf("formatted : %.*s\n", (int)v.size, (const char *)v.ptr);
    }

    /* 그릇이 모자라면? 자르지 않고 거부한다 */
    proven_byte_t small_buf[8];
    proven_u8str_t small = proven_u8str_borrow(small_buf, sizeof small_buf);
    proven_fmt_result_t r2 = proven_u8str_append_fmt(&small, "{}: {}",
```

```

PROVEN_ARG(host), PROVEN_ARG(port));

printf("into 8 bytes: %s (err=%d)\n",
      proven_is_ok(r2.err) ? "ok" : "refused", (int)r2.err);

/* 정렬과 자릿수 - 46장의 폭/정밀도에 해당한다 */
proven_u8str_t line = proven_u8str_borrow(buf, sizeof buf);
(void)proven_u8str_reset(&line);
proven_fmt_result_t r3 = proven_u8str_append_fmt(&line, "{:>10}|{:<10}|{:.3}|",
      PROVEN_ARG(host), PROVEN_ARG(host),
      PROVEN_ARG(load));

if (proven_is_ok(r3.err)) {
    proven_u8str_view_t v = proven_u8str_as_view(&line);
    printf("aligned : %.*s\n", (int)v.size, (const char *)v.ptr);
}
return 0;
}

```

실행 결과

```

formatted : example.org:8080 load=0.42
into 8 bytes: refused (err=2)
aligned   : |example.org|example.org|0.424|

```

이 예제는 화면이 아니라 **문자열로** 형식화한다 — 48장에서 본 `proven_printf`가 이 기계를 표준 출력에 연결한 판이다. 세 가지를 짚을 수 있다.

첫째, 서식화도 실패할 수 있다. 8바이트 그릇에 `example.org:8080`을 넣을 수 없으므로 거부됐다. 52장의 원칙이 여기서도 그대로다 — 자르느니 실패를 돌려준다. `snprintf`와 정반대의 기본값이다.

둘째, 서식 지정 문법이 조금 다르다. `{:>10}`의 `>`는 오른쪽 정렬, `{:<10}`은 왼쪽 정렬, `{:.3}`은 소수 셋째 자리다. 46장의 `%10s%-10s%.3f`와 대응하지만, **타입 글자가 없다는 점**이 다르다.

셋째, 반올림은 하되 자릿수는 지킨다. `load=0.42`는 `{:.2}`가 만든 것이다.

문 PROVEN_ARG가 정확히 무엇을 하는가?

답 `_Generic`으로 인자의 타입을 골라, 그 타입에 맞는 꼬리표를 붙인 작은 구조체를 만든다. 개념만 옮기면 이런 모양이다.

```

#define PROVEN_ARG(x) _Generic((x),
    int:      proven_arg_i32,
    double:   proven_arg_f64,
    const char *: proven_arg_cstr,
    bool:     proven_arg_bool
    /* ... */ )(x)

```

`_Generic`은 **컴파일 시간에** 가지를 고르므로 실행 시간에 타입을 판별하는 비용이 없다. 그리고 목록에 없는 타입을 넘기면 **컴파일 오류**가 난다 — 46장의 `printf`가 무엇이든 받아 주던 것과 정반대다.

✗ PROVEN_ARG 없이 값을 그냥 넘기기

```

proven_printf("count={}", count); /* 컴파일되지 않는다 */

```

꾸러미가 아닌 날값을 넘겼으므로 타입이 맞지 않아 빌드가 실패한다. 성가시게 느껴질 수 있지만 이것이 설계의 요점이다 — **감싸는 일을 잊으면 프로그램이 만들어지지 않는다.** 잊어도 컴파일되고 실행 중에 이상해지는 `printf`와의 차이가 여기에 있다.

△ “자리표시자에 타입이 없으니 느낄 것이다”

반대다. `printf`는 실행 중에 서식 문자열을 한 글자씩 해석하며 다음에 무엇을 꺼낼지 결정한다. `%d`도 서식을 훑는 것은 같지만, 각 인자의 타입은 이미 꼬리표로 정해져 있어 추측이 없다. 무엇보다 타입 불일치로 인한 UB가 없으므로 방어 코드도 필요 없다. 비용의 차이는 대개 무시할 만하고, 이 라이브러리의 실수 형식화는 정확성 쪽에 더 신경을 쓴 편이다 — 46장에서 본 `%f`의 반올림 규칙을 그대로 재현하는 것이 오히려 까다로운 일이다.

53.2 반대 방향 — 스캐너

파싱은 형식화의 거울이지만, 실패의 성격이 다르다. 형식화는 그릇이 모자랄 때만 실패하는데, 파싱은 입력이 기대와 다를 때마다 실패한다. 그리고 46장에서 본 대로 `sscanf`는 “몇 개나 성공했는가”만 알려 준다 — 어디서 왜 멈췄는지는 말해 주지 않는다.

`proven`의 스캐너는 커서를 가진 객체다. 뷰 위에 놓고, 하나씩 읽어 나간다. 읽을 때마다 결과가 꾸러미로 온다.

```
examples/ch53/lines.c
#include <proven.h>
#include <stdio.h>

/* 한 줄에서 "이름 나이" 꼴을 해석한다 - 실패는 값으로 드러난다. */
static void parse_line(const char *line)
{
    proven_scan_t sc = proven_scan_init(proven_u8str_view_from_cstr(line));

    proven_result_u8str_view_t name = proven_scan_str(&sc);
    if (!proven_is_ok(name.err)) {
        printf("[%s] -> no name found\n", line);
        return;
    }

    proven_result_i64_t age = proven_scan_i64(&sc);
    if (!proven_is_ok(age.err)) {
        printf("[%s] -> age is not a number\n", line);
        return;
    }

    printf("[%s] -> name %.s, age %lld\n", line,
           (int)name.val.size, (const char *)name.val.ptr, (long long)age.val);
}

int main(void)
{
    parse_line("alice 33");
    parse_line("bob thirty");
    parse_line(" ");
    return 0;
}
```

실행 결과

```
[alice 33] -> name alice, age 33
[bob thirty] -> age is not a number
[ ] -> no name found
```


세 입력이 세 갈래로 갈린 것이 핵심이다. bob thirty는 이름은 읽혔지만 나이에서 멈췄고, 공백뿐인 줄은 이름부터 실패했다. sscanf라면 둘 다 “항목 하나 성공” 또는 “0”으로 뭉뚱그려졌을 것이다.

커서를 가진다는 것에는 또 다른 이점이 있다. **어디까지 읽었는지**를 알 수 있어서, 남은 부분을 다른 방식으로 처리하거나 오류 메시지에 위치를 실을 수 있다. 22장에서 sscanf로 줄을 해석할 때 “몇 글자를 소비했는지 알 수 없다”고 했던 문제의 답이다.

문 스캐너에도 서식 문자열이 있는가?

답 있다 — proven_scan_fmt_cursor(&sc, "{}:{}", PROVEN_SCAN_ARG(&host),

PROVEN_SCAN_ARG(&port))처럼 쓴다. 형식화와 대칭이고, 인자는 값을 곳의 주소를 감싼 것이다. 다만 헤더가 정직하게 밝혀 두는 주의사항이 하나 있다 — 서식 중간에서 실패하면 **그 전까지 채워진 값은 이미 바뀌어 있다**는 것이다. 49장에서 배운 실패 원자성이 여기서는 보장되지 않으므로, 꼭 필요하다면 커서와 목적지를 미리 저장해 두었다가 되돌려야 한다. 계약을 숨기지 않고 문서에 적어 두는 것이 이 라이브러리의 방식이다.

● 파서가 관대하면 무슨 일이 생기는가

HTTP 요청 처리에서 반복적으로 드러난 문제가 있다. 서버와 프록시가 같은 요청을 **조금씩 다르게** 해석하면, 공격자가 그 틈으로 두 번째 요청을 몰래 끼워 넣을 수 있다(request smuggling). 한쪽은 헤더의 이상한 공백을 너그럽게 넘기고 다른 쪽은 엄격히 거부하는 식의 차이가 원인이었다. 교훈은 52장의 인코딩 이야기와 정확히 같다 — **애매한 입력은 고쳐 읽지 말고 거부하라**. 실패를 값으로 돌려주는 파서는 그 거부를 표현할 수단을 갖춘 파서이기도 하다.

≡ 복습 정리

형식화·파싱 요약.

하는 일	API	메모
화면에 한 줄	proven_println(fmt, ARG...)	에러 반환하되 강요 안 함
문자열로 형식화	proven_u8str_append_fmt(&s, ...)	모자라면 거부
자를 것을 허용	..._append_fmt_trunc	이름으로 의도를 밝힘
늘려 가며	..._append_fmt_grow(alloc, ...)	할당자 필요
인자 감싸기	PROVEN_ARG(x)	_Generic — 없는 타입은 컴파일 오류
스캐너 시작	proven_scan_init(view)	커서를 가진 객체
하나씩 읽기	proven_scan_i64/f64/str	꾸러미로 결과와 실패
서식으로 읽기	proven_scan_fmt_cursor(...)	중간 실패 시 부분 변경 주의

문자열을 담고, 만들고, 되읽는 어휘가 갖춰졌다. 다음은 **여럿을 담는** 도구 들이다 — 자라는 배열, 리스트, 링 버퍼, 해시 맵, 그리고 47장에서 예고한 “최악에도 보장이 있는” 알고리즘들.

54 컨테이너와 알고리즘

이 장이 끝나면

여럿을 담는 도구들이다 — 자라는 배열, 침습적 리스트, 링 버퍼, 해시 맵. 그리고 47장의 다섯째 버그(검사되지 않는 콜백)와 그 자리에 딸린 성능 함정에 대한 답으로, **최악에도 보장이 있는** 정렬과 공격을 견디는 해시를 본다. 32장의 배열과 38장의 구조체가 여기서 실전 부품이 된다.

돌아보기 32장에서 C의 배열은 크기가 컴파일 시간에 정해지고, 37장에서 `realloc` 으로 늘릴 수 있다고 배웠다. 그러면 “자라는 배열”을 직접 만들 때 가장 틀리기 쉬운 자리는 어디인가?

답 세 곳이다. 첫째, **늘릴 크기 계산** — 50장에서 본 곱셈 감김이 여기서 일어난다. 둘째, **실패했을 때의 상태** — `realloc`이 실패하면 원본 포인터는 그대로 유효한데, 반환값을 곧바로 원본 변수에 대입하는 흔한 코드는 그 원본을 잃어버린다(누수다). 셋째, **늘어난 뒤의 포인터** — 원소를 가리키던 포인터는 재할당 후 무효가 된다. 컨테이너를 직접 만들 때마다 이 셋을 다시 맞히는 것보다, 한 번 제대로 만든 것을 쓰는 편이 낫다.

54.1 자라는 배열

`proven_array_t`는 원소 크기와 정렬을 알고 있는 바이트 버퍼다. C에는 제네릭이 없으므로 타입은 매크로가 채워 준다.

```
examples/ch54/arr.c

#include <proven.h>
#include <stdio.h>

int main(void)
{
    proven_allocator_t alloc = proven_heap_allocator();

    proven_result_array_t made = PROVEN_ARRAY_INIT(alloc, int, 4);
    if (!proven_is_ok(made.err)) {
        printf("array creation failed\n");
        return 1;
    }
    proven_array_t arr = made.value;

    for (int i = 1; i <= 6; i += 1) {          /* 용량 4를 넘겨 자동으로 자란다 */
        if (!proven_is_ok(PROVEN_ARRAY_PUSH(&arr, int, i * i))) {
            printf("push failed\n");
            PROVEN_ARRAY_DESTROY(&arr);
            return 1;
        }
    }

    printf("count: %zu\n", arr.len);
    for (size_t i = 0; i < arr.len; i += 1) {
        printf("%d ", *PROVEN_ARRAY_GET(&arr, int, i));
    }
    printf("\n");

    PROVEN_ARRAY_DESTROY(&arr);
    return 0;
}
```

실행 결과

```
count: 6
1 4 9 16 25 36
```

PROVEN_ARRAY_INIT(alloc, int, 4)는 “int를 담을 배열, 초기 용량 4”라는 뜻이고, PROVEN_ARRAY_PUSH는 타입을 다시 적어 컴파일 시간에 맞는지 확인한다. 용량 4를 넘긴 다섯째 원소에서 배열은 스스로 늘어났다 — 그리고 그 성장은 할당자를 통해 일어난다(51장의 규칙 그대로, 배열이 자기가 만들어질 때 받은 할당자를 기억한다).

✗ 원소 포인터를 들고 있다가 push 하기

```
int *first = PROVEN_ARRAY_GET_MUT(&arr, int, 0);
(void)PROVEN_ARRAY_PUSH(&arr, int, 42); /* 여기서 버퍼가 옮겨갈 수 있다 */
*first = 7;                             /* 옛 주소에 쓴다 - 사용 후 해제 */
```

배열이 자라면 내용이 새 버퍼로 옮겨가고, 옛 주소를 가리키던 포인터는 무효가 된다. 37장에서 배운 사용 후 해제가 컨테이너 위에서 나타나는 형태다. 규칙은 하나다 — 컨테이너를 바꾼 뒤에는 인덱스로 다시 얻는다. 포인터가 아니라 인덱스를 들고 다니는 습관이 여기서 방어가 된다.

54.2 침습적 리스트와 링 버퍼

proven_list_t는 침습적(intrusive) 연결 리스트다 — 노드가 자료를 품는 것이 아니라, 자료 구조체 안에 링크 필드를 심는다. 38장에서 만든 struct 안에 proven_list_node_t link;를 한 칸 두는 식이다.

무엇이 좋은가. 리스트에 넣기 위해 따로 할당할 것이 없다 — 노드가 곧 자료이기 때문이다. 힙이 없는 환경에서도 리스트를 쓸 수 있고(56장), 같은 객체를 두 리스트에 동시에 매다는 것도 링크 필드를 둘 두면 된다. 대가는 자료 구조체가 리스트를 알아야 한다는 것이다.

proven_ring_t는 고정 크기 원형 버퍼다. 생산자와 소비자가 오가는 자리, 로그의 최근 N개, 오디오·센서 샘플처럼 지나간 것은 버려도 되는 흐름에 쓴다. 크기가 고정이므로 가득 차면 어떻게 할지가 계약의 일부다 — 여기서도 기본은 조용히 덮지 않고 알리는 쪽이다.

54.3 해시 맵, 그리고 공격받는 자료구조

proven_map_t는 열린 주소 방식(open addressing) 해시 맵이다. 키는 정수나 바이트열이고, 문자열 키는 두 가지 모드가 있다 — 빌린 키(호출자가 바이트를 살려 둔다)와 소유한 키(맵이 복사해 들고 있다). 51장의 소유·빌림 구분이 여기서도 그대로 나타난다.

examples/ch54/wordcount.c

```
#include <proven.h>
#include <stdio.h>

/* 낱말을 세어 정렬해 찍는다 - 맵과 배열과 정렬을 한 번에 */
typedef struct { proven_u8str_view_t word; int count; } entry_t;

/* 비교자는 전순서여야 한다: 동점도 일관되게 갈라 준다 (47장의 반례를 피한다) */
static int by_count_desc(const void *a, const void *b)
{
    const entry_t *x = a, *y = b;
    if (x->count != y->count) return (x->count < y->count) - (x->count > y->count);

    proven_size_t n = x->word.size < y->word.size ? x->word.size : y->word.size;
    int c = proven_memcmp(x->word.ptr, y->word.ptr, n);
    if (c != 0) return c;
    return (x->word.size > y->word.size) - (x->word.size < y->word.size);
}
```

```

}

int main(void)
{
    proven_allocator_t alloc = proven_heap_allocator();
    const char *text = "the quick fox the lazy dog the fox";

    /* 문자열 키 맵: 기본은 HashDoS 에 건디는 키드 해시를 쓴다 */
    proven_result_map_t made_map =
        proven_map_create(alloc, 16, PROVEN_KEY_TYPE_U8_OWNED, sizeof(int), alignof(int));
    if (!proven_is_ok(made_map.err)) return 1;
    proven_map_t counts = made_map.value;

    proven_u8str_view_t all = proven_u8str_view_from_cstr(text);
    proven_u8str_view_t space = proven_u8str_view_from_cstr(" ");
    proven_size_t start = 0;

    for (;;) {
        proven_size_t hit = proven_u8str_view_find(all, start, space);
        proven_size_t end = (hit == PROVEN_INDEX_NOT_FOUND) ? all.size : hit;
        proven_u8str_view_t w = proven_u8str_view_slice(all, start, end - start);

        const int *seen = proven_map_get(&counts, (proven_map_key_t){ .str = w });
        int next = seen ? *seen + 1 : 1;
        if (!proven_is_ok(proven_map_set(&counts, (proven_map_key_t){ .str = w }, &next)))
            break;

        if (hit == PROVEN_INDEX_NOT_FOUND) break;
        start = hit + 1;
    }
    printf("distinct words: %zu\n", counts.len);

    /* 세어 둔 것을 배열에 모아 정렬한다 */
    proven_result_array_t made_arr = PROVEN_ARRAY_INIT(alloc, entry_t, 8);
    if (!proven_is_ok(made_arr.err)) return 1;
    proven_array_t list = made_arr.value;

    const char *words[] = {"the", "quick", "fox", "lazy", "dog"};
    for (size_t i = 0; i < sizeof words / sizeof words[0]; i++) {
        proven_u8str_view_t w = proven_u8str_view_from_cstr(words[i]);
        const int *c = proven_map_get(&counts, (proven_map_key_t){ .str = w });
        entry_t e = { .word = w, .count = c ? *c : 0 };
        if (!proven_is_ok(PROVEN_ARRAY_PUSH(&list, entry_t, e))) break;
    }

    proven_array_sort(&list, by_count_desc); /* 최악에도 O(n log n) 보장 */
    for (size_t i = 0; i < list.len; i++) {
        const entry_t *e = PROVEN_ARRAY_GET(&list, entry_t, i);
        printf(" %.s = %d\n", (int)e->word.size, (const char *)e->word.ptr, e->count);
    }

    PROVEN_ARRAY_DESTROY(&list);
    proven_map_destroy(&counts);
    return 0;
}

```

실행 결과

```

distinct words: 5
the = 3

```

```
fox = 2
dog = 1
lazy = 1
quick = 1
```

이 예제 하나에 이 장의 도구가 다 들어 있다 — 맵으로 세고, 배열에 모으고, 정렬해서 찍는다. 낱말을 자르는 데는 52장의 뷰를 썼으므로 문자열 복사는 맵이 키를 소유할 때 한 번뿐이다.

● HashDoS — 해시 맵이 공격 대상이 된 사건

2011년, 여러 웹 프레임워크가 동시에 같은 취약점으로 무너졌다. 공격자가 **같은 버킷으로 몰리는 키**들을 골라 요청 하나에 수천 개의 매개변수로 실어 보내면, 평균 $O(1)$ 이던 삽입이 $O(n)$ 이 되고 전체가 $O(n^2)$ 으로 퇴화해 서버 한 대가 요청 몇 개로 멎었다. 해시 함수가 공개되어 있어 충돌을 **계산할 수 있었다**는 것이 원인이었다.

오늘의 처방은 **키드 해시(keyed hash)**다 — 프로세스마다 무작위 비밀을 뽑아 해시에 섞으면 공격자가 충돌을 미리 계산할 수 없다. proven의 proven_map_create가 문자열 키에 SipHash-2-4와 무작위 씨앗을 기본으로 쓰는 이유이고, 반대로 키가 전부 내 코드에서 오는 경우를 위해 더 빠른 FNV-1a를 쓰는 proven_map_create_trusted를 따로 둔 이유다. **기본값은 안전한 쪽, 빠른 쪽은 이름을 밝혀서** — 이 부에서 반복해 만난 원칙이다.

문 키드 해시를 쓰면 얼마나 느려지는가? 그리고 OS가 없으면 무작위 비밀은 어디서 오는가?

답 SipHash는 FNV-1a보다 느리지만 문자열 길이에 비례하는 정도이고, 맵 연산 전체에서 해시 계산이 차지하는 몫은 대개 크지 않다. 무작위 비밀은 운영체제의 난수원에서 한 번 뽑는다 — 그리고 OS가 없는 환경(56장)에서는 뽑을 곳이 없으므로 FNV-1a로 물러난다. 라이브러리는 이 사실을 숨기지 않고 문서에 적어 두는데, 근거가 분명하다: **공격자가 없는 곳에는 공격자 모델도 필요 없다**. 펌웨어 안에서 키를 고르는 외부인은 존재하지 않는다.

54.4 최악을 보장하는 정렬

47장에서 본 두 가지 문제 — 검사되지 않는 비교자, 그리고 최악에서 무너지는 알고리즘 — 을 여기서 함께 다룬다.

proven_array_sort는 **introsort**다. 빠른 퀵소트로 시작하되, 재귀가 너무 깊어지면 힙소트로 갈아탄다. 그래서 평균은 퀵소트만큼 빠르고 최악에도 $O(n \log n)$ 이 **보장**된다 — 47장에서 본 복잡도 공격이 통하지 않는다. 헤더의 문구를 그대로 옮기면 “ $O(n \log n)$ 은 평균이 아니라 보장”이다.

비교자 쪽은 언어가 도와줄 수 없으므로 계약을 문서와 예제로 밝힌다. 예제의 by_count_desc가 그 본보기다 — 개수로 내림차순, **동점이면 낱말로** 가른다. 47장의 반례(첫 글자만 보는 비교자)와 정확히 대비된다.

△ “동점은 어떻게 처리해도 상관없다”

상관있다. 비교자는 **전순서(total order)**를 이뤄야 한다 — 같으면 0, 일관되게 크고 작아야 하며, $\text{cmp}(a,b)$ 와 $\text{cmp}(b,a)$ 의 부호가 반대여야 한다. 이것을 어기면 결과가 뒤죽박죽이 되는 정도가 아니라, 구현에 따라 **배열 밖을 침범**할 수도 있다(파티션 알고리즘이 경계를 비교 결과로 판단하기 때문이다). “동점을 아무렇게나” 돌려주는 비교자는 그래서 버그이지 취향이 아니다.

54.5 바이트를 글자로 — 해시와 인코딩

같은 상자 안에 든 나머지 도구들도 짚어 둔다.

- **용도별 해시** — 맵의 내부용(빠른 혼합), 무결성 확인용(CRC-32), 암호학적 용도(SHA-256), 그리고 앞서 본 키드 해시(SipHash). 라이브러리가 이들을 구분해 두는 이유는 같은 “해시”라는 낱말이 전혀 다른 요구를 뜻하기 때문이다 — 빠르기만 하면 되는 자리에 SHA-256을 쓰면 낭비이고, 적대적 입력이 오는 자리에 FNV-1a를 쓰면 위험하다.
- **hex와 Base64** — 바이트를 텍스트로 옮기는 두 표준. 여기서도 원칙은 같다. 잘못된 입력(홀수 길이 hex, 잘못된 패딩)은 추측해 고치지 않고 `PROVEN_ERR_INVALID_ENCODING`으로 거부한다(52장의 그 규범이다).

≡ 복습 정리

컨테이너 요약.

도구	모양	맞는 자리	주의
<code>array</code>	자라는 연속 배열	순서 있는 목록	자라면 포인터 무효
<code>list</code>	침습적 연결 리스트	할당 없이 잇기	자료가 링크를 품는다
<code>ring</code>	고정 크기 원형	흐름·최근 N개	가득 찼을 때의 계약
<code>map</code>	열린 주소 해시	키로 찾기	키 소유 여부를 고른다
<code>array_sort</code>	<code>introsort</code>	무엇이든 정렬	비교자는 전순서
해시 4종	용도별	맵·무결성·암호·반공격	용도를 섞지 않는다

여기까지가 순수한 계산의 세계다 — 운영체제가 없어도 전부 돈다. 다음 장에서는 바깥으로 나간다. 파일, 스트림, 시간, 난수 — OS와 닿는 자리다.

55 바깥 세계 — 파일, 스트림, 시간, 난수

이 장이 끝나면

여기서부터는 운영체제와 닿는다. 파일을 열고 읽고 쓰는 일, 버퍼드 스트림, 시간을 읽고 형식화하는 일, 그리고 난수 — 용도에 따라 완전히 다른 물건이 되는 그 난수까지. 8장의 스트림 이야기와 22장의 입력 이야기가 여기서 실제 API로 완결된다.

돌아보기 8장에서 스트림이 “무엇에 연결됐는지 프로그램은 모른다”는 설계였고, 46장에서는 `fopen`이 실패를 널로 알린다고 했다. 그러면 파일 API에서 가장 자주 놓치는 실패는 무엇인가?

답 **부분 쓰기**다. 열기 실패는 눈에 잘 띄지만, `write`가 요청한 만큼 다 쓰지 않고 돌아오는 경우는 잊기 쉽다 — 디스크가 차거나, 시그널이 끼어들거나, 상대가 파이프인 경우에 실제로 일어난다. 그래서 이 라이브러리에는 두 판이 있다. `proven_fs_write`는 **실제로 쓴 바이트 수**를 돌려주고, `proven_fs_write_all`은 다 쓸 때까지 되풀이한 뒤 성공·실패만 알려 준다. 대부분의 코드가 원하는 것은 후자이고, 전자는 그 사실을 감추지 않기 위해 남아 있다.

55.1 파일 — 열고, 읽고, 닫기

examples/ch55/fileio.c

```
#include <proven.h>
#include <stdio.h>

int main(void)
{
    proven_allocator_t alloc = proven_heap_allocator();
    proven_u8str_view_t path = proven_u8str_view_from_cstr("proven_demo.txt");
    const char *text = "one\ntwo\nthree\n";

    /* 쓰기: 없으면 만들고, 있으면 비운다 */
    proven_result_file_t opened = proven_fs_open(
        alloc, path, PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
    if (!proven_is_ok(opened.err)) {
        printf("open for write failed (err=%d)\n", (int)opened.err);
        return 1;
    }
    proven_file_t f = opened.value;

    proven_mem_view_t src = {
        .ptr = (const proven_byte_t *)text,
        .size = proven_cstr_len(text)
    };
    proven_err_t e = proven_fs_write_all(f, src); /* 부분 쓰기를 되풀이해 준다 */
    printf("write_all : %s (%zu bytes)\n", proven_is_ok(e) ? "ok" : "failed", src.size);
    (void)proven_fs_close(f);

    /* 읽기 */
    opened = proven_fs_open(alloc, path, PROVEN_FS_READ);
    if (!proven_is_ok(opened.err)) return 1;
    f = opened.value;

    proven_result_size_t sz = proven_fs_size(f);
    printf("size      : %zu bytes\n", sz.value);

    proven_byte_t buf[64];
```

```

proven_result_size_t got = proven_fs_read(f, (proven_mem_mut_t){ .ptr = buf, .size = sizeof buf });
printf("read      : %zu bytes\n", got.value);

/* 읽은 것은 뷰로 다룬다 - 줄 단위로 자른다 */
proven_u8str_view_t all = { .ptr = buf, .size = got.value };
proven_u8str_view_t nl = proven_u8str_view_from_cstr("\n");
proven_size_t start = 0;
int line = 0;
while (start < all.size) {
    proven_size_t hit = proven_u8str_view_find(all, start, nl);
    proven_size_t end = (hit == PROVEN_INDEX_NOT_FOUND) ? all.size : hit;
    proven_u8str_view_t v = proven_u8str_view_slice(all, start, end - start);
    printf("  line %d   : %.*s\n", ++line, (int)v.size, (const char *)v.ptr);
    if (hit == PROVEN_INDEX_NOT_FOUND) break;
    start = hit + 1;
}
(void)proven_fs_close(f);

/* 없는 파일을 열면 실패가 값으로 온다 */
proven_result_file_t missing = proven_fs_open(
    alloc, proven_u8str_view_from_cstr("no_such_file.txt"), PROVEN_FS_READ);
printf("missing    : %s\n", proven_is_ok(missing.err) ? "opened" : "refused with an error code");

(void)proven_fs_remove(alloc, path);
return 0;
}

```

실행 결과

```

write_all : ok (14 bytes)
size      : 14 bytes
read      : 14 bytes
  line 1  : one
  line 2  : two
  line 3  : three
missing   : refused with an error code

```

몇 가지가 눈에 띈다.

경로도 뷰다. `proven_u8str_view_from_cstr(...)` — 문자열이 어디서 왔든 포인터와 길이로 다룬다(52장).

열기에 할당자가 필요하다. 시그니처의 첫 인자가 `scratch` 할당자인데, 경로를 운영체제가 요구하는 형태로 바꾸는 데 임시 기억이 필요할 수 있기 때문이다. 51장의 규칙이 여기서도 정직하게 지켜진다 — 할당할 수 있으면 할당자를 받는다.

읽은 것은 뷰가 된다. 버퍼와 읽은 바이트 수를 묶으면 그다음부터는 52장의 도구가 전부 쓸 수 있다. 예제가 줄을 가르는데 쓴 것이 그 방식이고, 복사는 한 번도 일어나지 않았다.

✗ size를 믿고 그만큼 읽었다고 가정하기

```

proven_result_size_t sz = proven_fs_size(f);
proven_byte_t *buf = malloc(sz.value);
(void)proven_fs_read(f, (proven_mem_mut_t){ buf, sz.value });
process(buf, sz.value);      /* 실제로 그만큼 읽혔는가? */

```

파일 크기와 이번 읽기가 가져온 양은 다르다. 파이프·터미널·네트워크 에서는 한 번에 조금씩 오고, 파일이라도 중간에 끝날 수 있다. `read`가 돌려준 수를 써야 하고, 그것이 이 라이브러리가 읽기 결과를

꾸러미로 돌려주는 이유다. 22장에서 “입력은 키보드가 아니라 스트림”이라 했던 그 사실이 여기서 실무 규칙이 된다.

55.2 스트림 — 버퍼를 둔 읽고 쓰기

시스템 호출은 비싸다. 한 바이트씩 `write`를 부르면 그 비용이 그대로 누적된다. 그래서 표준 C의 `FILE*`이 버퍼를 두었고(8장의 행 버퍼링 이야기), `proven`도 같은 자리에 **스트림**을 둔다 — 다만 두 가지가 다르다.

- 버퍼를 호출자가 준다. 스트림이 몰래 할당하지 않는다.
- 씻어내기(`flush`)의 실패가 값으로 온다. 닫을 때 조용히 삼키지 않는다.

이 둘은 같은 문제를 향한다. 버퍼가 있는 쓰기에서 진짜 실패는 `write`가 아니라 `flush`에서 드러나는데, 그 실패를 놓치면 “성공했다고 믿었지만 디스크에 없는” 자료가 생긴다. 데이터베이스와 파일 시스템이 가장 조심하는 자리이기도 하다(`proven_fs_sync`가 따로 있는 이유다).

55.3 시간 — 두 가지 다른 시계

시간에는 서로 다른 두 물건이 섞여 있다.

- **달력 시간** — “2026년 8월 5일 09시”처럼 사람이 읽는 시각. 사용자에게 보여 주고 기록에 남기는 데 쓴다. 시간대와 윤초와 서머타임이 얽힌다.
- **단조 시간(monotonic)** — 뒤로 가지 않는 눈금. **경과 시간**을 재는 데 쓴다.

둘을 섞으면 유명한 버그가 된다. 달력 시계로 경과 시간을 재면, 시스템이 시간을 맞추거나 서머타임이 바뀌는 순간 **음수 경과**가 나온다. 타임아웃 계산에 그 값이 들어가면 영원히 기다리거나 즉시 만료된다.

날짜 형식화는 46장에서 본 서식 문법을 그대로 쓴다 — “{year}-{month:0>2}-{day:0>2}”처럼 이름 있는 자리 표시자에 폭과 채움을 지정한다. `strftime`의 `%Y-%m-%d`와 달리 **무엇을 뜻하는 기호인지 외울 필요가 없다는 것**이 차이다.

△ “시간은 그냥 숫자니까 더하고 빼면 된다”

달력 시간에서는 아니다. 하루는 항상 86400초가 아니고(서머타임 전환일은 23시간 또는 25시간이다), 한 달의 길이는 제각각이며, 시간대는 정치적 결정으로 바뀐다. “한 달 뒤”는 산술이 아니라 달력 규칙이다. 반면 **단조 시간**의 차는 그냥 숫자로 다뤄도 된다 — 그것이 경과 시간 측정에 단조 시계를 쓰는 또 하나의 이유다.

55.4 난수 — 용도가 물건을 정한다

난수만큼 “같은 이름, 다른 요구”인 도구도 드물다. 라이브러리는 이것을 숨기지 않고 셋으로 갈라 둔다.

```
examples/ch55/rng.c
#include <proven.h>
#include <stdio.h>

int main(void)
{
    /* 재현 가능한 난수: 같은 씨앗 = 같은 수열. 시험과 시뮬레이션용이다 */
    proven_xoshiro256ss_t g;
    proven_xoshiro256ss_seed(&g, 12345);
    proven_rng_t rng = proven_xoshiro256ss_rng(&g);

    printf("seeded run 1:");
    for (int i = 0; i < 5; i++) printf(" %llu", (unsigned long long)proven_rng_below(rng, 100));
```

```

printf("\n");

proven_xoshiro256ss_seed(&g, 12345);          /* 같은 씨앗으로 되감는다 */
printf("seeded run 2:");
for (int i = 0; i < 5; i++) printf(" %llu", (unsigned long long)proven_rng_below(rng, 100));
printf("\n");

/* 범위 난수: 경계를 포함한다 */
proven_xoshiro256ss_seed(&g, 7);
printf("dice      :");
for (int i = 0; i < 8; i++) printf(" %lld", (long long)proven_rng_range(rng, 1, 6));
printf("\n");

/* 비밀에 쓸 난수는 여기서 얻지 않는다 - OS 난수원이 따로 있다 */
proven_byte_t key[16];
bool ok = proven_random_bytes(key, sizeof key);
printf("os entropy  : %s (%zu bytes requested)\n", ok ? "available" : "unavailable", sizeof key);
return 0;
}

```

실행 결과

```

seeded run 1: 74 13 96 4 55
seeded run 2: 74 13 96 4 55
dice       : 5 2 6 6 6 6 1 1
os entropy  : available (16 bytes requested)

```

재현 가능한 난수(`proven_xoshiro256ss_t`)는 시뮬레이션·게임·시험용이다. 씨앗이 같으면 수열이 같다 — 예제의 두 줄이 정확히 같은 이유이고, 실패한 시험을 다시 재현할 수 있게 해 주는 성질이기도 하다. 빠르지만 **예측 가능**하므로 비밀에는 절대 쓰지 않는다.

비밀용 난수(`proven_random_bytes`)는 운영체제의 암호학적 난수원에서 온다. 키, 토큰, 세션 식별자처럼 공격자가 맞으면 안 되는 값에 쓴다.

세 번째는 그 둘의 절충인 `proven_chacha_rng_t`로, OS 난수원에서 한 번 씨앗을 받아 암호학적으로 안전한 수열을 빠르게 이어 간다.

● 예측 가능한 난수가 만든 사고들

이 구분을 어긴 사고는 반복해서 일어났다. 온라인 카드 게임이 시각을 씨앗으로 쓴 탓에 패가 예측된 사례, 세션 식별자를 빠른 난수로 만들어 남의 계정에 들어갈 수 있었던 사례, 그리고 2008년 데비안의 OpenSSL 패치가 엔트로피 수집 코드를 지워 **생성 가능한 키가 몇만 개로 줄어든** 사건이 대표적이다. 마지막 것은 이미 만들어진 키를 전부 폐기해야 했다. 교훈은 라이브러리 문서의 한 줄로 요약된다 — **비밀에 쓸 난수는 오직 암호학적 난수원에서만 온다.**

문 운영체제가 없으면 비밀용 난수는 어떻게 되는가?

답 얻을 수 없으므로 `proven_random_bytes`는 **거짓을 돌려준다** — 그리고 이것이 중요한 설계 결정이다. 많은 라이브러리가 이럴 때 시각이나 주소 값으로 슬쩍 물러나는데, 그러면 “안전한 줄 알았는데 예측 가능” 최악의 상태가 된다. 이 라이브러리는 물러나지 않고 실패를 말한다. 보드에 진짜 엔트로피원(하드웨어 난수 발생기)이 있다면 그것을 등록해 쓰면 된다. **조용히 나쁜 것을 주느니 없다고 말한다** — 이 부에서 계속 만나 온 원칙의 또 다른 얼굴이다.

55.5 메모리 매핑

파일을 읽는 또 하나의 방식이 있다. 파일을 통째로 주소 공간에 걸어 두고 포인터로 접근하는 것 — `proven_mmap_*`이 그 창구다. 큰 파일을 무작위로 넘나들며 읽을 때 유리하고, 여러 프로세스가 같은 파일을 공유할 때도 쓴다.

대가도 분명하다. 매핑된 영역을 만지다 파일이 잘리면 프로그램이 신호를 받고 죽을 수 있고, 이식성도 파일 API보다 낮다. 그래서 이 도구는 “기본으로 쓰는 것”이 아니라 “이유가 있을 때 고르는 것”이다.

≡ 복습 정리

바깥 세계 요약.

하는 일	API	조심할 것
열기·닫기	<code>proven_fs_open/close</code>	scratch 할당자 필요
읽기	<code>proven_fs_read</code>	요청량 ≠ 읽은 양
쓰기	<code>proven_fs_write</code> / <code>_write_all</code>	부분 쓰기
디스크에 못박기	<code>proven_fs_sync</code>	<code>flush</code> ≠ <code>sync</code>
버퍼드 입출력	<code>proven_stream_*</code>	버퍼는 호출자가 준다
현재 시각	<code>proven_time_now_datetime</code>	경과 측정에는 쓰지 않는다
날짜 형식화	<code>proven_time_u8_fmt</code>	{year} 같은 이름 자리표시자
재현 난수	<code>proven_xoshiro256ss_*</code>	비밀에 쓰지 않는다
비밀용 난수	<code>proven_random_bytes</code>	없으면 거짓 — 물러나지 않는다
메모리 매핑	<code>proven_mmap_*</code>	이유가 있을 때만

이제 남은 것은 경계다 — 여러 일을 겹쳐 돌리는 방법, 그리고 운영체제가 아예 없는 자리. 마지막 장에서 이 부를 닫는다.

56 경계 — 겹쳐 돌리기, 그리고 OS가 없을 때

이 장이 끝나면

제11부의 마지막 장이다. 여러 일을 겹쳐 돌리는 두 가지 방법(스택리스 코루틴과 작업 시스템), 여러 갈래로 도는 프로그램에서 할당자와 포인터 출처가 왜 문제가 되는지, 그리고 운영체제가 아예 없는 자리에서 이 라이브러리가 어떤 모습이 되는지를 본다. 마지막 절은 **언제 쓰지 않는 것이 나은가**다.

돌아보기 10장에서 클록의 한계 때문에 코어가 여럿이 됐고, 거짓 공유 같은 사고가 생겼다고 했다. 그러면 “동시에 여러 일을 한다”는 말에는 몇 가지 서로 다른 뜻이 있는가?

답 적어도 둘이다. **겹쳐 돌리기**(concurrency)는 여러 일이 서로 기다리며 번갈아 진행되는 것으로, 코어가 하나여도 성립한다 — 입출력을 기다리는 동안 다른 일을 하는 서버가 그렇다. **동시에 돌리기**(parallelism)는 실제로 여러 코어가 같은 순간에 계산하는 것이다. 앞의 것은 **구조**의 문제이고 뒤의 것은 **성능**의 문제다. 이 장의 코루틴은 앞을, 작업 시스템은 뒤를 다룬다.

56.1 스택리스 코루틴 — 스레드 없이 겹쳐 돌리기

한 함수가 일을 하다 멈추고, 나중에 **그 자리에서** 다시 시작할 수 있으면 많은 것이 쉬워진다. 상태 기계를 손으로 짜는 대신 코드를 순서대로 쓸 수 있기 때문이다.

```
examples/ch56/coro.c

#include <proven.h>
#include <stdio.h>

/* 스택리스 코루틴: 진행 상태를 구조체 한 칸에 담고, 매크로가 switch 로
   그 자리에 되돌아간다. 스레드도, 별도 스택도, 할당도 없다. */
typedef struct {
    proven_coro_t co;      /* 어디까지 갔는지 기억하는 칸 */
    int          sent;
    int          limit;
} producer_t;

/* 0 을 돌려주면 "값 하나를 내놓고 잠시 멈춤", 1 이면 "다 끝남" */
static int produce(producer_t *p, int *out)
{
    PROVEN_CORO_BEGIN(&p->co);
    while (p->sent < p->limit) {
        *out = p->sent * 10;
        p->sent += 1;
        PROVEN_CORO_YIELD(&p->co); /* 여기서 나갔다가 다음 호출에 되돌아온다 */
    }
    PROVEN_CORO_END(&p->co);
}

int main(void)
{
    producer_t p = { .sent = 0, .limit = 4 };
    PROVEN_CORO_INIT(&p.co);

    int value = -1;
    while (!PROVEN_CORO_IS_DONE(&p.co)) {
        if (produce(&p, &value) == 0)
            printf("produced %d\n", value);
    }
}
```

```
printf("finished after %d values; coroutine state is %zu bytes\n",
      p.sent, sizeof(proven_coro_t));
return 0;
}
```

실행 결과

```
produced 0
produced 10
produced 20
produced 30
finished after 4 values; coroutine state is 4 bytes
```

핵심은 마지막 줄이다 — 이 코루틴이 기억하는 상태는 **4바이트**다. 별도 스택도, 스레드도, 할당도 없다. 매크로가 switch와 case로 재진입 지점을 만드는, C에서 오래된 요령(Duff's device의 사촌이다 — 27장에서 본 그 문법이 여기 다시 쓰인다)을 정리한 것이다.

대가도 분명하다. **지역 변수가 살아남지 않는다** — 멈췄다 돌아오면 사라지고 없으므로, 유지해야 할 상태는 구조체에 두어야 한다. 예제가 sent를 구조체 멤버로 둔 이유다. 그리고 switch로 구현되었으므로 PROVEN_CORO_YIELD를 다른 switch 안에 넣을 수 없다.

그래도 이 모델이 임베디드에서 사랑받는 이유는 뚜렷하다. 태스크 하나가 4바이트라면 수백 개를 띄워도 부담이 없고, 스택을 따로 잡지 않으니 기억 사용량을 컴파일 시간에 계산할 수 있다.

△ “코루틴은 스레드의 가벼운 버전이다”

하는 일이 아예 다르다. 스레드는 운영체제가 **마음대로 끼어들어** 전환 하지만(선점), 코루틴은 오직 자기가 YIELD라고 적은 자리에서만 멈춘다(협력). 그래서 코루틴들 사이에는 경쟁 조건이 생기지 않는다 — 두 코루틴이 같은 순간에 도는 일이 없기 때문이다. 대신 대가가 있다. 한 코루틴이 오래 계산하며 양보하지 않으면 나머지가 전부 굶는다. 그리고 코루틴은 코어를 여럿 쓰지 못한다 — 그것이 다음 절의 작업 시스템이 따로 있는 이유다.

56.2 작업 시스템 — 코어를 여럿 쓰기

proven_job_sys_t는 일감 큐와 작업자 스레드를 갖춘 작은 시스템이다. 큐의 크기가 **고정**이라는 점이 특징이다 — 넘치면 제출이 실패하고, 그 실패는 값으로 온다. 무한히 늘어나는 큐는 기억을 다 쓸 때까지 문제를 미루는 장치일 뿐이라는 판단이다.

✕ 닫기와 제출을 겹치기

```
/* 스레드 A */          /* 스레드 B */
proven_job_submit(sys, job); proven_job_system_destroy(sys);
```

헤더가 명시적으로 금지하는 무늬다. 올바른 순서는 **닫고(close), 생산자를 모두 합류시킨 뒤, 파괴(destroy)**다. 이런 순서 계약은 자료구조 안쪽에 락을 숨겨서는 해결되지 않는다 — 오히려 숨겨진 락이 있으면 “된다고 착각하는” 코드가 늘어난다. 이 라이브러리가 컨테이너에 락을 넣지 않고 **공유 변경은 호출자가 동기화한다**고 못박은 이유이기도 하다.

56.3 스레드, 할당자, 그리고 출처

여러 갈래로 도는 프로그램에서 할당자는 특별한 주의를 요구한다. 아레나 하나를 두 스레드가 함께 쓰면, 51장에서 본 “앞에서부터 잘라 주는” 그 단순한 동작이 곧바로 경쟁이 된다. 처방은 대개 **스레드마다 하나씩** 두는 것이다 — 그러면 동기화가 아예 필요 없다.

12장에서 이름만 보았던 포인터 출처(provenance)도 여기서 실무가 된다. 같은 주소라도 어느 할당에서 온 포인터인지가 의미를 가지므로, 어떤 할당자에서 얻은 블록을 다른 할당자에게 돌려주는 것은 (51장의 반례처럼) 주소가 우연히 맞더라도 계약 위반이다. 라이브러리의 이름이 여기서 왔다.

56.4 OS가 없을 때 — 프리스탠딩

이 라이브러리의 모양을 가장 크게 결정한 제약이 이것이다. 46장에서 표준 라이브러리가 얇은 이유로 든 바로 그 환경 — 운영체제도, 힙도, 파일도 없는 자리 — 에서 돌아야 한다는 요구다.

프리스탠딩 빌드에서 달라지는 것은 이렇다.

- platform/을 아예 넣지 않는다. 파일·시간·OS 난수가 사라진다.
- 할당은 정적 버퍼 위의 아레나로 한다(51장). malloc은 없다.
- 문자열은 _borrow로 스택·정적 버퍼 위에서 다룬다(52장).
- 패닉 처리기를 직접 등록한다 — 인쇄할 콘솔이 없을 수 있으므로, LED를 켜거나 위치독을 물리거나 재부팅하는 식이 된다.
- 실수 형식화는 큰 정수 연산을 쓰므로, 필요 없으면 통째로 뺄 수 있다.

앞 장들에서 “할당자를 인자로 받는다”, “뷰는 빌린 것이다”, “숨은 전역이 없다” 같은 규율이 왜 그렇게 집요했는지가 여기서 드러난다. 그 규율들은 취향이 아니라 **이 환경에서 돌기 위한 최소 조건**이었다.

● 같은 코드를 두 세계에서 — 그리고 그 대가

임베디드와 호스트 양쪽에서 도는 코드를 한 벌로 유지하는 것은 실제로 가치가 크다. 프로토콜 파서를 PC에서 테스트하고 그대로 펌웨어에 넣을 수 있다면, 디버깅 환경이 없는 보드 위에서 해매는 시간이 크게 줄기 때문이다. 많은 임베디드 팀이 “호스트에서 도는 시험용 빌드”를 별도로 유지하는 이유이고, 그 구조를 가능하게 하는 것이 정확히 **플랫폼 의존을 한 층에 가두는** 설계다. 대가는 API가 조금 더 격식을 갖추게 된다는 것 — 어디서나 통하는 코드는 어디에도 특화되지 않는다.

56.5 언제 쓰지 않는 것이 나은가

정직하게 달는다. 이 라이브러리가 답이 아닌 경우가 있다.

- **짧은 스크립트성 프로그램** — 스무 줄짜리 도구에 소유권 규율과 할당자 매개변수는 격식일 뿐이다. 표준 라이브러리로 충분하다.
- **이미 다른 규약을 가진 코드베이스** — 큰 프로젝트에는 대개 자체 문자열·컨테이너·에러 규약이 이미 있다. 두 규약이 섞이면 경계마다 변환 코드가 생기고, 그 경계가 새 버그의 자리가 된다.
- **C++·Rust처럼 이미 그 문제를 언어가 푸는 곳** — 소유권과 에러 전파를 언어가 다루는 환경에서 굳이 C 라이브러리를 엮을 이유는 적다.
- **표준 함수 하나면 끝나는 일** — 이 책이 46장에서 말한 그대로다. 옳은 태도는 “쓰지 않기”도 “무조건 쓰기”도 아니라 **계약을 알고 고르기**다.

문 그러면 이 부에서 배운 것 중 proven을 쓰지 않아도 남는 것은 무엇인가?

답 거의 전부다. 길이를 지니고 다니는 문자열, 자르지 않고 거부하는 쓰기, 값으로 돌아오는 에러, 시그니처에 드러나는 할당, 소유와 빌림의 구분, 검사 산술로 하는 크기 계산, 전순서를 지키는 비교자, 적대적 입력을 가정한 해시 — 이것들은 라이브러리가 아니라 **설계 원칙**이고, 어떤 C 코드에도 손으로 적용할 수 있다. 47장의 표를 다시 펴 보면 오른쪽 열이 전부 그런 항목이다. 이 부가 정말로 팔고 싶었던 것은 코드가 아니라 그 열이다.

≡ 복습 정리

제11부 전체 요약 — 문제와 답.

47장의 문제	답	장
크기를 모르는 문자열	뷰(ptr+size), 거부하되 자르지 않기	50·52
확인되지 않는 실패	값으로 오는 에러, [[nodiscard]]	49
서식 불일치	{ } 와 PROVEN_ARG (_Generic)	53
불분명한 소유권	할당자 매개변수, 소유 대 빌림	51
검사되지 않는 콜백	문서화된 계약, introsort, 키드 해시	54
바이트의 숨은 타입	proven_byte_t	50
OS가 없는 환경	platform 층 분리, 정적 아레나	56

이것으로 이 책이 1장에서 밝힌 약속을 지켰다 — C의 문제들을 먼저 보이고, 그에 대한 하나의 대답을 끝까지 보였다. 남은 두 장은 지면 밖의 이야기다. 다음 장에서 실무의 지형(빌드 도구, 버전 관리, 실제 프로젝트들)을 둘러보고, 마지막 장에서 이 책이 지나온 길을 정리한다.

제12부 — 닫으며

57 실전의 C — 도구, 프로젝트, 그리고 영역

이 장이 끝나면

책을 닫기 전에 지면 밖의 세계를 둘러본다. 실무의 두 필수 도구(make와 git), 오늘도 순수 C로 도는 대표 프로젝트들과 그들이 C를 고른 이유, 임베디드에서 C가 하는 일, 그리고 C의 한계를 실전에서 어떻게 넘는지 까지. 이 책이 가르친 언어가 실제로 어디에 서 있는지의 지도다.

돌아보기 43장에서 여러 파일 프로그램은 각 파일을 목적 파일로 만들어 두었다가 링크만 다시 하는 것이 관행이라 했다. 파일이 수백 개인 프로젝트에서 그 일을 사람이 손으로 할 수는 없을 텐데 — 무엇이 대신하는가?

답 빌드 도구다. 파일 사이의 의존 관계를 적어 두면, 무엇이 바뀌었는지 보고 다시 만들 것만 골라 만들어 준다. C 세계의 고전이자 여전한 표준이 make이고, 이 장이 그 도구부터 시작한다.

57.1 make — 무엇을 다시 만들 것인가

make의 착상은 한 줄이다 — **결과물이 재료보다 오래됐으면 다시 만든다.** 규칙을 적는 문법도 그 착상 그대로다:

```
hello: hello.c greet.c greet.h      # 목표: 재료들
cc -Wall -Wextra -o hello hello.c greet.c  # 만드는 법(탭으로 시작)
```

make hello라고 부르면 파일들의 시작을 비교해, 필요할 때만 명령을 실행한다. 파일 하나만 고쳤을 때 전체를 다시 컴파일하지 않는 것 — 14장의 릴레이와 43장의 분할이 실무에서 시간을 아끼는 방식이 이것이다.

make는 1976년에 만들어져 지금도 쓰인다. 문법의 과벽(명령 줄은 반드시 탭으로 시작해야 한다 — 만든 사람 스스로 “그때는 사용자가 열둘뿐이었다”고 회고한 실수다)까지 그대로 남았지만, “의존 관계를 적으면 필요한 것만 다시 만든다”는 착상은 이후 모든 빌드 도구의 뿌리가 됐다. 오늘의 큰 프로젝트는 CMake·Meson·Ninja 같은 도구를 없어 쓰지만, 그 도구들이 결국 만들어 내는 것도 대개 make류의 규칙이다.

57.2 git — 무엇이 언제 바뀌었나

두 번째 필수 도구는 **버전 관리**다. 코드의 모든 변경을 기록해 두고, 언제든 되돌리고, 여럿이 같은 코드를 고쳐도 합칠 수 있게 하는 도구 — 오늘의 사실상 표준이 git이다.

일상의 명령은 다섯이면 시작할 수 있다.

```
$ git init          # 이 폴더를 저장소로
$ git add hello.c    # 기록할 변경을 고른다
$ git commit -m "첫 프로그램"  # 이유와 함께 기록한다
$ git log           # 지나온 기록을 본다
$ git diff          # 지금 무엇이 바뀌었나
```

git이 이 책의 주제와 이어지는 대목이 하나 있다 — **git 자신이 순수 C로 쓰인 프로그램이다.** 2005년 리누스 토르발스가 리눅스 커널 개발을 위해 며칠 만에 초기판을 만들었고, 지금은 세계에서 가장 널리 쓰이는 개발 도구가 됐다. 8장에서 본 CRLF 경고를 내는 그 도구가, 이 책이 가르친 언어로 만들어져 있는 것이다.

57.3 오늘도 C로 도는 것들 — 그리고 왜

1장에서 “C는 어디에나 있다”고 했다. 이제 구체적인 이름과 이유를 적을 수 있다.

프로젝트	무엇	왜 C인가
Linux 커널	운영체제의 심장	하드웨어를 직접 다뤄야 하고, 런타임 의존이 없어야 하며, 모든 아키텍처에 컴파일러가 있다
SQLite	세계에서 가장 많이 배포된 데이터베이스	어디에나 이식되어야 하고(휴대폰·항공기·브라우저), 단일 소스 파일로 배포하는 극단적 단순함이 필요했다
FFmpeg	영상·음성 처리의 표준 도구	성능이 전부인 코덱 작업, 하드웨어 가속 API와의 직접 연동
Redis	메모리 기반 데이터 저장소	예측 가능한 지연시간과 메모리 제어 — 9장의 사다리를 직접 관리한다
curl	인터넷 전송의 만능 도구	모든 플랫폼·모든 기기에 올라가야 한다(자동차·TV·우주선까지)
CPython·Ruby 등	다른 언어의 구현체	언어 런타임은 결국 가장 낮은 층이 필요하고, C 인터페이스가 생태계의 공용어다
OpenSSL·zlib	암호화·압축 라이브러리	모든 언어에서 불러야 하므로 — C ABI가 사실상의 만국 공통 인터페이스

공통점을 뽑으면 C를 고르는 이유가 네 가지로 정리된다.

- **가장 낮은 층에 서야 할 때** — 운영체제, 드라이버, 런타임.
- **어디에나 옮겨야 할 때** — 새 칩이 나오면 C 컴파일러부터 만들어진다(2장의 이식성 혁명이 반세기째 유효하다).
- **성능과 자원을 손으로 통제해야 할 때** — 메모리 배치, 지연시간, 실행 파일 크기.
- **다른 모든 언어와 대화해야 할 때** — 파이썬·자바·리스트 어느 쪽에서든 C 함수는 부를 수 있다. C ABI가 언어들 사이의 국제 공용어다.

57.4 임베디드 — C의 본진

일상에서 눈에 안 띄지만 물량으로는 가장 큰 C의 영역이 임베디드다. 세탁기와 에어컨의 제어판, 자동차의 수십 개 제어 장치, 의료 기기, 드론과 로봇의 저수준 제어, 통신 모듈의 펌웨어 — 대부분이 C로 쓰인다.

이유는 그 환경의 제약에 있다. 메모리가 킬로바이트 단위이고, 운영체제가 없을 수도 있고(4장의 0번지가 인터럽트 벡터인 그 세계), 실행 시간이 예측 가능해야 하며(가비지 컬렉션 같은 예측 불가능한 멈춤이 허용되지 않는다), 전력이 제한된다. C는 이런 환경에서 **실행되지 않는 것에는 비용을 물리지 않는 언어**라 자연스럽게 맞는다 — 표준 라이브러리 없이 쓸 수 있고(46장의 프리스탠딩), 생성되는 코드가 예측 가능하다.

임베디드의 C에는 이 책의 지식이 유난히 직접적으로 쓰인다 — 특정 주소를 장치 레지스터로 다루기(4장), `volatile`로 최적화를 막기(11장), 비트 연산으로 하드웨어 플래그를 조작하기(5·24장), 동적 할당을 아예 쓰지 않기(37장), 정렬과 표현을 의식하며 통신 프로토콜을 다루기(3·39장).

57.5 한계를 넘는 법 — 실전의 요령

C의 결핍은 분명하다 — 이름 공간이 없고, 제네릭이 없고, 자원 정리가 자동이 아니고, 안전이 강제되지 않는다. 실전은 이것을 언어 밖의 규율과 도구로 메운다. 이 책이 지나온 것들을 실무 지침으로 모으면 이렇다.

- **이름 공간이 없다** → 접두어 규약(35장의 `proven_`)과 파일 수준 `static`(43장의 내부 연결).
- **제네릭이 없다** → 매크로로 코드 생성(44장의 `##`, `X` 매크로), 또는 `void *`와 크기·비교 함수를 받는 일반화(표준의 `qsort`가 그 방식이다), 또는 C11의 `_Generic` 선택.
- **자원 정리가 자동이 아니다** → 소유 규약을 이름과 문서로 명시하고 (37장), 한 함수의 정리 지점을 한 곳으로 모으는 관용구(정리 라벨로 `goto`하는 패턴은 커널에서 표준 관행이다).
- **안전이 강제되지 않는다** → 그물을 겹친다(15장): 경고 최대, 새니타이저, 두 컴파일러 교차, 정적 분석 도구, 그리고 검사가 내장된 부품(제11부).
- **표준 라이브러리가 얇다** → 필요한 층을 고르거나 만든다. `vendor`로 들여와 빌드를 단순하게 유지하는 것이 작은 프로젝트의 정석이다.
- **큰 프로젝트의 복잡도** → 컴파일 단위를 작게 나누고 헤더로 계약을 드러내며(43장), 테스트와 빌드를 자동화한다(이 장의 `make·git`).

● 섞어 쓰는 현실 — C는 사라지지 않고 층이 된다

오늘의 큰 시스템은 대개 한 언어로만 쓰이지 않는다. 성능이 필요한 핵심부는 C(또는 C++·Rust)로, 그 위의 논리는 파이썬·자바스크립트로 쓰는 층 구조가 일반적이다. 파이썬의 수치 계산 라이브러리가 빠른 이유는 그 밑에 C·포트란이기 때문이고, 브라우저가 영상을 재생하는 것도 밑에 C 코덱이 있기 때문이다. 1장에서 Rust와 Zig가 C의 영역을 잠식한다고 했지만, 실제로 벌어지는 일은 대체보다 공존에 가깝다 — 새 언어들도 기존 C 자산과 대화해야 하므로 C 인터페이스를 갖추고 태어난다. C를 읽고 쓸 줄 안다는 것은, 그래서 어느 층에서 일하든 아래층을 들여다볼 수 있다는 뜻이다.

문 이 책을 다 읽은 사람이 다음으로 만들어 볼 만한 것은 무엇인가?

답 작고 끝이 있는 것이 좋다. 텍스트 파일을 읽어 통계를 내는 도구, 간단한 설정 파일 파서(53장의 스캐너가 그대로 쓰인다), 자료구조 하나를 직접 구현해 보기(38장의 연결 구조), 좋아하는 작은 C 프로젝트의 소스를 읽고 한 곳 고쳐 보기. 마지막 것이 특히 권할 만하다 — 남의 코드를 읽는 능력이 실무에서 쓰는 시간의 대부분을 차지하고, 이 책은 그 읽기를 위한 눈을 기르는 데 상당 부분을 썼다.

지면 밖의 지형까지 둘러보았다. 다음 장에서 이 책이 지나온 길을 한 번 되짚고 책을 닫는다.

58 모던 C 총정리

이 장이 끝나면

마지막 장이다. 57장에서 실전의 지형까지 둘러보았으니, 이제 지나온 길을 정리한다. 이 책에서 자연스럽게 써 온 C23의 얼굴들을 모아 회고 하고, 모던 C의 관행을 한 장의 지침으로 정리하며, 여기서 어디로 갈 수 있는지 이야기한다. 전권 회고 문답으로 책을 닫는다.

돌아보기 1장에서 이 책은 “옛 어법의 관성으로 쓰인 입문서가 아니라 오늘의 C에서 출발하는 입문서”를 표방했다. 그러면 이 책을 다 읽은 지금, 독자가 배운 C는 옛 C와 무엇이 다른가?

답 문법의 목록이 다른 것이 아니라 **기준이 다르다**. 옛 어법의 책은 “이렇게 쓰면 동작한다”를 가르치고, 이 책은 “이렇게 쓰면 계약 안이다”를 가르쳤다 — 초기화하고 쓰기(20장), 경계와 크기를 함께 다루기(33장), 실패를 값으로 확인하기(41장), 계약 밖의 지름길을 피하기(42장), 도구로 검사하기(15장). C23의 새 문법들은 그 기준을 더 **쉽게 지키게** 해 주는 도구로 등장했다. 아래 회고가 그 목록이다.

58.1 회고 — 이미 써 온 C23

이 책은 새 표준을 따로 소개하지 않고, 필요한 자리에서 자연스럽게 썼다. 모아 보면 이만큼이다.

C23의 것	처음 만난 곳	무엇을 쉽게 만들었나
bool·true·false 키워드	26장	헤더 없이 판단을 값으로
nullptr	31장	NULL의 모호함 제거
2의 보수 확정	5장	부호 표현의 이식성 고민 종료
[[nodiscard]]	35·41장	실패 확인을 잊는 실수를 컴파일러가 차단
[[fallthrough]]	27장	switch 폴스루의 의도를 문서로
ckd_add 계열	42장	오버플로를 UB가 아니라 값으로
alignof	32장	정렬 요구를 언어 차원에서 질의
삼중자 제거	7장	문자 코드 시대의 흉터 정리
va_start의 완화	45장	가변 인자의 군더더기 제거

한 방향이 보인다 — **계약을 더 명시적으로, 실수를 더 빨리 잡히게**. 1장에서 “C 자신도 변화의 조짐을 보이고 있다”고 한 것의 구체적 내용이 이 표다.

58.2 모던 C 관행 — 한 장의 지침

이 책 전체를 실무 수칙으로 압축하면 이렇다.

빌드와 도구. 경고를 켜다(-Wall -Wextra). 시험 실행은 새니타이저와 함께(ASan·UBSan). 가능하면 두 컴파일러로 교차 검증한다(15장).

선언과 타입. 선언과 동시에 초기화한다. 바꾸지 않을 것은 const로 둔다. 크기가 계약인 자리에는 <stdint.h>의 고정 폭 타입을 쓴다. 크기·인덱스는 size_t 계열로 일관되게(25장의 부호 섞임 함정).

기억. 경계는 언제나 명시적으로 — 배열과 길이는 함께 다닌다. 포인터는 쓰기 전에 유효성을 확인한다. 동적 할당은 소유자를 정하고, 해제 후에는 포인터를 비운다. 가능하면 자동 수명으로 해결한다(36·37장).

오류. 실패는 값으로 알리고, 반환값은 확인한다. 내부 불변식은 assert, 바깥에서 온 것은 실행 중 검사(41장).

계약. “내 컴퓨터에서 돌아다”가 아니라 “추상 기계 위에서 옳은가”로 판단한다(12·42장).

부품. 표준 라이브러리의 계약을 알고 고르고, 반복되는 위험 지대에는 검사가 내장된 부품을 엮는다(46장·제11부).

문 여기서 어디로 가면 좋은가?

답 세 갈래를 권한다. **깊이** — 이 책이 문을 열어 둔 곳들: 자료구조와 알고리즘(38장의 연결 구조), 동시성(10장의 멀티코어와 C11 원자), 시스템 프로그래밍(파일·네트워크는 플랫폼 API의 세계다). **너비** — 이웃 언어들: C++·Rust·Zig를 볼 때, 이 책에서 배운 기억·계약·표현의 감각이 그대로 통역된다(1장에서 예고한 대로다). **실전** — 57장에서 이름을 든 프로젝트들의 소스를 직접 읽는 것. SQLite 나 curl 같은 잘 쓰인 C 코드는 이 책의 모든 주제가 실물로 살아 있는 교재다.

그리고 표준 문서 자체가 이제 읽을 수 있는 텍스트가 되었다는 것도 기억해 둘 만하다 — “계약서”라는 관점으로 보면, 그 무뚝뚝한 문서가 왜 그렇게 쓰였는지 이해된다.

58.3 다음에 읽을 책

이 책 다음에 무엇을 읽을 것인가. 답하기 전에 이 언어의 사정을 하나 짚어 두어야 한다. C는 입문서 다음 칸이 유난히 비어 있는 언어다.

C++ 쪽은 사정이 다르다. 게임 엔진을 비롯해 큰 응용을 C++로 짜는 자리가 많다 보니 입문서 위로 중급서·설계서·관용구 모음이 층층이 쌓여 있다. 반면 C는 여전히 중요한 곳(운영체제, 펌웨어, 인프라)에서 쓰이지만, 그 자리에서 일하는 사람들은 대개 이미 숙련자다. 그래서 시장에는 “처음 배우는 C”와 “표준 문서·소스 코드”만 남고, 그 사이를 이어 주는 책이 드물었다. 이 책이 굳이 컴퓨터 이야기부터 시작해 계약과 라이브러리 설계 까지 온 것도 그 빈칸을 의식했기 때문이다.

다행히 최근 몇 해 사이 이 칸을 메우는 책들이 나오고 있다. C11 이후의 변화와 C23의 확정이 새 책을 쓸 이유를 만들어 준 덕이다. 아래는 그중 읽어 볼 만한 것들이다.

표준을 기준으로 삼는 교재 — 옌스 구스테트, 『모던 C』

저자 옌스 구스테트(Jens Gustedt)는 프랑스 INRIA의 연구책임자이자 C 표준 위원회(ISO WG14)의 위원으로, C17(ISO/IEC 9899:2018)의 공동 편집자로 참여한 사람이다. 즉 **표준 문서를 만드는 쪽에서 쓴 교재**다.

내용은 난이도를 0에서 3까지 층으로 나누어, 기초부터 시작해 스레드와 원자적 연산, 메모리 일관성 모형까지 올라간다. 오래된 관행 대신 오늘의 표준을 기본값으로 삼는다는 점에서 이 책과 방향이 같고, 깊이는 훨씬 깊다. “C의 바이블”이라는 자리는 전통적으로 K&R의 것이지만, **오늘의 표준을 기준으로 한 단권 교재**로는 현재 이만한 것이 없다.

두 가지를 함께 알아 두면 좋다. 첫째, 한국어판(길벗, 2022)은 원서 2판을 옮긴 것이라 C17까지를 다룬다. C23을 다루는 것은 2024년에 나온 원서 **3판**이다. 둘째, 저자가 이 책의 전자판을 무료로 공개해 두었다 — INRIA의 공개 저장소(HAL)에서 받을 수 있다. 번역서를 읽다가 표현이 걸리거나 C23의 최신 내용이 필요하면 원서를 함께 펴 보는 방법이 있다는 뜻이다.

커리큘럼이 잘 짜인 정통 교과서 — K. N. 킹, C Programming: A Modern Approach

대학 교재로 널리 쓰인 책이고, 개념을 한 번에 쏟지 않고 여러 번에 걸쳐 조금씩 깊게 되돌아오는 **나선형** 구성이 특징이다(이 책이 택한 방식과 같다). 설명이 차분하고 예제가 정직해서, 입문서를 뒀 뒤 기초를 다시 탄탄히 다지고 싶을 때 좋은 선택이다.

아쉬운 점 둘. 정식 한국어판이 없고, 2판(2008)이 C89와 C99를 기준으로 한다. 다만 이 “오래됨”은 상황에 따라 장점이 되기도 한다 — 실무에서 C99까지만 허용하는 오래된 코드베이스나 임베디드 도구 체인을 다뤄야 한다면, 그 세계를 정확히 기준으로 삼은 책이 오히려 맞다.

설계와 패턴의 빈칸 — 크리스토퍼 프레세른, 『우아한 C언어 코딩 패턴』

원서는 오라일리에서 2022년에 나온 *Fluent C: Principles, Practices, and Patterns*다. 이 책이 다루는 것은 문법이 아니라 **구조에 대한 결정**이다 — 에러를 어떻게 돌려줄 것인가, 소유권과 수명을 어떻게 표시할 것인가, 유연한 인터페이스와 반복자를 어떻게 설계할 것인가. 이 책 제11부에서 계약이라는 이름으로 다룬 문제들이, 거기서는 패턴 목록의 형태로 정리되어 있다.

C를 배운 뒤 “이제 이걸로 뭘 어떻게 짜야 하지”라는 자리에서 막히는 독자에게 특히 맞다. 다만 평가가 갈리는 지점도 있다. 영어권 서평들은 공통적으로 **입문자용이 아니라는 점**을 강조한다 — 비자명한 프로그램을 하나쯤 써 본 뒤에 읽어야 값이 나온다는 것이다. 개발자 단체 ACCU의 서평은 “조건부 추천”을 주면서, 에러 처리에 두 장을 쓰면서도 시험(test)을 다루지 않는 점과 일부 패턴(지연 정리 같은)의 적절성에 의문을 제기했다. 패턴 목록을 규범이 아니라 **선택지의 목록**으로 읽으면 그 지적을 감안하고도 얻을 것이 많다.

컴파일러 쪽에서 본 C — 페터르 판데르린던, 『컴파일러 개발자가 들려주는 C 이야기』

원서는 *Expert C Programming: Deep C Secrets* (1994년)이다. 저자는 썬 마이크로시스템즈에서 14년간 컴파일러와 커널을 만든 사람이고, 그래서 설명이 **구현하는 쪽의 시선**으로 되어 있다. 선언을 읽는 법, 배열과 포인터가 왜 같지 않은지, 링크와 메모리 배치가 실제로 어떻게 이루어지는지 같은 주제들이 이 책에서 특히 선명하다. 저자 자신이 “모든 프로그래머의 두 번째 C 책”을 표방했을 만큼, 입문 다음의 자리를 겨냥한 책이기도 하다.

이 책의 또 다른 특징은 **재미있다**는 것이다. 영어권 독자들이 30년째 추천하는 이유의 절반은 유머와 일화이고, 1988년 인터넷 웹 사건 같은 실제 사고를 코드 수준에서 뜯어보는 대목이 대표적이다.

주의할 점 하나. 한국어판이 최근(2022)에 나왔을 뿐 **원서는 1994년 책**이다. 그래서 지금은 사라진 도구와 환경 이야기, 낡은 관행이 섞여 있고, 마지막의 C++ 소개도 오늘의 C++와는 거리가 멀다. 영어권 서평에서도 “고전이지만 유효한 부분을 골라 읽어야 한다”는 평이 함께 나온다. 이 책에서 얻을 것은 최신 정보가 아니라 **구현을 아는 사람의 사고방식**이다.

결들여 볼 만한 것

- 로버트 시코드(Robert Seacord)의 *Effective C*는 “입문 다음”을 정확히 겨냥한 또 한 권이고, 2판(2024)이 C23을 반영했다. 저자는 오랫동안 보안 코딩 표준 작업을 해 온 사람이라 위험한 관행에 대한 서술이 특히 단단하다. 다만 한국어판은 아직 없다.
- 그리고 결국 가장 정확한 참고 문헌은 **표준 문서 자체**다(앞 절에서 말한 대로다). 초안은 무료로 공개되어 있고, 이 책을 여기까지 읽은 독자라면 적어도 “무엇을 찾아볼 것인가”는 알게 되었을 것이다.

58.4 마지막으로

이 책은 2장에서 컴퓨터를 세 부품으로 그리는 것으로 시작했다. 그 그림은 곧 정직하게 무너졌고 — 기억은 사다리가 되고, 실행은 겹쳐지고, 편집자가 끼어들었다 — 그 자리에 남은 것이 **계약**이었다. C를 안다는 것은 문법을 외우는 일이 아니라 그 계약을 읽을 줄 아는 일이라는 것, 이 책이 하고 싶었던 말은 그것 하나다.

읽는 것만으로 여기까지 온 독자에게, 이제 남은 것은 하나뿐이다 — 읽은 것을 자기 코드로 써 보는 것. 좋은 여정이 되기를.

부록

부록 A — 연산자 우선순위와 결합

본문(17장)은 “표를 외우지 말고 괄호를 쓰라”고 권했다. 이 표는 외우기 위한 것이 아니라, 남의 코드를 읽다 막혔을 때 찾아보기 위한 것이다. 위쪽이 더 강하게 묶는다.

묶음	연산자	결합
후위	() [] . -> ++(후위) --(후위), 복합 리터럴	원→오
단항	++(전위) --(전위) + - ! ~ (타입) * & sizeof alignof	오→원
곱셈류	* / %	원→오
덧셈류	+ -	원→오
시프트	<< >>	원→오
관계	< <= > >=	원→오
상등	== !=	원→오
비트 AND	&	원→오
비트 XOR	^	원→오
비트 OR		원→오
논리 AND	&&	원→오
논리 OR		원→오
조건	?:	오→원
대입	= += -= *= /= %= &= ^= = <<= >>=	오→원
쉼표	,	원→오

특히 헛갈리는 세 자리를 적어 둔다.

- **비트 연산이 비교보다 약하다** — $a \& b == c$ 는 $a \& (b == c)$ 다. 비트 검사에는 괄호가 사실상 필수다.
- **시프트가 덧셈보다 약하다** — $a \ll 1 + 2$ 는 $a \ll 3$ 이다.
- **대입은 오른쪽 결합** — $a = b = 0$ 은 $a = (b = 0)$ 으로 읽힌다.

우선순위는 묶는 규칙일 뿐이고 평가의 시간 순서가 아니라는 것(17·29장) 을 함께 기억하면 된다.

부록 B — printf·scanf 서식 요약

19장에서 최소 집합(%d, %s, %)만 익혔다. 자주 쓰는 나머지를 모아 둔다. 서식과 인자의 타입이 어긋나면 계약 밖이라는 것(19·42장)을 잊지 않는다.

출력 변환

서식	인자 타입	뜻
%d %i	int	십진 정수
%u	unsigned int	부호 없는 십진
%x %X	unsigned int	16진(소문자·대문자)
%o	unsigned int	8진
%f %F	double	고정 소수 표기
%e %E	double	지수 표기
%g %G	double	짧은 쪽 자동 선택
%c	int(문자로 진급)	문자 하나
%s	char *	NUL 종단 문자열
%p	void *	포인터(형식은 구현 정의)
%zu	size_t	크기 값
%lld %llu	long long 계열	긴 정수
%%	—	퍼센트 글자

폭과 정밀도는 %8.3f(폭 8, 소수 3자리), %-10s(왼쪽 정렬), %.*s (길이를 인자로 — 8장의 뷰를 찍을 때 요긴하다)처럼 적는다.

입력 해석

sscanf 계열의 서식은 출력과 대체로 같되, 세 가지가 다르다.

- 인자는 **답을 곳의 주소여야 한다**(&n, 배열은 그대로).
- %s는 공백까지 읽고 멈추며, **반드시 최대 폭을 준다** — %99s처럼. 폭 없는 %s는 35장의 gets와 같은 위험이다.
- 반환값은 **해석에 성공한 항목 수**다. 확인하지 않으면 실패를 놓친다 (22·41장).

긴 정수·크기 타입에는 %lld, %zu 대신 입력 쪽 지정자를 정확히 맞춰야 하고, 정밀한 해석이 필요하다면 53장의 proven 스캐너처럼 실패가 값으로 드러나는 도구를 쓰는 편이 안전하다.

부록 C — 암묵 변환 요약

25장의 규칙을 표로 압축한 것이다.

정수 승격

bool, char, signed char, unsigned char, short, unsigned short, 비트 필드가 산술에 참여하면:

- 값들이 전부 int에 담기면 → int
- 아니면 → unsigned int

즉 산술은 int보다 좁은 타입 위에서 일어나지 않는다.

통상 산술 변환 (두 피연산자의 공통 타입)

1. 한쪽이 long double → 둘 다 long double
2. 아니고 한쪽이 double → 둘 다 double
3. 아니고 한쪽이 float → 둘 다 float
4. 둘 다 정수 → 각각 정수 승격 후:
 - 부호가 같으면 → 더 넓은 쪽
 - 부호 없는 쪽이 더 넓거나 같으면 → **부호 없는 쪽**
 - 부호 있는 쪽이 더 넓고 상대 값을 다 담으면 → 부호 있는 쪽
 - 그 밖 → 부호 있는 쪽의 부호 없는 판

실무 요약: **섞이면 넓은 쪽으로, 폭이 같으면 부호 없는 쪽이 이긴다.** 부호 섞인 비교는 피한다(경고를 켜 두면 컴파일러가 짚어 준다).

가변 인자의 기본 진급

원형에 타입이 없는 자리(...)로 넘어가는 인자:

- float → double
- bool·char·short 계열 → int(또는 unsigned int)

그래서 printf에 float 전용 서식이 없고, %f가 double을 받는다 (19·25·45장).

그 밖의 자주 만나는 변환

- **축소**(넓은 → 좁은 정수): 상위 비트가 잘린다(5장). 부호 있는 타입에서 값이 안 들어가면 구현 정의 동작이다.
- **실수** → **정수**: 소수부를 버린다(0을 향해). 범위를 벗어나면 계약 밖이다.
- **배열** → **포인터**: 대부분의 문맥에서 첫 원소 주소로 무너진다. 예외는 sizeof, alignof, &(33장).
- **함수** → **포인터**: 함수 이름은 함수 포인터로 무너진다.